

**MINISTÉRIO DA DEFESA
EXÉRCITO BRASILEIRO
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
CURSO DE GRADUAÇÃO EM ENGENHARIA CARTOGRÁFICA**

**VINICIUS HENEQUIM CORRÊA
DANIEL SANTANA**

**BUSCA DE GEOINFORMAÇÃO EM CATÁLOGO DE DADOS ESPACIAIS
UTILIZANDO MODELOS DE LINGUAGEM DE GRANDE PORTE COM
TOOL CALLING**

**RIO DE JANEIRO
2026**

©2026

INSTITUTO MILITAR DE ENGENHARIA

Praça General Tibúrcio, 80 – Praia Vermelha

Rio de Janeiro – RJ CEP: 22290-270

Este exemplar é de propriedade do Instituto Militar de Engenharia, que poderá incluí-lo em base de dados, armazenar em computador, microfilmар ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es) e do(s) orientador(es).

Corrêa, Vinicius Henequim; Santana, Daniel.

Busca de Geoinformação em Catálogo de Dados Espaciais Utilizando Modelos de Linguagem de Grande Porte com Tool Calling / Vinicius Henequim Corrêa e Daniel Santana. – Rio de Janeiro, 2026.

171 f.

Orientador(es): Ivanildo Barbosa.

Projeto Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia Cartográfica, 2026.

1. Modelos de Linguagem de Grande Porte. 2. Tool Calling. 3. Catálogo de Dados Espaciais. 4. Geoinformação. 5. Ollama. 6. LangChain. 7. Avaliação de Modelos. i. Barbosa, Ivanildo (orient.) ii. Título

VINICIUS HENEQUIM CORRÊA
DANIEL SANTANA

BUSCA DE GEOINFORMAÇÃO EM CATÁLOGO DE DADOS ESPACIAIS
UTILIZANDO MODELOS DE LINGUAGEM DE GRANDE PORTE COM
TOOL CALLING

Projeto Final de Curso apresentado ao Curso de Graduação em Engenharia Cartográfica do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia Cartográfica.

Orientador(es): Ivanildo Barbosa, DSc

Rio de Janeiro

2026

A Cristo Rei.

RESUMO

O acervo cartográfico da Diretoria de Serviço Geográfico (DSG) é organizado por nomenclaturas técnicas — Índice de Nomenclatura (INOM), Mapa Índice (MI), escalas e tipos de produto do Sistema Cartográfico Nacional — cujo domínio é uma barreira para usuários não especialistas. O 1º Centro de Geoinformação (1º CGEO) desenvolveu um protótipo de busca em linguagem natural com Saída Estruturada e um único modelo local (Phi-4 14B), que dependia de um dicionário de normalização e não dispunha de protocolo de avaliação. Este Projeto Final de Curso aprimorou a **camada de tradução linguística** dessa solução — a etapa em que a consulta livre é convertida em parâmetros estruturados de busca —, reimplementando o *backend* em Python e substituindo a Saída Estruturada por *Tool Calling* nativo, sem dicionário e sem exemplos no *prompt*. Para medir a tradução, construiu-se um *dataset* de 310 consultas, com gabarito definido por um manual de anotação e auditoria em três etapas. Um *pipeline* reproduzível avaliou quatro modelos abertos locais, via Ollama, e, como referência, três em nuvem, medindo acurácia, F1 por campo e latência, com testes de significância. Com *Tool Calling*, Gemma 4 E4B, Gemma 4 E2B e Qwen 3 4B tiveram acurácias estatisticamente indistinguíveis, de 60,2%, 56,5% e 54,2%. Com os mesmos modelos e consultas, numa só chamada, a Saída Estruturada foi mais precisa em três dos quatro modelos locais, e o ganho consistente do *Tool Calling* foi a recusa explícita das consultas fora do domínio. O método do protótipo acertou 58,8%, acurácia estatisticamente igual à dos melhores modelos locais, menores e mais de quinze vezes mais rápidos. Um lote de validação independente, de 1.929 consultas, repetiu a comparação com o Gemma 4 E4B: o *Tool Calling* recusou com mais precisão, e a Saída Estruturada extraiu melhor. A partir dos erros desse lote, desenvolveu-se uma nova versão (v3) do *Tool Calling*, com descrições corrigidas, ferramentas auxiliares determinísticas — identificação de nomes, normalização de códigos e escalas, resolução de datas — e um validador que devolve ao modelo os erros dos parâmetros antes da busca. Num segundo lote, de 945 consultas não usadas no desenvolvimento, com a configuração e o plano de análise fixados antes da rodada, ela acertou 94,4%, contra 87,8% da Saída Estruturada com as mesmas descrições e o mesmo validador, e 78,6% de uma arquitetura em duas etapas simulada, a melhor combinação das versões anteriores; sem as ferramentas nem o validador, que codificam o manual de anotação, o mesmo modelo acertava 75,2%. Na solução de uma chamada, recomenda-se o Gemma 4 E2B para estações com GPU modesta (latência mediana de 0,7 s em 4 GB de VRAM) e o Gemma 4 E4B para as demais; quanto à abordagem, o *Tool Calling* v3, medido só com o Gemma 4 E4B, em nuvem, ao custo de cerca de duas chamadas por consulta, e já configurado como padrão da API do *backend*. O *dataset* auditado, os lotes e o *pipeline* permitem repetir a comparação com novos modelos.

Palavras-chave: Modelos de Linguagem de Grande Porte. *Tool Calling*. Catálogo de Dados Espaciais. Geoinformação. Ollama. LangChain. Avaliação de Modelos.

ABSTRACT

The cartographic collection of the Brazilian Army Geographic Service Directorate (DSG) is organized by technical nomenclatures — the Nomenclature Index (INOM), the Index Map (MI), and the scales and product types of the National Cartographic System — whose mastery is a barrier for non-specialist users. The 1st Geoinformation Center (1º CGEO) developed a natural-language search prototype based on Structured Output and a single local model (Phi-4 14B), which relied on a normalization dictionary and had no evaluation protocol. This Final Course Project improved the **natural-language translation layer** of that solution — the step in which the user’s free-form query is converted into structured search parameters — by re-implementing the backend in Python and replacing Structured Output with native Tool Calling, with no normalization dictionary and no worked examples in the prompt. To measure translation quality, a 310-query dataset was built, with ground truth defined by an annotation manual and a three-stage audit. A reproducible pipeline evaluated four open models run locally through Ollama and, as a reference, three larger cloud-served models, measuring accuracy, per-field F1, and latency, with significance tests. With Tool Calling, Gemma 4 E4B, Gemma 4 E2B, and Qwen 3 4B reached statistically indistinguishable accuracy. With the same models and queries, in a single call, Structured Output was more accurate for three of the four local models, and the consistent gain of Tool Calling was the explicit decision to decline out-of-domain queries. The prototype’s method reached accuracy statistically equal to that of the best local models, which are smaller and more than fifteen times faster. An independent validation batch repeated the comparison with Gemma 4 E4B: Tool Calling declined more precisely, and Structured Output extracted better. From the errors in that batch, a new version (v3) of Tool Calling was developed, with corrected descriptions, deterministic auxiliary tools — name identification, code and scale normalization, date resolution — and a validator that returns parameter errors to the model before the search runs. On a second batch of 945 queries not used during development, with the configuration and the analysis plan fixed before the run, it reached 94.4% accuracy, against 87.8% for Structured Output given the same descriptions and the same validator, and 78.6% for a simulated two-stage architecture, the best combination of the earlier versions; without the tools or the validator, which encode the annotation manual, the same model reached 75.2%. For the single-call solution, Gemma 4 E2B is recommended for workstations with modest GPUs and Gemma 4 E4B where more VRAM is available; as to the approach, Tool Calling v3 is recommended, at the cost of about two calls per query; it was measured only with Gemma 4 E4B, in the cloud, and is the default configuration of the backend API. The audited dataset, the batches, and the pipeline allow the comparison to be repeated with new models.

Keywords: Large Language Models. Tool Calling. Spatial Data Catalog. Geoinformation. Ollama. LangChain. Model Evaluation.

LISTA DE ILUSTRAÇÕES

Figura 1 – Fluxo do mecanismo de <i>Tool Calling</i>	25
Figura 2 – Exemplo simplificado de definição de ferramenta para busca em catálogo de produtos cartográficos	25
Figura 3 – Estrutura Analítica do Projeto (EAP) adotada no aprimoramento . . .	59
Figura 4 – Arquitetura do protótipo de busca do 1º CGEO	60
Figura 5 – Arquitetura da solução aprimorada de busca	66
Figura 6 – Distribuição da latência da tradução nas rodadas completas, em escala logarítmica, por modelo	78
Figura 7 – Acurácia por consulta, por categoria e por modelo, nas rodadas completas	99
Figura 8 – F1-score por campo do <i>schema</i> de busca, por modelo, nas rodadas completas	101
Figura 9 – Distribuição da latência da tradução na estação de referência, em escala logarítmica	105
Figura 10 – Acurácia por consulta, por categoria, na estação de referência	105

LISTA DE QUADROS

Quadro 1 – Resumo dos requisitos que norteiam o escopo	36
Quadro 2 – Mapeamento entre limitações do protótipo e soluções adotadas	64
Quadro 3 – Classificação dos componentes do sistema final em relação ao protótipo	65
Quadro 4 – Cronograma de execução do projeto	74
Quadro 5 – Leituras aceitas para as expressões de tempo relativo (D = data de referência da execução)	135
Quadro 6 – Grupos de consultas do <i>dataset</i> e aspectos da tradução que exercitam	162
Quadro 7 – Decisões de adjudicação da auditoria do <i>dataset</i>	164

LISTA DE TABELAS

Tabela 1	– Comparativo entre consulta técnica e consulta em linguagem natural . . .	17
Tabela 2	– Modelos com suporte a <i>Tool Calling</i> disponíveis no Ollama (seleção) . . .	29
Tabela 3	– Estrutura de cada entrada do <i>dataset</i> de avaliação	38
Tabela 4	– Composição do <i>dataset</i> de avaliação por origem e família	40
Tabela 5	– Consultas por categoria e por camada (métricas principais; uma consulta pode ter mais de uma categoria)	41
Tabela 6	– Frequência de cada campo na leitura preferencial do gabarito (métricas principais)	42
Tabela 7	– Resultados da auditoria do <i>dataset</i>	43
Tabela 8	– Lote de validação: do alvo ao lote final, por família	46
Tabela 9	– Concordância na anotação às cegas do lote de validação	47
Tabela 10	– Ferramentas de inteligência artificial empregadas como apoio nos processos metodológicos	58
Tabela 11	– Ambiente de execução de cada rodada	76
Tabela 12	– Comparativo de desempenho entre os modelos avaliados	77
Tabela 13	– Latência da tradução, em milissegundos, por modelo	77
Tabela 14	– Tool Calling e Saída Estruturada sobre as mesmas consultas	79
Tabela 15	– Tool Calling e Saída Estruturada sobre as mesmas consultas — estação de referência (camadas P e N)	82
Tabela 16	– Gemma 4 E4B no lote de validação e nas 310 consultas: especificações v1 e v2	82
Tabela 17	– Comparações pareadas (Gemma 4 E4B)	86
Tabela 18	– Arquitetura em duas etapas e regra do objeto vazio, simuladas com as rodadas do Gemma 4 E4B, ao lado das quatro configurações executadas	88
Tabela 19	– Gemma 4 E4B no lote 2 (teste): os degraus da v3, o controle e as referências v1/v2	90
Tabela 20	– Comparações pareadas no lote 2 (Gemma 4 E4B)	91
Tabela 21	– Decomposição do acerto no lote 2: primeira decisão do modelo e correção pelo retorno do código	93
Tabela 22	– Teste A/B: <i>Tool Calling</i> (A) e Saída Estruturada (B) com o mesmo modelo, as mesmas consultas, a mesma especificação e o mesmo ambiente	96
Tabela 23	– F1-score por categoria de consulta e por modelo	98
Tabela 24	– Acurácia por camada de origem das consultas	100
Tabela 25	– F1-score por campo do <i>schema</i> de busca, por modelo	100
Tabela 26	– Ambiente de execução de cada rodada — estação de referência (camadas P e N)	103

Tabela 27 – Comparativo de desempenho entre os modelos avaliados — estação de referência (camadas P e N)	104
Tabela 28 – Latência da tradução, em milissegundos, por modelo — estação de referência (camadas P e N)	104
Tabela 29 – Teste de McNemar exato entre pares de modelos — estação de referência (camadas P e N)	104
Tabela 30 – Mesmo modelo nas rodadas completas e na estação de referência (camadas P e N, primeira repetição)	106
Tabela 31 – Qwen 3 4B executado duas vezes na estação de referência, com os mesmos pesos, prompt, ferramenta e data de referência (camadas P e N)	106
Tabela 32 – Ambiente de execução de cada rodada — referência em nuvem (Groq)	107
Tabela 33 – Comparativo de desempenho entre os modelos avaliados — referência em nuvem (Groq)	107
Tabela 34 – F1- <i>score</i> por categoria de consulta e por modelo — referência em nuvem (Groq)	107
Tabela 35 – Teste de McNemar exato entre pares de modelos	109
Tabela 36 – Concordância por campo entre o gabarito e a anotação independente (leitura preferencial, após a adjudicação)	165
Tabela 37 – Acurácia por categoria no lote de validação (Gemma 4 E4B)	165
Tabela 38 – F1 por campo no lote de validação (Gemma 4 E4B)	166

LISTA DE ABREVIATURAS E SIGLAS

1º CGEO	1º Centro de Geoinformação
API	<i>Application Programming Interface</i> (Interface de Programação de Aplicações)
BDGEx	Banco de Dados Geográficos do Exército
CGEO	Centro de Geoinformação
CPU	<i>Central Processing Unit</i> (Unidade Central de Processamento)
CSW	<i>Catalogue Service for the Web</i>
DSG	Diretoria de Serviço Geográfico
EAP	Estrutura Analítica do Projeto
ET-PCDG	Especificação Técnica para Produtos de Conjuntos de Dados Geoespaciais
GPU	<i>Graphics Processing Unit</i> (Unidade de Processamento Gráfico)
IBGE	Instituto Brasileiro de Geografia e Estatística
IC	Intervalo de confiança
INDE	Infraestrutura Nacional de Dados Espaciais
INOM	Índice de Nomenclatura
JSON	<i>JavaScript Object Notation</i>
LLM	<i>Large Language Model</i> (Modelo de Linguagem de Grande Porte)
MGB	Perfil de Metadados Geoespaciais do Brasil
MI	Mapa Índice
NLP	<i>Natural Language Processing</i> (Processamento de Linguagem Natural)
OGC	<i>Open Geospatial Consortium</i>
OM	Organização Militar
PFC	Projeto Final de Curso

REST	<i>Representational State Transfer</i>
RF	Requisito funcional
RNF	Requisito não funcional
SCN	Sistema Cartográfico Nacional
SQL	<i>Structured Query Language</i>
VRAM	<i>Video Random Access Memory</i> (memória da GPU)
WCS	<i>Web Coverage Service</i>
WFS	<i>Web Feature Service</i>
WMS	<i>Web Map Service</i>

SUMÁRIO

1	INTRODUÇÃO	16
1.1	Contextualização	16
1.2	Problema	17
1.3	Motivação	17
1.4	Lacuna Identificada	19
1.5	Objetivos	20
1.5.1	Objetivo Geral	20
1.5.2	Entregáveis do Projeto	20
1.6	Delimitação do Trabalho	21
1.7	Estrutura do Trabalho	21
2	REFERENCIAL TEÓRICO	23
2.1	Catálogos de Dados Espaciais	23
2.2	Tool Calling	24
2.2.1	Tool Calling <i>versus</i> Saída Estruturada	25
2.3	Modelos de Linguagem de Grande Porte e Execução Local	27
2.3.1	Vantagens da Execução Local	27
2.3.2	Ollama e Modelos com Suporte a Tool Calling	28
2.4	Orquestração com LangChain	28
2.5	Métricas de Avaliação	29
2.6	Trabalhos Relacionados	31
3	METODOLOGIA	34
3.1	Escopo do Projeto	34
3.1.1	Entregáveis	34
3.1.2	Exclusões	35
3.1.3	Requisitos que norteiam o escopo	36
3.2	Seleção de Modelos	36
3.2.1	Critérios de Seleção	36
3.2.2	Modelos Avaliados	36
3.2.3	Referência paralela em nuvem	37
3.3	Construção do <i>Dataset</i> de Avaliação	38
3.3.1	Estrutura de cada entrada	38
3.3.2	Categorias de Consultas	39
3.3.3	Camadas de Construção	39
3.3.4	Manual de Anotação e Leituras Múltiplas	40

3.4	Auditoria do <i>Dataset</i>	42
3.5	Lote de Validação Independente	44
3.6	Especificação v3: Ferramentas Auxiliares e Validação com Retorno	49
3.7	Protocolo de Avaliação	52
3.7.1	Visão Geral	52
3.7.2	Ambientes de Execução	52
3.7.3	Configuração dos Modelos	54
3.7.4	Linha de Base: Saída Estruturada	54
3.7.5	Métricas	55
3.7.6	Reprodutibilidade	57
3.8	Uso de Ferramentas de Inteligência Artificial	57
4	APRIMORAMENTO DO PROTÓTIPO: ESTRUTURA ANALÍ- TICA DO PROJETO	59
4.1	Análise do Protótipo do 1º CGEO	59
4.1.1	Visão Geral	59
4.1.2	Arquitetura do Protótipo	60
4.1.3	Contribuições do Protótipo	61
4.1.4	Limitações Identificadas	61
4.2	Elicitação de Requisitos	62
4.2.1	Requisitos Funcionais	62
4.2.2	Requisitos Não Funcionais	63
4.3	Planejamento da Migração	63
4.3.1	Mapeamento Limitação → Solução	64
4.3.2	Componentes Reaproveitados, Reescritos e Novos	64
4.4	Desenvolvimento	64
4.4.1	Camada de API: FastAPI	64
4.4.2	Camada de Orquestração: LangChain + Ollama	67
4.4.3	Ferramentas de Busca	68
4.4.4	Camada de Persistência	69
4.4.5	Pipeline de Avaliação	70
4.4.6	Registro de Abordagens, Implantação e Extensão	72
4.4.7	Ferramentas de Apoio ao Desenvolvimento	73
4.5	Cronograma	74
5	RESULTADOS	76
5.1	Configuração Experimental	76
5.2	Comparativo Geral entre Modelos	77
5.3	Tool Calling e Saída Estruturada	79
5.4	Lote de Validação Independente	81

5.5	Teste da v3 no Lote 2	89
5.6	Tool Calling × Saída Estruturada: o Teste A/B	95
5.7	Análise por Categoria de Consulta	98
5.8	Análise por Campo do <i>Schema</i>	100
5.9	Análise de Erros	101
5.9.1	Casos observacionais	103
5.10	Validação na Estação de Referência	103
5.11	Referência em Nuvem	106
5.12	Análise Estatística	108
5.13	Discussão	109
5.13.1	Recomendação de Modelo e de Abordagem para Uso Operacional	111
6	CONCLUSÃO	114
6.1	Síntese do Trabalho	114
6.2	Retomada dos Objetivos	115
6.3	Principais Resultados	115
6.4	Limitações do Trabalho	119
6.5	Trabalhos Futuros	122
6.6	Considerações Finais	124
	REFERÊNCIAS	126
	APÊNDICE A – DATASET DE AVALIAÇÃO	132
A.1	Manual de anotação	132
A.2	Consultas do protótipo (P01–P22)	137
A.3	Consultas redigidas pelos autores (N01–N40)	143
A.4	Geração programática da camada G	150
A.5	Consultas simples geradas por <i>templates</i> (G-S)	151
A.6	Consultas compostas geradas por <i>templates</i> (G-C)	153
A.7	Consultas com código MI/INOM geradas por <i>templates</i> (G-M)	155
A.8	Consultas com tempo relativo geradas por <i>templates</i> (G-T)	157
A.9	Consultas com ordenação geradas por <i>templates</i> (G-O)	158
A.10	Consultas ambíguas/informais adicionais (G-A)	160
A.11	Consultas amplas, com muitos produtos (G-P)	161
A.12	Consultas de fronteira do domínio (G-F)	161
A.13	Requisitos exercitados pelas consultas	162
A.14	Auditoria do <i>dataset</i>	163
A.15	Lote de validação: resultados complementares	163

APÊNDICE B – ESPECIFICAÇÃO DOS AMBIENTES DE EXECUÇÃO 167

B.1	Estação de referência	167
B.1.1	Comandos PowerShell de coleta	167
B.1.2	Saída coletada	168
B.2	Rodadas completas em GPU de nuvem	168
B.3	Referência em nuvem (Groq)	169

APÊNDICE C – REPOSITÓRIO DE CÓDIGO E REPRODUÇÃO DOS EXPERIMENTOS 170

1 INTRODUÇÃO

1.1 Contextualização

A geoinformação ocupa papel estratégico nas atividades de defesa, planejamento territorial, gestão ambiental e tomada de decisão em diferentes esferas do Estado brasileiro. O Brasil dispõe de uma vasta infraestrutura de produção e disseminação de dados espaciais, cujo arcabouço normativo é definido pelo Decreto n.º 6.666/2008, que instituiu a Infraestrutura Nacional de Dados Espaciais (INDE) (Brasil, 2008). A INDE tem por objetivo organizar, em âmbito federal, o acesso integrado a conjuntos de dados geoespaciais produzidos e mantidos pelos órgãos governamentais, promovendo a interoperabilidade e o compartilhamento dessas informações.

No contexto do Exército Brasileiro, a Diretoria de Serviço Geográfico (DSG) tem por missão orientar, no âmbito do Exército, as atividades referentes a imagens e informações geográficas, visando à produção cartográfica nas suas mais diversas formas, ao estabelecimento de normas e instruções, ao suprimento e à manutenção do material a seu cargo e às atividades decorrentes de convênios com órgãos da Administração Pública (Diretoria de Serviço Geográfico do Exército Brasileiro, 2025b). Para o cumprimento dessa missão, a DSG mantém o Banco de Dados Geográficos do Exército (BDGEx), desenvolvido para disponibilizar na Internet os dados geoespaciais da instituição, como determina o decreto que instituiu a INDE (XAVIER; MEYER; LUNARDI, 2014), e por meio do qual o acervo cartográfico oficial é disponibilizado para consulta tanto pelo público militar quanto pela sociedade civil (Diretoria de Serviço Geográfico do Exército Brasileiro, 2025a). A produção dos produtos cartográficos é realizada por cinco Centros de Geoinformação (CGEO) subordinados — localizados em Porto Alegre, Brasília, Olinda, Manaus e Rio de Janeiro — e segue a Especificação Técnica para Produtos de Conjuntos de Dados Geoespaciais (ET-PCDG) (Brasil. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico, 2016), em conformidade com o Perfil de Metadados Geoespaciais do Brasil (Perfil MGB) (Brasil. Comissão Nacional de Cartografia (CONCAR), 2009), cuja adesão é obrigatória para as Organizações Militares produtoras de dados geoespaciais.

Esses produtos são indexados e disponibilizados por meio de catálogos de dados espaciais compatíveis com padrões internacionais estabelecidos pelo *Open Geospatial Consortium* (OGC), em particular o protocolo *Catalogue Service for the Web* (CSW) (Open Geospatial Consortium, 2007). Tal padronização garante interoperabilidade técnica, mas armazena os conteúdos de forma estruturada conforme um modelo de dados rígido; consequentemente, consultas formuladas sobre dados semiestruturados (texto livre, sinônimos, abreviações) requerem processamento adequado para serem traduzidas aos campos técni-

cos do catálogo. Esta característica torna a recuperação de dados um processo intrincado para usuários sem formação na área cartográfica.

1.2 Problema

A busca em catálogos de dados espaciais institucionais exige que o usuário conheça previamente as convenções técnicas que organizam o acervo. No caso do acervo da DSG, isso implica saber utilizar o Índice de Nomenclatura (INOM), o Mapa Índice (MI), identificar corretamente as escalas disponíveis (1:1.000, 1:2.000, 1:5.000, 1:10.000, 1:25.000, 1:50.000, 1:100.000, 1:250.000), conhecer os tipos de produto do Sistema Cartográfico Nacional (SCN) (e.g., “SCN Carta Topográfica Matricial”, “MDT — RAM”, “SCN Carta Ortoimagem Banda P Pol HH”) e compreender as denominações dos Centros de Geoinformação responsáveis pela produção de cada folha.

Esta exigência constitui uma barreira de entrada considerável para usuários não especialistas — analistas, gestores e tomadores de decisão que precisam de acesso ágil à informação geoespacial, mas que não possuem formação técnica em cartografia. A interface atual, baseada em **busca por texto simples** (*full-text search*) contra os campos do banco, exige que o usuário conheça previamente os termos técnicos utilizados no acervo — e não usa qualquer forma de compreensão de linguagem natural. Um exemplo ilustrativo desta distinção é apresentado na Tabela 1.

Tabela 1 – Comparativo entre consulta técnica e consulta em linguagem natural

Modo atual (busca por texto simples)	Modo desejado (linguagem natural)
O usuário precisa digitar termos técnicos exatos que combinem com os campos do banco: “SCN Carta Topográfica Matricial 1:25.000 Rio de Janeiro 2023”	<i>“mapas do Rio de Janeiro em 25k publicados em 2023”</i>

Fonte: Adaptado de 1º Centro de Geoinformação (2025).

A barreira técnica imposta por esse modelo de interação resulta em subutilização do acervo disponível e restringe a capacidade de usuários não especialistas de acessar informações geoespaciais relevantes para suas atividades. Esta é uma limitação que afeta não apenas o acervo da DSG, mas a descoberta de dados em geoportais e catálogos de dados espaciais em geral (HU et al., 2015; LI et al., 2025).

1.3 Motivação

A Engenharia Cartográfica, enquanto disciplina, organiza-se em três eixos fundamentais: a **aquisição**, o **processamento** e a **disseminação** de dados geoespaciais.

Embora boa parte do ensino e da pesquisa concentre-se nos dois primeiros eixos — com foco em técnicas de levantamento, sensoriamento remoto e processamento digital —, a disseminação é a etapa que efetivamente conecta o acervo produzido aos seus usuários finais. É justamente nesse terceiro eixo que se insere o presente trabalho: facilitar o acesso e a disseminação dos produtos cartográficos do acervo da DSG, reduzindo a barreira técnica imposta pelas nomenclaturas especializadas e tornando o catálogo efetivamente utilizável por públicos não especialistas. Esta é a relação direta entre o trabalho desenvolvido e a Engenharia Cartográfica: traduzir, por meio de tecnologias de Processamento de Linguagem Natural, a riqueza do acervo cartográfico nacional em uma forma de acesso compatível com a linguagem cotidiana dos seus potenciais usuários.

Os Modelos de Linguagem de Grande Porte (LLMs, do inglês *Large Language Models*) representam um avanço substancial no campo do Processamento de Linguagem Natural (NLP). Baseados na arquitetura *Transformer* (VASWANI et al., 2017), esses modelos demonstram capacidade notável de compreender e gerar texto em linguagem natural, incluindo a habilidade de interpretar intenções implícitas, normalizar variações linguísticas e traduzir descrições informais para estruturas formais (BROWN et al., 2020; WEI et al., 2022).

Em particular, o mecanismo de *Tool Calling* — pelo qual um LLM pode decidir de forma autônoma quando e como invocar funções externas, estruturando os parâmetros necessários em formato JSON — abre perspectivas promissoras para a automação de tarefas de recuperação de informação (SCHICK et al., 2023; QIN et al., 2024). Este mecanismo é distinto da abordagem de Saída Estruturada (*Structured Output*), uma vez que o modelo não apenas formata uma resposta em esquema predefinido, mas decide ativamente quais ferramentas devem ser acionadas e com quais argumentos.

Um fator adicional de motivação é a viabilidade de execução local desses modelos. Plataformas como o Ollama (Ollama Team, 2023) permitem executar modelos de linguagem de código aberto — como os das famílias Qwen, Llama, Gemma e Mistral — diretamente em *hardware* local, sem dependência de serviços de nuvem. Isso é especialmente relevante para ambientes institucionais com restrições de privacidade, segurança da informação e custos operacionais, características típicas do contexto do Exército Brasileiro.

No âmbito da DSG, o 1º Centro de Geoinformação (1º CGEO) desenvolveu um protótipo inicial (1º Centro de Geoinformação, 2025), utilizando o modelo Phi-4 14B via Ollama com Saída Estruturada, confirmando a viabilidade da abordagem para o domínio cartográfico nacional, mas revelando limitações que motivaram o presente projeto.

1.4 Lacuna Identificada

Apesar da viabilidade demonstrada pelo protótipo do 1º CGEO, foram identificadas seis limitações que precisavam ser endereçadas para que a solução amadurecesse em direção ao uso operacional. Cada uma delas motivou uma frente de aprimoramento neste trabalho:

- **Backend em Node.js.** As bibliotecas de integração com LLMs, como o LangChain, têm versões mais completas em Python, o que limita a evolução da camada de tradução no *backend* original.
- **Ausência de *Tool Calling* real.** O protótipo utiliza *Structured Output*, no qual o modelo é forçado a devolver, a cada consulta, um objeto no formato do *schema* de saída, mesmo quando a consulta do usuário não contém informação para vários campos — o que pode induzir alucinações estruturadas (valores plausíveis, mas incorretos). O *Tool Calling*, ao contrário, delega ao modelo a decisão de quando invocar a ferramenta e quais argumentos preencher, o que pode ser mais adequado quando a consulta informa apenas parte dos campos — hipótese que o Capítulo 5 testa numa só chamada (Seção 5.3) e com ferramentas auxiliares e validação (Seção 5.5).
- **Uso de um único modelo (Phi-4 14B).** Sem comparação sistemática, não há evidência que sustente a escolha do modelo em relação a alternativas mais recentes, em um ecossistema de LLMs abertos que evolui rapidamente (e.g., Qwen 3 e Gemma 4).
- **Pré-processamento manual com dicionário *hardcoded*.** O protótipo depende de um dicionário fixo de normalizações (“25k” → “1:25.000”, “primeiro cgeo” → “1º Centro de Geoinformação”). O dicionário só cobre variações previstas em tempo de desenvolvimento e exige manutenção manual sempre que aparece uma forma nova. A solução de uma chamada (v1) delega a normalização ao LLM e elimina essa dependência; a v3 (Seção 3.6) devolve parte dela ao código, não como dicionário aplicado à consulta antes do modelo, mas como ferramentas que o modelo consulta e um validador que confere a saída.
- **Fallback por expressões regulares.** Quando o modelo falha, o protótipo aciona um mecanismo que tenta extrair parâmetros com regras rígidas. O *fallback* esconde as causas reais das falhas do modelo e não é passível de análise sistemática.
- **Ausência de métricas formais de avaliação.** O protótipo dispõe apenas de 22 casos de teste, pontuados por pesos atribuídos *ad hoc* aos campos esperados, sem penalizar campos preenchidos indevidamente. Sem um conjunto de referência (*ground truth*) mais amplo, métricas padronizadas e protocolo de medição, não há

como aferir objetivamente a qualidade da solução, nem sua evolução ao longo do tempo, à medida que novos modelos surjam.

O presente trabalho aprimorou o protótipo existente, endereçando diretamente cada uma dessas seis limitações.

1.5 Objetivos

1.5.1 Objetivo Geral

Aprimorar a **camada de tradução linguística** da solução de busca em linguagem natural sobre o acervo cartográfico da DSG — ou seja, a etapa em que uma consulta livre do usuário é convertida em parâmetros estruturados de busca — partindo do protótipo desenvolvido pelo 1º CGEO, por meio da reimplementação do *pipeline* com *Tool Calling* nativo, da avaliação comparativa de modelos de LLM abertos executáveis localmente via Ollama e da definição de métricas quantitativas que permitam aferir e comparar objetivamente o desempenho da tradução. A lógica de consulta ao banco e de resolução geográfica foi portada do protótipo e não é objeto de avaliação neste trabalho.

1.5.2 Entregáveis do Projeto

O projeto resultou nos seguintes entregáveis:

1. Uma nova arquitetura de *backend* em Python (FastAPI + LangChain), que substitui o *backend* Node.js do protótipo e implementa *Tool Calling* nativo via Ollama;
2. Um *dataset* de avaliação com 310 consultas de referência (*ground truth*) representativas do domínio cartográfico da DSG, acompanhado de manual de anotação e de registro de auditoria do gabarito (Capítulo 3 e Apêndice A);
3. Um *pipeline* de avaliação automatizado e reproduzível, com cálculo das métricas de acurácia, precisão, *recall*, F1-score e latência;
4. Um relatório comparativo entre quatro modelos de LLM com suporte a *Tool Calling* executáveis localmente via Ollama — Qwen 3 4B, Gemma 4 E4B, Gemma 4 E2B e Mistral Nemo 12B —, indicando o mais adequado para uso operacional no acervo da DSG, considerando o equilíbrio entre desempenho e requisitos de *hardware*;
5. Dois lotes de validação independentes, com o gabarito fixado antes da redação das consultas, e uma v3 do *Tool Calling*, com ferramentas auxiliares determinísticas e validação com retorno, desenvolvida sobre as 310 consultas e o primeiro lote e testada no segundo (Seções 3.5 e 3.6), que fundamenta a recomendação de abordagem; a

v3 está implementada no pacote, no avaliador e na API do *backend*, em que é a abordagem padrão.

Como referência, o mesmo *dataset*, o mesmo *prompt*, a mesma ferramenta e as mesmas métricas foram aplicados a três modelos de maior porte servidos em nuvem pela plataforma Groq. Esses modelos rodaram com uma repetição e com o menor esforço de raciocínio que cada um aceita: sem raciocínio no Qwen 3.8 27B, como os modelos locais, e com esforço baixo nos dois modelos GPT-OSS, que não permitem desligá-lo. Seus resultados são reportados à parte por não atenderem ao requisito de execução local.

1.6 Delimitação do Trabalho

O presente Projeto Final de Curso (PFC) tem como foco o aprimoramento do *pipeline* de tradução de consultas em linguagem natural para parâmetros técnicos de busca, compreendendo o componente de LLM e sua integração com o banco de dados geoespacial. Explicitamente fora do escopo deste trabalho estão: o desenvolvimento de interface gráfica (*frontend*) para o usuário final; a implantação em produção no sistema da DSG; a realização de pesquisa de opinião ou avaliação qualitativa com usuários; o treinamento ou ajuste fino (*fine-tuning*) de modelos de linguagem; a busca por coordenadas geográficas sem referência nominal; e o suporte a idiomas além do português. A delimitação completa do escopo é detalhada no Capítulo 3.

1.7 Estrutura do Trabalho

O presente trabalho está organizado em seis capítulos, dos quais este é o primeiro. O Capítulo 2 apresenta o referencial teórico, cobrindo os fundamentos de catálogos de dados espaciais, o mecanismo de *Tool Calling* e sua relação com a Saída Estruturada, os Modelos de Linguagem de Grande Porte e sua execução local com o Ollama, a orquestração com LangChain, as métricas de avaliação empregadas e os trabalhos relacionados. O Capítulo 3 detalha a metodologia adotada: o escopo do trabalho (entregáveis e exclusões), os requisitos que o norteiam, a seleção dos modelos comparados, a construção e a auditoria do *dataset* de avaliação, os dois lotes de validação, as especificações v2 e v3 e o protocolo de avaliação, incluindo os ambientes de execução e o cálculo das métricas. O Capítulo 4 descreve a execução do aprimoramento: a análise do protótipo de referência do 1º CGEO, a elicitação de requisitos, o planejamento da migração e o desenvolvimento da solução por camadas (API, orquestração, ferramenta de busca e persistência), além do *pipeline* de avaliação. O Capítulo 5 apresenta os resultados dos experimentos com os quatro modelos locais, a comparação entre *Tool Calling* e Saída Estruturada, o lote de validação, o teste da v3 no lote 2, a validação na estação de referência, o resultado de referência em nuvem

e a discussão comparativa que fundamenta a recomendação de modelo e de abordagem. O Capítulo 6 sintetiza as conclusões do trabalho, apresenta suas limitações e propõe direções para trabalhos futuros. Por fim, o Apêndice A reúne o *dataset* de avaliação, o manual de anotação e o registro da auditoria do gabarito; o Apêndice B especifica os ambientes de execução; e o Apêndice C descreve o repositório do código-fonte e os passos para a reprodução dos experimentos.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos que sustentam o trabalho. A organização segue a ordem em que os conceitos são utilizados na metodologia: primeiro, os *catálogos de dados espaciais* (acervo a ser consultado); em seguida, o mecanismo de *Tool Calling* (estratégia central do projeto) e sua distinção em relação à *Saída Estruturada* adotada pelo protótipo de referência; depois, os *Modelos de Linguagem de Grande Porte* executados localmente por meio do Ollama e a orquestração com o LangChain; as *métricas de avaliação* empregadas para comparar os modelos; e, por fim, os *trabalhos relacionados*.

2.1 Catálogos de Dados Espaciais

A produção cartográfica brasileira é regida pelo Sistema Cartográfico Nacional (SCN), instituído pelo Decreto-Lei n.º 243/1967, que fixa as diretrizes e bases da cartografia brasileira e as escalas-padrão da cartografia sistemática terrestre (Brasil, 1967); as normas técnicas da cartografia nacional, incluindo os padrões de exatidão, foram estabelecidas pelo Decreto n.º 89.817/1984 (Brasil, 1984). As folhas da cartografia sistemática são identificadas pelo Índice de Nomenclatura (INOM) e pelo número do Mapa Índice (MI). A Especificação Técnica para Produtos de Conjuntos de Dados Geoespaciais (ET-PCDG) (Brasil. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico, 2016), mantida pela DSG, detalha os requisitos técnicos dos produtos, como cartas topográficas, cartas ortoimagem, modelos digitais de elevação e ortoimagens.

A Infraestrutura Nacional de Dados Espaciais (INDE), instituída pelo Decreto n.º 6.666/2008 (Brasil, 2008), organiza, em âmbito federal, o acesso integrado a conjuntos de dados geoespaciais produzidos e mantidos pelos órgãos governamentais. A INDE é constituída por *nós* provedores — entendidos como pontos de acesso institucionais que catalogam e disponibilizam dados conforme o Perfil de Metadados Geoespaciais do Brasil (Perfil MGB) (Brasil. Comissão Nacional de Cartografia (CONCAR), 2009). O decreto determina que os dados geoespaciais produzidos pelos órgãos do Poder Executivo federal sejam disseminados na Internet com seus metadados; para atendê-lo, a DSG desenvolveu o Banco de Dados Geográficos do Exército (BDGEx) (XAVIER; MEYER; LUNARDI, 2014), por meio do qual o acervo da DSG é disponibilizado para consulta pela sociedade (Diretoria de Serviço Geográfico do Exército Brasileiro, 2025a). A obrigatoriedade de adesão das Organizações Militares produtoras de dados geoespaciais ao Perfil MGB e ao SCN é parte das atribuições legais decorrentes do referido decreto.

Do ponto de vista técnico, a INDE e o BDGEx adotam os padrões do OGC para a disponibilização dos dados, incluindo os serviços *Web Map Service* (WMS), *Web Feature*

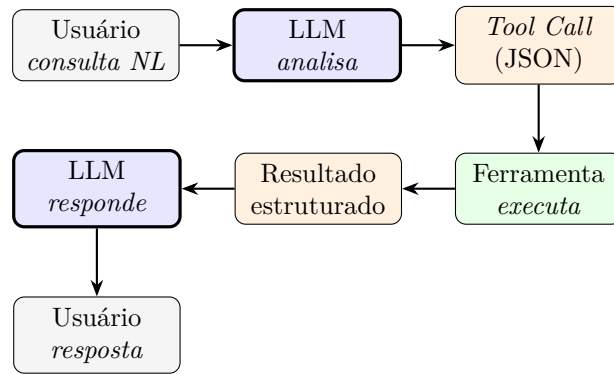
Service (WFS), *Web Coverage Service* (WCS) e *Catalogue Service for the Web* (CSW). O protocolo CSW (Open Geospatial Consortium, 2007) é o padrão internacional para publicação e descoberta de metadados de recursos geoespaciais e define operações de consulta (como *GetRecords*) que permitem a busca por título, data, tipo, área de abrangência geográfica e palavras-chave. O catálogo exposto pela interface CSW, embora tecnicamente robusto, organiza os metadados em campos estruturados, segundo um modelo de dados rígido; conseqüentemente, consultas em texto livre (com sinônimos, abreviações e grafias informais) requerem processamento adequado para serem traduzidas aos campos técnicos do catálogo.

2.2 Tool Calling

O mecanismo de *Tool Calling* (também denominado *Function Calling*) é a capacidade de um Modelo de Linguagem de Grande Porte (LLM) de, ao processar uma consulta do usuário, identificar que a resposta requer a execução de uma função externa e gerar uma chamada estruturada para essa função, incluindo todos os parâmetros necessários em formato JSON (SCHICK et al., 2023; QIN et al., 2024). Trata-se, portanto, de uma capacidade que estende o modelo para além da geração textual pura, permitindo que ele atue como o componente decisório de um sistema que orquestra ferramentas determinísticas (consultas a bancos de dados, chamadas a APIs, execuções de código).

O fluxo completo de uma interação com *Tool Calling* pode ser descrito da seguinte forma: o **usuário** formula uma consulta em linguagem natural; o **LLM** analisa a consulta e, com base nas definições de ferramentas (*tools*) fornecidas no *prompt* do sistema, decide que uma ou mais ferramentas devem ser invocadas; o LLM gera uma **chamada de ferramenta** (*tool call*) em formato JSON, especificando o nome da função e seus argumentos; o **sistema de orquestração** (e.g., LangChain) intercepta a chamada, executa a função correspondente com os argumentos fornecidos e retorna o resultado ao LLM; por fim, o LLM incorpora o resultado da execução em seu contexto e formula a resposta final ao usuário. A Figura 1 ilustra este fluxo de forma esquemática.

No protocolo de *Tool Calling* adotado pelo Ollama e pelo LangChain, as ferramentas são definidas por um *schema* JSON que especifica o nome da função, sua descrição e os parâmetros aceitos, com seus tipos e descrições. O LLM recebe essas definições como parte do contexto do sistema e as utiliza para gerar chamadas estruturalmente válidas. Um exemplo de definição de ferramenta para a busca em um catálogo de dados espaciais é apresentado na Figura 2.

Figura 1 – Fluxo do mecanismo de *Tool Calling*

Fonte: Elaborado pelos autores.

```

{
  "name": "buscar_catalogo",
  "description": "Busca produtos cartograficos no catalogo",
  "parameters": {
    "type": "object",
    "properties": {
      "keyword": {"type": "string",
        "description": "Codigo MI, INOM ou nome da carta"},
      "scale": {
        "type": "string",
        "enum": ["1:25.000", "1:50.000", "1:100.000", ...],
        "description": "Escala cartografica"},
      "state": {
        "type": "string",
        "description": "Nome do estado (normalizado)"},
      "productType": {"type": "string",
        "description": "Tipo de produto SCN"}
    }
  }
}

```

Figura 2 – Exemplo simplificado de definição de ferramenta para busca em catálogo de produtos cartográficos

Fonte: Elaborado pelos autores.

2.2.1 Tool Calling *versus* Saída Estruturada

É importante distinguir *Tool Calling* da abordagem de Saída Estruturada (*Structured Output*), conceito frequentemente confundido com o primeiro. Na Saída Estruturada, o modelo é instruído a retornar sua resposta em um formato JSON validado por um *schema* predefinido (e.g., Pydantic em Python (COLVIN, 2019) ou Zod em TypeScript): o modelo *sempre* retorna uma saída estruturada, independentemente da consulta, e a decisão se restringe a quais campos preencher. Bibliotecas como o Instructor (LIU, 2023)

interceptam a saída do LLM e a forçam a corresponder ao *schema*, repetindo a chamada em caso de saída inválida.

A abordagem de *Structured Output* tem uma limitação fundamental: o modelo é sempre compelido a preencher a estrutura predefinida, mesmo quando a consulta do usuário não contém informação suficiente para todos os campos, o que pode levar a alucinações estruturadas — valores plausíveis mas incorretos inseridos para satisfazer o *schema*. Além disso, o modelo não pode invocar ferramentas no meio da resposta: o resultado de uma verificação só lhe chega como nova tentativa, depois de emitido o objeto inteiro, como no Instructor — o arranjo do controle com Saída Estruturada da v3 (Seção 3.6).

No *Tool Calling*, em contraste, o modelo *decide* de forma autônoma se e quando invocar uma ferramenta, selecionando entre múltiplas ferramentas disponíveis. O modelo pode responder diretamente ao usuário sem invocar ferramentas, ou pode encadear múltiplas chamadas em sequência para resolver uma tarefa complexa. Para o caso de uso deste trabalho, o *Tool Calling* pode ser mais adequado do que a Saída Estruturada por permitir que o modelo trate consultas ambíguas de forma mais flexível, decida quais parâmetros de busca são realmente relevantes e potencialmente encadeie operações de validação e refinamento da busca — a vantagem prática, contudo, só se estabelece empiricamente. O Capítulo 5 faz essa verificação, comparando as duas abordagens com os mesmos modelos, as mesmas consultas e as mesmas métricas numa só chamada (Seção 5.3) e com o encadeamento de ferramentas auxiliares e de validação da v3 (Seção 5.5).

Modelos com *thinking*.

Uma parte dos modelos abertos recentes — notadamente Gemma 4 e Qwen 3 — incorpora, opcionalmente, uma etapa de raciocínio interno (*thinking*) antes de emitir a resposta ou a chamada de ferramenta. Nesse modo, o modelo produz internamente uma sequência de tokens de raciocínio que não aparecem na saída, o que aumenta a latência total mas tende a melhorar a qualidade das decisões de *Tool Calling* em consultas com múltiplos critérios. Neste trabalho, o modo *thinking* foi mantido **desativado** em todos os modelos locais avaliados — por parâmetro da API nos modelos que declaram essa capacidade e, no Qwen 3, pela adoção da *build* oficial sem raciocínio —, garantindo a comparabilidade dos tempos de resposta e das decisões de *Tool Calling* entre modelos que oferecem e que não oferecem essa etapa (no resultado paralelo em nuvem, adotou-se o menor esforço de raciocínio aceito por cada modelo); a configuração adotada para cada modelo é detalhada no Capítulo 3. A avaliação com o modo *thinking* ativado, que permitiria medir o ganho de qualidade obtido em troca da latência adicional, não foi realizada e fica como trabalho futuro.

2.3 Modelos de Linguagem de Grande Porte e Execução Local

Os Modelos de Linguagem de Grande Porte (LLMs) baseiam-se, em sua quase totalidade, na arquitetura *Transformer* (VASWANI et al., 2017), que substituiu as arquiteturas recorrentes (RNN, LSTM) por um mecanismo puramente baseado em atenção (*self-attention*). Essa mudança permitiu o processamento paralelo de sequências longas e a captura de dependências distantes no texto, viabilizando o treinamento em corpora de grande escala. Para o presente trabalho, a propriedade mais relevante dos LLMs modernos é a capacidade de seguir instruções complexas em linguagem natural (*instruction following*), obtida sobretudo pelo ajuste fino com instruções e com *feedback* humano (OUYANG et al., 2022) — que é o fundamento sobre o qual o *Tool Calling* se assenta: o modelo precisa entender a consulta, interpretar o *schema* das ferramentas disponíveis e decidir como populá-las.

2.3.1 Vantagens da Execução Local

A execução local de LLMs, em contraste com o uso de APIs de serviços na nuvem, apresenta vantagens relevantes para contextos institucionais como o da DSG (Ollama Team, 2023; ggml contributors, 2023): **privacidade e segurança**, pois as consultas e os dados não trafegam por redes externas — fator crítico para organizações com restrições de sigilo; **ausência de custos recorrentes por *token***, eliminando dependência de orçamento variável; **controle e auditabilidade**, com a organização mantendo a versão exata do modelo utilizado; e **disponibilidade offline**, com o sistema funcionando independentemente da conectividade.

Essa abordagem também traz **desvantagens** que devem ser reconhecidas: (i) o **acesso limitado aos modelos de fronteira** — os LLMs proprietários mais capazes não têm os pesos publicados e ocupam a maior parte das primeiras posições em *benchmarks* de chamada de funções, como o BFCL (PATIL et al., 2025), embora alguns modelos abertos os superem em posições intermediárias; (ii) o **custo de infraestrutura** desloca-se do modelo de assinatura para o de *hardware*: a organização precisa adquirir e manter servidores com GPU e VRAM suficientes, além do custo operacional (energia, refrigeração, atualização de *drivers*); (iii) o **ciclo de atualização** dos modelos passa a ser responsabilidade interna — novas versões exigem repetir a avaliação sistemática.

Uma característica mitigadora é que o LangChain, adotado neste trabalho, permite **trocar o modelo local por um de terceiros** (ou vice-versa) com alteração mínima no código de aplicação — basta substituir o *wrapper* ChatOllama por, por exemplo, ChatOpenAI ou ChatAnthropic. Essa portabilidade permite que a decisão local × nuvem seja **gerencial**, e não técnica, no futuro. Neste trabalho, essa mesma troca (ChatOllama → ChatOpenAI, apontado para o serviço Groq) foi empregada para obter um resultado

paralelo, em nuvem, com modelos de maior porte, reportado à parte como referência no Capítulo 5.

2.3.2 Ollama e Modelos com Suporte a Tool Calling

O Ollama (Ollama Team, 2023) é uma plataforma de código aberto que simplifica a instalação, a execução e o gerenciamento de modelos de linguagem localmente, expondo uma API compatível com o padrão OpenAI — o que facilita a integração com *frameworks* como o LangChain. Internamente, o Ollama utiliza o *backend llama.cpp* para inferência, com modelos armazenados no formato de arquivo GGUF (ggml contributors, 2023), sucessor do formato GGML, que comporta diferentes tipos de quantização (como Q4_0 e Q4_K_M), capazes de reduzir a precisão numérica dos pesos do modelo (tipicamente de 16 bits para cerca de 4 bits por peso) com impacto pequeno na qualidade das respostas (FRANTAR et al., 2023). No presente trabalho, foram adotadas quantizações de 4 bits — Q4_K_M, considerada um bom equilíbrio entre compressão e qualidade, ou a quantização equivalente da *build* publicada de cada modelo (as *tags* exatas constam da Seção 3.2) —, o que reduz o consumo de VRAM em cerca de quatro vezes em relação ao formato FP16. Como regra prática, modelos quantizados em Q4_K_M demandam aproximadamente 0,5 GB de VRAM por bilhão de parâmetros, viabilizando a execução de modelos de até cerca de 14B em GPUs de uso doméstico.

A Tabela 2 apresenta os principais modelos disponíveis no Ollama com suporte a *Tool Calling*, entre os quais foram selecionados os modelos comparados neste trabalho (Seção 3.2). A coluna “Tamanho aprox. (4 bits)”, que corresponde aos pesos e não inclui a memória de contexto, serviu de referência para selecionar modelos compatíveis com a estação de referência especificada na Seção 3.7.2 do Capítulo 3; modelos cuja VRAM exigida ultrapassa a disponível ainda podem ser executados com *offload* parcial para CPU, ao custo de maior latência.

2.4 Orquestração com LangChain

O LangChain (CHASE, 2022) é um *framework* de código aberto para o desenvolvimento de aplicações baseadas em LLMs, fornecendo abstrações para os componentes comuns de um *pipeline*: *wrappers* de modelos de linguagem, *prompts*, *chains* (sequências de operações), agentes, ferramentas e memória. Para o presente trabalho, o LangChain é utilizado para (a) gerenciar a integração com o Ollama via `ChatOllama`; (b) ligar ao modelo, pelo método `bind_tools()`, a ferramenta de busca descrita por seu *schema* JSON; (c) obter, em uma única chamada `invoke()` na solução v1 e num laço de chamadas na v3, as chamadas de ferramenta geradas pelo modelo e, no ciclo completo, devolver-lhe o resultado da busca para a resposta final; e (d) servir de interface única com os modelos

Tabela 2 – Modelos com suporte a *Tool Calling* disponíveis no Ollama (seleção)

Modelo	Parâmetros	Tamanho aprox. (4 bits)	Organização
Gemma 4 E2B	2B efet.	~4 GB ^a	Google DeepMind
Gemma 4 E4B	4B efet.	~5,7 GB ^a	Google DeepMind
Qwen3 (4B)	4B	~2,3 GB	Alibaba Cloud
Llama 3.2 (3B)	3B	~2 GB	Meta AI
Qwen3 (8B)	8B	~5 GB	Alibaba Cloud
Mistral Nemo (12B)	12B	~6,6 GB	Mistral AI e NVIDIA
Gemma 4 (26B MoE)	26B (4B ativ.)	~15–17 GB	Google DeepMind

Fonte: Compilado pelos autores a partir de Ollama Team (2023) e da documentação oficial dos modelos.

Nota: ^a Tamanho da *build* com quantização consciente de treinamento (QAT, 4 bits) publicada no Ollama (`gemma4:e2b-it-qat` e `gemma4:e4b-it-qat`). “Efet.” indica parâmetros efetivos; o total armazenado é maior (4,6B no E2B, segundo o Ollama), o que explica, em parte, o tamanho acima da regra prática.

no *pipeline* de avaliação.

O LangGraph (LangChain AI, 2024), extensão do LangChain que permite modelar fluxos de execução como grafos de estados, com ciclos e ramificações condicionais, é indicado quando o fluxo exige iteração explícita sobre resultados intermediários. Não foi necessário neste trabalho: na solução v1, a tradução da consulta se resolve em uma única decisão do modelo (chamar ou não a ferramenta, e com quais argumentos), e o ciclo completo é implementado explicitamente no módulo de *pipeline*, conforme detalhado na Seção 4.4 do Capítulo 4; na v3 (Seção 3.6), que itera sobre ferramentas auxiliares e o retorno de um validador, o laço é curto, tem limites fixos e também é escrito explicitamente, sem grafo de estados.

2.5 Métricas de Avaliação

Definir bem as métricas de avaliação importa por dois motivos práticos. Primeiro, elas permitem ao usuário final (a DSG e as OM subordinadas) tomar uma **decisão informada de qual modelo implantar** — há um *trade-off* entre qualidade de extração e latência, e diferentes contextos operacionais podem priorizar aspectos distintos. Segundo, elas viabilizam a **reavaliação periódica** da solução à medida que novos modelos surjam — o *pipeline* de avaliação (entregável do projeto, descrito na Seção 4.4 do Capítulo 4) foi projetado para ser rerodado com custo marginal baixo sobre qualquer LLM compatível com *Tool Calling*.

As métricas adotadas, todas oriundas da literatura de recuperação de informação (MANNING; RAGHAVAN; SCHÜTZE, 2008), devem ser lidas como **pontos de vista complementares sobre o mesmo resultado**: cada uma revela um aspecto (correção total, correção por campo, tempo de resposta) e as decisões operacionais nascem da análise

conjunta, não do ranking de uma métrica isolada. Modelos diferentes podem vencer em métricas diferentes.

Nota sobre o critério de comparação.

A comparação entre a resposta do modelo e o *ground truth* é feita **campo a campo** sobre o objeto JSON estruturado, e não byte a byte: enums (`scale`, `productType`, `supplyArea`, `project`) e campos de ordenação (`sortField`, `sortDirection`) exigem correspondência exata; *strings* livres (`keyword`, `state`, `city`) são normalizadas antes da comparação (*lowercase*, remoção de acentos, substituição da pontuação por espaço e colapso de espaços repetidos); intervalos de data são expressos pelos limites `start` e `end` em ISO 8601 e exigem coincidência exata desses limites; e `limit` é comparado como inteiro. Quando a consulta admite mais de uma leitura legítima (por exemplo, “últimos 3 meses” como noventa dias ou como três meses de calendário), o gabarito lista explicitamente as leituras aceitas, e a resposta é considerada correta se coincidir com alguma delas; os critérios de anotação constam do Capítulo 3, Seção 3.3. Detalhes do protocolo constam do Capítulo 3, Seção 3.7.

A **acurácia** mede a proporção de consultas para as quais o modelo extraiu corretamente *todos* os parâmetros esperados, sem acrescentar parâmetros ausentes do gabarito, em relação ao total de consultas avaliadas; nas consultas cujo gabarito prevê que nenhuma ferramenta seja chamada, a resposta é correta quando o modelo não a chama:

$$\text{Acurácia} = \frac{\text{Número de consultas completamente corretas}}{\text{Total de consultas}}.$$

Esta é a métrica mais estrita, pois qualquer erro em qualquer parâmetro conta como uma consulta incorreta.

Para a análise por parâmetro (campo a campo) do *schema* de busca, utilizam-se as métricas clássicas de **precisão**, **recall** e **F1-score**:

$$\text{Precisão} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F_1 = 2 \cdot \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}},$$

onde *TP* (*true positives*) são os parâmetros corretamente extraídos, *FP* (*false positives*) são parâmetros extraídos incorretamente, e *FN* (*false negatives*) são parâmetros esperados que não foram extraídos. O *F1-score* é a média harmônica entre precisão e *recall* e é especialmente útil quando ambas as métricas são igualmente importantes.

A **latência** é o tempo de resposta do modelo para uma consulta. Na avaliação, mede-se o tempo da tradução — do envio da consulta ao modelo até o recebimento da resposta com as chamadas de ferramenta —, que é a etapa comparada entre os modelos. É uma métrica operacional importante para a viabilidade de uso em produção e é reportada por mínimo, mediana, média, percentil 95 e máximo sobre o conjunto de consultas

avaliadas. Para diagnóstico, o tempo de execução da ferramenta (a consulta ao banco) é registrado à parte, quando a busca é executada, e não entra na comparação entre modelos.

2.6 Trabalhos Relacionados

Este trabalho se situa na interseção de três linhas: as interfaces de linguagem natural para bancos de dados, o uso de LLMs em geoinformação e na descoberta de dados geoespaciais, e a avaliação da chamada de ferramentas por LLMs. O ponto de partida direto, o protótipo do 1º CGEO (1º Centro de Geoinformação, 2025), é analisado no Capítulo 4.

Interfaces de linguagem natural para bancos de dados.

Na formulação *text-to-SQL*, o modelo gera a consulta SQL completa a partir da pergunta e do esquema do banco. O *dataset* Spider (YU et al., 2018) consolidou essa avaliação com cerca de dez mil perguntas sobre 200 bancos de 138 domínios, exigindo generalização para esquemas não vistos no treino. Com os LLMs, a abordagem passou a depender sobretudo do *prompt* e do mapeamento entre a pergunta e o esquema, e a ambiguidade e a subespecificação da linguagem natural figuram entre os problemas centrais apontados pela literatura (LIU et al., 2025). O presente trabalho adota uma formulação mais restrita: o modelo não escreve SQL nem vê o esquema do banco; apenas preenche os parâmetros tipados de uma ferramenta fixa, e a consulta é montada pelo *backend*, com os valores passados como parâmetros. Essa restrição elimina o SQL gerado pelo modelo, e com ele o risco de consulta inválida ou injetada, e permite validar cada argumento contra os enumerados do domínio, ao custo de atender só às buscas que a ferramenta representa.

LLMs em geoinformação.

Mai et al. (2024) discutem as oportunidades e os desafios dos modelos de fundação para a inteligência artificial geoespacial, e Manvi et al. (2024) mostram que o conhecimento geográfico contido nos LLMs pode ser aproveitado em tarefas de predição, desde que o *prompt* traga contexto do OpenStreetMap, e não apenas coordenadas. Em outra direção, os LLMs passaram a atuar como núcleo de decisão de sistemas que executam tarefas de SIG: no SIG autônomo de Li e Ning (2023), o protótipo LLM-Geo, construído sobre o GPT-4, gera e executa o código que coleta, analisa e visualiza dados espaciais; no GeoGPT (ZHANG et al., 2024), o modelo planeja e encadeia ferramentas de SIG a partir de pedidos em linguagem natural. Nesses sistemas, o modelo tem alta autonomia. Este trabalho ocupa a outra ponta desse espectro: pouca autonomia do modelo — na v1, uma única decisão, chamar ou não a ferramenta de busca, e com quais argumentos; na v3, um

laço curto com ferramentas auxiliares determinísticas e um validador, sempre com uma única ferramenta de busca —, modelos abertos executados localmente e uma avaliação quantitativa sobre um *dataset* auditado.

Descoberta de dados geoespaciais.

A dificuldade de encontrar dados em geoportais e catálogos é anterior aos LLMs: Hu et al. (2015) a atribuem, em parte, à heterogeneidade dos metadados publicados por diferentes contribuidores e propõem harmonização de tópicos e busca semântica para geoportais baseados em *Linked Data*. Com os LLMs, Ning et al. (2025) propõem um agente que escolhe a fonte de dados numa lista predefinida e gera o programa que a recupera, guiado por um manual de cada fonte, e Li et al. (2025) combinam um grafo de conhecimento, construído a partir de cinco grandes portais, com LLMs para responder a perguntas de descoberta de dados, motivados pelo pouco suporte semântico dos portais existentes. No caso da DSG, o catálogo é único e já estruturado, e o problema se reduz a traduzir a intenção do usuário nos filtros desse catálogo; por isso a solução dispensa grafo de conhecimento e geração de código e se concentra na qualidade dessa tradução.

Avaliação da chamada de ferramentas.

Os trabalhos sobre uso de ferramentas por LLMs vieram acompanhados de *benchmarks* próprios: o API-Bank (LI et al., 2023), com diálogos anotados sobre dezenas de APIs executáveis; o ToolBench, do ToolLLM (QIN et al., 2024), com mais de 16 mil APIs reais; o APIBench, do Gorilla (PATIL et al., 2024); e o *Berkeley Function Calling Leaderboard* (BFCL) (PATIL et al., 2025), que compara a chamada gerada com a esperada pela sua árvore sintática, avalia também a capacidade de o modelo se abster e conclui que os melhores modelos vão bem em chamadas de um único turno, mas ainda não em memória, decisões dinâmicas e raciocínio de longo prazo. Esses *benchmarks* são predominantemente em inglês, de domínio geral e com muitas ferramentas. Nenhum cobre o cenário deste trabalho — uma única ferramenta de busca, consultas em português com o vocabulário técnico da cartografia sistemática (MI, INOM, escalas, centros produtores) e modelos pequenos executados localmente —, o que motivou a construção de um *dataset* próprio, com gabarito auditado e comparação campo a campo semelhante, em espírito, à comparação estrutural do BFCL.

Ferramentas auxiliares e retorno.

O *Tool Calling* não precisa ser uma única chamada: no ReAct (YAO et al., 2023), o modelo alterna raciocínio e ações, e o resultado de cada ação alimenta o passo seguinte. A correção a partir desse retorno depende de ele ser externo: o CRITIC (GOU et al.,

2024) verifica a saída com ferramentas e a revisa, e o *self-debugging* (CHEN et al., 2024) corrige consultas a partir do resultado da execução, inclusive no Spider; já a autocorreção sem sinal externo (MADAAN et al., 2023) pode não ajudar e até piorar o raciocínio (HUANG et al., 2024). Os *benchmarks* recentes medem essas capacidades separadamente: o ToolSandbox (LU et al., 2025) identifica a canonicalização de argumentos — datas relativas, lugares — como uma das maiores dificuldades e usa um usuário simulado; o When2Call (ROSS; MAHABALESHWARKAR; SUHARA, 2025) mostra que decidir entre chamar, perguntar ou recusar é uma capacidade própria, fraca em modelos pequenos; o τ -bench (YAO et al., 2025) avalia o estado final de diálogos com usuário simulado; e o CRITICTOOL (HUANG et al., 2025) classifica os erros de chamada que o modelo precisa perceber e corrigir. Todo ajuste guiado pelos erros de um conjunto rotulado — por lições em linguagem natural acumuladas a partir de sucessos e falhas, como no Reflexion (SHINN et al., 2023) e no ExpeL (ZHAO et al., 2024), ou por otimização automática de *prompts* (YANG et al., 2024; KHATTAB et al., 2024) — corre o risco de sobreajustar ao conjunto em que se mede, o que exige um conjunto de teste que o desenvolvimento não tenha visto (DWORK et al., 2015). A v3 deste trabalho (Seção 3.6) aplica essas ideias num domínio restrito: ferramentas auxiliares determinísticas, validação com retorno e pedido de esclarecimento com usuário simulado, acrescentados em degraus e avaliados num lote de teste independente.

3 METODOLOGIA

Este capítulo descreve o método de trabalho adotado para o aprimoramento da solução de busca em linguagem natural sobre o acervo cartográfico da DSG. O trabalho parte de um protótipo de referência desenvolvido pelo 1º CGEO, descrito e analisado em detalhe no Capítulo 4, e adota a estratégia de **substituição dirigida**: identificam-se os componentes responsáveis pelas limitações documentadas pelo cliente e substituem-se esses componentes por equivalentes mais robustos, preservando o que já funciona (esquema do banco, conjunto de parâmetros de busca, lógica das consultas espaciais).

O capítulo apresenta os elementos transversais ao projeto: o escopo (entregáveis e exclusões), os modelos comparados, a construção e a auditoria do *dataset* de avaliação, os dois lotes de validação, as especificações v2 e v3 e o protocolo de medição. A execução do aprimoramento — análise do protótipo, requisitos, planejamento da migração, desenvolvimento por camadas e cronograma — é apresentada no Capítulo 4.

Foco da avaliação: a camada de tradução linguística.

O objeto de avaliação deste trabalho é o desempenho do LLM na **tradução de uma consulta em linguagem natural para os parâmetros estruturados** de busca no catálogo. A execução dessa consulta contra o banco (PostgreSQL/PostGIS), a resolução de nomes geográficos e a interpretação das *bounding boxes* das cartas são componentes herdados do protótipo do 1º CGEO; sua correção é assumida como *caixa-preta* e não é medida. Métricas sobre o *conjunto de produtos retornados* ficam fora do escopo (Seção 3.1.2) e são apontadas como direção de trabalho futuro no Capítulo 6. O LLM é comparado contra um *ground truth* de **parâmetros extraídos**, não de produtos recuperados.

3.1 Escopo do Projeto

3.1.1 Entregáveis

O projeto entregou:

- uma nova arquitetura de *backend* em Python (FastAPI + LangChain), que substituiu o *backend* Node.js do protótipo e implementa *Tool Calling* nativo via Ollama;
- um *dataset* de avaliação com 310 consultas de referência (*ground truth*), distribuídas em seis categorias de consulta e uma de consultas fora do domínio, acompanhado de um manual de anotação e de um relatório de auditoria;

- um *pipeline* de avaliação automatizado, que executa o *dataset* sobre cada modelo e produz as métricas de acurácia, precisão, *recall*, F1-score e latência, com os testes estatísticos correspondentes;
- um relatório comparativo entre quatro modelos de LLM com suporte a *Tool Calling*, executados localmente, com a recomendação fundamentada do modelo mais adequado para uso operacional sobre o acervo da DSG, complementado por uma referência paralela com modelos maiores servidos em nuvem;
- dois lotes de validação independentes (Seções 3.5 e 3.6) e a v3 do *Tool Calling*, com ferramentas auxiliares e validação com retorno, desenvolvida sobre as 310 consultas e o primeiro lote e testada no segundo contra um controle com Saída Estruturada; a v3 está implementada no pacote (`src/pfc_busca/v3.py`), no avaliador e na API do *backend*, em que é a abordagem padrão.

3.1.2 Exclusões

Estão fora do escopo deste trabalho:

- **Avaliação da qualidade de recuperação em nível de produto:** uma vez extraídos os parâmetros, o conjunto de produtos retornado depende do motor de busca (`buscar_catalogo`) e da qualidade do banco cartográfico, ambos herdados do protótipo. O trabalho **não mede** precisão e *recall* em nível de produto. A execução SQL permanece no *pipeline* apenas como demonstração de que a arquitetura fecha ponta a ponta;
- **Resolução avançada de nomes geográficos:** o cruzamento entre *strings* livres (estado, município) e as *bounding boxes* das cartas depende de recursos externos (malha do IBGE, Nominatim/OSM) e da qualidade das geometrias no banco; é tratado como trabalho futuro (Capítulo 6);
- desenvolvimento de interface gráfica (*frontend*) para o usuário final;
- implantação em produção em ambiente da DSG ou de suas OM subordinadas;
- pesquisa de opinião ou avaliação qualitativa com usuários;
- treinamento ou ajuste fino (*fine-tuning*) de modelos de linguagem;
- busca por coordenadas geográficas brutas (*bounding box*) sem referência nominal a um lugar;
- suporte a idiomas além do português brasileiro.

3.1.3 Requisitos que norteiam o escopo

O escopo é norteado por requisitos funcionais (RF) e não funcionais (RNF) elicitados a partir da análise do protótipo e das conversas com o cliente. O Quadro 1 traz o resumo; a discussão completa está no Capítulo 4 (Seção 4.2).

Quadro 1 – Resumo dos requisitos que norteiam o escopo

Requisitos Funcionais (RF)		Requisitos Não Funcionais (RNF)	
RF1	Receber consulta em linguagem natural via REST	RNF1	Execução totalmente local (privacidade)
RF2	Traduzir consulta em parâmetros do <i>schema</i>	RNF2	Latência mediana operacional aceitável
RF3	Executar busca no PostgreSQL/PostGIS	RNF3	Conformidade a ET-PCDG e Perfil MGB
RF4	Selecionar modelo por requisição (<i>dev only</i>)	RNF4	Reprodutibilidade dos experimentos
RF5	Registrar traços para as métricas	RNF5	Manutenibilidade (módulos, testes)
		RNF6	Substituibilidade de modelo por configuração

Fonte: Elaborado pelos autores. Detalhes no Cap. 4, Seção 4.2.

Requisitos ideais são mensuráveis, mas alguns dos RNF — em particular o RNF2, latência — dependem de fatores externos ao escopo deste PFC (*hardware* do ambiente de implantação, carga concorrente). Por isso os valores de latência são reportados nas condições de cada ambiente de execução (Seção 3.7.2), sem limiar operacional fixado *a priori*.

3.2 Seleção de Modelos

3.2.1 Critérios de Seleção

Os modelos foram selecionados segundo os critérios: (i) disponibilidade no Ollama com suporte nativo a *Tool Calling*, verificada no catálogo oficial da biblioteca (Ollama Team, 2023), que mantém rankings de popularidade e uma etiqueta explícita de compatibilidade com *tool calling*; (ii) disponibilidade em versões quantizadas compatíveis com estações de trabalho modestas; (iii) lançamento ou atualização entre 2024 e 2026; e (iv) diversidade de arquiteturas e organizações desenvolvedoras.

3.2.2 Modelos Avaliados

Foram avaliados quatro modelos abertos, executados localmente pelo Ollama:

1. **Qwen 3 4B** (Qwen Team, 2025) (Alibaba Cloud), *tag* `qwen3:4b-instruct-2507-q4_K_M`: 4 bilhões de parâmetros, quantização Q4_K_M, 2,3 GB — a menor *build* dos quatro; mesmo assim, carregado com o contexto, não coube inteiro nos 4 GB de VRAM da estação de referência, onde o Ollama alocou na GPU 66% do modelo carregado (Tabela 26). Usou-se a *build* oficial sem raciocínio interno (*Instruct-2507*), pela razão explicada na Seção 3.7.3;
2. **Gemma 4 E4B** (Gemma Team, 2026) (Google DeepMind), *tag* `gemma4:e4b-it-qat`: variante *edge* de cerca de 4 bilhões de parâmetros efetivos, com quantização consciente de treinamento. É a menor *build* publicada do E4B (~5,7 GB) e não cabe em 4 GB de VRAM — a variante Q4_K_M ocupa ~9 GB —, o que exige *offload* parcial para a CPU na estação de referência;
3. **Gemma 4 E2B** (Gemma Team, 2026), *tag* `gemma4:e2b-it-qat`: variante menor da mesma família, com cerca de 2 bilhões de parâmetros efetivos (~4,0 GB), incluída como alternativa de menor custo computacional ao E4B, que não cabe na VRAM da estação de referência; o E2B também recebe *offload* parcial nessa estação. Permite observar, dentro de uma mesma família, o efeito do tamanho do modelo;
4. **Mistral Nemo 12B** (Mistral AI, 2024) (Mistral AI e NVIDIA), *tag* `mistral-nemo:12b`: 12 bilhões de parâmetros (~6,6 GB), com suporte consolidado a *Tool Calling* e bom desempenho em português. Representa a faixa superior de tamanho executável com *offload* parcial numa estação modesta.

3.2.3 Referência paralela em nuvem

Como referência — e não como candidatos à implantação, uma vez que o requisito RNF1 exige execução local —, a mesma avaliação foi executada sobre três modelos maiores servidos pelo provedor Groq e disponíveis com suporte a *Tool Calling* em setembro de 2026: `qwen/qwen3.8-27b`, `openai/gpt-oss-20b` e `openai/gpt-oss-120b` (Qwen Team, 2026; OpenAI, 2025). Os três têm pesos abertos, sob licença Apache 2.0, mas exigem mais memória do que a estação de referência oferece. O objetivo é situar os modelos locais em relação a modelos de maior porte sob o mesmo *prompt*, a mesma ferramenta, o mesmo *dataset* e as mesmas métricas. A troca de provedor usa exatamente a portabilidade descrita no Capítulo 2: o `ChatOllama` do LangChain é substituído pelo `ChatOpenAI` apontado para o *endpoint* compatível do provedor, sem alteração no restante do *pipeline*. Esses resultados são sempre reportados à parte e identificados como execução em nuvem.

3.3 Construção do *Dataset* de Avaliação

A comparação objetiva dos modelos exige um *dataset* de referência representativo do espectro de consultas que usuários do catálogo tenderiam a formular, cobrindo diferentes níveis de complexidade e combinações de parâmetros, e cujo gabarito seja **auditável**: cada resposta esperada deve poder ser justificada a partir do texto da consulta e de regras escritas, e qualquer leitor deve poder refazer essa verificação.

O *dataset* serve exclusivamente para **avaliação**: nenhum modelo foi treinado ou ajustado com ele, e cada modelo recebe as consultas uma a uma, sem exemplos resolvidos (*zero-shot*). A qualidade que importa é, portanto, a do gabarito — se cada resposta esperada está certa e se as leituras aceitas são todas as leituras legítimas da consulta —, e ela se apoia em três mecanismos: regras de anotação escritas antes da anotação (Seção 3.3.4), construção rastreável em camadas (Seção 3.3.3) e uma auditoria com checagens automáticas, anotação independente às cegas, adjudicação registrada e conferência humana de uma amostra (Seção 3.4).

3.3.1 Estrutura de cada entrada

Cada consulta do *dataset* é registrada com os campos da Tabela 3, acrescidos de **origem** (P, N ou G), **familia** (P, N, GS ... GF) e **trocas** (leituras estruturais alternativas, das quais **alternativas** é a expansão); valores alternativos e campos opcionais ficam marcados no próprio **esperado**. O arquivo `dataset.json` e o Apêndice A são gerados a partir da mesma fonte, de modo que não podem divergir.

Tabela 3 – Estrutura de cada entrada do *dataset* de avaliação

Campo	Descrição
id	Identificador (P01–P22, N01–N40, GS001 ... GF007)
fonte	Rastreabilidade: arquivo e linha de origem (camada P), autores (camada N) ou função geradora, modelo de frase e semente (camada G)
categorias	Uma ou mais das categorias S, C, M, T, O, A; ou F (fora do domínio), que não se combina com as demais
consulta	Texto em linguagem natural
esperado	Leitura preferencial do gabarito: parâmetros de busca na forma canônica, com períodos relativos expressos por regras nomeadas
alternativas	Demais leituras aceitas, quando a consulta admite mais de uma (Seção 3.3.4)
espera_tool_call	Falso quando o comportamento correto é não chamar a ferramenta
observacional	Verdadeiro quando o caso fica fora das métricas principais
notas	Justificativa das leituras alternativas e observações do anotador

Fonte: Elaborada pelos autores.

3.3.2 Categorias de Consultas

O *dataset* é organizado em seis categorias, alinhadas aos exemplos do protótipo e às demandas do cliente: **simples** (S, um único parâmetro: “cartas de São Paulo”); **compostas** (C, dois ou mais parâmetros: “ortoimagens de Manaus em 1:50.000”); **com código MI/INOM** (M: “INOM SG-22-X-A”); **com referência temporal relativa** (T: “nos últimos 3 meses”, “depois de 2022”); **com ordenação** (O: “as 3 cartas mais antigas”); e **ambíguas ou informais** (A: siglas, grafias sem acento, abreviações e expressões qualitativas). Uma consulta pode pertencer a mais de uma categoria. Uma sétima categoria, **fora do domínio** (F: “cartas de Marte”, “qual a previsão do tempo para amanhã em Manaus?”), reúne as consultas em que a resposta correta é não chamar a ferramenta; ela mede a capacidade de o modelo recusar pedidos que o catálogo não atende.

3.3.3 Camadas de Construção

O *dataset* tem 310 consultas, construídas em três camadas complementares (Tabela 4):

1. **Camada P (protótipo, 22 casos)**: as consultas do arquivo de testes do protótipo (`backend/evaluation/test-cases.ts`), cada uma rastreada até a linha de origem. A anotação original da equipe do 1º CGEO é mantida como leitura preferencial — o *baseline* não é reanotado —, e as leituras alternativas acrescentadas seguem o manual de anotação, com justificativa registrada. Os gabaritos de datas relativas do arquivo original, calculados para uma data de referência do início de 2025, são substituídos pelas regras nomeadas da Seção 3.3.4, que se resolvem na data de referência de cada rodada.
2. **Camada N (autores, 40 casos)**: redigidos para preencher lacunas de distribuição — em especial a categoria Simples, ausente na camada P — e para exercitar ambiguidades e casos de fronteira do domínio.
3. **Camada G (geração programática, 248 casos)**: consultas sintetizadas pela combinação de catálogos fechados do domínio — 27 UFs, 12 municípios de referência, 8 escalas do SCN, 9 tipos de produto das ET-PCDG, 5 CGEOs, 9 projetos, códigos MI e INOM, 15 expressões de tempo (14 relativas e o intervalo absoluto “entre 2022 e 2023”) e padrões de ordenação — em modelos de frase do português corrente (com preposição adequada a cada UF e apelidos usuais de escala, como “25k” ou “escala de 25 mil”). O gabarito é conhecido **por construção**: a frase é montada a partir dos parâmetros que deve produzir. A geração é determinística (semente 42) e rejeita, dentro da camada G, qualquer consulta que repita outra após normalização; a construção do *dataset* verifica ainda que nenhuma consulta gerada repita, após

normalização, uma das camadas P e N, e é interrompida se isso ocorrer. A camada inclui sete consultas de fronteira do domínio (família G-F): seis fora do domínio e uma observacional.

Tabela 4 – Composição do *dataset* de avaliação por origem e família

Origem / família	Consultas	Observacionais	Com mais de uma leitura aceita
Protótipo do 1º CGEO (P)	22	0	11
Autores (N)	40	3	15
Geradas — simples (G-S)	61	0	0
Geradas — compostas (G-C)	55	0	3
Geradas — código MI/INOM (G-M)	28	0	0
Geradas — tempo relativo (G-T)	41	0	20
Geradas — ordenação (G-O)	19	0	16
Geradas — informais (G-A)	24	0	2
Geradas — amplas (G-P)	13	0	0
Geradas — fronteira (G-F)	7	1	0
Total	310	4	67

Fonte: Elaborada pelos autores a partir de `dataset.json`.

Justificativa da geração programática.

Consultas escritas à mão são caras de redigir e anotar e tendem a variar sobre um pequeno conjunto de padrões familiares aos autores. A geração por modelos de frase sobre catálogos fechados oferece cobertura sistemática de combinações raras (pares UF $\times$ escala pouco frequentes, por exemplo), escala sem custo adicional de anotação e volume suficiente para estimar intervalos de confiança e comparar modelos com testes de significância. A contrapartida é a regularidade linguística, que pode favorecer os modelos: por isso a acurácia é também desagregada por camada (Tabela 24), o que permite distinguir o desempenho nas consultas geradas do desempenho nas consultas redigidas por pessoas (P e N).

3.3.4 Manual de Anotação e Leituras Múltiplas

O gabarito é construído segundo um manual de anotação escrito (Apêndice A.1) que fixa seis princípios — evidência explícita no texto, forma canônica, leituras múltiplas explícitas, tratamento das consultas fora do domínio, consistência e casos observacionais — e regras para cada um dos doze campos da ferramenta.

O princípio mais importante para a confiabilidade da avaliação é o das **leituras múltiplas explícitas**. Uma parte das consultas admite, legitimamente, mais de uma resposta correta: “cartas de São Paulo” pode referir-se ao estado ou à capital; “produtos

Tabela 5 – Consultas por categoria e por camada (métricas principais; uma consulta pode ter mais de uma categoria)

Categoria	P	N	G	Total
Simples (S)	0	11	89	100
Compostas (C)	22	21	137	180
Código MI/INOM (M)	10	5	42	57
Tempo relativo (T)	6	9	45	60
Ordenação (O)	4	6	19	29
Ambíguas/informais (A)	16	19	162	197
Fora do domínio (F)	0	2	6	8

Fonte: Elaborada pelos autores.

publicados nos últimos 3 meses” pode significar noventa dias ou três meses de calendário; “as cartas mais recentes” pode ser ordenado pela data de publicação ou pela de criação, pois a consulta não diz qual. Um gabarito que escolhesse uma única leitura penalizaria como erro uma resposta razoável e mediria a coincidência com a preferência do anotador, não a qualidade da tradução. Nesses casos o gabarito lista **todas** as leituras aceitas, por três mecanismos: *valor alternativo* (o campo aceita mais de um valor), *campo opcional* (pode faltar sem erro, como `limit = 1` em “a carta mais recente”) e *leitura estrutural alternativa* (o mesmo trecho pode ocupar campos diferentes, como `state` ou `city`). As expressões de tempo relativo são registradas como regras nomeadas, resolvidas na data de referência da rodada — registrada no manifesto e informada ao modelo no *prompt* de sistema — com as leituras aceitas do Quadro 5.

A ambiguidade não é, porém, tratada com leniência genérica: toda leitura aceita está escrita no gabarito, justificada por uma regra do manual e visível no Apêndice A; fora delas, a comparação é estrita (Seção 3.7.5). Das 306 consultas das métricas principais, a Tabela 4 indica quantas admitem mais de uma leitura.

Consultas fora do domínio e casos observacionais.

Oito consultas estão fora do domínio (N36, N37 e seis da família G-F): o comportamento correto é determinado — não chamar a ferramenta — e elas entram nas métricas principais, na categoria F. Quatro consultas são observacionais (N38, N39, N40 e GF007), porque o comportamento correto não é determinável a partir do texto: valor impossível de representar (“escala 1:10.000.000”, “cartas do estado 42”) ou pedido de resposta discutível (“cartas topo no oceano atlântico”, “cartas do futuro (ano 2050)”). Elas são executadas e o comportamento de cada modelo é reportado à parte; as métricas principais cobrem as 306 consultas restantes.

Tabela 6 – Frequência de cada campo na leitura preferencial do gabarito (métricas principais)

Campo	Ocorr.	Campo	Ocorr.
keyword	62	project	11
scale	84	publicationPeriod	51
productType	44	creationPeriod	10
state	130	sortField	29
city	27	sortDirection	29
supplyArea	40	limit	21

Fonte: Elaborada pelos autores.

3.4 Auditoria do *Dataset*

A confiabilidade das métricas depende diretamente da confiabilidade do gabarito. Por isso o *dataset* foi submetido a uma auditoria em três etapas, repetidas até não restar divergência sem decisão registrada. O procedimento, as ferramentas e os resultados completos estão no Apêndice A.14 e no repositório ([data/auditoria/](#)).

1. **Checagens automáticas (pfc-auditar)**: unicidade de identificadores e de consultas (após normalização de caixa, acentos e pontuação); validade de todos os valores contra os enumerados da ferramenta; resolução de todas as regras de tempo relativo em datas diversas, inclusive anos bissextos e viradas de ano; rastreabilidade das consultas da camada P até a linha citada do arquivo de origem; um **anotador por regras**, que localiza na consulta o trecho que justifica cada campo do gabarito e aponta campo sem evidência, evidência sem campo e valor divergente; e uma verificação de **consistência** de convenções entre casos (a mesma expressão, a mesma anotação).
2. **Anotação independente às cegas**: todas as 310 consultas foram anotadas novamente, a partir do zero, por um modelo de linguagem de propósito geral de grande porte (Claude Opus 5.5, da Anthropic), que não integra o conjunto de modelos avaliados. A anotação foi feita em dez lotes de 31 consultas, em sessões independentes, cada uma com acesso apenas ao manual de anotação, à definição da ferramenta, à data de referência e ao próprio lote — sem acesso ao gabarito, ao gerador ou aos demais lotes. Para cada consulta, o anotador registrou a decisão (chamar a ferramenta, não chamar ou caso observacional), a leitura preferencial, as leituras alternativas previstas pelo manual e uma justificativa por referência às regras.
3. **Adjudicação**: cada consulta com divergência em qualquer das medidas de concordância recebeu uma decisão registrada — gabarito mantido, corrigido ou ampliado com leitura alternativa — com justificativa por referência ao manual. O gabarito

anterior a cada decisão fica registrado, de modo que a concordância anterior à adjudicação pode ser recalculada a partir do repositório.

A concordância entre o gabarito e a anotação independente foi medida em rigor crescente (Tabela 7): a decisão sobre chamar a ferramenta, com o kappa de Cohen (COHEN, 1960; ARTSTEIN; POESIO, 2008), que desconta a concordância esperada ao acaso; a compatibilidade das leituras nos dois sentidos (a leitura do anotador é aceita pelo gabarito e a leitura preferencial do gabarito é aceita pelo anotador); a identidade da leitura preferencial; a presença e o valor de cada campo; e a detecção de ambiguidade, isto é, se a consulta admite mais de uma leitura. A coluna “antes da adjudicação” mede a concordância propriamente dita; a coluna “após” mostra o que restou depois das decisões.

Tabela 7 – Resultados da auditoria do *dataset*

Verificação	Antes da adjudicação	Após
<i>Checagens automáticas</i>		
Problemas estruturais (ids, duplicatas, enumerados, datas)	—	0
Consultas da camada P conferidas na linha de origem	—	22/22
Apontamentos do anotador por regras sem resolução	—	0
Expressões com anotação inconsistente sem justificativa	—	0
<i>Anotação independente às cegas (310 consultas)</i>		
Decisão (chamar / não chamar / observacional)	302/310 (97,4%)	310/310 (100,0%)
kappa de Cohen	0,658	1,000
Leituras compatíveis nos dois sentidos	300/310 (96,8%)	309/310 (99,7%)
Leitura preferencial idêntica ^a	294/298 (98,7%)	294/298 (98,7%)
Presença de cada campo na leitura preferencial ^{a,b} (kappa)	0,999	0,999
Valor do campo, quando ambos o anotam ^a	534/537 (99,4%)	534/537 (99,4%)
Consulta com mais de uma leitura aceita ^a	296/298 (99,3%)	297/298 (99,7%)
kappa de Cohen	0,981	0,990
Consultas com divergência em alguma medida, sem decisão registrada	14	0
Revisão humana de amostra estratificada: gabarito confirmado	—	40/40 (100,0%)

Fonte: Elaborada pelos autores a partir de `data/auditoria/`. (a) Sobre as consultas em que gabarito e anotador pedem a chamada da ferramenta nas métricas principais. (b) Doze campos por consulta.

As medidas de campo e de ambiguidade mostram concordância quase perfeita já antes da adjudicação. A principal divergência foi de classificação: o gabarito tratava como observacionais oito consultas fora do domínio cujo comportamento correto é determinado

pelo próprio manual (não chamar a ferramenta). A decisão foi corrigir o gabarito e incluí-las nas métricas principais, na categoria F. As demais divergências foram uma leitura ampliada (N33, em que “cartografia sistemática” passou a admitir a leitura sem o campo *project*, pois a expressão não consta da descrição da ferramenta) e cinco divergências mantidas com justificativa, entre elas três escolhas de leitura preferencial dentro do mesmo conjunto de leituras aceitas, que não afetam a pontuação. O Quadro 7 do Apêndice A.14 lista essas 14 decisões e mais uma, originada nas checagens de consistência (P14, mantida), num total de 15 consultas adjudicadas.

Duas ressalvas delimitam o alcance da auditoria. O anotador por regras foi escrito pelos autores e, portanto, verifica evidência e consistência, mas não constitui uma anotação independente. A anotação às cegas é independente do gabarito, mas foi feita por um modelo de linguagem, e não por uma pessoa: ela identifica divergências com alta sensibilidade, e a decisão sobre cada uma é tomada na adjudicação, por referência ao manual. Na camada G, além disso, o gabarito é conhecido por construção, e a auditoria verifica sobretudo se a frase gerada sustenta, em português corrente, os parâmetros que a geraram.

Para compensar em parte essa ressalva, uma amostra estratificada de 40 consultas (semente 42), sorteada entre as camadas P, N e G, foi conferida manualmente por um dos autores, que confirmou o gabarito de todas elas (Tabela 7). Em síntese, a resposta à pergunta “como saber se o gabarito está correto” é dada por números: antes de qualquer correção, uma anotação feita do zero, sem acesso ao gabarito, concordou com ele em 99,4% dos valores de campo anotados por ambos (534 de 537), teve kappa de 0,999 na presença dos campos e de 0,981 na identificação das consultas ambíguas, e encontrou leitura compatível em 300 das 310 consultas; a única divergência sistemática — a classificação de oito consultas fora do domínio — foi corrigida na adjudicação, e nenhuma divergência ficou sem decisão registrada.

3.5 Lote de Validação Independente

As 310 consultas deixam três perguntas que só um segundo conjunto responde: se as conclusões dependem da regularidade da camada G; se a recusa, medida sobre oito consultas fora do domínio, se sustenta numa amostra maior; e se o desempenho do *Tool Calling* reflete a abordagem ou a especificação dada ao modelo. Para isso foi construído um **lote de validação** (daqui em diante, também **lote 1**) de 1.929 consultas novas, independente das 310, usado no Capítulo 5 com o modelo local de maior acurácia com *Tool Calling*. O lote difere das 310 em três pontos de método: os valores vêm de catálogos reais; o gabarito de cada consulta é fixado *antes* de o texto existir; e a consulta só entra no lote se uma anotação às cegas, feita sem acesso ao gabarito, for compatível com ele.

Cada etapa tem *script*, semente e arquivos versionados no repositório, e a metodologia completa está em `docs/lote_validacao.md`.

Fontes reais.

Os metadados públicos do catálogo do BDGEx foram coletados pelo serviço CSW (OGC CSW 2.0.2) em 28/09/2026: 31.190 registros, dos quais saem nomes de folha (7.838 distintos), códigos MI e INOM, escalas, tipos de produto, CGEOs e anos. A lista oficial de municípios do IBGE (5.571) fornece os municípios. Nas consultas compostas, metade das combinações de escala, tipo, CGEO e folha é tirada de uma mesma carta do acervo; o lugar é sorteado à parte.

Alvos e gabarito por construção.

Foram sorteados 1.950 *alvos* (semente 2026) em oito famílias (Tabela 8). Cada alvo especifica o gabarito, segundo o mesmo manual de anotação, a forma de superfície de cada valor (“25k”, “1:25000”, “3o cgeo”, “terceiro cgeo”, sigla ou nome da UF, INOM compacto em minúsculas...), o que a consulta não pode dizer, um registro de linguagem (formal, direto, coloquial, pergunta, telegráfico, sem acento, com erro de digitação ou com contexto de uso) e uma persona (do oficial que planeja uma operação ao cidadão sem conhecimento técnico). O manual recebeu extensões que não alteram nenhuma anotação das 310 (Apêndice A.1, “Extensões para o lote de validação”): regras de tempo relativo parametrizadas (“últimos N meses”, “desde AAAA” para qualquer N e ano), municípios acompanhados da sigla da UF e as *armadilhas de vocabulário* (“carta de vinhos”, “folha de pagamento”, “escala de plantão”), que estão fora do domínio.

Duas mudanças de composição respondem diretamente às perguntas acima. A categoria F passa a ter 336 consultas, em nove subtipos, o que permite medir a recusa como classificação — precisão, *recall* e F1 — e a taxa de falsa recusa nas consultas do domínio. E uma nova categoria, **subespecificadas** (E, 146 consultas), reúne pedidos de produtos do acervo sem nenhum critério representável no *schema* (“quero ver as cartas do acervo”, “cartas do Nordeste”, “mapas para uma trilha”), em que duas respostas são aceitas: buscar sem filtros ou não buscar. Como prevê o manual (“pedir esclarecimento ou recusar”), a pontuação aceita qualquer não busca, e não só o pedido de esclarecimento: a recusa de uma consulta E também conta como acerto, embora a especificação v2, descrita adiante, diga que “consultas vagas sobre o acervo não são fora do escopo”. A regra favorece sobretudo a Saída Estruturada v2, que marcou **fora_do_escopo** em 100,0% das consultas E; com essas recusas contadas como erro, a sua acurácia no lote seria de 64,9%, e não de 72,4% (Seção 5.4). Um único critério (“ortoimagens”) continua sendo consulta simples, que exige a busca, como nas 310. Ao todo, 471 consultas do lote admitem mais de uma leitura aceita.

Redação controlada.

Os alvos foram embaralhados e divididos em 15 tarefas de 130, cada uma redigida por um agente de linguagem (Claude Opus 5.5, da Anthropic, que não integra os modelos avaliados), que recebeu apenas o que a consulta deveria e não deveria dizer, o registro e a persona — não o gabarito — e a instrução de soar como uma pessoa digitando numa caixa de busca, variando a construção de uma mensagem para outra. Checagens automáticas conferem, em cada consulta, a presença literal das superfícies exigidas, o verbo do período (publicação, criação ou nenhum) e a ausência de marcadores de campos que o alvo não tem; a deduplicação descarta consultas que repitam, após normalização, uma das 310 ou outra do lote.

Anotação às cegas e filtro.

Todas as consultas redigidas foram anotadas de novo, a partir do zero, por outros 15 agentes, com o mesmo formato e as mesmas referências da anotação independente das 310 (Seção 3.4): o manual, a definição da ferramenta, a data de referência e a lista de municípios do IBGE. As consultas chegaram embaralhadas e com identificadores neutros, sem família, persona ou gabarito. Uma amostra aleatória de 20% (390 consultas) recebeu uma segunda anotação, por outros três agentes, que não participa do filtro. Entra no lote a consulta cuja anotação é compatível com o gabarito nos dois sentidos — o critério da auditoria das 310. Das 1.950 consultas redigidas, 1.929 (98,9%) formam o lote; as descartadas, com o motivo, estão na Tabela 8 e no repositório.

Tabela 8 – Lote de validação: do alvo ao lote final, por família

Família	Testa	Alvos	Checagens	Anotação às cegas	Lote final
VS	Simples (um critério)	280	−3	0	277
VC	Compostas	360	0	−1	359
VM	Códigos MI/INOM	160	0	0	160
VT	Tempo	260	−1	0	259
VO	Ordenação	200	−1	0	199
VA	Leituras múltiplas	200	−7	0	193
VE	Subespecificadas	150	−4	0	146
VF	Fora do domínio	340	−4	0	336
Total		1.950	−20	−1	1.929

Fonte: Elaborada pelos autores. Checagens: descartadas nas checagens automáticas e na deduplicação. Anotação às cegas: descartadas por incompatibilidade entre o gabarito por construção e a anotação A.

A concordância, medida sobre todas as consultas redigidas antes do filtro (Tabela 9), foi de kappa 1,000 na decisão (buscar, buscar ou esclarecer, não buscar), 99,8% de leituras compatíveis e 100,0% dos valores de campo; entre os dois anotadores, 100,0% de leituras compatíveis. Esses números são altos por construção — os alvos exigem superfícies explícitas, e o manual decide as ambiguidades previstas — e devem ser lidos com

a mesma ressalva da auditoria das 310, agravada: redatores e anotadores são modelos da mesma família. Eles mostram que cada consulta sustenta o seu gabarito segundo o manual, de forma reproduzível por anotadores independentes; não mostram que pessoas leriam do mesmo modo. Dois sinais indicam que a anotação não reproduziu o gabarito por outra via: na amostra dupla, os anotadores concordam mais entre si do que com a construção, e as justificativas que escreveram são textualmente distintas. O texto das consultas é sintético — não há registro de consultas reais de usuários do BDGEx disponível —, e essa é a principal limitação do lote.

Tabela 9 – Concordância na anotação às cegas do lote de validação

Medida	Construção $\times$ A (n=1.950)	A $\times$ B (n=390)
Decisão (buscar / buscar ou esclarecer / não buscar), kappa	1,000	1,000
Leituras compatíveis nos dois sentidos	99,8%	100,0%
Leitura preferencial idêntica	99,8%	100,0%
Presença dos campos, kappa	0,999	1,000
Valor igual quando ambos preenchem	100,0%	100,0%
Deteção de leituras múltiplas, kappa	0,967	1,000

Fonte: Elaborada pelos autores. Medidas de `audit.concordancia`, as mesmas da auditoria das 310 consultas, sobre todas as consultas redigidas (antes do filtro). A: anotação de todas as consultas; B: segunda anotação independente de uma amostra aleatória de 20%.

Especificação v2.

A análise de erros do *Tool Calling* do Gemma 4 E4B nas 310 consultas mostrou que os dois erros mais frequentes contrariam instruções que o *prompt* v1 já dava. Em 161 das 918 execuções (17,5%), o modelo não buscou numa consulta do domínio, embora a instrução 1 diga que “um único parâmetro basta” e a 2 só permita deixar de chamar a ferramenta quando a consulta não tratar de produtos do acervo; em 154 delas pediu esclarecimento — às vezes de informação que já tinha, como “preciso saber qual é o ano atual”. Em 72, acrescentou o tipo de produto a “cartas” ou “mapas”, contra a instrução 3 (“não complete campos por suposição”). Outros vêm de exemplos que a descrição trazia ou de convenções do manual que ela não informava: em 21, o modelo acrescentou ao código um sufixo, em geral o do exemplo da descrição (“2901” virou “2901-2-NE”), ou usou “folha” como *keyword*; em 18, resolveu “este ano” como um período que começa hoje. Em 21, deixou a escala sem o ponto de milhar, embora o enumerado da definição traga a grafia com o ponto. A descrição v1 também não informava outras convenções do gabarito: que “última atualização” indica a data de criação, que um período que vai até hoje tem fim e que uma escala qualitativa como “grande escala” deve ser preenchida com uma das escalas que ela abrange.

Para separar o efeito da abordagem do efeito da especificação, as duas abordagens foram avaliadas também numa **v2**, que muda apenas a especificação: descrições dos parâmetros que explicitam as convenções do manual (quando preencher cada campo, a

forma canônica, o que não deduzir), sem os códigos de exemplo da v1 e com a lista das siglas das 27 UFs; no *Tool Calling*, uma segunda ferramenta, `recusar_consulta`, e a instrução de chamar exatamente uma das duas, sem pedir esclarecimento; na Saída Estruturada, as mesmas descrições, a mesma proibição de pedir esclarecimento e um campo `fora_do_escopo`, que torna a recusa representável. A v2 não ficou livre de exemplos copiáveis: a descrição de `keyword` traz “sf22yd” → “SF-22-Y-D”, que é o código da consulta N35 das 310 com o seu gabarito, e a de `limit`, “a carta mais recente” → 1. A descrição de `state` pede o nome por extenso, mas troca a regra com direção da v1 (“Normalizar siglas: ‘RJ’ → ‘Rio de Janeiro’”) por uma lista de pares (“AC Acre, AL Alagoas”, e assim por diante), e os dois *prompts* v2 perderam a frase da instrução 1 da v1 segundo a qual siglas, abreviações e grafias sem acento “são esperadas e devem ser reconhecidas e normalizadas”. Tipos, enumerados e mecanismos não mudam, e as duas v2 recebem as mesmas descrições dos 12 parâmetros; o texto que enquadra a recusa, porém, difere. Só o *Tool Calling* é instruído a buscar “mesmo que de forma vaga”, lê na descrição da busca que “uma consulta sem nenhum parâmetro reconhecível ainda é uma busca” e, na da ferramenta de recusa, que ela não serve para consultas vagas sobre o acervo; na Saída Estruturada, recusar é responder `fora_do_escopo` igual a `true` “e nada mais”, e a descrição do campo diz que “consultas vagas sobre o acervo não são fora do escopo”. A comparação entre as duas v2 mede, portanto, o mecanismo junto com esse texto, sobretudo na recusa.

Como a v2 foi desenhada a partir dos erros nas 310, esperava-se que o seu resultado nesse conjunto, de dentro da amostra, fosse otimista. Não foi: nas 310, o *Tool Calling* v2 acertou 47,4% e a Saída Estruturada v2, 68,0%, abaixo das v1 no mesmo conjunto (60,1% e 75,2%). No domínio, a diferença entre a v2 e a v1 foi, no *Tool Calling*, de -13,1 p.p. nas 310 e de +4,4 p.p. no lote e, na Saída Estruturada, de -10,1 e -12,2 p.p.: dentro da amostra, a v2 foi pior do que fora dela no *Tool Calling* e parecida na Saída Estruturada. Das consultas do domínio que o *Tool Calling* v1 acertava, o v2 errou 46,6% nas 310, contra 30,1% no lote, em boa parte porque os erros de forma que a v2 introduziu, como a sigla no lugar do nome da UF e o prefixo “MI” na *keyword*, atingem formas que se repetem na camada G, como “cartas de” seguido de uma UF e “folha MI” seguida do código e da UF. Os consertos dentro da amostra existem, mas são menores que as quebras: nas 310, o *Tool Calling* v2 acertou 25 consultas em que o v1 não buscava e 18 em que o v1 buscava com erro, mas errou 82 que o v1 acertava. A medida que vale continua sendo a do lote, que o desenho da v2 não viu (Seção 5.4).

Execução e medidas.

O Gemma 4 E4B foi executado na mesma T4 das rodadas completas, com uma repetição, nas quatro configurações (*Tool Calling* e Saída Estruturada, v1 e v2) sobre o lote e nas duas v2 sobre as 310. Além das métricas da Seção 3.7.5, o lote permite

medir a recusa como classificação (positivo: consulta fora do domínio; previsto positivo: não buscou), o comportamento na categoria E, a acurácia nas consultas com uma e com mais de uma leitura aceita e a acurácia por registro de linguagem. As comparações entre configurações usam o teste de McNemar exato e, para a diferença de acurácia e de F1, um *bootstrap* pareado com 2.000 reamostragens das consultas.

3.6 Especificação v3: Ferramentas Auxiliares e Validação com Retorno

Os resultados do lote de validação (Seção 5.4) mostraram que os erros do *Tool Calling* com a v2 são, em boa parte, de forma — sigla no lugar do nome da UF, prefixo no código, limite em plurais — e reversíveis por código, e que o *Tool Calling* recusa com precisão muito maior que a Saída Estruturada. A v3 pergunta o que o *Tool Calling* ganha quando se usa o que ele tem de próprio: chamar funções no meio da resposta e receber um retorno delas. Ela acrescenta à v2, em degraus, as peças abaixo, descritas em `docs/v3.md` no repositório.

Descrições corrigidas.

As descrições dos parâmetros são as da v2 com os efeitos colaterais medidos no lote 1 corrigidos: a sigla aparece só como entrada (“MT” → “Mato Grosso”), o limite no singular exige ordenação, o código vai sem prefixo e sem sufixo acrescentado, o município vai com o nome inteiro, e o tipo de produto só é preenchido com evidência na consulta (“na dúvida, não preencha”).

Ferramentas auxiliares.

Quatro funções determinísticas que o modelo pode chamar antes da busca: `identificar_nome`, que diz se um trecho da consulta é UF (inclusive sigla), município (pela lista do IBGE, com a UF), folha do acervo (pelo índice do BDGEx) ou região, e dá a forma canônica; `normalizar_codigo`, que devolve o código MI ou INOM sem prefixo e com hífen e diz se ele consta no acervo; `normalizar_escala`, que converte a menção à escala para o enumerado do SCN; e `resolver_periodo`, que resolve uma expressão de tempo pela tabela do manual, tirando a aritmética de datas do modelo. A elas se soma `pedir_esclarecimento`, que o *prompt* e a descrição da ferramenta reservam às consultas sem nenhum critério (o código não impõe a restrição), com um usuário simulado de resposta fixa: não tem outros detalhes, e a busca pode seguir com o que foi dito. Uma pergunta que o modelo escreve como texto recebe a mesma resposta, como receberia numa conversa, e uma resposta só em texto, sem pergunta nem chamada, recebe uma

vez o pedido “Responda chamando uma ferramenta”. Chamadas que o modelo escreve como texto, em vez de emití-las como *tool call* — na sintaxe de função, em JSON ou no formato nativo do Gemma —, são interpretadas como chamadas e contadas em todas as configurações v3, inclusive na de uma chamada; na v1 e na v2, a mesma resposta conta como não busca. Essa leitura é parte do sistema medido e teria de acompanhar a v3 numa implantação. Cada consulta tem no máximo seis chamadas ao modelo, três tentativas de busca e duas perguntas.

Retorno: validação e recusa contestada.

Antes de executar, a ferramenta de busca confere os parâmetros e devolve ao modelo, como resultado da chamada, os erros de forma (sigla no lugar do nome, prefixo no código, escala fora do enumerado, UF ou palavra genérica como *keyword*), que impedem a busca, e avisos de evidência: campo preenchido sem trecho correspondente na consulta (tipo de produto a partir de “cartas”, limite sem número, estado não citado, período que não corresponde à expressão pela tabela do manual) e campo ausente cuja evidência está na consulta (UF ou município citado, código, expressão de tempo). Depois de um aviso, repetir os mesmos parâmetros confirma a busca: o aviso orienta, não obriga. A recusa também tem retorno: se a consulta cita um critério forte do catálogo — tipo de produto, código, escala, CGEO, projeto, UF ou município —, a primeira recusa é devolvida ao modelo com a lista desses critérios e a lembrança de que a finalidade declarada pelo usuário (uma obra, um trabalho) não impede a busca; uma segunda recusa vale. A regra contestaria 12 das 336 consultas fora do domínio do lote 1 (“o que é um MDT”, “PIB da Bahia”), que o modelo precisa então confirmar.

Como o validador codifica as convenções do manual, ele foi testado contra as respostas corretas: aplicado à leitura preferencial do gabarito de todas as consultas de desenvolvimento, não pode reclamar de nenhuma. A primeira versão reclamava de 51 das 1593 respostas certas do lote 1; a versão congelada, de 0 no lote 1 e de 0 das 298 nas 310. A mesma codificação é a principal ressalva da v3: as ferramentas e o validador implementam as convenções que definem o gabarito — para datas, a mesma tabela que resolve as regras de tempo —, de modo que o resultado mede o conjunto do modelo com o código do domínio, e não só o modelo. Por isso a análise separa a primeira decisão — a primeira busca ou recusa, antes de qualquer retorno do validador — da resposta final. No *Tool Calling*, essa primeira decisão já vem depois das ferramentas auxiliares, que também são código do domínio: o que elas acrescentam aparece nos degraus, e o que o modelo faz só com a especificação é medido pelo degrau das descrições corrigidas.

Degraus e controle.

Cada configuração acrescenta uma peça à anterior: o *Tool Calling* v1 (a solução do Capítulo 4); a v2; a v3 só com as descrições corrigidas, ainda numa chamada; com as ferramentas auxiliares, o pedido de esclarecimento com usuário simulado e o pedido para responder chamando uma ferramenta, num laço de chamadas; e com o retorno, que é a v3 completa. Os degraus são cumulativos e medidos numa só ordem: cada diferença é o efeito da peça dadas as anteriores. O controle é a Saída Estruturada com as mesmas descrições v3, o mesmo validador e a mesma recusa contestada, aplicados como nova tentativa, do modo como o protótipo fazia com o Instructor. Ele não recebe as ferramentas auxiliares, o usuário simulado, o pedido para responder chamando uma ferramenta nem a leitura de chamadas escritas como texto, e o seu *prompt* não traz a instrução, dada só ao *Tool Calling*, de que nome de lugar desconhecido não é motivo de recusa. Se ele empatar com a v3, o ganho da v3 é das descrições e do validador, que os dois recebem; se ficar abaixo, a diferença mede o mecanismo de *Tool Calling* junto com as ferramentas auxiliares, que só ele chama no meio da resposta, e com as demais peças que só ele tem, sem separá-los. Um controle que desse à Saída Estruturada, antes da resposta, o que as ferramentas auxiliares devolvem separaria o acesso a esse código do mecanismo; ele não foi executado. As referências externas são a melhor configuração v1 ou v2 isolada no lote 2 e a arquitetura em duas etapas simulada na Seção 5.4 (o *Tool Calling* v2 decide se busca e a Saída Estruturada v1 extrai os parâmetros), calculada no lote 2 com as rodadas das duas configurações.

Desenvolvimento, teste e pré-registro.

As ferramentas, as descrições e o validador foram desenvolvidos só com as 310 consultas e o lote 1, numa amostra estratificada de 160 consultas do lote 1 executada no Gemma 4 E4B local (o mesmo arquivo de modelo da T4), em ciclos registrados no repositório (`docs/v3_desenvolvimento.md`). Como cada ciclo corrigia erros observados nessas mesmas consultas, os números de desenvolvimento são de dentro da amostra e só são citados como registro do processo, não como resultado. Uma variante com orientações escritas a partir dos erros de desenvolvimento — regras curtas em arquivos *markdown*, injetadas no *prompt* quando pertinentes à consulta, na linha das memórias de lições de agentes (ZHAO et al., 2024) — foi testada e descartada no desenvolvimento, antes do teste, por não ter efeito mensurável: com e sem elas, a v3 acertou o mesmo número de consultas, 155 das 160 (4 acertadas só com elas e 4 só sem elas), e a primeira decisão foi um pouco melhor sem elas. Retirá-las simplifica o desenho, mas não o livra do risco de sobreajuste: as regras do validador também foram escritas em ciclos sobre os erros dessas mesmas consultas, e é o lote 2 que mede quanto isso pesa.

O teste é um **lote 2**, construído com a mesma receita do lote 1 — outra semente e outros agentes redatores e anotadores, da mesma família de modelos —, com 945 consultas (163 fora do domínio e 65 subespecificadas; concordância na decisão com kappa de 1,000 e 99,6% de leituras compatíveis) e nenhuma delas lida no desenvolvimento (DWORK et al., 2015); os relatórios dos anotadores do lote 2 citaram trechos de algumas dezenas de consultas, e nada da v3 foi escrito ou alterado a partir deles. Uma checagem automática confirmou que nenhum exemplo das descrições, das ferramentas ou dos *prompts* coincide com uma consulta inteira do lote 2; o que coincide é vocabulário do domínio, como “100k” e “mais recente”. Como os dois lotes sortearam nomes e códigos das mesmas listas que as ferramentas consultam (a lista de municípios do IBGE e o índice do BDGEx) e exigem na consulta a superfície literal de cada valor, o teste não mede o alcance das ferramentas em nomes e códigos fora dessas listas. Antes da rodada de teste, a v3 foi congelada: o caderno do Colab grava o *hash* SHA-256 dos *prompts*, das ferramentas e do código e dos dados das ferramentas, e se recusa a rodar com outra versão; o plano de análise — a comparação principal com o controle, os degraus, as referências e a decomposição entre primeira decisão e resposta final — foi fixado e versionado no repositório (`docs/v3.md`) antes da rodada; qualquer outra análise do lote 2, como a leitura dos erros e a acurácia por categoria, é exploratória, foi feita depois do teste e não alterou a v3.

3.7 Protocolo de Avaliação

3.7.1 Visão Geral

Cada modelo executa todas as consultas do *dataset*, em ordem fixa, sob condições controladas e reproduzíveis. A execução é automatizada pelo comando `pfc-avaliar` (Capítulo 4), que grava um registro por consulta e repetição — consulta, gabarito resolvido, parâmetros emitidos pelo modelo, *tool calls* brutos, texto de resposta, latência, contagem de *tokens* e classe de erro — e um manifesto de reprodutibilidade por rodada.

3.7.2 Ambientes de Execução

Estação de referência.

A estação que define o cenário de implantação modesta tem a seguinte configuração (especificação completa no Apêndice B):

- **CPU:** Intel Core i7-12700H (12.^a geração, 14 núcleos / 20 *threads*, base 2,3 GHz);
- **RAM:** 32 GB DDR4 a 3200 MHz;
- **GPU dedicada:** NVIDIA GeForce RTX 3050 Laptop GPU com 4 GB de VRAM;

- **Disco:** SSD NVMe de 512 GB;
- **Sistema operacional:** Microsoft Windows 11 Pro (versão 10.0.26200, 64 bits);
- **Software:** Python 3.14.4; Ollama 0.34.0; LangChain 1.x (`langchain-core` 1.6.4, `langchain-ollama` 1.1.0, cliente `ollama` 0.6.2); PostgreSQL 16.4 com PostGIS 3.4 em contêiner Docker, usado apenas na demonstração ponta a ponta.

Com 4 GB de VRAM, nenhum dos modelos avaliados executa integralmente em GPU: mesmo o Qwen 3 4B, o menor deles, teve 66% do modelo carregado (pesos e contexto) alocado na VRAM, e o Gemma 4 E2B, 38%; o restante executa na CPU, com penalidade de latência (Seção 5.10).

Rodadas completas.

As rodadas completas dos quatro modelos locais, com três repetições por modelo, foram executadas numa GPU de nuvem NVIDIA T4 com 16 GB nominais de VRAM (15 GB utilizáveis, conforme o manifesto e a Tabela 11), suficiente para cada modelo inteiro, com Ollama (na versão registrada no manifesto de cada rodada), as *tags* e quantizações da Seção 3.2 (para o Qwen 3 4B e o Gemma 4 E2B, as mesmas da validação na estação de referência), o mesmo *prompt* de sistema e a mesma definição da ferramenta, executados pelo comando `pfc-avaliar`. Isso separa o efeito do modelo do efeito do *offload* e evita ocupar a estação de referência durante horas de execução. A qualidade da tradução depende essencialmente do modelo, e não do *hardware* — diferenças numéricas entre execução em GPU e em CPU podem alterar respostas isoladas, o que a validação na estação de referência quantifica —; a latência, ao contrário, depende fortemente do *hardware* e é reportada separadamente por ambiente.

Validação na estação de referência.

As 62 consultas das camadas P e N foram executadas também na estação de referência pelos dois modelos de menor porte (Qwen 3 4B e Gemma 4 E2B), com uma repetição, para medir a latência no cenário de implantação modesta e verificar a concordância das respostas com as rodadas completas. As duas rodadas foram feitas na mesma sessão, primeiro a do Qwen 3 4B; antes da rodada do Gemma 4 E2B, todos os modelos residentes no Ollama foram descarregados. Os dois modelos foram, assim, carregados a frio, com VRAM livre semelhante no início da rodada (3,0 GB e 3,1 GB). O manifesto registra a VRAM livre no início da rodada e a parcela do modelo que o Ollama alocou na GPU, porque a latência nessa estação depende diretamente desses dois valores.

Referência em nuvem.

Os três modelos do Groq (Seção 3.2) executaram as 310 consultas com uma repetição, respeitando os limites de taxa do provedor.

3.7.3 Configuração dos Modelos

Todos os modelos foram avaliados com a mesma configuração: **temperatura zero**; limite de **1024 tokens** de saída; **nenhum contexto de conversações anteriores**; **nenhum exemplo resolvido no prompt** (*zero-shot*) e **nenhum dicionário de normalização**; o mesmo *prompt* de sistema, com a data de referência injetada; e a mesma definição da ferramenta `buscar_catalogo` (as especificações v2 e v3, avaliadas só com o Gemma 4 E4B, mudam o *prompt* e as descrições; Seções 3.5 e 3.6). Nas configurações v1 e v2, não há nova tentativa nem mecanismo de *fallback*: cada falha do modelo é registrada e contada. A v3 acrescenta, por desenho, o retorno de um validador e, no controle com Saída Estruturada, novas tentativas; cada retorno fica registrado no traço da execução, e a análise separa a primeira decisão da resposta final.

Modo *thinking*.

O modo de raciocínio interno foi desativado por parâmetro (`think=false` na API do Ollama) nos modelos que o declaram, para comparabilidade das latências. Na *build* padrão do Qwen 3 (`qwen3:4b`) não foi possível desativá-lo de forma limpa — nem pela diretiva `/no_think` no *prompt* nem pelo parâmetro `think=false` —: nos testes preliminares, o modelo gerou centenas de *tokens* de raciocínio por consulta, com latências de dezenas de segundos, e, nas consultas em que esgotou o limite de 1024 *tokens* de saída, não chegou a emitir a chamada de ferramenta. Adotou-se, por isso, a *build* oficial sem raciocínio (*Qwen3-4B-Instruct-2507*), de mesmo porte e mesma quantização. Na referência em nuvem, o esforço de raciocínio foi fixado no mínimo aceito por cada modelo (Groq, 2026) — `none` para o `qwen/qwen3.8-27b` e `low` para o `openai/gpt-oss-20b` e o `openai/gpt-oss-120b`; o valor `none` desliga o raciocínio, e a família GPT-OSS não permite desligá-lo —, o que é registrado no manifesto.

3.7.4 Linha de Base: Saída Estruturada

Para medir o que a troca da Saída Estruturada pelo *Tool Calling* rendeu, os modelos foram avaliados também com a abordagem do protótipo, sobre as mesmas consultas e com as mesmas métricas. Nessa linha de base, a chamada de ferramenta é substituída por um pedido de resposta estruturada: o *prompt* de sistema é o mesmo, sem as duas instruções que tratam de chamar ou não a ferramenta, e passa a pedir um objeto JSON

no formato do *schema* dos parâmetros, incluído no próprio *prompt* com as mesmas descrições e os mesmos enumerados da ferramenta; o Ollama é usado no modo JSON. A temperatura, o limite de *tokens*, a data de referência e a ausência de exemplos e de dicionário são os da configuração avaliada. Duas diferenças decorrem da própria abordagem: o modelo não tem como recusar — toda consulta resulta em busca, como no protótipo —, e os enumerados não são impostos por uma ferramenta, de modo que uma grafia fora da forma canônica conta como erro. Valores vazios que o modelo emite para campos ausentes (texto vazio, limite zero), e que a aplicação ignoraria, são tratados como ausentes, o que favorece a linha de base. A decodificação restrita ao *schema*, que o Ollama também oferece, foi testada e descartada: com ela, os modelos fechavam os intervalos de datas logo depois do primeiro limite, um artefato da gramática que penalizaria a linha de base.

Método do protótipo.

Uma segunda linha de base reproduz a solução anterior tal como foi entregue, portada para Python a partir do código do protótipo: o modelo Phi-4 14B; o dicionário de normalização aplicado à consulta antes do modelo, com as suas 159 entradas; o *prompt* original, em inglês, com dez exemplos resolvidos; o *schema* com o campo obrigatório de justificativa e os enumerados do protótipo, validado a cada resposta, com até três novas tentativas quando a validação falha; e o *fallback* por expressões regulares. Três diferenças são deliberadas: a temperatura é zero, e não 0,6; os valores padrão de ordenação que o protótipo acrescenta a toda busca não contam como parâmetros extraídos; e a validação de estados e municípios contra o banco não é feita. A saída é traduzida para o vocabulário do PFC pela mesma correspondência usada para alinhar o gabarito da camada P (“SCN Carta Topografica Matricial” para “SCN Carta Topográfica Matricial”, por exemplo).

Execução.

As linhas de base foram executadas com uma repetição: as dos modelos locais e o método do protótipo, na mesma T4 das rodadas completas, pelo caderno do Colab do repositório; a Saída Estruturada dos modelos em nuvem, no Groq, com a data de referência e o esforço de raciocínio das rodadas com *Tool Calling* de cada modelo. Os dois modelos de menor porte foram comparados também na estação de referência, sobre as camadas P e N.

3.7.5 Métricas

Comparação.

A resposta do modelo é comparada com o gabarito campo a campo (Capítulo 2): enumerados exigem igualdade exata; *strings* livres (**keyword**, **state**, **city**) são compa-

radas após normalização de caixa, acentos e pontuação; períodos exigem coincidência exata dos limites; `limit` é inteiro. Quando o gabarito tem leituras múltiplas, a resposta é comparada com cada leitura aceita e avaliada contra a de melhor casamento: correta se coincidir com alguma; por campo, contra a leitura que maximiza acertos menos erros. Campos opcionais ausentes não contam como erro.

Acurácia por consulta.

Uma consulta é correta se o conjunto de parâmetros emitidos coincide com uma leitura aceita — nem campos a mais, nem a menos. Nas consultas cujo gabarito é não chamar a ferramenta, correta é a resposta sem *tool call*. A acurácia é a proporção de execuções corretas e é reportada com intervalo de confiança de 95% pelo método de Wilson (WILSON, 1927), adequado a proporções com amostras moderadas. Como as repetições de uma mesma consulta não são observações independentes — numa mesma sessão, elas quase sempre coincidem —, o intervalo usa como tamanho da amostra o número de consultas, e não o de execuções.

Precisão, *recall* e F1 por campo.

Para cada campo: verdadeiro positivo (TP) quando o campo é emitido e corresponde a um valor aceito; falso positivo (FP) quando é emitido e não corresponde, ou não deveria ter sido emitido; falso negativo (FN) quando é obrigatório no gabarito e não foi emitido. Um valor errado conta, portanto, como FP. O F1 médio é ponderado pelo número de ocorrências de cada campo no gabarito; as médias *macro* e *micro* ficam registradas no relatório de cada rodada, no repositório.

Latência.

Tempo da chamada de tradução: do envio da consulta ao modelo até o recebimento da resposta com os *tool calls*, medido com relógio monotônico. Na v3 e no seu controle, que podem fazer mais de uma chamada, vai do envio da consulta à decisão final e inclui todas as chamadas ao modelo e a execução das ferramentas auxiliares e do validador; na arquitetura em duas etapas simulada, é a soma das duas chamadas quando há busca e só a da decisão quando não há. O tempo de execução da ferramenta (SQL) é registrado à parte e não entra na comparação entre modelos. Reportam-se mínimo, mediana, média, percentil 95 e máximo, por ambiente de execução.

Comparação entre modelos.

Diferenças de acurácia entre dois modelos, sobre as mesmas consultas, são testadas pelo teste de McNemar exato (MCNEMAR, 1947), adequado para comparar dois classifi-

cadres avaliados sobre as mesmas instâncias (DIETTERICH, 1998); o teste usa apenas os pares discordantes (consultas em que só um dos modelos acerta). Pela mesma razão, o teste usa uma observação por consulta: o resultado de cada consulta é o da maioria das suas repetições.

Análise de erros.

Cada resposta incorreta é classificada em: campo inventado (emitido sem estar no gabarito), campo omitido, valor errado, erro misto, não chamou a ferramenta quando devia, chamou quando não devia e argumentos fora do *schema* (tipo inválido ou campo inexistente). Erros de infraestrutura (*timeout*, indisponibilidade) são separados dos erros do modelo e não entram nas estatísticas de latência. Como discutido no Capítulo 2, **não se busca um ranking único**: cada métrica revela um aspecto da tradução, e a recomendação final pondera qualidade e latência.

3.7.6 Reprodutibilidade

Cada rodada registra em manifesto: *tag* e *digest* do modelo no Ollama, versões do Ollama e do LangChain, configuração de inferência, data de referência, *hardware* e os *hashes* SHA-256 do *dataset* e da definição da ferramenta. A pontuação é sempre recalculada contra o gabarito vigente: uma correção de gabarito não exige nova execução dos modelos, e uma consulta cujo texto tenha mudado é descartada da pontuação, com registro. O *dataset*, o gerador, o manual, os relatórios de auditoria, os registros de execução e os *scripts* estão no repositório do projeto (Apêndice C).

3.8 Uso de Ferramentas de Inteligência Artificial

Os modelos de linguagem avaliados (Seção 3.2) são o objeto do estudo. Além deles, ferramentas de inteligência artificial foram empregadas como apoio nos processos metodológicos do trabalho, listados na Tabela 10 com a ferramenta, a finalidade e a forma de controle do resultado. Em nenhum desses processos a ferramenta definiu o gabarito: nas 310 consultas, ele foi definido pelos autores segundo o manual de anotação, e a anotação por modelo serviu para medir a concordância e apontar divergências, adjudicadas pelos autores; nos lotes, o gabarito foi fixado por construção, antes da redação, a partir de catálogos reais sorteados por código. Os agentes que redigiram e anotaram as consultas dos lotes são de uma família de modelos que não integra os modelos avaliados, e essa dependência é uma limitação registrada (Seção 3.5). Todos os números de resultado vêm de macros geradas por código a partir dos registros das rodadas.

Tabela 10 – Ferramentas de inteligência artificial empregadas como apoio nos processos metodológicos

Processo	Ferramenta	Finalidade	Controle do resultado
Desenvolvimento do <i>software</i> (<i>backend</i> , <i>pipeline</i> de avaliação, especificações v2 e v3)	Claude Code, com o modelo Claude Opus 5.5 (Anthropic)	escrita e depuração de código e de testes, a partir da especificação dos autores	testes automatizados, revisão pelos autores, manifestos de cada rodada e repositório público
Anotação independente das 310 consultas (Seção 3.4)	Claude Opus 5.5	segunda anotação, às cegas, só com o manual e a definição da ferramenta	concordância medida contra o gabarito; divergências adjudicadas pelos autores; revisão humana de uma amostra
Redação e anotação das consultas dos lotes de validação (Seção 3.5)	agentes com Claude Opus 5.5	redigir consultas a partir de alvos com gabarito fixado antes; anotá-las às cegas	checagens automáticas, anotação às cegas com filtro de compatibilidade e segunda anotação de uma amostra
Auditoria e análise dos erros das rodadas	agentes com Claude Opus 5.5	classificar erros e propor causas, com verificação por agentes independentes	todos os números recalculados por código; causas não testadas registradas como hipóteses

Fonte: Elaborado pelos autores.

4 APRIMORAMENTO DO PROTÓTIPO: ESTRUTURA ANALÍTICA DO PROJETO

Este capítulo apresenta a execução do projeto de aprimoramento da solução de busca em linguagem natural sobre o acervo cartográfico da DSG. Como se trata de um projeto de engenharia sobre um sistema existente, o capítulo é organizado segundo uma **Estrutura Analítica do Projeto (EAP)**, que decompõe o aprimoramento em quatro grandes etapas, sequenciais mas com pontos de realimentação: (1) *Análise do Protótipo*; (2) *Elicitação de Requisitos*; (3) *Planejamento da Migração*; e (4) *Desenvolvimento*. A EAP é apresentada graficamente na Figura 3 e norteia tanto a redação deste capítulo quanto o cronograma de execução, apresentado ao final.

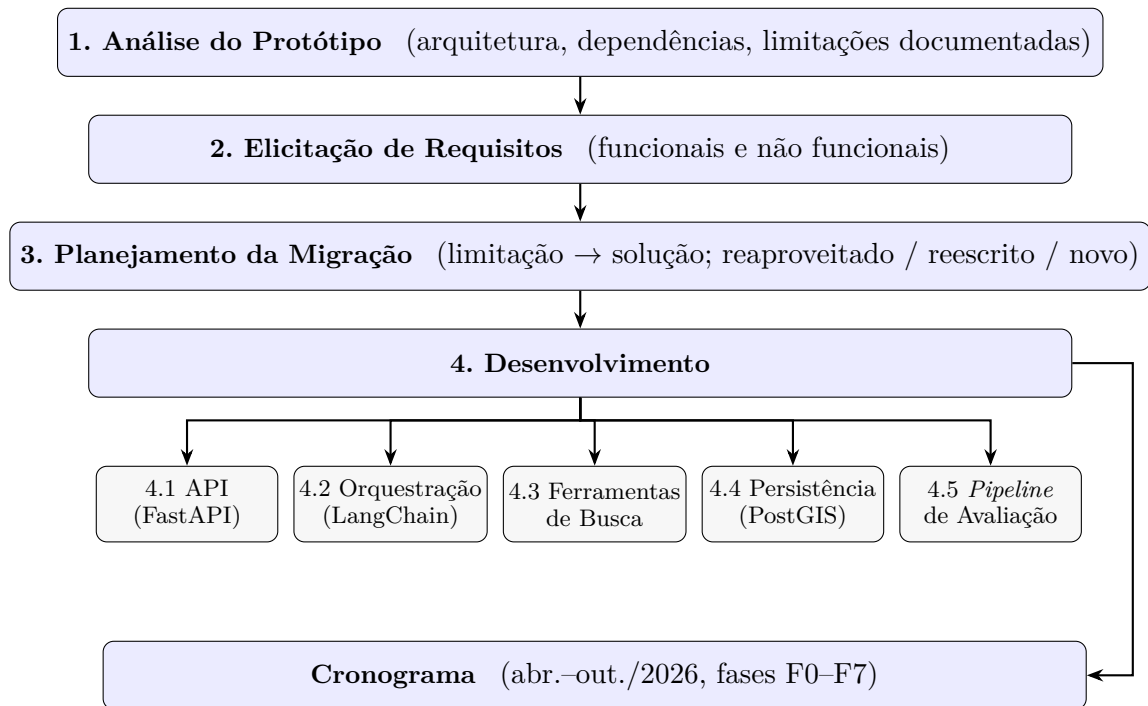


Figura 3 – Estrutura Analítica do Projeto (EAP) adotada no aprimoramento

Fonte: Elaborado pelos autores.

4.1 Análise do Protótipo do 1º CGEO

4.1.1 Visão Geral

O 1º Centro de Geoinformação (1º CGEO), por meio de sua Divisão de Levantamento “Gen Augusto Tasso Fragoso”, desenvolveu um protótipo funcional de busca em linguagem natural sobre o catálogo de dados geoespaciais sob sua guarda (1º Centro de

Geoinformação, 2025). O protótipo foi concebido como prova de conceito para avaliar a viabilidade do uso de LLMs locais no contexto cartográfico brasileiro e está disponível publicamente no repositório GitHub do 1º CGEO. Este protótipo é o ponto de partida e a referência de projeto do presente trabalho; seus 22 casos de teste compõem a camada P do *dataset* de avaliação.

4.1.2 Arquitetura do Protótipo

A arquitetura do protótipo é composta por quatro camadas principais, conforme ilustrado na Figura 4.

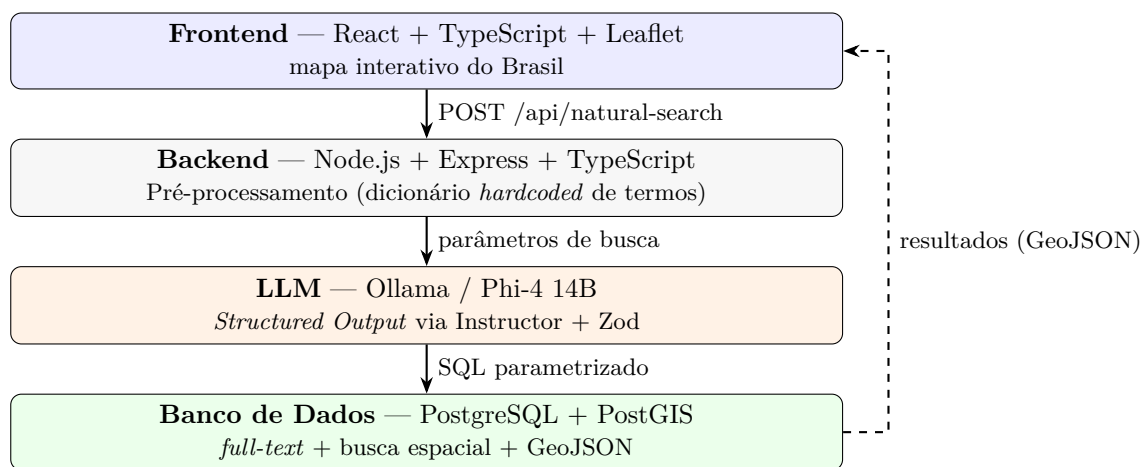


Figura 4 – Arquitetura do protótipo de busca do 1º CGEO

Fonte: Adaptado de 1º Centro de Geoinformação (2025).

Frontend. Implementado em React com TypeScript, utiliza a biblioteca Leaflet para exibição de um mapa interativo do Brasil. O usuário digita sua consulta em linguagem natural em um campo de texto, e os resultados são exibidos em lista paginada e como polígonos no mapa (a geometria GeoJSON de cada folha).

Backend. Implementado em Node.js com Express e TypeScript, recebe as consultas via rota REST (`POST /api/natural-search`), aplica pré-processamento de termos e encaminha a consulta ao serviço de LLM, cujo resultado é convertido em SQL parametrizado para o banco PostgreSQL/PostGIS.

Pré-processamento. Antes de enviar a consulta ao LLM, o protótipo aplica um módulo de pré-processamento (`preprocessor.ts`) que normaliza variações ortográficas comuns no domínio cartográfico. O módulo utiliza um dicionário *hardcoded* de sinônimos e variações, com mapeamentos como “25k” → “1:25.000” ou “primeiro cgeo” → “1º Centro de Geoinformação”.

Serviço de LLM com *Structured Output*. O núcleo do protótipo é o serviço de LLM (`service.ts`), que utiliza o modelo *Phi-4 14B* (ABDIN et al., 2024), executado

localmente via Ollama. A integração é feita pela biblioteca Instructor para TypeScript, que usa Zod para definir o *schema* de saída. O *schema* prevê um campo obrigatório `reasoning`, no qual o modelo justifica a extração de cada parâmetro, seguido dos doze campos de busca: `keyword`, `scale`, `productType`, `state`, `city`, `supplyArea`, `project`, `publicationPeriod`, `creationPeriod`, `sortField`, `sortDirection` e `limit`. O *prompt* do sistema fornece ao modelo exemplos detalhados (*few-shot prompting*) para guiar a extração.

Fallback com expressões regulares. Em caso de falha do LLM (exceção após `max_retries = 3`), o protótipo aciona um mecanismo de *fallback* baseado em expressões regulares (`utils.ts`), que tenta extrair diretamente do texto escala, tipo de produto, área de suprimento, projeto, ordenação, limite e períodos de publicação ou de criação.

Banco de dados. O banco utiliza PostgreSQL com PostGIS para armazenar os metadados dos produtos cartográficos e suas geometrias (*bounding boxes* das folhas). As consultas, montadas no serviço `search.ts` e executadas pela biblioteca `pg-promise`, combinam pesquisa textual (*full-text search*) com operações espaciais do PostGIS para retornar resultados em GeoJSON. O repositório do protótipo contém o esquema do banco (`postgres/init.sql`, com as tabelas `datasets`, `estados`, `municipios` e `areas_suprimento`), mas não os dados do acervo.

4.1.3 Contribuições do Protótipo

O protótipo representa contribuição pioneira no contexto brasileiro, demonstrando que: (i) a abordagem baseada em LLMs locais é tecnicamente viável para o domínio cartográfico; (ii) o modelo Phi-4 14B, via Ollama, é capaz de extrair parâmetros de busca relevantes a partir de consultas em linguagem natural em português; (iii) a combinação de *few-shot prompting* com *Structured Output* produz resultados utilizáveis para a maioria das consultas; e (iv) a integração de LLM com PostgreSQL/PostGIS para busca espacial é viável em ambiente controlado.

4.1.4 Limitações Identificadas

O próprio cliente identificou explicitamente seis limitações do protótipo, que constituem as frentes de aprimoramento deste Projeto Final de Curso:

1. **Backend em Node.js.** O ecossistema Node.js tem suporte consideravelmente menor do que Python para bibliotecas de IA e aprendizado de máquina; as bibliotecas de integração com LLMs (LangChain, LlamaIndex, Instructor) têm versões mais completas em Python.

2. **Apenas Saída Estruturada.** A abordagem força o modelo a sempre retornar a estrutura predefinida, independentemente da consulta, e não permite encadear ferramentas nem adaptar o comportamento ao resultado de operações anteriores.
3. **Modelo único e defasado (Phi-4 14B).** O uso de um único modelo impede a seleção embasada do mais adequado ao caso de uso; o ecossistema evolui rapidamente, e a escolha precisa ser revista à luz dos modelos mais recentes.
4. **Pré-processamento manual com dicionário *hardcoded*.** O protótipo mantém um dicionário fixo de 159 pares (“25k” → “1:25.000”, “primeiro cgeo” → “1º Centro de Geoinformação”, “carta ortoimg” → “SCN Carta Ortoimagem”) que é aplicado **antes** de o texto chegar ao LLM. Isso traz três problemas: (i) o dicionário só cobre variações *previstas em tempo de desenvolvimento* — grafias novas que emergem no uso operacional (“dois cgeo” em vez de “segundo cgeo”, “25 mil” em vez de “25k”) falham silenciosamente; (ii) sua manutenção é **manual** e depende do desenvolvedor observar erros no *log* e propor novas entradas; (iii) o pré-processamento pode **deturpar** consultas legítimas quando um termo do dicionário aparece em contexto ambíguo. A delegação ao LLM, adotada na solução v1, elimina o dicionário fixo em troca do conhecimento semântico do modelo, que reconhece variações inéditas sem alteração de código.
5. **Fallback com expressões regulares.** Sua presença é sintoma de falhas do LLM que não foram sistematicamente documentadas, dificultando a análise de causa raiz.
6. **Ausência de métricas de avaliação.** O protótipo dispõe apenas de 22 casos de teste, pontuados por pesos atribuídos *ad hoc* aos campos esperados, sem penalizar campos preenchidos indevidamente, e não há protocolo de medição, o que impede comparar objetivamente modelos, *prompts* ou configurações.

4.2 Elicitação de Requisitos

A análise do protótipo e as conversas com o cliente conduziram ao seguinte conjunto de requisitos para a solução aprimorada.

4.2.1 Requisitos Funcionais

- RF1** Receber, por uma rota REST, uma consulta em linguagem natural em português.
- RF2** Traduzir a consulta em parâmetros técnicos compatíveis com os campos do catálogo (`keyword`, `scale`, `productType`, `state`, `city`, `supplyArea`, `project`, `publicationPeriod`, `creationPeriod`, `sortField`, `sortDirection`, `limit`).

- RF3** Executar a busca técnica resultante no banco PostgreSQL/PostGIS, preservando a lógica de filtros e a saída em GeoJSON do protótipo.
- RF4** Permitir a seleção, em tempo de requisição e em ambiente de desenvolvimento, do modelo de LLM utilizado para a tradução, para comparação interativa entre modelos.
- RF5** Registrar, para cada requisição, os parâmetros extraídos, a sequência de *tool calls*, os tempos de cada etapa e eventuais erros, em formato adequado ao cálculo automatizado das métricas.

4.2.2 Requisitos Não Funcionais

- RNF1 Execução totalmente local:** nenhuma chamada a serviços de LLM em nuvem (privacidade e segurança).
- RNF2 Latência aceitável:** a latência por consulta deve ser compatível com uso interativo; ela é medida e reportada por modelo, de modo que o limite operacional possa ser fixado pelo cliente conforme o contexto de implantação.
- RNF3 Conformidade com normas:** aderência ao Perfil de Metadados Geoespaciais do Brasil (Perfil MGB) e às ET-PCDG no que diz respeito à estrutura dos metadados consultados.
- RNF4 Reprodutibilidade:** *tags* dos modelos, versões das bibliotecas e *hashes* do *dataset* e da definição da ferramenta registrados em cada rodada de avaliação.
- RNF5 Manutenibilidade:** código modular, com testes automatizados das funções de busca e do cálculo das métricas.
- RNF6 Substituibilidade de modelo:** troca do modelo sem alteração da camada de orquestração, apenas por configuração.

4.3 Planejamento da Migração

A migração não parte do zero: o protótipo fornece decisões de projeto já validadas (estrutura do banco, catálogo de parâmetros, lógica de *queries* espaciais) que são aproveitadas — o esquema do banco sem alterações; o catálogo de parâmetros e a lógica das consultas com os ajustes indicados no Quadro 3. A estratégia adotada é, portanto, de **substituição dirigida**: identificar os componentes responsáveis pelas limitações documentadas e substituí-los, preservando o que já funciona.

4.3.1 Mapeamento Limitação → Solução

O Quadro 2 relaciona cada limitação identificada na Seção 4.1.4 ao componente do novo sistema que a endereça.

Quadro 2 – Mapeamento entre limitações do protótipo e soluções adotadas

#	Limitação	Solução adotada	Componente afetado
1	<i>Backend</i> em Node.js	Reimplementação em Python com FastAPI	Nova API REST
2	Apenas <i>Structured Output</i>	<i>Tool Calling</i> nativo via LangChain	Serviço de LLM
3	Modelo único (Phi-4 14B)	Avaliação comparativa de quatro modelos atuais (Qwen 3 4B, Gemma 4 E2B, Gemma 4 E4B, Mistral Nemo 12B)	Configuração e testes
4	Pré-processamento manual	Normalização delegada ao LLM, orientada pelas descrições do <i>schema</i> da ferramenta	Remoção do pré-processador
5	<i>Fallback</i> via regex	Falhas registradas como métrica, sem <i>fallback</i> oculto	Serviço de LLM
6	Ausência de métricas	<i>Dataset</i> auditado com gabarito + <i>pipeline</i> de avaliação automatizado	Novo módulo de avaliação

Fonte: Elaborado pelos autores.

Nota: Soluções da v1, descrita neste capítulo. Nas linhas 4 e 5, a v3 (Seção 3.6), que é a abordagem padrão da API (Seção 4.4.6), acrescenta ferramentas auxiliares determinísticas, que o modelo chama quando precisa, e o retorno de um validador, com cada intervenção registrada no traço da execução.

4.3.2 Componentes Reaproveitados, Reescritos e Novos

O Quadro 3 classifica cada componente do sistema final em relação ao protótipo.

4.4 Desenvolvimento

A arquitetura da solução aprimorada é apresentada na Figura 5, com as camadas detalhadas nas subseções seguintes.

4.4.1 Camada de API: FastAPI

O FastAPI (RAMÍREZ, 2018) é o *framework* da API REST, que expõe duas rotas. A rota POST `/api/search` recebe um objeto JSON com o campo `query` (consulta

Quadro 3 – Classificação dos componentes do sistema final em relação ao protótipo

Componente	Categoria	Observação
Esquema do banco PostgreSQL/PostGIS	Reaproveitado	Mesmo esquema do protótipo (<code>init.sql</code> , quatro tabelas), sem alterações
Conjunto de parâmetros de busca	Reaproveitado	Os mesmos doze campos, compatíveis com a lógica das consultas SQL; enumerados recortados (9 tipos de produto e 9 projetos, contra 30 e 15 no protótipo) e grafados conforme as ET-PCDG
Lógica das consultas SQL	Reaproveitado	Portada de <code>search.ts</code> ; aceita períodos com um só limite; sem o filtro por retângulo do mapa e a paginação, próprios do <i>frontend</i>
<i>Frontend</i> React/Leaflet	Fora do escopo	Não é parte deste trabalho
API REST	Reescrito	Node.js/Express → Python/FastAPI
Serviço de LLM	Reescrito	<i>Structured Output</i> (Instructor/Zod) → <i>Tool Calling</i> (LangChain)
Camada de acesso ao banco	Reescrito	<code>pg-promise</code> → <code>psycopg</code>
Pré-processamento de termos	Eliminado	Função transferida ao LLM
<i>Fallback</i> com regex	Eliminado	Falhas viram métricas
Semente sintética do banco	Novo	Apenas para a demonstração ponta a ponta; nenhuma métrica depende dela
<i>Dataset</i> de avaliação	Novo	310 consultas com gabarito auditado (Cap. 3 e Ap. A)
<i>Pipeline</i> de avaliação	Novo	Comandos <code>pfc-dataset</code> , <code>pfc-auditar</code> , <code>pfc-avaliar</code> e <code>pfc-relatorio</code>
Ferramentas auxiliares e validador (v3)	Novo	<code>ferramentas.py</code> e <code>v3.py</code> (Seção 3.6); no pacote, no avaliador e na API, como abordagem padrão
Registro de abordagens, comando <code>pfc</code> e integração contínua	Novo	<code>abordagens.py</code> , <code>cli.py</code> e <code>.github/workflows/testes.yml</code> , com o guia <code>docs/como_estender.md</code> (Seção 4.4.6)

Fonte: Elaborado pelos autores.

em linguagem natural) e devolve os parâmetros técnicos extraídos pelo LLM, a sequência de *tool calls*, os produtos cartográficos encontrados (total e itens, com geometria em GeoJSON), a resposta textual ao usuário, as latências de cada etapa (tradução, execução da SQL, resposta final e total) e eventuais erros e avisos, atendendo a RF1, RF2 e RF3 e expondo os traços de RF5, cujo registro para o cálculo das métricas é feito pelo *pipeline* de avaliação (Seção 4.4.5). A rota `GET /api/health` informa a versão do servidor Ollama, a disponibilidade do banco, o modelo configurado e a abordagem no ar (Seção 4.4.6). A

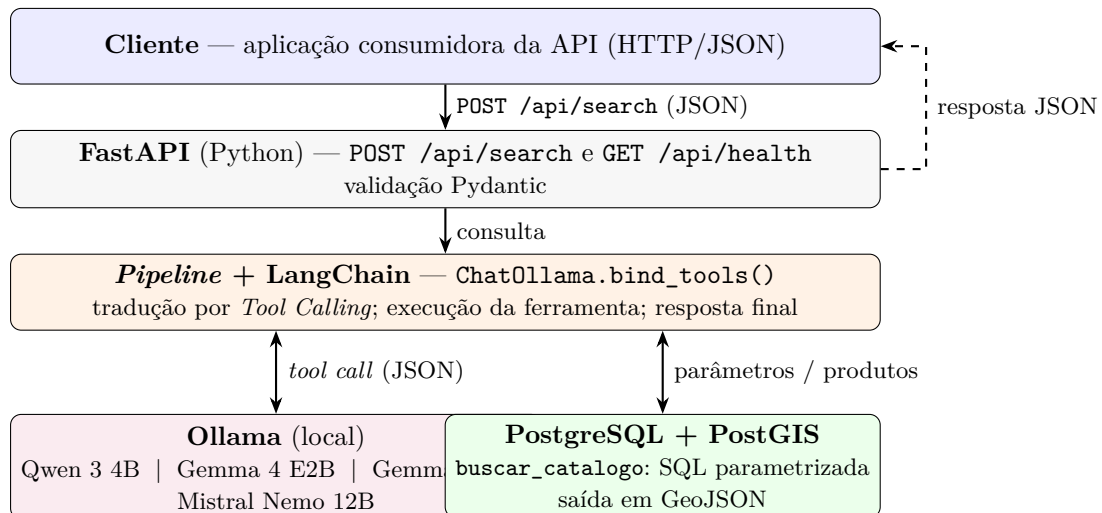


Figura 5 – Arquitetura da solução aprimorada de busca

Fonte: Elaborado pelos autores.

escolha do FastAPI justifica-se pela validação automática das requisições via Pydantic, pela documentação OpenAPI (Swagger) gerada automaticamente e pela integração direta com o ecossistema Python de IA. A camada de API é mantida minimalista; toda a lógica de orquestração reside no módulo de *pipeline*. O *pipeline* de avaliação não passa pela API: invoca diretamente a camada de tradução, de modo que a latência medida corresponde à chamada ao modelo, sem o custo da camada HTTP da própria API (a chamada ao servidor do Ollama permanece incluída).

Seleção de modelo.

O serviço aceita um campo opcional `model` na requisição (RF4), habilitado apenas no ambiente de desenvolvimento por variável de ambiente; fora dele, a requisição que solicita um modelo diferente do configurado é recusada. Na configuração operacional, o modelo é fixado por variável de ambiente no servidor: alternar o modelo entre requisições implicaria desalocar e recarregar pesos na VRAM, com custo de segundos por chamada e problemas de concorrência em cenários multiusuário.

Tratamento de falhas.

Na solução v1, falhas do LLM não são recuperadas por regras determinísticas nem por novas tentativas: cada falha é **registrada como métrica**, o que permite atribuir corretamente os erros ao modelo (limitação 5 da Seção 4.1.4). Na v3 (Seção 3.6), um validador devolve ao modelo os erros dos parâmetros e, em parte dos casos, o valor a usar; cada retorno fica no traço da execução, e a análise separa a primeira decisão da resposta final. As falhas são classificadas em duas famílias: **erros de infraestrutura**

(tempo esgotado, servidor ou modelo indisponível, falha de execução) e **erros do modelo** (argumentos fora do *schema*, chamada a ferramenta inexistente). Uma chamada com argumentos fora do tipo esperado é preservada como veio, e a comparação campo a campo com o gabarito contabiliza o erro. Para uma versão de produção destinada aos usuários finais, é desejável um caminho de *fallback* suave (por exemplo, delegar a consulta bruta à busca textual do PostgreSQL), preservando comportamento útil diante de falhas; essa evolução é apontada como trabalho futuro no Capítulo 6.

Consultas sem resposta no acervo.

O sistema distingue três desfechos, e em nenhum deles um produto pode ser inventado. Quando a consulta está fora do domínio do catálogo (“cartas de Marte”, “qual a previsão do tempo para amanhã em Manaus?”), o modelo não chama a ferramenta e responde, em uma frase, por que o pedido está fora do escopo; nenhuma busca é executada, e esse comportamento é medido pela categoria F do *dataset*. Quando a busca encontra produtos, a lista devolvida vem sempre do banco, e o modelo apenas a resume. Quando nenhum produto atende aos filtros, a API devolve a lista vazia e uma mensagem fixa com os critérios usados, sugerindo retirar ou ampliar algum deles, sem nova chamada ao modelo. Pedidos com valores que a ferramenta não representa (“escala 1:10.000.000”, “cartas do estado 42”) são tratados no *dataset* como casos observacionais, e o comportamento de cada modelo diante deles é relatado no Capítulo 5.

4.4.2 Camada de Orquestração: LangChain + Ollama

A integração com o Ollama é feita pela classe `ChatOllama` do LangChain (CHASE, 2022), que encapsula a API nativa do Ollama: envia a consulta ao modelo junto com a definição da ferramenta e devolve, de forma estruturada, as chamadas de ferramenta que o modelo decidiu gerar — desativando, por parâmetro (`reasoning=False`, que envia `think=false` à API do Ollama), o modo *thinking* nos modelos que declaram essa capacidade. O objeto `ChatOllama` é instanciado com o nome do modelo, temperatura zero, limite de 1024 *tokens* de saída e o *base URL* do Ollama; a ligação da ferramenta é feita pelo método `bind_tools()`, que recebe a definição de `buscar_catalogo` em JSON Schema; na v1, a invocação é uma única chamada `invoke()`, cujo retorno traz a lista de *tool calls*, e, na v3 (Seção 3.6), o módulo `v3.py` repete a invocação num laço explícito, com no máximo seis chamadas ao modelo e três tentativas de busca por consulta. Na versão 1.x do LangChain, adotada neste trabalho, o antigo `AgentExecutor` foi descontinuado; como a etapa avaliada na v1 é exatamente essa única decisão do modelo (chamar ou não a ferramenta, e com quais argumentos), o ciclo completo — executar a ferramenta e devolver o resultado ao modelo para a resposta final — é implementado explicitamente no módulo de *pipeline*, sem necessidade de um grafo de estados como o LangGraph (LangChain AI, 2024).

No caso do Qwen 3 4B, empregou-se a *build* oficial sem raciocínio, de mesmo porte e mesma quantização (*tag qwen3:4b-instruct-2507-q4_K_M*), porque na *build qwen3:4b* o raciocínio não pôde ser desligado nem pela diretiva */no_think* nem pelo parâmetro *think=false* (Capítulo 3, Seção 3.7.3).

A mesma camada admite outro provedor sem alteração do restante do código: substituindo *ChatOllama* por *ChatOpenAI*, apontado para um *endpoint* compatível com a API da OpenAI, o *bind_tools()* e a interpretação das *tool calls* permanecem idênticos. Essa portabilidade, discutida no Capítulo 2, foi empregada para obter o resultado paralelo com modelos maiores servidos em nuvem pelo Groq (Seção 4.4.5), sem qualquer efeito sobre a configuração local avaliada.

4.4.3 Ferramentas de Busca

A solução v1 expõe ao LLM uma única ferramenta, *buscar_catalogo*, em linha direta com a funcionalidade do protótipo. Ela é descrita por um *schema* JSON explícito — nome, descrição geral e os doze parâmetros de busca (*keyword*, *scale*, *productType*, *state*, *city*, *supplyArea*, *project*, *publicationPeriod*, *creationPeriod*, *sortField*, *sortDirection*, *limit*), cada um com tipo e, quando aplicável, enumeração dos valores válidos; dez deles (todos exceto *sortField* e *sortDirection*) trazem na própria descrição instruções de preenchimento e normalização (por exemplo, “25k” corresponde a “1:25.000”) — mantido em um único módulo do repositório (*schema.py*). O mesmo módulo é a fonte dos campos e tipos usados na validação dos argumentos devolvidos pelo modelo e dos enumerados usados na validação do *dataset*. Manter o *schema* explícito, em vez de derivá-lo de *docstrings*, garante que o texto lido pelo modelo é exatamente o que está versionado no repositório; seu *hash* é registrado no manifesto de cada rodada de avaliação.

As descrições dos parâmetros são o único lugar em que o modelo recebe exemplos de normalização: o *prompt* de sistema apenas instrui a converter cada valor para a forma canônica descrita na ferramenta, sem dicionário de termos nem exemplos resolvidos (*few-shot*). As 159 entradas do dicionário do protótipo são, assim, substituídas pela capacidade semântica do modelo (limitação 4). A v3 (Seção 3.6) devolve parte dessa normalização ao código, de outra forma: em vez de reescrever a consulta antes do modelo com pares previstos à mão, oferece ao modelo ferramentas que ele consulta quando precisa, derivadas do manual e de catálogos (a lista de municípios do IBGE e o índice do BDGEx), e um validador que confere a saída e cujo retorno fica registrado; o custo de manter esse código junto com o catálogo é discutido no Capítulo 6. A função Python que implementa a ferramenta recebe os argumentos decididos pelo modelo e executa a consulta parametrizada descrita na Seção 4.4.4.

Ferramentas auxiliares.

Ferramentas como `validar_inom` e `validar_mi` (verificação sintática de códigos INOM e MI antes da consulta ao banco) e `listar_tipos_produto` (retorno dos valores válidos de `productType`, útil quando o modelo precisa deduzir o tipo a partir de um sinônimo informal) não integram a solução v1, cujo objeto é a decisão única do modelo sobre `buscar_catalogo`. Ferramentas desse tipo — `identificar_nome`, `normalizar_codigo`, `normalizar_escala` e `resolver_periodo` — foram acrescentadas na v3 (Seção 3.6), a partir dos erros das 310 consultas e do lote 1, e o seu efeito é medido na Seção 5.5. A v3 está implementada no pacote e no avaliador (`src/pfc_busca/v3.py` e `ferramentas.py`) e integrada à API descrita neste capítulo, em que é a abordagem padrão; a solução v1 continua disponível por configuração (Seção 4.4.6).

4.4.4 Camada de Persistência

O esquema do banco é o do protótipo: PostgreSQL com PostGIS (PostGIS Development Team, 2001), com as quatro tabelas definidas em `postgres/init.sql` — `datasets` (metadados e geometria de cada produto, com um vetor de busca textual gerado a partir dos campos descritivos), `estados`, `municipios` e `areas_suprimento` —, reaproveitado sem alterações. Como o repositório do protótipo contém apenas esse esquema, sem os dados do acervo, a demonstração ponta a ponta povoa o mesmo esquema com uma **semente sintética**, gerada por *script* com semente fixa e rotulada como sintética nos comentários de cada tabela: 27 unidades da federação representadas como retângulos numa grade, 14 municípios, as cinco áreas de suprimento dos CGEO e 117 produtos. A semente serve apenas para demonstrar que a arquitetura fecha — da consulta em linguagem natural ao resultado em GeoJSON —; **nenhuma métrica deste trabalho depende do banco**, pois a avaliação incide sobre os parâmetros extraídos pelo modelo (Capítulo 3). O ambiente é levantado por Docker Compose, com um contêiner PostgreSQL 16 com PostGIS 3.4 que aplica o esquema e a semente na inicialização.

A camada de acesso foi reimplementada em Python com `psycopg`, em substituição à biblioteca `pg-promise` do protótipo. A lógica de construção das consultas, portada de `backend/src/services/search.ts`, foi preservada: busca textual (*full-text search* em português) combinada com a identificação de códigos MI e INOM por expressão regular; filtros exatos de escala, tipo de produto e projeto; intervalos de datas de publicação e de criação; interseções espaciais (`ST_Intersects`) com estados, municípios e áreas de suprimento; ordenação e limite de resultados. Para preservar a fidelidade ao protótipo, as mudanças limitaram-se à sintaxe do *driver*, à restrição da direção de ordenação a `ASC/DESC`, à aceitação de períodos com apenas um dos limites (`start` ou `end`), admitidos pelo novo *schema*, e à retirada do filtro por retângulo envolvente e da paginação, próprios do *frontend* de mapa (fora do escopo); o limite de resultados vem do parâmetro `limit` da

ferramenta, como no protótipo, com valor padrão de 10 e teto de 100. Todas as consultas são parametrizadas: o modelo nunca escreve SQL, apenas preenche os parâmetros da ferramenta, e a função monta a consulta com os valores passados como argumentos do *driver*.

Resolução dos campos geográficos: fora do escopo.

Os campos `state` e `city` extraídos pelo LLM são *strings* livres em português, que a consulta casa por nome com as tabelas de estados e municípios antes da interseção espacial com a geometria de cada folha. A correção desse resultado depende de geometrias reais das unidades territoriais (malhas do IBGE, Nominatim/OSM) e da qualidade das geometrias dos produtos importadas do catálogo (PYCSW), o que constitui uma frente de trabalho própria e **fica fora do escopo deste PFC**: a avaliação mede se o LLM extraiu o nome correto do estado ou do município, e não se a consulta espacial resultante retorna o conjunto exato de folhas. O tratamento completo dessa questão é apontado como trabalho futuro no Capítulo 6.

4.4.5 Pipeline de Avaliação

O *pipeline* de avaliação automatiza a execução do *dataset* de avaliação (Capítulo 3, Seção 3.3; Apêndice A) sobre cada modelo e a produção dos números apresentados no Capítulo 5. É composto por quatro comandos, declarados como pontos de entrada do pacote Python:

- **pfc-dataset**: constrói, a partir de uma fonte única — os casos das camadas P e N, mantidos em módulo próprio, e o gerador da camada G, com semente fixa —, o arquivo JSON executado pelo avaliador e os blocos do Apêndice A, de modo que o texto e o arquivo avaliado não podem divergir; os enumerados e campos de cada gabarito são validados contra o *schema* da ferramenta;
- **pfc-auditar**: executa as checagens automáticas da auditoria do *dataset* — estrutura (identificadores únicos, consultas não repetidas, enumerados válidos, expressões de tempo relativo que resolvem em diferentes datas de referência); rastreabilidade (toda consulta tem fonte, e as da camada P aparecem literalmente na linha citada do arquivo de casos de teste do protótipo); evidência, por meio de um anotador baseado em regras, escrito sem acesso ao gabarito, que aponta campos anotados sem evidência no texto da consulta, evidências sem campo anotado e valores divergentes; e consistência, exigindo a mesma anotação para a mesma expressão em todos os casos. O comando também compara o gabarito com a anotação independente feita às cegas e lista cada divergência; esses apontamentos alimentam a adjudicação descrita no Capítulo 3;

- **pfc-avaliar**: executa as consultas sobre um modelo, em ordem fixa e sequencialmente, e grava um registro por consulta $\times$ repetição em formato JSON Lines (texto da consulta, data de referência, *tool calls*, argumentos, tempos, *tokens* e erro classificado). A execução é retomável: se interrompida, é continuada pelo mesmo comando, sem repetir o que já foi registrado. Cada rodada gera um **manifesto de reprodutibilidade** (RNF4) com a *tag* e o *digest* do modelo, as versões de *software* (Python, Ollama, LangChain), o *hardware*, a data de referência injetada no *prompt* e os *hashes* SHA-256 do *dataset* e do *schema* da ferramenta. A pontuação é **sempre recalculada contra o gabarito vigente**: o registro guarda a resposta do modelo e a data de referência, e as métricas (Capítulo 3, Seção 3.7.5) são refeitas a partir do *dataset* atual. Assim, uma correção de gabarito decidida na auditoria não exige nova execução dos modelos, e registros cuja consulta teve o texto alterado são descartados e listados no resumo;
- **pfc-relatorio**: consolida os resultados dos modelos nas tabelas LaTeX e nas figuras do Capítulo 5 (comparativo geral, F1 por categoria e por campo, latência), calcula o intervalo de confiança de Wilson para a acurácia e o teste exato de McNemar pareado entre modelos, ambos com uma observação por consulta, e extrai uma amostra de erros representativos por tipo para a análise qualitativa.

Linha de base.

O comando **pfc-avaliar** aceita também a opção `--abordagem saida_estruturada`, que troca a chamada de ferramenta pelo pedido de um objeto JSON no formato do *schema* (Capítulo 3, Seção 3.7.4), e a opção `--abordagem prototipo`, que reproduz o método do protótipo, para a comparação entre as abordagens apresentada no Capítulo 5. As especificações v2 e v3 (Seções 3.5 e 3.6) são escolhidas pela mesma opção (por exemplo, `--abordagem tool_calling_v3`), e a opção `--dataset` aponta para o arquivo de cada lote. Os registros das linhas de base ficam em diretórios próprios e são pontuados pelas mesmas métricas.

Provedor alternativo e resultado paralelo.

O comando **pfc-avaliar** aceita a opção `--provedor groq`, que substitui ChatOllama por ChatOpenAI apontado para o *endpoint* compatível do Groq, mantendo o mesmo *prompt*, a mesma ferramenta, o mesmo *dataset* e as mesmas métricas. Com essa opção foram avaliados três modelos maiores servidos em nuvem (`qwen/qwen3.8-27b`, `openai/gpt-oss-20b` e `openai/gpt-oss-120b`), com uma repetição e com o esforço de raciocínio fixado no menor valor aceito por cada um (`none` para o `qwen/qwen3.8-27b` e `low` para o `openai/gpt-oss-20b` e o `openai/gpt-oss-120b`): o Qwen 3.8 27B foi

avaliado sem raciocínio, como os modelos locais, e os dois GPT-OSS, que não permitem desligá-lo, com esforço baixo. Esses resultados são gravados em diretório próprio, com manifesto rotulado como execução em nuvem, e reportados à parte no Capítulo 5 como referência: por dependerem de serviço externo, estão fora do requisito de execução local (RNF1) e não concorrem à recomendação de modelo para implantação. Duas particularidades do provedor são tratadas explicitamente: as respostas de limite de taxa do plano gratuito são aguardadas e a chamada é repetida — assim como, até três vezes, as falhas transitórias de conexão ou do servidor, antes de serem registradas como erro de infraestrutura —, registrando-se apenas a latência da tentativa respondida; e o servidor valida os argumentos contra o *schema* e rejeita a *tool call* inválida, o que é contado como erro do modelo, com os argumentos gerados — quando recuperáveis da mensagem de rejeição — avaliados campo a campo, como na execução local.

4.4.6 Registro de Abordagens, Implantação e Extensão

Além da solução v1, o repositório entrega a estrutura com que todas as configurações avaliadas no Capítulo 5 são executadas, implantadas e testadas, e com que uma versão nova de *Tool Calling* ou de Saída Estruturada é acrescentada sem alterar o código das já avaliadas:

- **Registro único de abordagens** (`src/pfc_busca/abordagens.py`): cada uma das nove configurações — o *Tool Calling* v1, v2 e os três degraus da v3, a Saída Estruturada v1, v2 e o controle da v3, e o método do protótipo — é uma entrada com nome, família (TC ou SE), versão da especificação, prefixo da pasta de resultados, a fábrica do tradutor e as funções que dão o texto do *prompt* e a definição das ferramentas, cujos *hashes* vão para o manifesto de cada rodada. O avaliador, a API e o comando `pfc` leem daí. Os módulos das versões avaliadas ficam congelados, porque os *hashes* das rodadas dependem do texto deles: o registro só os importa e envolve, e um teste de regressão confere que o código atual reproduz os *hashes* de *prompt* e de ferramenta de todas as rodadas válidas do repositório (os da v3 dependem do índice de folhas do BDGEx, sem o qual esses casos são pulados).
- **Abordagem escolhida por configuração**: na API, as variáveis `PFC_ABORDAGEM` e `PFC_MODELO_PADRAO` escolhem a abordagem e o modelo, com padrão na configuração recomendada, o *Tool Calling* v3 com o Gemma 4 E4B (Seção 5.13); a v1 continua disponível (`PFC_ABORDAGEM=tool_calling`), e a abordagem passa a ser trocada por configuração, como o modelo (RNF6). Um nome fora do registro impede a subida. O `/api/health` informa a abordagem, a família, a versão, o modelo, os *hashes* de *prompt* e de ferramenta — os mesmos do manifesto — e as rodadas do repositório feitas com essa mesma configuração, o que permite verificar se o que está no ar é o

que foi avaliado. Sem um dado de que as ferramentas da v3 dependem, como esse índice, que não é versionado, a API sobe em modo degradado, com aviso no *log*, no `/api/health` e em cada resposta. Com a v3, a SQL é executada uma vez, com os parâmetros da busca que encerra o laço de chamadas, e o pedido de esclarecimento continua respondido pelo usuário simulado da avaliação: levá-lo ao usuário exige conversa em múltiplos turnos (Capítulo 6). Um *Dockerfile* empacota a API, e o Docker Compose a sobe com o banco, com o Ollama no *host*.

- **Comandos:** o comando `pfc` reúne os do *pipeline* de avaliação em subcomandos com os mesmos argumentos (`pfc avaliar`, `pfc relatorio`, `pfc conferir`, `pfc ab`, entre outros) e acrescenta `pfc abordagens`, que lista o registro e os *hashes*, e `pfc api`, que sobe a API com a abordagem e o modelo escolhidos. O `pfc conferir` confere, antes da consolidação, os *hashes*, a completude e a data de referência de cada rodada, e o `pfc ab` gera o teste A/B da Seção 5.6.
- **Integração contínua** (`.github/workflows/testes.yml`): a cada *push* e *pull request*, roda o `ruff` e o `pytest` em Python 3.12 e constrói a imagem da API, que precisa subir e responder o `/api/health` sem Ollama e sem banco. Nenhum teste depende do Ollama, do Groq ou do banco: a API é testada com tradutores falsos.
- **Guia de extensão** (`docs/como_estender.md`): o caminho que a v3 percorreu, escrito para a próxima versão — módulo novo, sem editar os congelados; entrada no registro; testes; ciclos só nos conjuntos de desenvolvimento; congelamento pelo *hash*; checagem de contaminação; plano de análise gravado antes da rodada; conferência; análise, inclusive o teste A/B; e números levados ao texto por macros geradas —, com a regra de que um conjunto de teste testa uma vez: uma v4 desenhada depois deste trabalho não pode ser testada no lote 2 e precisa de um terceiro lote, construído com a mesma receita.

4.4.7 Ferramentas de Apoio ao Desenvolvimento

Em apoio ao desenvolvimento e à reprodutibilidade, foram empregados: `git`, para controle de versão; `pytest`, com testes automatizados que cobrem o *schema* da ferramenta, a montagem das consultas SQL, a resolução das expressões de tempo relativo, as invariantes do *dataset*, o cálculo das métricas e a geração dos relatórios (RNF5); `ruff`, para análise estática do código; Docker Compose, para subir o contêiner PostgreSQL/PostGIS de forma reprodutível; e o arquivo `pyproject.toml`, que declara as dependências e os comandos do pacote, instalados com `pip` em ambiente virtual Python. O repositório do projeto é público (Apêndice C).

4.5 Cronograma

O projeto foi executado entre abril e outubro de 2026, em oito fases (F0–F7) alinhadas às etapas da EAP, conforme o Quadro 4.

Quadro 4 – Cronograma de execução do projeto

Fase	Atividade	Período	Marco
F0	Análise do protótipo: código-fonte, dependências, esquema do banco e limitações documentadas	abr/2026	Limitações mapeadas
F1	Elicitação de requisitos e planejamento da migração (limitação → solução; componentes reaproveitados, reescritos e novos)	mai/2026	Requisitos e plano de migração
F2	Referencial teórico; seleção dos modelos; definição das métricas e do protocolo de avaliação	mai–jun/2026	Protocolo de avaliação
F3	Construção do <i>dataset</i> de avaliação: camadas P, N e G e respectivo gabarito (Apêndice A)	jun–jul/2026	310 consultas anotadas
F4	Prova de conceito do <i>Tool Calling</i> sobre consultas representativas, com modelo local e em nuvem	ago/2026	Viabilidade do <i>Tool Calling</i> demonstrada
F5	Implementação: API, orquestração, ferramenta <code>buscar_catalogo</code> , persistência e <i>pipeline</i> de avaliação, com testes automatizados	set/2026	Sistema ponta a ponta
F6	Experimentos (modelos locais, validação na estação de referência e resultado paralelo em nuvem); auditoria do <i>dataset</i> ; análise de erros; lote 1 e especificação v2; desenvolvimento da v3 e teste no lote 2	set/2026	Tabelas e figuras do Cap. 5
F7	Redação dos resultados e da conclusão; preparação da defesa	set–out/2026	Defesa (out/2026)

Fonte: Elaborado pelos autores.

A fase F6 foi conduzida em três ambientes, com os mesmos comandos de avaliação: (i) as rodadas completas dos quatro modelos locais, com Ollama em uma GPU de nuvem (NVIDIA T4, 16 GB), as mesmas *tags* de modelo e as versões de *software* registradas no manifesto de cada rodada (Tabela 11); (ii) um subconjunto de validação (camadas P e N) na estação de referência, com RTX 3050 Laptop de 4 GB de VRAM (Capítulo 3, Seção 3.7.2); e (iii) o resultado paralelo com os modelos servidos pelo Groq, fora do requisito de execução local e reportado à parte. As rodadas dos dois lotes de validação, com o Gemma 4 E4B, usaram a mesma T4; os ciclos de desenvolvimento da v3 rodaram com o mesmo arquivo de modelo, localmente (Capítulo 3, Seção 3.6). A configuração de cada ambiente e os resultados correspondentes são apresentados no Capítulo 5 (Seção 5.1).

A configuração do *hardware* da estação de referência foi coletada por meio dos comandos PowerShell apresentados no Apêndice B; a saída, transcrita de forma condensada, está na Seção B.1.2 do mesmo apêndice.

5 RESULTADOS

Este capítulo apresenta os resultados da execução do protocolo descrito no Capítulo 3 sobre o *dataset* do Apêndice A. Todos os valores são produzidos pelo *pipeline* de avaliação (Seção 4.4.5) a partir dos registros de execução, com a pontuação recalculada contra o gabarito vigente; cada rodada carrega um manifesto com a *tag* do modelo (e, nas rodadas locais, o *digest*), as versões de *software*, o *hardware*, a data de referência e os *hashes* do *dataset* e da definição da ferramenta.

5.1 Configuração Experimental

Os resultados vêm de três conjuntos de rodadas, descritos na Seção 3.7.2: as **rodadas completas** dos quatro modelos locais, com três repetições, executadas com Ollama numa GPU de nuvem NVIDIA T4; a **validação na estação de referência**, com as 62 consultas das camadas P e N; e a **referência em nuvem**, com três modelos de maior porte servidos pelo Groq, com uma repetição. Além delas, as linhas de base com Saída Estruturada foram executadas na mesma T4, no Groq e na estação de referência (Seção 5.3). As Tabelas 11, 26 e 32 transcrevem dos manifestos o ambiente de cada rodada. Todas as rodadas desses três conjuntos executaram as mesmas 310 consultas, com o mesmo *prompt* de sistema e a mesma definição da ferramenta, e foram pontuadas contra o gabarito final; as dos dois GPT-OSS, feitas antes da adjudicação da auditoria, que alterou apenas gabaritos, foram repontuadas, como descrito na Seção 3.7.6. As rodadas do Gemma 4 E4B nos dois lotes de validação, com as especificações v1, v2 e v3, são descritas nas Seções 3.5 e 3.6 e apresentadas nas Seções 5.4 e 5.5.

Tabela 11 – Ambiente de execução de cada rodada

Modelo	GPU / máquina	Servidor	Rep.	Exec.	Na GPU	VRAM livre
Qwen 3 4B	Tesla T4, 15360 MiB	Ollama 0.34.4	3	930	100%	14,6 GB
Gemma 4 E4B	Tesla T4, 15360 MiB	Ollama 0.34.4	3	930	100%	14,6 GB
Gemma 4 E2B	Tesla T4, 15360 MiB	Ollama 0.34.4	3	930	100%	14,6 GB
Mistral Nemo 12B	Tesla T4, 15360 MiB	Ollama 0.34.4	3	930	100%	14,6 GB

Fonte: Elaborado pelos autores a partir de `results/<modelo>/manifesto.json`. Na GPU: parcela do modelo carregado (pesos e contexto) alocada na VRAM pelo Ollama, medida após o aquecimento; VRAM livre: no início da rodada, com os demais modelos descarregados. Data de referência: 2026-09-14. Python 3.13.15; langchain-core 1.6.5, langchain-ollama 1.1.0.

Cada modelo foi avaliado com temperatura zero, limite de 1024 *tokens* de saída, sem contexto de conversações anteriores, sem exemplos no *prompt* (*zero-shot*) e sem dicionário de normalização, com o mesmo *prompt* de sistema e a mesma definição da ferramenta `buscar_catalogo`; o modo de raciocínio foi desativado nos modelos locais que o oferecem (Seção 3.7.3). As *tags* exatas são `qwen3:4b-instruct-2507-q4_K_M`,

`gemma4:e4b-it-qat`, `gemma4:e2b-it-qat` e `mistral-nemo:12b`. Nas rodadas completas, as métricas principais cobrem 306 consultas, pontuadas em cada uma das três repetições — 918 execuções por modelo —, incluindo as da categoria F, em que a resposta correta é não chamar a ferramenta; as consultas observacionais são reportadas à parte (Seção 5.9.1). Nenhuma execução das rodadas completas terminou em erro de infraestrutura.

5.2 Comparativo Geral entre Modelos

A Tabela 12 sumariza as métricas principais por modelo, e a Tabela 13 detalha a distribuição da latência da tradução.

Tabela 12 – Comparativo de desempenho entre os modelos avaliados

Modelo	Acurácia	IC 95%	Precisão	Recall	F1	Lat. med. (ms)
Qwen 3 4B	54,2%	48,6%–59,7%	0,697	0,743	0,666	828
Gemma 4 E4B	60,2%	54,7%–65,6%	0,853	0,763	0,793	725
Gemma 4 E2B	56,5%	50,9%–62,0%	0,821	0,760	0,765	397
Mistral Nemo 12B	26,8%	22,1%–32,0%	0,804	0,326	0,451	2391

Fonte: Elaborado pelos autores. IC 95%: intervalo de confiança de Wilson para a acurácia, com n igual ao número de consultas, pois as repetições de uma mesma consulta não são independentes. Precisão, *recall* e F1 ponderados pela frequência de cada campo no gabarito. Latência mediana da chamada de tradução, no ambiente de cada rodada.

Tabela 13 – Latência da tradução, em milissegundos, por modelo

Modelo	n	Mín.	Mediana	Média	p95	Máx.
Qwen 3 4B	918	66	828	868	1403	3117
Gemma 4 E4B	918	405	725	813	1230	4849
Gemma 4 E2B	918	224	397	420	703	1242
Mistral Nemo 12B	918	316	2391	2651	5860	16085

Fonte: Elaborado pelos autores. Tempo da chamada de tradução, medido no ambiente de cada rodada (Tabela 11); execuções com erro de infraestrutura excluídas.

Qualidade.

Três dos quatro modelos ficaram num mesmo patamar de acurácia: 60,2% para o Gemma 4 E4B, 54,2% para o Qwen 3 4B e 56,5% para o Gemma 4 E2B, com intervalos de confiança que se sobrepõem amplamente. Pelo teste de McNemar, com uma observação por consulta (Seção 5.12), nenhuma das diferenças entre esses três é estatisticamente significativa. O Mistral Nemo 12B, o maior dos quatro, ficou muito abaixo, com 26,8% (IC 95%: 22,1% a 32,0%), e a diferença para cada um dos outros três é significativa. O tamanho do modelo, portanto, não explica o desempenho na tarefa: a Seção 5.9 mostra que a queda do Mistral Nemo decorre quase inteiramente de um modo de falha no uso da ferramenta.

Precisão e *recall*.

Com acurácias próximas, os três modelos do patamar superior chegam a F1 ponderados diferentes: 0,793 no Gemma 4 E4B, 0,765 no Gemma 4 E2B e 0,666 no Qwen 3 4B. O Gemma 4 E4B teve a maior precisão (0,853) e o maior *recall* (0,763) dos três. O Qwen 3 4B teve a menor precisão (0,697), porque acrescenta com mais frequência campos que a consulta não pediu (Seção 5.9); como cada campo a mais conta como falso positivo, seu F1 cai mais do que sua acurácia.

Latência.

Na T4, o Gemma 4 E2B foi o mais rápido, com mediana de 0,4 s e percentil 95 de 0,7 s; o Gemma 4 E4B e o Qwen 3 4B tiveram medianas de 0,7 s e 0,8 s, com percentis 95 de 1,2 s e 1,4 s; e o Mistral Nemo 12B, 2,4 s, cerca de 6,0 vezes a mediana do Gemma 4 E2B (Figura 6). Esses tempos valem para uma GPU com VRAM suficiente para cada modelo inteiro; na estação de referência, com 4 GB, a latência é maior e depende da parcela do modelo que cabe na GPU (Seção 5.10).

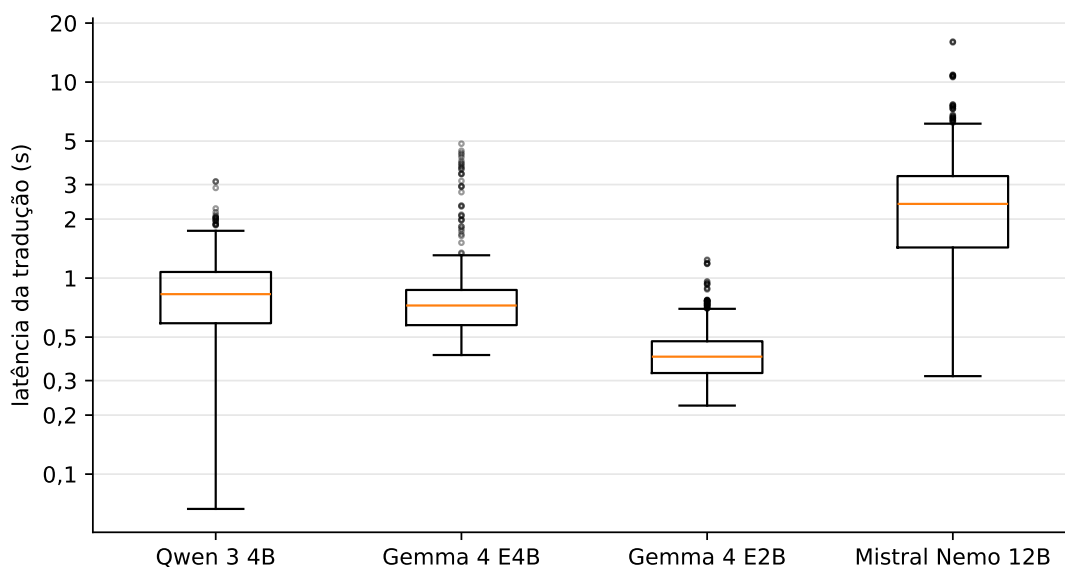


Figura 6 – Distribuição da latência da tradução nas rodadas completas, em escala logarítmica, por modelo

Fonte: Elaborado pelos autores.

Estabilidade entre repetições.

Numa mesma sessão da T4, as três repetições produziram exatamente a mesma resposta em 306 das 306 consultas para o Qwen 3 4B, em 306 para o Gemma 4 E2B, em

306 para o Mistral Nemo 12B e em 287 para o Gemma 4 E4B. Com temperatura zero e o mesmo ambiente, a variação entre repetições foi, portanto, pequena; a Seção 5.10 mostra que ela é maior entre sessões diferentes na estação de referência.

5.3 Tool Calling e Saída Estruturada

A troca da Saída Estruturada pelo *Tool Calling* é a principal mudança em relação ao protótipo. Para medir o que ela rendeu, as mesmas consultas foram executadas com as duas linhas de base da Seção 3.7.4: a Saída Estruturada com os mesmos modelos, o mesmo *prompt* e as mesmas descrições dos parâmetros, que isola o mecanismo, e o método do protótipo, com o Phi-4 14B, o dicionário de normalização e os exemplos resolvidos, que reproduz a solução anterior. Os modelos locais e o método do protótipo executaram as linhas de base na mesma T4 das rodadas completas; os modelos em nuvem, no Groq, cada um com a data de referência da sua rodada com *Tool Calling*. A Tabela 14 resume a comparação sobre as 306 consultas pontuadas, com os modelos em ordem crescente de tamanho.

Tabela 14 – Tool Calling e Saída Estruturada sobre as mesmas consultas

Modelo	Parâm.	Abordagem	Acurácia	Sem F	F1	Recusa F	Campo a mais	<i>p</i>	<i>p</i> sem F
Gemma 4 E2B	2B ef.	Tool Calling Saída Estruturada	56,5% 53,9%	56,4% 55,4%	0,765 0,792	62,5% 0,0%	17,1% 26,8%	0,488	0,839
Qwen 3 4B	4B	Tool Calling Saída Estruturada	54,2% 60,1%	53,0% 61,7%	0,666 0,808	100,0% 0,0%	37,6% 27,2%	0,038	0,001
Gemma 4 E4B	4B ef.	Tool Calling Saída Estruturada	60,2% 75,2%	59,2% 77,2%	0,793 0,902	100,0% 0,0%	10,3% 6,7%	< 0,001	< 0,001
Mistral Nemo 12B	12B	Tool Calling Saída Estruturada	26,8% 52,0%	24,8% 53,4%	0,451 0,727	100,0% 0,0%	7,4% 8,7%	< 0,001	< 0,001
GPT-OSS 20B (Groq)	21B (MoE)	Tool Calling Saída Estruturada	89,2% 77,5%	88,9% 79,5%	0,946 0,919	100,0% 0,0%	2,3% 4,7%	< 0,001	< 0,001
GPT-OSS 120B (Groq)	117B (MoE)	Tool Calling Saída Estruturada	89,9% 87,3%	89,6% 89,6%	0,956 0,964	100,0% 0,0%	4,0% 3,0%	0,268	1,000
Phi-4 14B	14B	Método do protótipo	58,8%	60,4%	0,856	0,0%	28,2%	—	—

Fonte: Elaborado pelos autores. Sem F: acurácia nas consultas das métricas principais fora da categoria F. Recusa F: proporção das execuções da categoria F sem busca; na Saída Estruturada a aplicação sempre busca. Campo a mais: proporção das execuções fora da categoria F com algum campo ausente do gabarito. *p*: teste de McNemar exato entre as duas abordagens do mesmo modelo, com uma observação por consulta (maioria das repetições). Parâm.: parâmetros (ef.: efetivos; MoE: mistura de especialistas, com parte dos parâmetros ativa a cada *token*). Modelos locais e método do protótipo: GPU T4, *Tool Calling* com três repetições e linhas de base com uma, data de referência de 14/09/2026. Modelos (Groq): execução em nuvem, uma repetição, data de referência de 24/09/2026.

Modelos locais.

Com os mesmos modelos, a Saída Estruturada foi mais precisa em três dos quatro: 75,2% contra 60,2% no Gemma 4 E4B, 60,1% contra 54,2% no Qwen 3 4B e 52,0% contra 26,8% no Mistral Nemo 12B, com diferenças significativas pelo teste de McNemar (*p*-valor inferior a 0,001, 0,038 e inferior a 0,001, respectivamente); no Gemma 4 E2B, as duas empataram (56,5% com *Tool Calling* e 53,9% com Saída Estruturada; *p*-valor de

0,488). A diferença vem, em boa parte, da liberdade de não buscar que o *Tool Calling* dá ao modelo. Das 62 consultas que só a Saída Estruturada acertou no Gemma 4 E4B, 32 foram consultas em que o modelo com *Tool Calling* não buscou, quase sempre por pedir esclarecimento; no Mistral Nemo 12B, foram 98 de 109, as chamadas descritas em texto (Seção 5.9). No Qwen 3 4B, que quase sempre busca, a vantagem veio da própria extração: com Saída Estruturada, ele acrescentou menos campos não pedidos (27,2% das execuções fora da categoria F, contra 37,6% com *Tool Calling*).

Modelos de maior porte.

Nos modelos em nuvem, o quadro foi outro. No GPT-OSS 20B, o *Tool Calling* foi claramente superior: 89,2% contra 77,5%, com diferença significativa mesmo sem as consultas fora do domínio (88,9% contra 79,5%; p -valor inferior a 0,001). Com Saída Estruturada, esse modelo omitiu parâmetros que a consulta trazia — “produtos na escala 1:1000”, com a escala grafada sem ponto, resultou num objeto vazio — e acrescentou o estado a partir do município (`state` “Bahia” em “cartas de Salvador”), o tipo de preenchimento para completar a estrutura que motivou a troca de abordagem (Seção 2.2). No GPT-OSS 120B, as duas abordagens empataram fora da categoria F (89,6% com *Tool Calling* e 89,6% com Saída Estruturada), e a diferença total, 89,9% contra 87,3%, veio das consultas fora do domínio.

A recusa.

Em todos os modelos, o *Tool Calling* permitiu recusar o que o catálogo não atende: com ele, os modelos da tabela não buscaram em nenhuma das oito consultas fora do domínio, exceto o Gemma 4 E2B, que recusou 62,5% delas; com Saída Estruturada, a aplicação buscava em todas. Os modelos devolveram, em parte delas, um objeto vazio — em nenhuma delas, no Mistral Nemo 12B, e em todas, no GPT-OSS 120B —, mas a aplicação do protótipo busca a partir de qualquer objeto. Uma aplicação que tratasse como recusa a resposta sem nenhum campo do *schema* separaria boa parte desses casos: no lote de validação, com o Gemma 4 E4B, essa resposta apareceu em 78,9% das consultas fora do domínio e em 0,5% das consultas do domínio, e a regra levaria a Saída Estruturada a 75,9% no lote e a 77,1% nas 310 (Seção 5.4). As consultas fora do domínio em que o modelo preencheu algum filtro, quase sempre a *keyword*, continuariam sendo buscadas, e, nas que a regra separasse, a aplicação não teria o que responder ao usuário. O *Tool Calling* tornou essa decisão explícita e mensurável, e o custo dessa liberdade apareceu na outra ponta: as consultas legítimas em que o modelo decidiu não buscar.

O efeito do porte.

Não há tendência de o *Tool Calling* melhorar a extração à medida que o modelo cresce: a Saída Estruturada foi melhor em três modelos locais, de 4 a 12 bilhões de parâmetros; as duas empataram no menor deles e no maior modelo em nuvem; e o *Tool Calling* foi melhor no GPT-OSS 20B. O efeito parece depender do modelo — possivelmente de como cada um foi ajustado para chamar ferramentas e para responder em JSON —, e não do porte.

O método do protótipo.

O método do protótipo acertou 58,8% (IC 95%: 53,2% a 64,2%), acurácia estatisticamente igual à dos três melhores modelos locais com *Tool Calling* (p -valor de 0,805 contra o Gemma 4 E4B, 0,614 contra o Gemma 4 E2B e 0,206 contra o Qwen 3 4B), com F1 de 0,856. Para isso, usou um modelo de 14 bilhões de parâmetros, com mediana de 12,6 s por consulta na T4, contra 0,4 s a 0,8 s dos modelos locais com *Tool Calling*: em 295 das 306 consultas, a primeira resposta do modelo, quase vazia, foi rejeitada pela validação do *schema* e exigiu nova tentativa. O dicionário alterou o texto de 68 consultas antes de chegarem ao modelo, e o *fallback* por expressões regulares foi acionado em 10. Em relação ao protótipo como foi entregue, a solução aprimorada chegou, portanto, à mesma acurácia com modelos de 2 a 4 bilhões de parâmetros efetivos, mais de quinze vezes mais rápidos, sem dicionário nem exemplos, e capazes de recusar o que o catálogo não atende. A comparação com a Saída Estruturada nos mesmos modelos mostra, contudo, que esse resultado se deve à troca de modelo, e não ao *Tool Calling* de uma chamada: com Saída Estruturada, os mesmos modelos locais seriam mais precisos. A Seção 5.5 mostra que o quadro muda com a v3, em que o *Tool Calling* chama ferramentas auxiliares no meio da resposta.

Na estação de referência.

A mesma comparação, feita na estação sobre as 59 consultas pontuadas das camadas P e N (Tabela 15), confirmou o empate no Gemma 4 E2B (47,5% com *Tool Calling* e 45,8% com Saída Estruturada) e não mostrou diferença significativa no Qwen 3 4B nessa amostra menor (49,2% e 50,8%).

5.4 Lote de Validação Independente

O lote de validação (Seção 3.5) repetiu a comparação numa amostra seis vezes maior e independente das 310 consultas, com o Gemma 4 E4B, o modelo local de maior

Tabela 15 – Tool Calling e Saída Estruturada sobre as mesmas consultas — estação de referência (camadas P e N)

Modelo	Parâm.	Abordagem	Acurácia	Sem F	F1	Recusa F	Campo a mais	<i>p</i>	<i>p</i> sem F
Gemma 4 E2B	2B ef.	Tool Calling	47,5%	45,6%	0,719	100,0%	15,8%	1,000	1,000
		Saída Estruturada	45,8%	47,4%	0,782	0,0%	28,1%		
Qwen 3 4B	4B	Tool Calling	49,2%	47,4%	0,733	100,0%	31,6%	1,000	0,629
		Saída Estruturada	50,8%	52,6%	0,815	0,0%	22,8%		

Fonte: Elaborado pelos autores. Sem F: acurácia nas consultas das métricas principais fora da categoria F. Recusa F: proporção das execuções da categoria F sem busca; na Saída Estruturada a aplicação sempre busca. Campo a mais: proporção das execuções fora da categoria F com algum campo ausente do gabarito. *p*: teste de McNemar exato entre as duas abordagens do mesmo modelo, com uma observação por consulta (maioria das repetições). Parâm.: parâmetros (ef.: efetivos; MoE: mistura de especialistas, com parte dos parâmetros ativa a cada *token*).

acurácia com *Tool Calling*, em quatro configurações: as duas abordagens com a especificação da solução (v1) e com a especificação v2, que explicita as convenções do manual e dá às duas uma forma de recusar. A Tabela 16 resume o resultado nos dois conjuntos; a Tabela 17, as comparações pareadas; e a Tabela 18, duas combinações simuladas a partir dessas rodadas, discutidas adiante.

Tabela 16 – Gemma 4 E4B no lote de validação e nas 310 consultas: especificações v1 e v2

Conjunto	Configuração	Acurácia	IC 95%	F1	Recusa F	Falsa recusa	F1 recusa	Múltiplas leituras
Lote de validação (n=1929)	Tool Calling v1	61,5%	59,3%–63,6%	0,761	100,0%	32,1%	0,592	34,6%
	Saída Estruturada v1	62,2%	60,0%–64,3%	0,892	0,0%	0,0%	0,000	72,0%
	Tool Calling v2	62,8%	60,7%–65,0%	0,840	99,7%	0,9%	0,980	46,5%
	Saída Estruturada v2	72,4%	70,4%–74,4%	0,860	100,0%	17,8%	0,723	61,1%
310 consultas (n=306)	Tool Calling v1	60,1%	54,6%–65,5%	0,793	100,0%	18,8%	0,222	26,9%
	Saída Estruturada v1	75,2%	70,0%–79,7%	0,902	0,0%	0,0%	0,000	52,2%
	Tool Calling v2	47,4%	41,9%–53,0%	0,796	100,0%	0,0%	1,000	40,3%
	Saída Estruturada v2	68,0%	62,6%–73,0%	0,859	100,0%	14,4%	0,271	53,7%

Fonte: Elaborado pelos autores. Gemma 4 E4B, GPU T4. Acurácia por consulta, com IC de Wilson. Recusa F: proporção das consultas fora do domínio sem busca (recall da recusa). Falsa recusa: proporção das consultas do domínio (fora das categorias F e E) sem busca. F1 recusa: média harmônica da precisão e do recall da recusa. Múltiplas leituras: acurácia nas consultas do domínio com mais de uma leitura aceita. Na v1, a Saída Estruturada não tem como recusar. A v2 foi desenhada a partir dos erros nas 310 consultas: nesse conjunto, o resultado da v2 é de dentro da amostra e, mesmo assim, ficou abaixo do da v1 nas duas abordagens.

O empate é efeito da composição.

Com a especificação v1, as duas abordagens empataram no lote: 61,5% com *Tool Calling* e 62,2% com Saída Estruturada (*p*-valor de 0,656). O empate esconde duas diferenças de sinal contrário. Nas 1.447 consultas do domínio, a Saída Estruturada acertou 75,5% e o *Tool Calling*, 48,9%; nas 336 consultas fora do domínio, o *Tool Calling* recusou 100,0%, e a Saída Estruturada, que não tem como recusar, nenhuma. A acurácia geral é, portanto, uma média das duas, ponderada pela proporção de consultas fora do domínio, e há uma proporção em que as duas abordagens se igualam: 18,0% no lote (IC 95%: 16,3% a 19,8%) e 15,3% nas 310. O lote tem 17,4% de consultas fora do domínio, perto do ponto

de equilíbrio, daí o empate; as 310 têm 2,6%, daí a vantagem da Saída Estruturada na Seção 5.3. Trocar só a composição inverte o vencedor: com 2,6% de consultas fora do domínio, como nas 310, o lote daria 54,6% ao *Tool Calling* e 73,4% à Saída Estruturada; com 17,4%, como no lote, as 310 dariam 66,2% e 63,7%. Com a v1, o *Tool Calling* só é mais preciso que a Saída Estruturada se mais de 18,0% das consultas reais estiverem fora do catálogo, e isso pela pontuação do protocolo, em que o objeto vazio da Saída Estruturada conta como busca. Se a aplicação tratasse como não busca a resposta sem nenhum campo do *schema*, o ponto de equilíbrio subiria para 51,0% no lote e 42,0% nas 310.

A falsa recusa.

No domínio, o *Tool Calling* v1 deixou de buscar em 464 consultas (32,1%), contrariando a instrução do *prompt* de que um único parâmetro basta. Pela leitura das respostas, 434 delas são pedidos de esclarecimento, 23 são recusas — algumas afirmam ao usuário, sem ter buscado, que o acervo não tem o produto —, 5 delegam a chamada ao usuário (“utilize a ferramenta `buscar_catalogo` informando o parâmetro `state` como ‘Tocantins’”) e 2 descrevem a chamada sem fazê-la. Os pedidos são sobretudo do tipo de produto e do local, e só uma minoria pede o que a consulta já dizia; o pedido da data corrente (“preciso saber qual é o ano atual”), frequente nas 310 (Seção 5.9), é minoria nas consultas de tempo do lote. A falsa recusa depende da consulta: foi de 44,5% nas consultas com um só campo e de 12,0% nas com quatro ou mais, de 54,7% nas de tempo relativo e de 1,8% nas com código, e de 15,6% nos registros direto e telegráfico, contra 39,0% nos demais. As 310, de consultas mais curtas e diretas, a subestimam: nelas, a taxa foi de 18,8%. Nas consultas em que o *Tool Calling* não buscou, a Saída Estruturada acertou 316. Se o *Tool Calling* tivesse buscado nelas com a resposta da Saída Estruturada, sua acurácia no domínio iria de 48,9% a 70,8%: a falsa recusa explica 82,3% da diferença no domínio (+21,8 de +26,5 p.p.). O restante é extração: nas 983 consultas do domínio em que as duas buscaram, a Saída Estruturada acertou 78,9%, e o *Tool Calling*, 72,0% (p -valor $< 0,001$), sobretudo porque o *Tool Calling* acrescentou o tipo de produto a “cartas” e “mapas” em 13,1% das buscas cujo gabarito não tem tipo, contra 0,6% da Saída Estruturada; em F1 ponderado, a distância nessas consultas é pequena (0,915 contra 0,927). Na categoria E, em que buscar sem filtros e não buscar são aceitos, o mesmo comportamento conta a favor: o *Tool Calling* v1 deixou de buscar em 97,3% das consultas subespecificadas, quase sempre pedindo esclarecimento; se essas não buscas contassem como erro, sua acurácia geral cairia de 61,5% para 54,1%.

A v2: a recusa resolvida, a forma quebrada.

A especificação v2 fez o que foi desenhada para fazer quanto à recusa: a falsa recusa do *Tool Calling* caiu para 0,9%, e o F1 da recusa subiu de 0,592 para 0,980. A acurácia, porém, quase não mudou (62,8%; p -valor de 0,277 contra a v1), e o saldo esconde movimentos grandes nos dois sentidos. A v2 acertou 222 consultas em que a v1 não buscava (+11,5 p.p.) e 55 em que a v1 buscava com campos errados (+2,9 p.p.), mas errou 251 que a v1 acertava (-13,0 p.p.), 37 delas subespecificadas, em que passou a buscar com filtros indevidos (-1,9 p.p.). Todo o ganho vem, portanto, de buscar onde a v1 não buscava: nas 987 consultas em que as duas versões buscaram, a acurácia do *Tool Calling* caiu de 71,7% para 55,7%, porque a v2 introduziu erros de forma que a v1 não cometia. A lista de siglas das UFs, incluída na descrição de **state** para evitar confusões como “AM” com Amapá, virou formato de saída: o *Tool Calling* v2 escreveu a sigla, no lugar do nome ou antes dele, em 41,4% das execuções com estado (a v1, em 0,0%). A causa provável é que a lista, sem direção, substituiu a regra da v1 (“Normalizar siglas: ‘RJ’ → ‘Rio de Janeiro’”), e os *prompts* v2 perderam a instrução de reconhecer e normalizar siglas. A regra do singular (“a carta mais recente” → 1) virou **limit** em plurais (8,8% das buscas sem limite no gabarito, contra 0,2%). E, com a instrução nova de copiar o código “exatamente como aparece”, o modelo passou a manter o prefixo “MI” na *keyword* (89,4% das consultas com “MI”, contra 10,6%), embora a mesma descrição, como a da v1, mande escrever o código sem o prefixo. Outros erros novos têm valores que coincidem com os termos citados nas proibições da v2: depois de “palavras genéricas (‘carta’, ‘folha’, ‘mapa’) nunca são *keyword*”, a *keyword* genérica, que a v1 não usava no lote, apareceu em buscas do *Tool Calling* v2, e, depois de “regiões (‘Nordeste’, ‘Sul’, ‘Amazônia Legal’) não são estados”, o nome de região em **state** ficou mais frequente. Essas causas são hipóteses: a associação é correlacional, e nenhuma frase da v2 foi retirada para testá-la. São erros de forma, reversíveis sem o gabarito: três normalizações determinísticas — sigla para o nome da UF, prefixo retirado da *keyword* e *keyword* genérica descartada — levariam o *Tool Calling* v2 a 72,5% e a Saída Estruturada v2 a 76,3%, um diagnóstico, e não uma configuração avaliada.

Nas 310 consultas, a v2 piorou as duas abordagens (*Tool Calling* de 60,1% para 47,4%; Saída Estruturada de 75,2% para 68,0%), embora tenha sido desenhada a partir dos erros nesse conjunto. No *Tool Calling*, as parcelas são as mesmas do lote, com pesos diferentes: as falsas recusas consertadas valeram +8,2 p.p. (+11,5 no lote) e os campos consertados, +5,9 (+2,9), de modo que a falsa recusa menor das 310 é quase compensada por mais consertos de campo; a diferença entre os conjuntos está nas consultas quebradas, que custaram -26,8 p.p., contra -13,0 no lote. A v2 errou 46,6% das consultas do domínio que a v1 acertava nas 310, contra 30,1% no lote, porque os erros de forma atingem as construções mais frequentes da camada G, com o nome da UF (“cartas do Acre”) ou com

o código MI (“mi 2901”); a composição das 310, com mais consultas desses dois tipos, explica boa parte desse excesso. As consultas fora do domínio não entram nessa diferença, porque o *Tool Calling* já as recusava todas na v1. Na Saída Estruturada, entram: a recusa ganhou +8 consultas nas 310, contra +336 no lote, e não compensou a perda no domínio (-30; no lote, -177). O resultado dentro da amostra, que se esperava otimista, não o foi no *Tool Calling*: no domínio, a diferença entre a v2 e a v1 foi de -13,1 p.p. nas 310, contra +4,4 p.p. no lote; na Saída Estruturada, foi parecida nos dois conjuntos (-10,1 e -12,2 p.p.). Com as mesmas três normalizações, o *Tool Calling* v2 chegaria a 73,5%. A lição de desenho é que acrescentar instruções a um modelo pequeno não é monotônico: cada regra nova conserta o erro que a motivou e pode criar outro, e só um conjunto que o desenho não viu mede o saldo.

Especificação × mecanismo.

As duas v2 recebem as mesmas descrições dos doze parâmetros, mas o texto que enquadra a recusa difere. O *Tool Calling* deve buscar quando a consulta pedir produtos do acervo “mesmo que de forma vaga”, com os parâmetros vazios se nenhum for reconhecível, e a ferramenta de recusa avisa que não serve para consultas vagas; a Saída Estruturada, quando a consulta não tratar do acervo, deve responder só com **fora_do_escopo** verdadeiro, “e nada mais”, e o campo avisa que consultas vagas não estão fora do escopo. A comparação entre as duas v2 mede, portanto, o mecanismo junto com esse texto. A recusa é melhor no *Tool Calling*: as duas recusaram praticamente todas as consultas fora do domínio, mas a ferramenta **recusar_consulta** teve precisão de 96,3%, contra 56,7% do campo **fora_do_escopo**, que a Saída Estruturada marcou em 17,8% das consultas do domínio, apesar da regra de buscar com os parâmetros que a consulta permite, “mesmo que seja um só ou nenhum”. Essa falsa recusa se concentra nas consultas com um só campo (34,8%; 88,7% das que trazem só o município e 72,1% das que trazem só o estado), nas de região e nas de escala qualitativa. O mesmo gradiente já aparecia no *Tool Calling* v1, como pedido de esclarecimento (44,5% das consultas com um só campo), e só o *Tool Calling* v2 o suprime (1,9%): é uma tendência do modelo, que o canal e o texto de recusa do *Tool Calling* v2 contêm. A Saída Estruturada v2 marcou fora do escopo também todas as subespecificadas, o que a pontuação da categoria E aceita como não busca, mas a própria especificação v2 exclui; contadas como erro, sua acurácia cairia para 64,9%.

A extração é melhor na Saída Estruturada: nas 1.189 consultas do domínio em que as duas buscaram, ela acertou 77,0%, contra 55,8% do *Tool Calling* (p -valor $< 0,001$), com F1 ponderado de 0,931 contra 0,867. Com a v1, a distância era bem menor: no subconjunto de 898 consultas em que as quatro configurações buscaram, a vantagem da Saída Estruturada na acurácia foi de +7,0 p.p. na v1 e de +19,6 p.p. na v2. Os erros de forma da v2 — sigla, prefixo e limite — aparecem nas duas abordagens e são, portanto,

da especificação, mas o *Tool Calling* os amplificou. Nas 238 consultas com estado em que as duas buscaram, a sigla da UF apareceu, sozinha ou antes do nome, em 34,9% das respostas do *Tool Calling*, contra 21,0% das da Saída Estruturada (p -valor $< 0,001$; nas 310, 41,4% contra 23,4%, p -valor $< 0,001$), e só no *Tool Calling* ela aparece antes do nome, copiada da lista da descrição; o prefixo, em 89,4% das consultas com “MI”, contra 25,7%; e o limite, em 8,8% das buscas sem limite no gabarito, contra 1,9%. O tipo de produto acrescentado a “cartas” é de outra natureza: vem do mecanismo, e não da especificação. Já na v1, com o mesmo texto nas duas abordagens, o *Tool Calling* o acrescentou em 13,1% das buscas cujo gabarito não tem tipo, contra 0,6% da Saída Estruturada, e a proibição explícita da v2 não o reduziu: nas mesmas buscas, o *Tool Calling* v2 o acrescentou em 16,7%, contra 13,1% da v1 (p -valor 0,004), e a Saída Estruturada v2, em 0,7% das suas. No lote 2, porém, a descrição v3, que manda não preencher o tipo na dúvida, o reduziu (Seção 5.5): o acréscimo vem do mecanismo, mas a especificação pode contê-lo. Numa só chamada, o canal de ferramentas em si funcionou: só 13 das 1.929 respostas do *Tool Calling* v2 trouxeram a chamada escrita no texto, na sintaxe própria do Gemma, sem que o Ollama a reconhecesse, e 1 veio com argumentos fora do tipo. Com este modelo e uma só chamada, o *Tool Calling* recusa melhor e extrai pior (com a v3, o quadro muda; Seção 5.5); não há sinal de que a implementação esteja errada, e sim de que o Gemma 4 E4B preenche mais campos quando responde pela ferramenta do que quando responde em JSON.

Tabela 17 – Comparações pareadas (Gemma 4 E4B)

Conjunto	Par (A × B)	Só A	Só B	p	B – A, acurácia (IC)	B – A, F1 (IC)
Lote	TC v1 × SE v1	419	433	0,656	+0,7 p.p. (-2,4 a +3,8)	0,131 (0,114 a 0,148)
Lote	TC v2 × SE v2	153	338	$< 0,001$	+9,6 p.p. (+7,4 a +11,8)	0,020 (0,006 a 0,033)
Lote	TC v1 × TC v2	251	277	0,277	+1,3 p.p. (-0,8 a +3,7)	0,079 (0,061 a 0,098)
Lote	SE v1 × SE v2	271	468	$< 0,001$	+10,2 p.p. (+7,5 a +13,0)	-0,032 (-0,044 a -0,020)
Lote	TC v1 × SE v2	146	357	$< 0,001$	+10,9 p.p. (+8,9 a +13,2)	0,099 (0,081 a 0,119)
310	TC v1 × SE v1	16	62	$< 0,001$	+15,0 p.p. (+9,8 a +20,6)	0,106 (0,072 a 0,147)
310	TC v2 × SE v2	30	93	$< 0,001$	+20,6 p.p. (+14,1 a +27,1)	0,063 (0,024 a 0,098)
310	TC v1 × TC v2	82	43	$< 0,001$	-12,7 p.p. (-19,9 a -6,2)	0,001 (-0,044 a 0,052)
310	SE v1 × SE v2	66	44	0,045	-7,2 p.p. (-13,4 a -0,7)	-0,043 (-0,076 a -0,011)
310	TC v1 × SE v2	49	73	0,037	+7,8 p.p. (+1,0 a +14,4)	0,064 (0,023 a 0,110)

Fonte: Elaborado pelos autores. Só A / Só B: consultas em que só uma das configurações acerta. p : McNemar exato, uma observação por consulta. IC: bootstrap pareado (2.000 reamostragens das consultas), percentis 2,5 e 97,5. TC: *Tool Calling*; SE: Saída Estruturada.

Duas etapas: a decisão do *Tool Calling*, a extração da Saída Estruturada.

Se o *Tool Calling* recusa melhor e a Saída Estruturada extrai melhor, cada mecanismo pode ficar com a parte que faz melhor: o *Tool Calling* v2 decide se a consulta é do catálogo e, quando decide buscar, os parâmetros vêm da Saída Estruturada v1. Como as duas configurações foram executadas sobre as mesmas consultas, com temperatura zero, e a segunda etapa é exatamente a chamada da Saída Estruturada v1, essa

arquitetura pode ser simulada com as rodadas existentes, sem chamada nova ao modelo. A Tabela 18 a compara com as quatro configurações executadas e com uma alternativa de uma só chamada: a Saída Estruturada v1 em que a resposta sem nenhum campo do *schema* conta como não busca. No lote, as duas etapas acertaram 79,5%, contra 72,4% da melhor configuração executada, a Saída Estruturada v2; 269 consultas foram acertadas só pelas duas etapas, e 132, só pela Saída Estruturada v2 (p -valor $< 0,001$; diferença de +7,1 p.p., IC 95%: +5,1 a +9,1). A vantagem persiste contra a Saída Estruturada v2 com as três normalizações (+3,2 p.p.; p -valor 0,002). Nas 310, as duas etapas acertaram 77,8%, contra 75,2% da Saída Estruturada v1, a melhor configuração executada nesse conjunto (+2,6 p.p., IC 95%: +1,0 a +4,3; p -valor 0,008). A combinação herda o melhor de cada etapa. No domínio, acerta praticamente o mesmo que a Saída Estruturada v1 (75,3% contra 75,5% no lote), porque o *Tool Calling* v2 quase nunca recusa uma consulta legítima (falsa recusa de 0,9%); fora dele, recusa 99,7% das consultas, com a precisão de 96,3% da ferramenta de recusa, e tem uma mensagem para o usuário. Por isso, supera a Saída Estruturada v1 a partir de 0,1% de consultas fora do domínio, enquanto a escolha entre as configurações v1 isoladas dependia de essa proporção ficar acima ou abaixo de 18,0%. A regra do objeto vazio chega a 75,9% no lote e a 77,1% nas 310, recusando 78,9% das consultas fora do domínio, com precisão de 97,4%; perde para as duas etapas no lote (72 consultas acertadas só pelas duas etapas, contra 3; p -valor $< 0,001$), empata nas 310 (2 contra 0; p -valor 0,500) e não tem o que responder ao usuário. O custo das duas etapas é a segunda chamada sempre que há busca: somadas as duas latências, a mediana foi de 1,46 s no lote (percentil 95 de 2,82 s) e de 1,61 s nas 310, contra 0,72 s e 0,76 s da Saída Estruturada v1. As duas combinações são simulações *post hoc*, e não *pipelines* executados: a regra do objeto vazio foi definida depois de ver os dados, e a latência das duas etapas é uma soma, sem medir cache nem sobreposição das chamadas.

A avaliação contou erros de verdade?

Uma auditoria adversarial sorteou 160 execuções erradas do lote (40 por configuração, estratificadas por família) e julgou cada uma pelo manual: 155 (96,9%; IC 95%: 92,9% a 98,7%) são erros reais do modelo, 5 têm gabarito discutível, e nenhuma tem gabarito que contrarie o manual. O número de gabaritos discutíveis depende do critério: vai de 3, se só for discutível a resposta cuja forma nenhuma regra escrita do manual cobre, a 6, se também o for a resposta que daria a mesma busca de uma leitura aceita ou que acertaria sob as convenções não informadas discutidas abaixo; os erros reais nunca ficam abaixo de 154. A comparação estrita também não penaliza, na prática, respostas inócuas: comparando a SQL gerada por cada resposta errada com a das leituras aceitas, no máximo 5,2% delas, conforme a configuração, dariam a mesma busca. Esse máximo depende de duas aproximações na comparação das datas; sem elas, a proporção vai de 1,2% a 3,3%, e contar essas respostas como acerto não mudaria a ordem das configurações. Os erros

Tabela 18 – Arquitetura em duas etapas e regra do objeto vazio, simuladas com as rodadas do Gemma 4 E4B, ao lado das quatro configurações executadas

Conjunto	Configuração	Acurácia	IC 95%	Domínio	Recusa F	Precisão da recusa	Falsa recusa	Latência (s)
Lote de validação (n=1929)	Tool Calling v1	61,5%	59,3%–63,6%	48,9%	100,0%	42,0%	32,1%	0,86
	Saída Estruturada v1	62,2%	60,0%–64,3%	75,5%	0,0%	—	0,0%	0,72
	Tool Calling v2	62,8%	60,7%–65,0%	53,4%	99,7%	96,3%	0,9%	0,94
	Saída Estruturada v2	72,4%	70,4%–74,4%	63,2%	100,0%	56,7%	17,8%	0,61
	SE v1, objeto vazio como não busca	75,9%	74,0%–77,8%	75,5%	78,9%	97,4%	0,5%	0,72
	Duas etapas (TC v2 decide, SE v1 extrai)	79,5%	77,7%–81,3%	75,3%	99,7%	96,3%	0,9%	1,46
310 consultas (n=306)	Tool Calling v1	60,1%	54,6%–65,5%	59,1%	100,0%	12,5%	18,8%	0,72
	Saída Estruturada v1	75,2%	70,0%–79,7%	77,2%	0,0%	—	0,0%	0,76
	Tool Calling v2	47,4%	41,9%–53,0%	46,0%	100,0%	100,0%	0,0%	0,81
	Saída Estruturada v2	68,0%	62,6%–73,0%	67,1%	100,0%	15,7%	14,4%	0,71
	SE v1, objeto vazio como não busca	77,1%	72,1%–81,5%	77,2%	75,0%	100,0%	0,0%	0,76
	Duas etapas (TC v2 decide, SE v1 extrai)	77,8%	72,8%–82,1%	77,2%	100,0%	100,0%	0,0%	1,61

Fonte: Elaborado pelos autores. Gemma 4 E4B, GPU T4. As duas últimas linhas de cada conjunto são simulações *post hoc* sobre as rodadas existentes, sem chamada nova ao modelo. Duas etapas: o *Tool Calling* v2 decide se busca; quando busca, valem os parâmetros da Saída Estruturada v1 na mesma consulta, e a latência é a soma das duas chamadas. Objeto vazio como não busca: a resposta da Saída Estruturada v1 sem nenhum campo do *schema* conta como recusa (regra definida depois de ver os dados). Acurácia com IC de Wilson. Domínio: acurácia fora das categorias F e E. Recusa F: proporção das consultas fora do domínio sem busca. Precisão da recusa: proporção de consultas fora do domínio entre as que ficaram sem busca (—: nenhuma ficou). Falsa recusa: proporção das consultas do domínio sem busca. Latência: mediana. TC: *Tool Calling*; SE: Saída Estruturada.

de forma da v2, ao contrário, não são inócuos, e alguns são piores que omitir o campo. Pela busca por substring, dos valores de *state* em sigla do *Tool Calling* v2, 35,3% não casam nenhuma UF, 6,7% casam só uma UF errada (“SP” casa “Espírito Santo”) e 50,4% casam a certa e outras (“AM” casa “Amazonas”, “Amapá” e “Pernambuco”); e todas as execuções erradas com o prefixo “MI” na *keyword* (103/103) dão uma busca que não encontra a folha pedida.

A ressalva que fica é inversa: 154 consultas do lote dependem de convenções do manual que a descrição v1 não informa — “atualização” como pista de criação, o fim obrigatório nos períodos “até hoje”, “cartas” seguido de um nome, no registro telegráfico, lido só como município, e a escala qualitativa com valor obrigatório. Aceitar nelas a leitura natural converteria em acerto 30 erros do *Tool Calling* v1 e 35 da Saída Estruturada v1, contra 15 e 4 das v2, cuja descrição explicita a maior parte delas: as convenções não informadas penalizam mais as duas v1, e explicitá-las foi uma das motivações da v2. Com essa leitura, as acurácias seriam de 63,0% e 64,0% na v1 e de 63,6% e 72,6% na v2; a vantagem do *Tool Calling* v2 sobre o v1 cairia de +1,3 para +0,6 p.p., e a Saída Estruturada v1 passaria o *Tool Calling* v2.

Categorias e registros.

As Tabelas 37 e 38, no Apêndice A.15, trazem a acurácia por categoria e o F1 por campo. Excluídas as consultas fora do domínio e as subespecificadas, a Saída Estruturada v1 foi a melhor configuração em todos os oito registros de linguagem; as vantagens aparentes do *Tool Calling* em registros como “direto” e “sem acento” vêm de esses registros concentrarem as consultas fora do domínio. Os registros sem acento e telegráfico

ficam abaixo dos demais nas quatro configurações (na Saída Estruturada v1, 66,6% contra 77,8%; p -valor $< 0,001$). Todas as configurações acertam menos nas consultas com mais de uma leitura aceita do que nas de leitura única: 34,6% contra 55,8% no *Tool Calling* v1, 72,0% contra 77,2% na Saída Estruturada v1, 46,5% contra 56,7% no *Tool Calling* v2 e 61,1% contra 64,2% na Saída Estruturada v2. A vantagem da Saída Estruturada cresce nas leituras múltiplas — a diferença entre as diferenças é de +16,1 p.p. na v1 (IC 95%: +9,9 a +21,9) e de +7,1 p.p. na v2 (IC 95%: +1,0 a +13,1) —, mas não por efeito da ambiguidade em si. As leituras múltiplas se concentram em algumas famílias, e, comparadas dentro de cada família, a diferença desaparece na v2; na v1, vem sobretudo da não busca do *Tool Calling*, mais frequente nessas consultas, e essa não busca, na maior parte, não pede ao usuário que escolha entre as leituras previstas no gabarito. As quedas mais fortes, em dois subtipos da família VA — escala qualitativa e região acompanhada de outro critério, este com uma única leitura aceita —, também vêm de abstenção: o *Tool Calling* v1 deixa de buscar, e a Saída Estruturada v2 declara a consulta fora do escopo.

5.5 Teste da v3 no Lote 2

A v3 (Seção 3.6) foi testada no lote 2, de 945 consultas que o desenvolvimento não leu, com o Gemma 4 E4B na mesma T4, uma repetição e a configuração congelada: o caderno do Colab conferiu o *hash* da v3 antes de rodar, e o manifesto de cada rodada registra os *hashes* do *prompt* e das ferramentas da sua configuração, o mesmo *commit* do código, o mesmo arquivo de modelo e a mesma versão do Ollama. As análises seguem o plano fixado antes da rodada (`docs/v3.md`); as medidas do plano que não estão nas tabelas, como os efeitos de forma de cada configuração, estão em `results/lote2/consolidado/lote2.md`. As que não estavam no plano — a comparação das primeiras decisões do *Tool Calling* v3 e do controle, a do degrau de uma chamada com a primeira decisão do controle, o acerto nas consultas com chamada escrita como texto, a comparação entre desenvolvimento e teste, a análise por categoria e a leitura dos erros — foram feitas depois do teste, são exploratórias e vêm marcadas como tais; nenhuma alterou a v3. Com a correção de Holm para as nove comparações do plano, as que eram significativas continuam significativas. A Tabela 19 traz as medidas dos degraus, do controle e das referências; a Tabela 20, as comparações pareadas; e a Tabela 21, a decomposição entre a primeira decisão e a resposta final.

A comparação principal.

O *Tool Calling* v3 acertou 94,4% das consultas (IC 95%: 92,7% a 95,7%), e o controle, a Saída Estruturada com as mesmas descrições e o mesmo validador, 87,8% (IC 95%: 85,6% a 89,8%). 95 consultas foram acertadas só pelo *Tool Calling* e 33, só pela

Tabela 19 – Gemma 4 E4B no lote 2 (teste): os degraus da v3, o controle e as referências v1/v2

Degrau	Configuração	Acurácia	IC 95%	Domínio	F1 recusa	Falsa recusa	E	F1 campos	Latência (s)	Chamadas
0	Tool Calling v1	58,6%	55,5%–61,7%	45,9%	0,562	35,4%	95,4%	0,731	0,85	1,00
1	Tool Calling v2	62,1%	59,0%–65,2%	53,3%	0,991	0,4%	64,6%	0,842	0,91	1,00
2	Tool Calling v3d (descrições v3)	75,2%	72,4%–77,9%	70,3%	0,985	0,7%	67,7%	0,896	0,90	1,01
3	Tool Calling v3a (+ ferramentas auxiliares)	83,1%	80,5%–85,3%	79,8%	0,943	1,8%	86,2%	0,929	1,37	1,82
4	Tool Calling v3 (+ retorno)	94,4%	92,7%–95,7%	94,4%	0,969	0,3%	92,3%	0,981	1,47	2,06
Controle	Saída Estruturada v3 (controle)	87,8%	85,6%–89,8%	84,4%	0,901	5,0%	95,4%	0,946	0,78	1,23
Referência	Saída Estruturada v1	61,3%	58,1%–64,3%	74,8%	0,000	0,0%	66,2%	0,895	0,72	1,00
Referência	Saída Estruturada v2	73,8%	70,9%–76,5%	65,4%	0,738	16,2%	100,0%	0,857	0,61	1,00
Simulação	Duas etapas (TC v2 decide, SE v1 extrai)	78,6%	75,9%–81,1%	74,6%	0,991	0,4%	69,2%	0,900	1,45	1,81
310 (desenv.)	Tool Calling v3 (+ retorno)	97,7%	95,4%–98,9%	98,3%	0,857	0,0%	—	0,993	1,38	2,14
310 (desenv.)	Saída Estruturada v3 (controle)	94,1%	90,9%–96,2%	94,0%	0,615	3,4%	—	0,974	0,77	1,21

Fonte: Elaborado pelos autores. Gemma 4 E4B (`gemma4:e4b-it-qat`), GPU T4, uma repetição, data de referência 2026-09-24. Lote 2 ($n = 945$ consultas; conjunto de teste, nenhuma consulta lida no desenvolvimento): rodadas em `results/lote2`, repontuadas contra `lote_validacao_2.json`, uma execução por consulta; $n = \dots$ ao lado do nome: rodada com menos consultas que o conjunto. Degraus: cada configuração acrescenta uma peça à anterior (0: v1; 1: v2; 2: descrições v3; 3: ferramentas auxiliares e usuário simulado; 4: retorno do validador e recusa contestada). Controle: Saída Estruturada com as descrições v3 e o mesmo retorno, sem as ferramentas no meio da resposta. Duas etapas: simulação *post hoc* com as rodadas do próprio lote 2 (o *Tool Calling* v2 decide se busca; quando busca, valem os parâmetros da Saída Estruturada v1; latência somada). Acurácia por consulta, com IC de Wilson. Domínio: acurácia fora das categorias F e E. F1 recusa: média harmônica da precisão e do recall da recusa em F. Falsa recusa: proporção das consultas do domínio sem busca. E: acurácia nas subespecificadas (buscar sem filtros ou não buscar). F1 campos: F1 ponderado dos campos. Latência: mediana por consulta, com todas as chamadas ao modelo. Chamadas: média de chamadas ao modelo por consulta (v1 e v2: uma por construção; duas etapas: uma ou duas). As linhas 310 (desenv.) são de desenvolvimento: as 310 consultas ($n = 306$; rodadas em `results/v3_310`) orientaram o desenho das descrições, das ferramentas e do validador, e o resultado nelas é de dentro da amostra, otimista por construção.

Saída Estruturada (p -valor $< 0,001$); a diferença de acurácia, Saída Estruturada menos *Tool Calling*, foi de -6,6 p.p. (IC 95%: -8,9 a -4,2). Como o controle recebe as mesmas descrições, o mesmo validador e a mesma recusa contestada, a diferença mede o que só o *Tool Calling* v3 tem: o mecanismo junto com as ferramentas auxiliares, que ele chama no meio da resposta e que também são código do domínio, o usuário simulado, o pedido para responder chamando uma ferramenta, a leitura das chamadas escritas como texto e a instrução do seu *prompt* de que nome de lugar desconhecido não é motivo de recusa, que o *prompt* do controle não traz. O desenho não separa o mecanismo dessas peças. A vantagem vem toda das consultas do domínio, em que o *Tool Calling* acertou 94,4%, contra 84,4%: a Saída Estruturada v3 declarou fora do escopo 5,0% delas (36/717), contra 0,3% (2/717) do *Tool Calling*, e errou mais parâmetros nas que buscou; por isso, a precisão da recusa foi de 98,7% no *Tool Calling* e de 81,9% na Saída Estruturada. Nas outras categorias, a Saída Estruturada acertou mais: recusou todas as consultas fora do domínio, e o *Tool Calling* v3, 95,1% (155/163); nas subespecificadas, acertou 95,4%, contra 92,3%, quase sempre declarando-as fora do escopo (93,8%), o que a pontuação da categoria E aceita, embora o seu *prompt* diga que consulta vaga não é fora do escopo.

Os degraus.

Cada peça da v3 acrescentou acurácia, com diferença significativa sobre o degrau anterior; o passo da v1 para a v2 não foi significativo. Os degraus são cumulativos e foram medidos numa só ordem: cada diferença é o efeito de uma peça dadas as anteriores. Da v1 para a v2, o *Tool Calling* foi de 58,6% para 62,1% (p -valor 0,058), o mesmo quadro do lote 1: a recusa resolvida e os erros de forma introduzidos. As descrições corrigidas desses erros levaram a 75,2% (+13,1 p.p.; p -valor < 0,001): a sigla no lugar do nome da UF, que o *Tool Calling* v2 escreveu em 36,8% das execuções com estado, desapareceu (0,0%), o prefixo no código caiu de 89,8% para 2,0%, e o tipo de produto acrescentado a “cartas”, de 22,6% para 5,2% das buscas sem tipo no gabarito. O degrau inclui também a leitura das chamadas escritas como texto, que a v2 contava como não busca; no *Tool Calling* v2, foram só 12 consultas. O degrau das ferramentas auxiliares, que acrescenta com elas o usuário simulado, o pedido para responder chamando uma ferramenta e um *prompt* que explica quando chamar cada uma, levou a 83,1% (+7,8 p.p.; p -valor < 0,001): o tipo acrescentado caiu para 0,6%, o limite sem pedido de 6,5% para 1,4%, e a acurácia nas subespecificadas subiu de 67,7% para 86,2%. O retorno do validador, por fim, levou a 94,4% (+11,3 p.p.; 119 consultas acertadas só com o retorno, contra 12; p -valor < 0,001).

Tabela 20 – Comparações pareadas no lote 2 (Gemma 4 E4B)

Comparação	Par (A × B)	n	Só A	Só B	p	B – A, acurácia (IC)	B – A, F1 (IC)
Principal	TC v3 × SE v3	945	95	33	< 0,001	-6,6 p.p. (-8,9 a -4,2)	-0,035 (-0,047 a -0,025)
Degraus	TC v1 × TC v2	945	126	159	0,058	+3,5 p.p. (0,0 a +7,0)	0,111 (0,083 a 0,142)
Degraus	TC v2 × TC v3d	945	26	150	< 0,001	+13,1 p.p. (+10,5 a +15,8)	0,053 (0,042 a 0,066)
Degraus	TC v3d × TC v3a	945	55	129	< 0,001	+7,8 p.p. (+5,1 a +10,5)	0,033 (0,018 a 0,047)
Degraus	TC v3a × TC v3	945	12	119	< 0,001	+11,3 p.p. (+9,1 a +13,8)	0,052 (0,042 a 0,064)
Secundárias	Melhor v1/v2 isolada (SE v2) × TC v3	945	29	224	< 0,001	+20,6 p.p. (+17,8 a +23,7)	0,124 (0,107 a 0,143)
Secundárias	Duas etapas × TC v3	945	25	174	< 0,001	+15,8 p.p. (+13,1 a +18,4)	0,081 (0,068 a 0,096)
Secundárias	Duas etapas × SE v3	945	53	140	< 0,001	+9,2 p.p. (+6,6 a +12,1)	0,046 (0,031 a 0,061)
Secundárias	Melhor v1/v2 isolada (SE v2) × SE v3	945	29	162	< 0,001	+14,1 p.p. (+11,4 a +16,9)	0,089 (0,072 a 0,106)
310 (desenv.)	TC v3 × SE v3	306	15	4	0,019	-3,6 p.p. (-6,5 a -1,0)	-0,018 (-0,031 a -0,007)

Fonte: Elaborado pelos autores. Plano de análise fixado antes da rodada (docs/v3.md). Lote 2 (n = 945 consultas; conjunto de teste, nenhuma consulta lida no desenvolvimento): rodadas em **results/lote2**; n : consultas presentes nas duas configurações (as comparações usam só essas). Só A / Só B: consultas em que só uma das configurações acerta. p : McNemar exato, uma observação por consulta. IC: bootstrap pareado (2.000 reamostragens das consultas, semente 42), percentis 2,5 e 97,5; F1: F1 ponderado dos campos. Melhor v1/v2 isolada: a de maior acurácia no lote 2 entre TC v1, SE v1, TC v2 e SE v2 (Saída Estruturada v2). Duas etapas: simulação *post hoc* (TC v2 decide, SE v1 extrai). TC: *Tool Calling*; SE: Saída Estruturada. 310 (desenv.): as 310 consultas de desenvolvimento, dentro da amostra que orientou a v3 (não é resultado de teste).

Contra as referências.

O *Tool Calling* v3 superou a melhor configuração v1 ou v2 isolada no lote 2, a Saída Estruturada v2 (73,8%; +20,6 p.p., IC 95%: +17,8 a +23,7), e a arquitetura em duas etapas simulada com as rodadas do lote 2, que acertou 78,6% (+15,8 p.p., IC 95%: +13,1 a +18,4; 174 consultas acertadas só pelo *Tool Calling* v3, contra 25). A Saída

Estruturada v3 também superou as duas (+14,1 e +9,2 p.p.). A troca entre extração e recusa que as configurações v1 e v2 impunham (Seção 5.4) muda de forma com a v3: o *Tool Calling* v3 acerta mais que a Saída Estruturada v3 nas consultas do domínio e recusa com a precisão da ferramenta de recusa, mas recusa menos consultas fora do domínio — *recall* de 95,1%, contra 100,0% do *Tool Calling* v2, do degrau de uma chamada, do controle e das duas etapas, e F1 da recusa de 0,969, contra 0,991 das duas etapas —, e custa cerca de duas chamadas ao modelo por consulta.

Quanto é do modelo e quanto é do código.

Os degraus são essa medida: só com a especificação — as descrições corrigidas, numa chamada —, o modelo acertou 75,2%; o degrau das ferramentas auxiliares acrescentou +7,8 p.p., e o retorno do validador, +11,3 p.p. A decomposição do plano separa só o retorno do resto. A primeira decisão — a primeira busca ou recusa, antes de qualquer retorno do validador — vem, no *Tool Calling* v3, depois das chamadas às ferramentas auxiliares e das perguntas ao usuário simulado e acertou 83,0% das consultas, praticamente o mesmo que o degrau sem retorno (83,1%); a da Saída Estruturada v3 acertou 74,6%. A primeira decisão do *Tool Calling* v3 não mede, portanto, o modelo sem o código: inclui o que as ferramentas devolvem — UFs e municípios pela lista do IBGE e folhas pelo índice do BDGEx, as mesmas listas de que o lote 2 sorteou nomes e códigos, e datas pela mesma tabela do gabarito. O retorno acrescentou mais ao controle (+13,2 p.p.) que ao *Tool Calling* (+11,4 p.p.): consertou 125 consultas na Saída Estruturada, sem estragar nenhuma, e 112 no *Tool Calling*, em que estragou 4. A vantagem do *Tool Calling* sobre o controle já está, portanto, na primeira decisão, e o retorno a reduz um pouco: numa comparação exploratória, fora do plano, 146 primeiras decisões estavam certas só no *Tool Calling* e 67, só na Saída Estruturada (diferença de -8,4 p.p., Saída Estruturada menos *Tool Calling*; IC 95%: -11,3 a -5,4; p -valor $< 0,001$). Também fora do plano, o degrau de uma chamada, com as mesmas descrições v3 e sem ferramentas auxiliares, ficou perto da primeira decisão do controle (75,2% e 74,6%): o que separa as abordagens aparece no degrau das ferramentas auxiliares. Das consultas que o retorno consertou no *Tool Calling*, 60 (53,6%) receberam do validador o valor a usar — a UF por extenso, o município citado, as datas pela tabela do manual, o código na forma canônica —, 31 (27,7%) só a ordem de retirar um campo sem evidência ou de corrigir a sua forma, 11 a indicação do campo sem o valor e 10 a contestação de uma recusa; na Saída Estruturada, 43,2% das consertadas receberam o valor. Nas que receberam o valor, o acerto é do código que codifica o manual, e não do modelo; nas demais, o modelo fez a correção a partir do retorno. É por isso que a v3 deve ser lida como o conjunto do modelo com esse código.

Tabela 21 – Decomposição do acerto no lote 2: primeira decisão do modelo e correção pelo retorno do código

Medida	TC v3a	TC v3	SE v3	TC v3 (310, desenv.)	SE v3 (310, desenv.)
Consultas com traço	945	945	945	306	306
Acurácia da primeira decisão	83,0%	83,0%	74,6%	90,5%	76,8%
Acurácia final	83,1%	94,4%	87,8%	97,7%	94,1%
Final – primeira (p.p.)	+0,1	+11,4	+13,2	+7,2	+17,3
Consertadas pelo retorno	1	112	125	23	53
dando o valor	0	60	54	13	35
mandando remover um campo	0	19	20	4	6
corrigindo a forma	1	12	4	3	1
apontando o campo, sem o valor	0	11	21	3	6
contestando a recusa	0	10	26	0	5
Estragadas pelo retorno	0	4	0	1	0

Fonte: Elaborado pelos autores. Gemma 4 E4B. Lote 2 (n = 945 consultas; conjunto de teste, nenhuma consulta lida no desenvolvimento): rodadas em **results/lote2**. Primeira decisão: os parâmetros da primeira chamada a **buscar_catalogo**, ou a recusa, se veio antes de qualquer busca (no *Tool Calling*, pelo traço gravado; na Saída Estruturada, o primeiro objeto do histórico de tentativas), pontuada com o mesmo gabarito da resposta final. Consertadas: primeira decisão errada e final certa; estragadas: o contrário. Cada consertada é atribuída ao retorno mais informativo que recebeu antes da decisão final, pelas mensagens gravadas do validador: dando o valor (a UF ou o município citados, as datas pela tabela do manual, o código ou a escala na forma canônica, o município do IBGE); mandando remover um campo sem evidência na consulta; corrigindo a forma (valor fora do enumerado, período malformado, objeto fora do *schema*); apontando o campo a preencher ou a trocar, sem o valor; contestando a recusa de uma consulta que cita critério do catálogo. No TC v3a não há validação: o retorno é só o da chamada malformada. Colunas 310: consultas de desenvolvimento, dentro da amostra que orientou a v3 (não é resultado de teste).

As chamadas escritas como texto.

Em 236 consultas (25,0%), o Gemma escreveu ao menos uma chamada de ferramenta no texto da resposta — na sintaxe de função, em JSON ou no formato nativo do Gemma —, em vez de emití-la pelo canal de ferramentas, e a v3 a interpretou como chamada; numa contagem exploratória, 230 delas (97,5%) terminaram certas. Isso ocorreu em 235 consultas no degrau das ferramentas auxiliares e em 7 no de uma chamada só; nas configurações v1 e v2, a mesma resposta conta como não busca. Quando o modelo responde só com texto, sem chamada e sem pergunta, o laço da v3 lhe envia, uma vez, o pedido “Responda chamando uma ferramenta”. As duas coisas são parte do sistema medido, e não um detalhe de implementação: a leitura depende do formato do Gemma e da versão do Ollama e precisa ir para a produção junto com a v3. Quanto a v3 acertaria sem ela não se sabe, porque o diálogo seria outro.

O custo.

O *Tool Calling* v3 fez, em média, 2,06 chamadas ao modelo por consulta, e sua latência mediana na T4 foi de 1,47 s (percentil 95 de 4,96 s), contra 0,85 s do *Tool Calling* v1, 0,78 s da Saída Estruturada v3 e 1,45 s das duas etapas. O modelo perguntou ao usuário simulado em 133 consultas (14,1%), 81 delas por texto, mais que as 65 subespecificadas,

as únicas sem nenhum critério, a que o *prompt* reserva a pergunta. O usuário simulado é uma simplificação: responde sempre que não tem outros detalhes e que a busca pode seguir com o que foi dito, e um usuário real poderia acrescentar critérios ou confirmar que o pedido não é do acervo. A recusa contestada, a regra que devolve ao modelo a recusa de uma consulta com critério do catálogo, agiu em 20 consultas: nas do domínio, 10 terminaram em busca correta; nas fora do domínio, 7 tiveram a recusa confirmada e 3 viraram busca. Esse é o custo da regra, e uma parte da queda do *recall* da recusa; a outra parte já aparece no degrau das ferramentas auxiliares (96,3% de recusa, contra 100,0% no degrau anterior), que acrescenta também o usuário simulado, cuja resposta diz que a busca pode seguir, e a instrução de conferir com `identificar_nome` o lugar que o modelo não reconhece; o desenho não separa essas causas.

Desenvolvimento e teste (exploratório).

Nas consultas de desenvolvimento, a configuração congelada acertara 155 das 160 (Seção 3.6); nas 310, que também orientaram o desenho e dão um resultado de dentro da amostra, acertou 97,7%, contra 94,1% do controle (15 consultas acertadas só pelo *Tool Calling*, contra 4; *p*-valor 0,019); no lote 2, 94,4%. Os conjuntos têm composições diferentes — as 310 têm 8 consultas fora do domínio e nenhuma subespecificada, e a amostra de desenvolvimento é estratificada —, e a comparação menos afetada pela composição é a do domínio: o *Tool Calling* v3 foi de 98,3% nas 310 a 94,4% no lote 2, e o controle, de 94,0% a 84,4%. A queda existe, como se esperava de regras depuradas sobre os erros do desenvolvimento, e foi maior no controle. Nas 310 e no lote 2, o *Tool Calling* v3 ficou acima do controle, e os dois acima das duas etapas e de todas as configurações v1 e v2; a ordem entre estas, porém, mudou: nas 310, a melhor configuração isolada foi a Saída Estruturada v1 (75,2%), e o *Tool Calling* v2 (47,4%) ficou abaixo do v1 (60,1%); no lote 2, a melhor foi a Saída Estruturada v2 (73,8%). Os degraus intermediários não foram executados no desenvolvimento.

O que o *Tool Calling* v3 ainda erra (exploratório).

Fora do plano, a leitura dos erros do *Tool Calling* v3, feita depois do teste e sem alterar a v3, mostra três grupos. O maior é a *keyword* preenchida com o que não é nome de carta: a região como único lugar (“cartas do Nordeste”, “do Pantanal”), nas consultas subespecificadas; o nome do projeto repetido na *keyword*; a escala escrita como *keyword* (“100k”); e o município em *keyword* no lugar de *city*. O validador não confere se a *keyword* é região, projeto ou escala, e esses erros passaram sem aviso. O segundo são as consultas fora do domínio em que o modelo buscou (recusou 155/163); em 3 delas, a busca veio depois de a recusa contestada devolver ao modelo uma recusa correta, e o erro é custo de uma regra do validador, e não lacuna dele. O terceiro, os períodos e os lugares:

expressões de tempo que a tabela do manual não cobre e estados ou municípios omitidos. Em parte dos erros, o validador avisou e o modelo repetiu os mesmos parâmetros, o que, por desenho, confirma a busca. As lacunas são corrigíveis por código da mesma natureza, e a contestação pode ser ajustada, mas fazê-lo a partir destes erros transformaria o lote 2 em desenvolvimento; ficam como trabalho futuro, a medir num conjunto novo.

5.6 Tool Calling × Saída Estruturada: o Teste A/B

As seções anteriores compararam as duas abordagens em cada conjunto. Esta reúne essas evidências num teste A/B, sem rodada nova: a Tabela 22 traz os 19 pares de rodadas em que o mesmo modelo executou os dois modos sobre as mesmas consultas, com a mesma especificação e no mesmo ambiente. No **Modo A**, o *Tool Calling* nativo, a definição das ferramentas em JSON Schema é ligada ao modelo pelo `bind_tools()` do LangChain, e o modelo decide, pelo canal de *tool calls* do Ollama ou, na nuvem, do Groq, se chama cada ferramenta e com quais argumentos (Seção 4.4); na v3, a chamada escrita no texto da resposta também é lida como chamada (Seção 5.5). No **Modo B**, a Saída Estruturada, o JSON Schema dos mesmos parâmetros vai no *prompt*, o Ollama (ou o Groq) responde no modo JSON, e o objeto devolvido é validado pelo mesmo modelo Pydantic que valida os argumentos da *tool call* no Modo A (`ParametrosBusca`, em `schema.py`). O modelo Pydantic valida a resposta, mas não restringe a geração: a decodificação restrita ao *schema* foi testada e descartada (Seção 3.7.4). Na v2 e na v3, o Modo B tem também o campo `fora_do_escopo`, e o controle da v3 recebe o retorno do validador como nova tentativa. Nas 310, em que o *Tool Calling* v1 tem três repetições, cada consulta vale pela maioria delas, como no teste de McNemar; por isso, a acurácia do Gemma 4 E4B com *Tool Calling* na tabela, 60,1%, difere ligeiramente da Tabela 14 (60,2%), que usa a média das execuções.

Acurácia.

Pelo teste de McNemar, a Saída Estruturada foi melhor em 6 dos 19 pares, o *Tool Calling* em 5, e os dois empataram nos demais (fora do desenvolvimento, 5 e 2 dos 15); qual modo vence depende da especificação e, na v1, também do modelo. Numa só chamada, com a v1, a Saída Estruturada extrai melhor, e o *Tool Calling* recusa: nos modelos locais, ela foi melhor em três dos quatro nas 310 (Seção 5.3) e empatou nos dois lotes, em que acertou 75,5% das consultas do domínio no lote 1 e 74,8% no lote 2, contra 48,9% e 45,9% do *Tool Calling*; no lote 1, essa diferença vem sobretudo das falsas recusas do *Tool Calling* e é compensada pelas consultas fora do domínio, que a Saída Estruturada v1 não tem como recusar (Seção 5.4). Na estação de referência, com a v1, os dois modos empataram nos três modelos medidos; nos modelos de maior porte, em nuvem, o *Tool Calling* v1 foi

Tabela 22 – Teste A/B: *Tool Calling* (A) e Saída Estruturada (B) com o mesmo modelo, as mesmas consultas, a mesma especificação e o mesmo ambiente

Conjunto	Modelo	Espec.	n	Acurácia (%) TC / SE	SE – TC, p.p. (IC 95%)	p	Domínio (%) TC / SE	F1 recusa TC / SE	Latência (s) TC / SE	p95 (s) TC / SE	Chamadas TC / SE
310 T4	Gemma 4 E2B	v1	306	56,5 / 53,9	-2,6 (-8,8 a +4,2)	0,488	56,4 / 55,4	0,238 / 0,000	0,40 / 0,47	0,70 / 0,86	1,00 / 1,00
	Qwen 3 4B	v1	306	54,2 / 60,1	+5,9 (+0,7 a +10,8)	0,038	53,0 / 61,7	0,552 / 0,000	0,83 / 0,69	1,40 / 1,97	1,00 / 1,00
	Gemma 4 E4B	v1	306	60,1 / 75,2	+15,0 (+9,8 a +20,6)	< 0,001	59,1 / 77,2	0,222 / 0,000	0,72 / 0,76	1,23 / 1,60	1,00 / 1,00
	Mistral Nemo 12B	v1	306	26,8 / 52,0	+25,2 (+18,3 a +32,0)	< 0,001	24,8 / 53,4	0,079 / 0,000	2,39 / 1,00	5,86 / 3,39	1,00 / 1,00
310 (P e N) Estação	Gemma 4 E2B	v1	59	47,5 / 45,8	-1,7 (-15,3 a +11,9)	1,000	45,6 / 47,4	0,333 / 0,000	0,71 / 0,76	1,20 / 1,55	1,00 / 1,00
	Qwen 3 4B	v1	59	49,2 / 50,8	+1,7 (-13,6 a +16,9)	1,000	47,4 / 52,6	0,400 / 0,000	2,60 / 1,97	5,70 / 5,12	1,00 / 1,00
	Gemma 4 E4B	v1	59	52,5 / 61,0	+8,5 (-3,4 a +20,3)	0,302	50,9 / 63,2	0,222 / 0,000	3,42 / 3,29	6,63 / 7,13	1,00 / 1,00
310 (P e N) Estação	Gemma 4 E2B	v3 [†]	59	78,0 / 55,9	-22,0 (-35,6 a -6,8)	0,007	78,9 / 54,4	0,250 / 0,222	1,75 / 0,80	4,62 / 2,17	2,46 / 1,59
	Gemma 4 E4B	v3 [†]	59	98,3 / 84,7	-13,6 (-22,0 a -5,1)	0,008	98,2 / 84,2	1,000 / 0,444	5,63 / 3,38	15,43 / 13,08	2,17 / 1,24
310 Groq	GPT-OSS 20B	v1	306	89,2 / 77,5	-11,8 (-16,7 a -7,2)	< 0,001	88,9 / 79,5	0,889 / 0,000	— / —	— / —	1,00 / 1,00
	Qwen 3.8 27B	v1	293*	94,9 / 94,5	-0,3 (-2,4 a +1,7)	1,000	95,2 / 95,2	0,667 / 0,000	— / —	— / —	1,00 / 1,00
	GPT-OSS 120B	v1	306	89,9 / 87,3	-2,6 (-6,5 a +1,3)	0,268	89,6 / 89,6	0,727 / 0,000	— / —	— / —	1,00 / 1,00
310 T4	Gemma 4 E4B	v2 [†]	306	47,4 / 68,0	+20,6 (+14,1 a +27,1)	< 0,001	46,0 / 67,1	1,000 / 0,271	0,81 / 0,71	1,31 / 1,47	1,00 / 1,00
	Gemma 4 E4B	v3 [†]	306	97,7 / 94,1	-3,6 (-6,5 a -1,0)	0,019	98,3 / 94,0	0,857 / 0,615	1,38 / 0,77	2,66 / 2,67	2,14 / 1,21
Lote 1 T4	Gemma 4 E4B	v1	1.929	61,5 / 62,2	+0,7 (-2,4 a +3,8)	0,656	48,9 / 75,5	0,592 / 0,000	0,86 / 0,72	1,48 / 1,45	1,00 / 1,00
	Gemma 4 E4B	v2	1.929	62,8 / 72,4	+9,6 (+7,4 a +11,8)	< 0,001	53,4 / 63,2	0,980 / 0,723	0,94 / 0,61	1,45 / 1,48	1,00 / 1,00
Lote 2 T4	Gemma 4 E4B	v1	945	58,6 / 61,3	+2,6 (-1,6 a +6,9)	0,255	45,9 / 74,8	0,562 / 0,000	0,85 / 0,72	1,41 / 1,44	1,00 / 1,00
	Gemma 4 E4B	v2	945	62,1 / 73,8	+11,6 (+7,9 a +14,9)	< 0,001	53,3 / 65,4	0,991 / 0,738	0,91 / 0,61	1,40 / 1,47	1,00 / 1,00
	Gemma 4 E4B	v3	945	94,4 / 87,8	-6,6 (-8,9 a -4,2)	< 0,001	94,4 / 84,4	0,969 / 0,901	1,47 / 0,78	4,96 / 2,55	2,06 / 1,23

Fonte: Elaborado pelos autores. Cada linha é um par de rodadas do mesmo modelo, sobre as mesmas consultas, com a mesma especificação e no mesmo ambiente: A, *Tool Calling* (TC); B, Saída Estruturada (SE); na v3, o TC completo (descrições, ferramentas auxiliares e retorno do validador) e o controle SE v3, com as mesmas descrições e o mesmo retorno. Linhas: 310, T4: rodadas completas dos modelos locais na GPU T4 (**results/<modelo>/** e **results/se-<modelo>/**), *Tool Calling* com três repetições e Saída Estruturada com uma; 310 (P e N), estação: estação de referência (RTX 3050 Laptop, 4096 MiB), camadas P e N (**results/estacao/**); 310, Groq: referência em nuvem (**results/groq-<modelo>/** e **results/groq-se-<modelo>/**); Lote 1: lote de validação (1.929 consultas pontuadas; **results/lote/**), independente do desenho da v2; Lote 2: conjunto de teste da v3 (945 consultas pontuadas; **results/lote2/**), nenhuma delas lida no desenvolvimento; [†] desenvolvimento: as 310 (P e N) consultas (**results/estacao/**) orientaram o desenho da v3, e o resultado nelas é de dentro da amostra, otimista por construção; [†] desenvolvimento: as 310 consultas (**results/v2_310/** e **results/v3_310/**) orientaram o desenho da v2 e da v3, e o resultado nelas é de dentro da amostra, otimista por construção; * Qwen 3.8 27B (Groq): rodada de Saída Estruturada incompleta (293 das 306 consultas pontuadas com *Tool Calling*); a linha usa só as 293 presentes nas duas. Acurácia por consulta; com repetições, cada consulta vale pela maioria delas. SE – TC: diferença de acurácia, com IC 95% por *bootstrap* pareado (2.000 reamostragens das consultas, semente 42). *p*: McNemar exato, uma observação por consulta. Domínio: acurácia fora das categorias F e E. F1 recusa: média harmônica da precisão e do *recall* da recusa nas consultas fora do domínio (na v1, a SE não tem como recusar). Latência: mediana e percentil 95 da tradução, por execução e com todas as chamadas ao modelo, só nos pares cujas duas rodadas têm, nos manifestos, a mesma GPU, a mesma versão do Ollama e a mesma parcela do modelo na GPU (em quatro deles, TC e SE rodaram em sessões diferentes, com a mesma GPU); —: nuvem, com *hardware* do provedor, não controlado. Chamadas: média de chamadas ao modelo por consulta (v1 e v2: uma, por construção).

melhor no GPT-OSS 20B e empatou nos outros dois. Com a v2, os dois modos podem recusar, e o *Tool Calling* recusa muito melhor — F1 da recusa de 0,980, contra 0,723, no lote 1, e de 0,991, contra 0,738, no lote 2 —, mas a Saída Estruturada extrai melhor e venceu nos três conjuntos, inclusive nas 310, de dentro da amostra (+9,6 p.p. no lote 1 e +11,6 p.p. no lote 2, Saída Estruturada menos *Tool Calling*), numa comparação que mede o mecanismo junto com o texto que enquadra a recusa, diferente nos dois modos (Seção 5.4). Com a v3, o sinal se inverte: o *Tool Calling* acertou 94,4% das consultas do lote 2, contra 87,8% do controle (-6,6 p.p., IC 95%: -8,9 a -4,2), com o mesmo sinal nas 310, de dentro da amostra (*p*-valor 0,019), e nas 62 consultas P e N executadas na estação de referência, também de dentro da amostra, com o Gemma 4 E4B e com o Gemma 4 E2B; com este, o *Tool Calling* v3 acertou 78,0% delas, contra 55,9% do controle e 47,5%

do *Tool Calling* v1, primeira indicação de que a v3 não depende do modelo em que foi desenvolvida. O controle recebe as mesmas descrições e o mesmo validador, mas não as ferramentas auxiliares nem as outras peças que só o laço do *Tool Calling* tem, e o desenho não separa o mecanismo dessas peças (Seção 5.5). A evidência de ganho do *Tool Calling* na acurácia é, portanto, condicional: numa chamada, com os modelos locais, ele não extrai melhor que a Saída Estruturada; com o que só o laço da v3 lhe dá — as ferramentas auxiliares, chamadas no meio da resposta, e o código do domínio que elas trazem —, ele a supera.

Tempo.

A latência só entra na tabela quando as duas rodadas do par têm, nos manifestos, a mesma GPU, a mesma versão do Ollama e a mesma parcela do modelo na GPU; no Groq, cujo *hardware* não é controlado, fica de fora. Numa chamada, os dois modos têm o mesmo custo em ordem de grandeza: uma chamada ao modelo por consulta e, na T4, medianas abaixo de um segundo nos modelos de até 4 bilhões de parâmetros efetivos, com diferenças de décimos de segundo, em geral a favor da Saída Estruturada (a mais rápida em todos os pares da T4 cujas duas rodadas foram feitas na mesma sessão; o *Tool Calling* só foi mais rápido em pares das 310 com as rodadas em sessões diferentes e no Gemma 4 E2B da estação). A exceção é o Mistral Nemo 12B, com mediana de 2,39 s com *Tool Calling*, contra 1,00 s com Saída Estruturada: com a ferramenta, ele deixou de emitir a chamada na maior parte das consultas e respondeu em texto, em geral descrevendo a chamada (Seção 5.9), com respostas mais longas. Com a v3, o ganho de acurácia tem custo: o *Tool Calling* fez 2,06 chamadas ao modelo por consulta no lote 2, contra 1,23 do controle, e a sua mediana foi 1,89 vezes a do controle (1,47 s contra 0,78 s). Na estação de referência, com 4 GB de VRAM e só parte do modelo na GPU, a v3 teve mediana de 1,75 s com o Gemma 4 E2B (percentil 95 de 4,62 s), contra 0,80 s do controle e 0,71 s do *Tool Calling* v1; com o Gemma 4 E4B, de que coube na GPU uma parcela menor, a mediana foi de 5,63 s (percentil 95 de 15,43 s), contra 3,42 s com a v1. Essas rodadas foram feitas com a GPU da estação compartilhada com outros programas, cuja ocupação o manifesto registra.

O que a acurácia não mede.

Três diferenças a favor do *Tool Calling* ficam fora da pontuação ou só entram nela junto com outras peças. A primeira é a mensagem de recusa: a consulta fora do domínio recebe do modelo uma resposta ao usuário — na v1, uma frase sobre por que o pedido está fora do escopo (Seção 4.4); na v2 e na v3, o argumento obrigatório `motivo` da ferramenta `recusar_consulta`, uma frase dirigida ao usuário —, enquanto a Saída Estruturada v1 não tem como recusar, e as v2 e v3 devolvem só o campo `fora_do_escopo`,

sem nada a dizer ao usuário. A segunda é a pergunta ao usuário: na v3, a ferramenta `pedir_esclarecimento` dá ao modelo um canal para perguntar, separado da busca e da recusa, que a Saída Estruturada não tem; ele foi exercitado só com o usuário simulado, de resposta fixa, e usado além das consultas subespecificadas (Seção 5.5); numa chamada, a mesma liberdade apareceu como falsa recusa, com pedidos de esclarecimento em consultas que bastavam para buscar. A terceira é chamar ferramentas no meio da resposta: o modelo consulta as ferramentas auxiliares quando precisa e recebe o retorno do validador como resultado da própria chamada de busca, com cada passo registrado no traço da execução, enquanto a Saída Estruturada só recebe o retorno como nova tentativa, depois de uma resposta completa. A terceira só aparece na acurácia da v3, e junto com o código das ferramentas; o conteúdo da mensagem e o da pergunta não são pontuados, e medir o seu valor para o usuário exige o estudo com usuários apontado no Capítulo 6.

5.7 Análise por Categoria de Consulta

A Tabela 23 apresenta o F1 ponderado por categoria, e a Figura 7, a acurácia por consulta. A categoria F aparece só na figura: como o gabarito dessas consultas não tem campos, a acurácia nela é a taxa de recusa correta.

Tabela 23 – F1-score por categoria de consulta e por modelo

Categoria	Qwen 3 4B	Gemma 4 E4B	Gemma 4 E2B	Mistral Nemo 12B
Simples (n=100)	0,691	0,874	0,844	0,443
Compostas (n=180)	0,654	0,794	0,763	0,431
Código MI/INOM (n=57)	0,949	0,878	0,929	0,362
Tempo relativo (n=60)	0,725	0,547	0,645	0,450
Ordenação (n=29)	0,775	0,650	0,691	0,530
Ambíguas/informais (n=197)	0,640	0,825	0,744	0,429
Média ponderada	0,666	0,793	0,765	0,451

Fonte: Elaborado pelos autores. Uma consulta pode pertencer a mais de uma categoria; n = consultas distintas da categoria nas métricas principais (cada uma pontuada em todas as repetições). As consultas fora do domínio (categoria F), cujo gabarito é não chamar a ferramenta, não têm campos e aparecem na Figura de acurácia por categoria.

O que é fácil.

Entre as categorias com campos, as consultas simples e as com código MI/INOM tiveram as maiores acurácias nos três modelos do patamar superior: nas simples, 79,0% no Gemma 4 E2B, 77,3% no Gemma 4 E4B e 74,0% no Qwen 3 4B; nas com código, 91,2%, 78,9% e 77,2%. Na categoria F, o Qwen 3 4B, o Gemma 4 E4B e o Mistral Nemo 12B recusaram todas as oito consultas fora do domínio, o que confirma que o *Tool Calling* permite ao modelo decidir não buscar, algo que a Saída Estruturada do protótipo não permitia; no Mistral Nemo 12B, que deixou de chamar a ferramenta também na maior

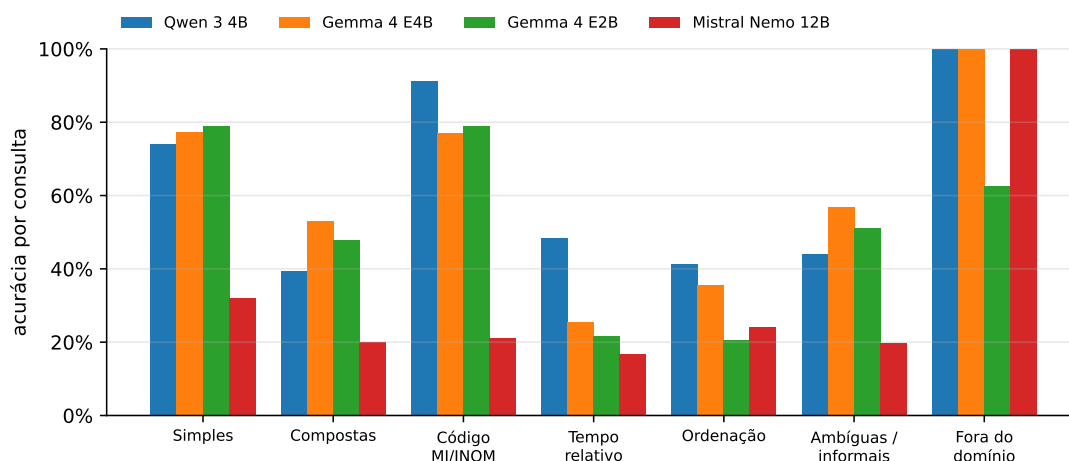


Figura 7 – Acurácia por consulta, por categoria e por modelo, nas rodadas completas

Fonte: Elaborado pelos autores.

parte das consultas legítimas (Seção 5.9), esse acerto é pouco informativo. O Gemma 4 E2B acertou 62,5% delas: em três, chamou a ferramenta em todas as repetições, buscando “Marte” como **keyword** em “cartas de Marte”, “Argentina” como **state** em “cartas da Argentina” e a própria frase como **keyword** em “mapa do tesouro pirata”.

O que é difícil para todos.

As consultas compostas, de tempo relativo e de ordenação concentraram as menores acurácias. Nos dois modelos Gemma, tempo relativo e ordenação foram as categorias mais difíceis (25,6% e 35,6% no E4B; 21,7% e 20,7% no E2B). O Qwen 3 4B teve o melhor desempenho nessas duas categorias (48,3% e 41,4%) e, entre os três, a menor acurácia nas compostas (39,4%); nas compostas, que exigem vários campos corretos ao mesmo tempo, nenhum modelo passou de 53,1%. Essas quedas indicam dificuldades da tarefa, e não de um modelo específico: resolver uma expressão temporal em datas ISO e decidir se, e por qual data, ordenar são as etapas em que a tradução mais depende de raciocínio além do reconhecimento de nomes.

Onde os modelos divergem.

Nas consultas com código MI ou INOM, o Qwen 3 4B foi o mais consistente (91,2%, contra 78,9% do Gemma 4 E2B e 77,2% do Gemma 4 E4B). Nas consultas compostas e nas ambíguas e informais, o Gemma 4 E4B teve as maiores acurácias (53,1% e 56,9%).

Por camada de origem.

A Tabela 24 desagrega a acurácia pelas camadas do *dataset*. Nos três modelos do patamar superior, a acurácia nas consultas do protótipo (camada P) foi de 45,5% (Qwen 3 4B), 37,9% (Gemma 4 E4B) e 40,9% (Gemma 4 E2B); nas redigidas pelos autores (camada N), de 51,4%, 55,9% e 51,4%; e nas geradas (camada G, com 247 consultas pontuadas), de 55,5%, 62,9% e 58,7%. As consultas do protótipo, as mais longas e com mais campos por consulta, foram as mais difíceis para os modelos locais.

Tabela 24 – Acurácia por camada de origem das consultas

Camada	Consultas	Qwen 3 4B	Gemma 4 E4B	Gemma 4 E2B	Mistral Nemo 12B
Protótipo (P)	22	45,5%	37,9%	40,9%	9,1%
Autores (N)	37	51,4%	55,9%	51,4%	29,7%
Geradas (G)	247	55,5%	62,9%	58,7%	27,9%

Fonte: Elaborado pelos autores. Consultas distintas das métricas principais; acurácia sobre todas as repetições.

5.8 Análise por Campo do *Schema*

A Tabela 25 e a Figura 8 apresentam o F1 por campo do *schema*.

Tabela 25 – F1-score por campo do *schema* de busca, por modelo

Campo	Ocorr.	Qwen 3 4B	Gemma 4 E4B	Gemma 4 E2B	Mistral Nemo 12B
keyword	186	0,776	0,883	0,713	0,430
scale	252	0,526	0,875	0,841	0,415
productType	132	0,684	0,635	0,871	0,554
state	390	0,818	0,920	0,795	0,489
city	81	0,412	1,000	0,892	0,667
supplyArea	120	0,950	0,897	0,947	0,431
project	33	0,541	0,806	0,909	0,000
publicationPeriod	153	0,693	0,424	0,548	0,455
creationPeriod	30	0,625	0,378	0,429	0,462
sortField	87	0,400	0,641	0,513	0,377
sortDirection	87	0,400	0,641	0,651	0,314
limit	42	0,358	0,794	0,824	0,444

Fonte: Elaborado pelos autores. Ocorr. = ocorrências do campo no gabarito das execuções pontuadas (consultas × repetições).

O campo `supplyArea` teve F1 alto nos três modelos do patamar superior (0,950 no Qwen 3 4B, 0,897 no Gemma 4 E4B e 0,947 no Gemma 4 E2B). Nos demais campos de identificação, os modelos Gemma foram mais consistentes: o E4B teve F1 de 1,000 em `city` e de 0,883 em `keyword`, e o E2B, 0,909 em `project`, enquanto o Qwen 3 4B ficou em 0,412 em `city` e 0,541 em `project`. São campos em que a tarefa se reduz a reconhecer e normalizar um nome, justamente o que o dicionário fixo do protótipo fazia; aqui, a normalização foi feita pelo próprio modelo, a partir das descrições da ferramenta.

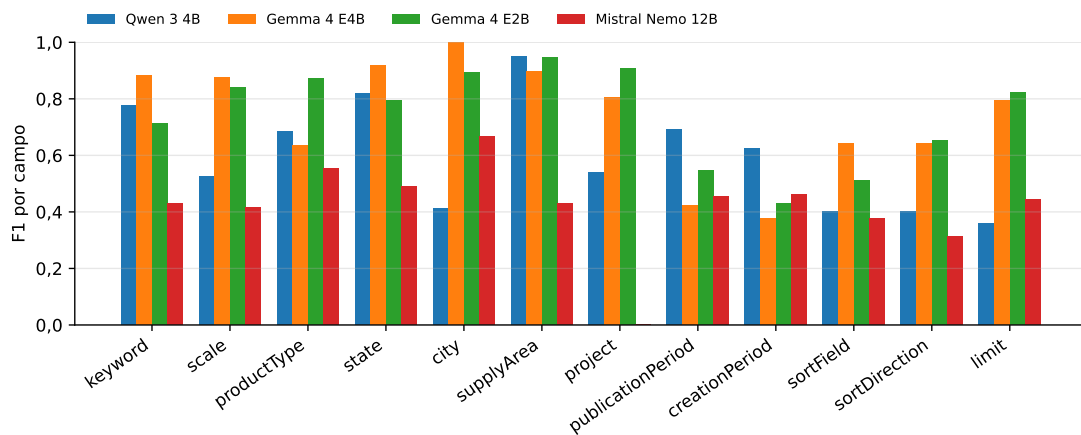


Figura 8 – F1-score por campo do *schema* de busca, por modelo, nas rodadas completas

Fonte: Elaborado pelos autores.

Os campos de período e de ordenação concentraram as perdas. O F1 de `publicationPeriod` ficou entre 0,424 (Gemma 4 E4B) e 0,693 (Qwen 3 4B), e os de `sortField` e `sortDirection` não passaram de 0,651 em nenhum dos quatro modelos. O campo `creationPeriod` aparece em poucas consultas, o que torna seu F1 instável.

5.9 Análise de Erros

A classificação dos erros (Seção 3.7.5) mostra um modo de falha dominante diferente em cada modelo. Os exemplos citados são da primeira repetição e podem ser conferidos nos registros de execução de cada modelo (`results/<modelo>/execucoes.jsonl`); o arquivo `results/consolidado/erros_representativos.md` do repositório traz uma amostra aleatória de até quatro erros de cada tipo por modelo.

Mistral Nemo 12B: a chamada descrita em texto.

O Mistral Nemo 12B deixou de chamar a ferramenta em 561 das 918 execuções pontuadas. Em 357 delas, a resposta cita a ferramenta `buscar_catalogo` e descreve, em texto corrido, a chamada que faria. Para “preciso da carta MI 2965-2-NE do segundo cgeo”, por exemplo, o modelo respondeu que chamaria a ferramenta com `keyword` “2965-2-NE” e `supplyArea` “2º Centro de Geoinformação”, mas não emitiu a chamada estruturada. Em outras 117, pediu mais informações ao usuário ou descreveu os filtros sem chamar a ferramenta. Nos casos de chamada descrita em texto, o modelo identifica a ferramenta e os parâmetros, mas não emite a chamada no canal de `tool calls` da integração com o Ollama; como o protocolo das configurações v1 e v2 não interpreta texto livre como chamada, cada um deles conta como falha (a v3 passa a interpretá-lo; Seção 5.5).

Gemma 4 E4B e E2B: o pedido de esclarecimento.

Os dois modelos Gemma deixaram de chamar a ferramenta em 161 (E4B) e 87 (E2B) execuções, quase sempre pedindo ao usuário que especificasse melhor a busca — 139 e 72 casos, respectivamente. No Gemma 4 E2B, isso ocorreu sobretudo em consultas amplas, com um único critério, como “cartas topográficas” e “produtos em escala 1:50.000”. O Gemma 4 E4B fez o mesmo em consultas amplas, como “ortoimagens disponíveis” e “mapas de pe”, e também em consultas com tempo relativo, em que pediu a data ou o ano apesar de a data de referência constar do *prompt* (“produtos da semana passada do terceiro cgeo”, “ortoimagens de 2 anos atrás”). O Gemma 4 E2B cometeu ainda um erro próprio, o campo acrescentado mais frequente nas suas respostas: copiar para **keyword** palavras comuns da consulta, como “mdt” em “mdt do para em 25k” ou “mapeamento detalhado” em “mapeamento detalhado (25k ou 50k) de brasil”.

Qwen 3 4B: a recusa indevida e os campos acrescentados.

O Qwen 3 4B chamou a ferramenta com mais regularidade, mas declarou fora do escopo 27 execuções de consultas legítimas, como “primeiro mapeamento feito em cuiaba” e “ultima atualizacao do para”. Seu erro mais frequente, porém, foi acrescentar campos não pedidos, sobretudo ordenação e limite (**sortField**, **sortDirection** e **limit** são os campos que ele mais acrescentou sem que a consulta os pedisse): para “todas as cartas em escala maior que 100k do rio grande do sul”, emitiu ordenação decrescente por data de publicação e limite de 100 resultados, e omitiu a escala.

Datas relativas.

Os erros em períodos seguem padrões comuns aos modelos: tratar “esse ano” como começando na data de referência, e não em 1º de janeiro; reduzir “2 anos atrás” a um único dia; e trocar o trimestre-calendário por três meses corridos, ou pelo trimestre errado, em “trimestre passado” e “último trimestre do ano passado”. A Tabela 25 mostra o efeito desses erros no F1 dos campos de período.

Os tipos de erro têm, assim, remédios diferentes. A chamada descrita em texto do Mistral Nemo é uma limitação da integração do modelo com o canal de ferramentas. O pedido de esclarecimento dos modelos Gemma e a recusa indevida do Qwen são decisões de chamar ou não, sensíveis à redação do *prompt*. Os erros de período e de ordenação são erros de raciocínio sobre a consulta. Nenhum deles decorre de valores fora do *schema*: não houve, em nenhum modelo, argumento com campo inexistente ou tipo inválido.

5.9.1 Casos observacionais

As quatro consultas observacionais testam o comportamento diante de pedidos atípicos: duas trazem um valor que a ferramenta não consegue representar, e duas têm resposta discutível. O Qwen 3 4B foi o único que chamou a ferramenta nas duas consultas com valor impossível, em 6 das 12 execuções observacionais: para “escala 1:10.000.000”, emitiu a escala 1:10.000, que existe no enumerado mas não corresponde ao pedido; para “cartas do estado 42”, emitiu `state` “Rio Grande do Sul”, um valor sem nenhum apoio no texto. Os demais modelos não chamaram a ferramenta nesses dois casos. Nas consultas de resposta discutível — “cartas topo no oceano atlântico” e “cartas do futuro (ano 2050)” —, os modelos em geral também não chamaram a ferramenta; o Gemma 4 E2B buscou cartas topográficas sem restrição de área na primeira delas.

5.10 Validação na Estação de Referência

As 62 consultas das camadas P e N foram executadas na estação de referência pelos dois modelos de menor porte, Qwen 3 4B e Gemma 4 E2B, com uma repetição e a mesma data de referência das rodadas completas. As duas rodadas foram feitas em sequência, na mesma sessão da estação, e começaram sem outro modelo residente no Ollama — antes da rodada do Gemma 4 E2B, o *pipeline* descarregou os modelos residentes —, de modo que cada modelo foi carregado a frio, com a VRAM que a estação tinha livre (Tabela 26). Com 4 GB de VRAM, nenhum dos dois coube inteiro na GPU: o Ollama alocou na VRAM 66% do Qwen 3 4B carregado, contando pesos e contexto, e 38% do Gemma 4 E2B; o restante executou na CPU.

As Tabelas 27 e 28 e as Figuras 9 e 10 resumem a validação na estação.

Tabela 26 – Ambiente de execução de cada rodada — estação de referência (camadas P e N)

Modelo	GPU / máquina	Servidor	Rep.	Exec.	Na GPU	VRAM livre
Qwen 3 4B	RTX 3050 Laptop, 4096 MiB	Ollama 0.34.0	1	62	66%	3,0 GB
Gemma 4 E2B	RTX 3050 Laptop, 4096 MiB	Ollama 0.34.0	1	62	38%	3,1 GB

Fonte: Elaborado pelos autores a partir de `results/<modelo>/manifesto.json`. Na GPU: parcela do modelo carregado (pesos e contexto) alocada na VRAM pelo Ollama, medida após o aquecimento; VRAM livre: no início da rodada, com os demais modelos descarregados. Data de referência: 2026-09-14. Python 3.14.4; langchain-core 1.6.4, langchain-ollama 1.1.0.

Qualidade.

Nas 59 consultas pontuadas, os dois modelos tiveram acurácias estatisticamente indistinguíveis: 49,2% para o Qwen 3 4B (IC 95%: 36,8% a 61,6%) e 47,5% para o Gemma 4 E2B (IC 95%: 35,3% a 60,0%). Entre as consultas acertadas por apenas um deles, 8 foram do Qwen e 7 do Gemma, com *p*-valor de 1,000 no teste de McNemar exato (Tabela 29).

Tabela 27 – Comparativo de desempenho entre os modelos avaliados — estação de referência (camadas P e N)

Modelo	Acurácia	IC 95%	Precisão	Recall	F1	Lat. med. (ms)
Qwen 3 4B	49,2%	36,8%–61,6%	0,741	0,762	0,733	2596
Gemma 4 E2B	47,5%	35,3%–60,0%	0,795	0,722	0,719	709

Fonte: Elaborado pelos autores. IC 95%: intervalo de confiança de Wilson para a acurácia, com n igual ao número de consultas, pois as repetições de uma mesma consulta não são independentes. Precisão, *recall* e F1 ponderados pela frequência de cada campo no gabarito. Latência mediana da chamada de tradução, no ambiente de cada rodada.

Tabela 28 – Latência da tradução, em milissegundos, por modelo — estação de referência (camadas P e N)

Modelo	n	Mín.	Mediana	Média	p95	Máx.
Qwen 3 4B	59	1251	2596	2857	5702	8098
Gemma 4 E2B	59	372	709	777	1198	1439

Fonte: Elaborado pelos autores. Tempo da chamada de tradução, medido no ambiente de cada rodada (Tabela 26); execuções com erro de infraestrutura excluídas.

Tabela 29 – Teste de McNemar exato entre pares de modelos — estação de referência (camadas P e N)

Modelo A	Modelo B	Só A acerta	Só B acerta	<i>p</i> -valor
Qwen 3 4B	Gemma 4 E2B	8	7	1,0000

Fonte: Elaborado pelos autores. Consultas discordantes entre os dois modelos nas métricas principais, com o resultado de cada consulta dado pela maioria das suas repetições; *p*-valor bicaudal exato.

Latência.

A latência, ao contrário, separou nitidamente os dois modelos. O Qwen 3 4B teve mediana de 2,6 s e percentil 95 de 5,7 s; o Gemma 4 E2B, 0,7 s e 1,2 s, cerca de 3,7 vezes mais rápido, apesar da menor parcela na GPU. Dois fatores são compatíveis com essa diferença: o Gemma 4 E2B computa com cerca de 2 bilhões de parâmetros efetivos (Seção 3.2), e suas respostas foram mais curtas, com mediana de 35 *tokens* de saída, contra 50 do Qwen 3 4B. Em relação à T4, a latência mediana do Qwen 3 4B subiu de 0,8 s para 2,6 s, e a do Gemma 4 E2B, de 0,4 s para 0,7 s.

Comportamento.

Na categoria F, a acurácia foi de 100,0% para o Qwen 3 4B e de 100,0% para o Gemma 4 E2B: os dois recusaram as consultas fora do domínio das camadas P e N. Os modos de falha foram os mesmos das rodadas completas: o Qwen declarou fora do escopo pedidos como “atualizações deste mês”, e o Gemma pediu mais critérios em consultas como “cartas topográficas” e “produtos publicados nos últimos 3 meses”, contrariando a

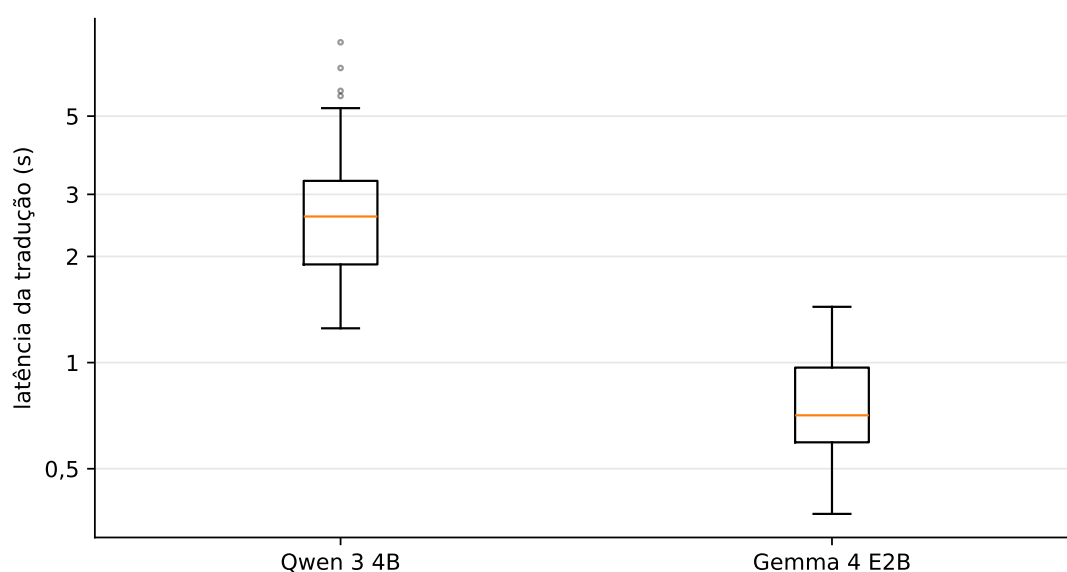


Figura 9 – Distribuição da latência da tradução na estação de referência, em escala logarítmica

Fonte: Elaborado pelos autores.

instrução do *prompt* de sistema de que um único parâmetro basta para chamar a ferramenta.

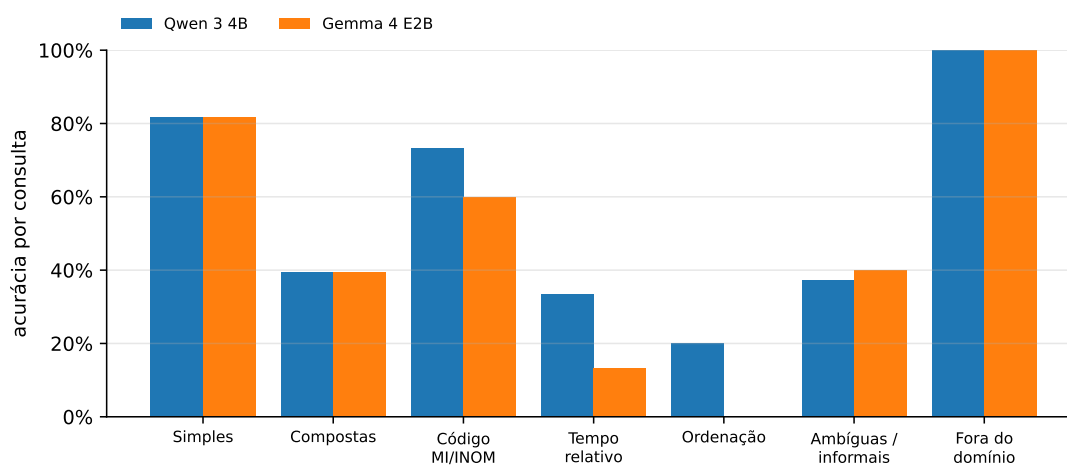


Figura 10 – Acurácia por consulta, por categoria, na estação de referência

Fonte: Elaborado pelos autores.

Comparação com as rodadas completas.

A Tabela 30 compara, para os mesmos modelos e consultas, as respostas da estação com as da primeira repetição das rodadas completas. A resposta foi idêntica em 50

das 59 consultas para o Qwen 3 4B e em 53 para o Gemma 4 E2B, e a pontuação coincidiu em 57 e 57, respectivamente. A acurácia nessas consultas foi de 49,2% e 47,5% nas rodadas completas, e de 49,2% e 47,5% na estação. A qualidade da tradução, portanto, praticamente não mudou com o *hardware*; a latência, sim.

Tabela 30 – Mesmo modelo nas rodadas completas e na estação de referência (camadas P e N, primeira repetição)

Modelo	n	Resposta idêntica	Mesma pontuação	Acurácia (rodada completa)	Acurácia (estação)
Qwen 3 4B	59	50	57	49,2%	49,2%
Gemma 4 E2B	59	53	57	47,5%	47,5%

Fonte: Elaborado pelos autores. Resposta idêntica: mesmos parâmetros emitidos (ou nenhuma chamada nos dois ambientes).

Reexecução na mesma estação.

O Qwen 3 4B já havia executado as mesmas consultas na estação em 14 de setembro de 2026, com os mesmos pesos, *prompt*, ferramenta, data de referência e versão do Ollama. A Tabela 31 compara as duas execuções: mesmo com temperatura zero, a resposta se repetiu em 51 das 59 consultas, e a pontuação, em 56. A explicação mais provável são diferenças numéricas de ponto flutuante na inferência, que dependem da divisão do modelo entre GPU e CPU e da ordem das operações e bastam para mudar a escolha de um *token* em respostas isoladas. A variação é maior entre sessões na estação, com parte do modelo na CPU, do que entre as repetições de uma mesma sessão na T4 (Seção 5.2), e as três repetições das rodadas completas, feitas numa mesma sessão, não a capturam: elas medem apenas a variação dentro da sessão, que foi pequena, o que recomenda ler as diferenças de acurácia entre modelos à luz dos intervalos de confiança e do teste de McNemar.

Tabela 31 – Qwen 3 4B executado duas vezes na estação de referência, com os mesmos pesos, prompt, ferramenta e data de referência (camadas P e N)

Modelo	n	Resposta idêntica	Mesma pontuação	Acurácia (24/set)	Acurácia (14/set)
Qwen 3 4B	59	51	56	49,2%	47,5%

Fonte: Elaborado pelos autores. Resposta idêntica: mesmos parâmetros emitidos (ou nenhuma chamada nos dois ambientes).

5.11 Referência em Nuvem

Três modelos de maior porte servidos pelo Groq, `qwen/qwen3.8-27b`, `openai/gpt-oss-20b` e `openai/gpt-oss-120b`, executaram, com uma repetição, o

mesmo *prompt* de sistema, a mesma ferramenta e as mesmas métricas, com o menor esforço de raciocínio que aceitam (*none* para o *qwen/qwen3.8-27b* e *low* para o *openai/gpt-oss-20b* e o *openai/gpt-oss-120b*) e data de referência de 24 de setembro de 2026 (Apêndice B.3). As Tabelas 33 e 34 trazem as métricas da referência em nuvem. Eles não atendem ao requisito de execução local (RNF1) e servem como referência de quanto modelos maiores, sem restrição de *hardware*, acertam nas mesmas consultas; a latência inclui a rede e não é comparável à dos modelos locais.

Tabela 32 – Ambiente de execução de cada rodada — referência em nuvem (Groq)

Modelo	GPU / máquina	Servidor	Rep.	Exec.	Na GPU	VRAM livre
Qwen 3.8 27B (Groq)	servidores do provedor	Groq (API)	1	310	—	—
GPT-OSS 20B (Groq)	servidores do provedor	Groq (API)	1	310	—	—
GPT-OSS 120B (Groq)	servidores do provedor	Groq (API)	1	310	—	—

Fonte: Elaborado pelos autores a partir de `results/<modelo>/manifesto.json`. Na GPU: parcela do modelo carregado (pesos e contexto) alocada na VRAM pelo Ollama, medida após o aquecimento; VRAM livre: no início da rodada, com os demais modelos descarregados. Data de referência: 2026-09-24. Python 3.14.4; langchain-core 1.6.4, langchain-ollama 1.1.0, langchain-openai 1.6.6.

Tabela 33 – Comparativo de desempenho entre os modelos avaliados — referência em nuvem (Groq)

Modelo	Acurácia	IC 95%	Precisão	Recall	F1	Lat. med. (ms)
Qwen 3.8 27B (Groq)	95,1%	92,1%–97,0%	0,974	0,994	0,984	695
GPT-OSS 20B (Groq)	89,2%	85,2%–92,2%	0,959	0,945	0,946	530
GPT-OSS 120B (Groq)	89,9%	86,0%–92,8%	0,965	0,951	0,956	655

Fonte: Elaborado pelos autores. IC 95%: intervalo de confiança de Wilson para a acurácia, com *n* igual ao número de consultas, pois as repetições de uma mesma consulta não são independentes. Precisão, *recall* e F1 ponderados pela frequência de cada campo no gabarito. Latência mediana da chamada de tradução, no ambiente de cada rodada.

Tabela 34 – F1-score por categoria de consulta e por modelo — referência em nuvem (Groq)

Categoria	Qwen 3.8 27B (Groq)	GPT-OSS 20B (Groq)	GPT-OSS 120B (Groq)
Simples (n=100)	0,985	0,985	0,979
Compostas (n=180)	0,983	0,937	0,950
Código MI/INOM (n=57)	1,000	0,985	0,986
Tempo relativo (n=60)	0,979	0,938	0,933
Ordenação (n=29)	0,982	0,837	0,926
Ambíguas/informais (n=197)	0,983	0,952	0,951
Média ponderada	0,984	0,946	0,956

Fonte: Elaborado pelos autores. Uma consulta pode pertencer a mais de uma categoria; *n* = consultas distintas da categoria nas métricas principais (cada uma pontuada em todas as repetições). As consultas fora do domínio (categoria F), cujo gabarito é não chamar a ferramenta, não têm campos e aparecem na Figura de acurácia por categoria.

Sobre as mesmas 306 consultas pontuadas dos modelos locais, o Qwen 3.8 27B acertou 95,1% (IC 95%: 92,1% a 97,0%), o GPT-OSS 120B, 89,9% (IC 95%: 86,0% a 92,8%), e o GPT-OSS 20B, 89,2% (IC 95%: 85,2% a 92,2%), contra 60,2% do melhor

modelo local, o Gemma 4 E4B. O Qwen 3.8 27B, avaliado sem raciocínio, como os modelos locais, superou os dois GPT-OSS, que raciocinaram com esforço baixo, com diferença significativa pelo teste de McNemar exato (p -valor de 0,005 contra o GPT-OSS 20B e de 0,007 contra o GPT-OSS 120B); entre os dois GPT-OSS, a diferença não foi significativa (p -valor de 0,880). A distância para os modelos locais é maior justamente nas categorias difíceis para eles: nas consultas compostas e de tempo relativo, o Qwen 3.8 27B acertou 93,3% e 91,7%, e o GPT-OSS 20B, 84,4% e 83,3%, contra no máximo 53,1% e 48,3% entre os modelos locais. A ordenação separou os modelos em nuvem: foi a categoria mais difícil para os dois GPT-OSS, com 48,3% no 20B e 79,3% no 120B, mas não para o Qwen 3.8 27B, que acertou 93,1% dessas consultas. Os dois GPT-OSS recusaram todas as consultas fora do domínio; o Qwen 3.8 27B recusou sete das oito e, para “quero um mapa bonito”, chamou a ferramenta sem nenhum parâmetro. Por camada, o Qwen 3.8 27B acertou 86,4% das consultas da camada P, 86,5% das da camada N e 97,2% das da camada G; o GPT-OSS 120B, 68,2%, 81,1% e 93,1%; e o GPT-OSS 20B, 77,3%, 83,8% e 91,1%.

A referência mostra que a tarefa, com a mesma ferramenta, o mesmo *prompt* e o mesmo gabarito, é resolvível com alta acurácia por modelos de maior porte, e o resultado do Qwen 3.8 27B mostra que isso não depende do raciocínio: sem ele, na mesma configuração dos modelos locais, esse modelo teve a maior acurácia da comparação. A distância para os modelos locais reflete, assim, sobretudo a capacidade dos modelos — porte e geração —, e não um defeito do protocolo ou do gabarito; quanto dela o raciocínio reduziria nos modelos locais não foi medido.

5.12 Análise Estatística

A Tabela 35 traz o teste de McNemar exato entre cada par de modelos das rodadas completas, com uma observação por consulta: o resultado da maioria das suas três repetições (Seção 3.7.5). Entre os três modelos do patamar superior, nenhuma diferença é significativa: o p -valor foi de 0,101 entre Qwen 3 4B e Gemma 4 E4B, de 0,569 entre Qwen 3 4B e Gemma 4 E2B e de 0,305 entre os dois Gemma. Entre o Mistral Nemo 12B e cada um dos outros três, o p -valor foi inferior a 0,001. Tratar as três repetições como pares independentes triplicaria, na prática, as consultas discordantes, que são quase sempre as mesmas nas três repetições de uma sessão, e exageraria a significância das diferenças; pela mesma razão, os intervalos de confiança da Tabela 12 usam como tamanho da amostra o número de consultas.

Tabela 35 – Teste de McNemar exato entre pares de modelos

Modelo A	Modelo B	Só A acerta	Só B acerta	<i>p</i> -valor
Qwen 3 4B	Gemma 4 E4B	45	63	0,1014
Qwen 3 4B	Gemma 4 E2B	52	59	0,5692
Qwen 3 4B	Mistral Nemo 12B	108	24	< 0,0001
Gemma 4 E4B	Gemma 4 E2B	53	42	0,3049
Gemma 4 E4B	Mistral Nemo 12B	125	23	< 0,0001
Gemma 4 E2B	Mistral Nemo 12B	118	27	< 0,0001

Fonte: Elaborado pelos autores. Consultas discordantes entre os dois modelos nas métricas principais, com o resultado de cada consulta dado pela maioria das suas repetições; *p*-valor bicaudal exato.

5.13 Discussão

Qualidade e latência.

Os resultados não apontam um modelo dominante em qualidade entre Qwen 3 4B, Gemma 4 E4B e Gemma 4 E2B: as acurácias são estatisticamente indistinguíveis, e cada um tem pontos fortes próprios — o Qwen nos códigos MI/INOM, no tempo relativo e na ordenação; o Gemma 4 E4B na precisão, no *recall*, nas consultas compostas e nas informais; o Gemma 4 E2B nas consultas simples e na latência. O Gemma 4 E4B teve a maior acurácia e o maior F1 dos três, embora a diferença não seja significativa. A latência os separa com mais clareza: o Gemma 4 E2B foi o mais rápido tanto na T4 quanto na estação de referência, onde sua mediana ficou abaixo de um segundo.

O que o *Tool Calling* entregou.

Numa só chamada, a comparação direta da Seção 5.3 não confirma a hipótese de que o *Tool Calling* melhoraria a extração dos parâmetros: com os mesmos modelos locais, a Saída Estruturada foi mais precisa em três dos quatro, e o *Tool Calling* só a superou no GPT-OSS 20B. O que ele entregou de forma consistente foi a decisão explícita de buscar ou não: com *Tool Calling*, três dos quatro modelos locais e dois dos três modelos em nuvem recusaram todas as oito consultas fora do domínio, o Gemma 4 E2B recusou 62,5% delas e o Qwen 3.8 27B, 87,5%, enquanto a Saída Estruturada, na forma avaliada nas 310 e pontuada literalmente, leva toda consulta a uma busca. O lote 1 mostrou que esse limite não é do mecanismo: com um campo de recusa, a Saída Estruturada v2 recusou 100,0% das consultas fora do domínio, embora com precisão de só 56,7%, e o objeto vazio da Saída Estruturada v1 funcionaria como recusa em 78,9% delas (Seção 5.4). O que fica como próprio do *Tool Calling* é a precisão da recusa — 96,3% com a v2 — e a resposta que ela dá ao usuário. Essa liberdade tem custo: os modelos locais também deixaram de buscar em consultas legítimas, e esse é o principal motivo de a Saída Estruturada acertar mais neles; no lote 1, o Gemma 4 E4B com a v1 deixou de buscar em 32,1% das consultas do

domínio. Em relação ao protótipo como foi entregue, a solução chegou à mesma acurácia com modelos menores e muito mais rápidos, sem dicionário — a normalização de siglas, grafias sem acento, apelidos de escala e nomes de projeto foi feita pelo próprio modelo, nas duas abordagens —, e toda falha passou a ser registrada e classificada, o que permitiu identificar o modo de falha de cada modelo, inclusive o do Mistral Nemo 12B, que um mecanismo de *fallback* teria mascarado. Isso vale para a solução de uma chamada (v1). A v3 devolve ao sistema código de normalização, não como um dicionário aplicado à consulta antes do modelo, mas como ferramentas que o modelo consulta e um validador que confere a saída e recupera parte das falhas, com cada intervenção registrada no traço da execução. E o seu teste mostra que o quadro depende de como o mecanismo é usado: quando o *Tool Calling* passa a chamar ferramentas auxiliares no meio da resposta e a receber o retorno de um validador, ele supera a Saída Estruturada que recebe as mesmas descrições e o mesmo validador, mas não as ferramentas auxiliares, nas consultas do domínio e na precisão da recusa, embora recuse uma parte menor das consultas fora do domínio (95,1%, contra 100,0%); o desenho não separa o mecanismo dessas ferramentas (Seção 5.5).

O que continua difícil.

Com uma só chamada, datas relativas, ordenação e consultas com muitos campos permanecem difíceis para todos os modelos locais, e a ordenação o é também para os dois GPT-OSS. Parte da dificuldade é inerente à tarefa: nem sempre a consulta diz se a ordem é por publicação ou por criação, e o gabarito aceita as duas leituras, mas os modelos ainda assim acrescentam ou omitem campos. Outra parte é sensível ao *prompt* e à abordagem: a recusa indevida e o pedido de esclarecimento em consultas amplas contrariam uma instrução explícita e são candidatos naturais a ajuste de *prompt* ou a exemplos no *prompt*, que este trabalho não usou por desenho. O ajuste funcionou no *Tool Calling*: no lote 1, a instrução da v2 de nunca pedir esclarecimento, com uma ferramenta própria para a recusa, levou a falsa recusa do Gemma 4 E4B de 32,1% para 0,9%. A Saída Estruturada não resolve o problema por si: na v1, sem opção de não buscar, ela sempre busca, mas, quando ganhou um campo de recusa, na v2, recusou 17,8% das consultas do domínio, com o mesmo gradiente — mais nas consultas com um só campo (Seção 5.4). Com a v3, em que o código assume a aritmética de datas e confere os campos, essas categorias subiram: numa análise por categoria exploratória, feita depois do teste, o Gemma 4 E4B com a v3 acertou, no lote 2, 91,4% das consultas de tempo relativo, 97,0% das de ordenação e 93,6% das compostas, contra 22,3%, 27,3% e 49,1% com a v1.

Quando o acervo não tem a resposta.

O sistema foi avaliado também no que acontece quando não há produto a devolver. Nas oito consultas fora do domínio, a resposta correta era não buscar, e três dos quatro

modelos locais não buscaram em nenhuma delas; o Gemma 4 E2B buscou nas que citavam “Marte”, “Argentina” e “mapa do tesouro pirata”, com esses termos como parâmetros. Nas consultas com valor que a ferramenta não representa, só o Qwen 3 4B chamou a ferramenta, com valores substituídos (Seção 5.9.1). E, quando a busca é legítima mas nenhum produto atende aos filtros, a resposta ao usuário não depende do modelo: é uma mensagem fixa com os critérios usados (Seção 4.4). Na abordagem do protótipo, ao contrário, toda consulta resulta em busca.

Hardware.

Com 4 GB de VRAM, nenhum modelo coube inteiro na GPU da estação, e a latência depende diretamente da parcela alocada. A qualidade, por outro lado, praticamente não mudou entre a estação e a T4. Para a DSG, isso significa que a escolha do modelo pode ser feita pela qualidade medida em qualquer ambiente, e que o *hardware* de implantação define a latência: uma GPU com 6 a 8 GB de VRAM já comportaria inteiros o Qwen 3 4B e o Gemma 4 E2B.

5.13.1 Recomendação de Modelo e de Abordagem para Uso Operacional

Para o uso operacional sobre o acervo da DSG, com a solução v1 do Capítulo 4, que usa *Tool Calling* numa só chamada, a recomendação depende do *hardware* de implantação. Em estações com GPU modesta, como a de referência, recomenda-se o **Gemma 4 E2B**: sua acurácia não difere significativamente da dos outros dois modelos do patamar superior (Seção 5.12), seu F1 é próximo ao do melhor modelo local, e ele teve a menor latência nos dois ambientes, inclusive na estação de 4 GB. Sua fraqueza é a recusa das consultas fora do domínio, em que chamou a ferramenta em parte dos casos; nessa configuração, convém que a aplicação trate como sem resultado as buscas cujos parâmetros não correspondam ao acervo. Em estações com mais VRAM, o **Gemma 4 E4B** é preferível: teve a maior acurácia e o maior F1 dos modelos locais, recusou todas as consultas fora do domínio e, na T4, teve latência comparável à do Qwen 3 4B; na estação de referência, de que só uma parte coube na GPU, teve mediana de 3,42 s com a solução v1 nas 62 consultas P e N. No lote de validação, porém, o Gemma 4 E4B com a solução do Capítulo 4, o *Tool Calling* v1, deixou de buscar em 32,1% das consultas do domínio (464/1.447), taxa que chega a 39,0% fora dos registros direto e telegráfico (Seção 5.4); os parágrafos seguintes tratam dessa limitação. O **Qwen 3 4B** é a alternativa quando predominarem consultas com códigos MI/INOM, tempo relativo ou ordenação, categorias em que teve o melhor desempenho, ao custo de uma latência maior na estação. O Mistral Nemo 12B não é recomendado na configuração avaliada, pela falha sistemática no uso do canal de ferramentas.

A comparação da Seção 5.3 e o lote de validação (Seção 5.4) acrescentam a essa recomendação uma escolha de arquitetura. Isoladas, as configurações executadas obrigariam a escolher entre extração e recusa: a Saída Estruturada v1 extrai melhor — 75,2% com o Gemma 4 E4B nas 310, a melhor configuração local executada —, mas leva toda consulta a uma busca, inclusive as que o catálogo não atende; o *Tool Calling* recusa esses pedidos e responde ao usuário, ao custo de falsas recusas, na v1, ou de erros de forma, na v2; e a Saída Estruturada v2 recusa com pouca precisão. A arquitetura em duas etapas simulada no lote 1, em que o *Tool Calling* v2 decide se busca ou recusa e, quando busca, os parâmetros vêm da Saída Estruturada v1, foi, entre as configurações v1 e v2, a de maior acurácia: acertou 79,5% no lote 1 e 77,8% nas 310, acima das quatro configurações executadas nos dois conjuntos (Tabela 18), com a precisão de recusa do *Tool Calling* v2 (96,3%) e, no domínio, a extração da Saída Estruturada v1. Como supera a Saída Estruturada v1 a partir de 0,1% de consultas fora do domínio, a escolha deixa de depender de quanto o uso real terá de pedidos fora do catálogo, proporção que não se conhece. Entre as configurações v1 e v2, seria essa a recomendada, ao custo de uma segunda chamada ao modelo sempre que há busca (na T4, mediana somada de 1,46 s no lote 1, contra 0,72 s da Saída Estruturada v1). Essa arquitetura tem ressalvas próprias: a combinação foi simulada *post hoc* sobre rodadas já feitas, e não executada como um *pipeline*; foi medida com um único modelo, o Gemma 4 E4B; e o lote 1 é sintético, redigido e anotado por modelos da mesma família. Se for preciso manter uma só chamada, há duas alternativas: a Saída Estruturada v1 com a regra do objeto vazio (75,9% no lote 1), abaixo das duas etapas e sem mensagem de recusa para o usuário, e o *Tool Calling* com a parte de decisão da v2 — a ferramenta de recusa e a instrução de nunca pedir esclarecimento — e descrições sem os erros de forma da v2, que o lote 2 mediu como o degrau v3d (parágrafo seguinte). A v2 inteira não é recomendada em nenhuma das duas abordagens: o *Tool Calling* v2 acertou 47,4% nas 310 e, mesmo com as três normalizações, 72,5% no lote 1, e a Saída Estruturada v2 recusa com precisão de 56,7% e declara fora do escopo todas as consultas subespecificadas.

O teste da v3 no lote 2 (Seção 5.5) muda a recomendação de arquitetura. O *Tool Calling* v3 — as descrições corrigidas, as ferramentas auxiliares e o retorno do validador — acertou 94,4% das consultas de teste, acima do controle, a Saída Estruturada com as mesmas descrições e o mesmo validador (87,8%), e da arquitetura em duas etapas simulada com as rodadas do mesmo lote (78,6%), com precisão de recusa de 98,7% e falsa recusa de 0,3%, mas com *recall* da recusa de 95,1%, contra 100,0% do controle e das duas etapas. Recomenda-se, portanto, para a DSG, o ***Tool Calling* v3 com o Gemma 4 E4B**, medido só com esse modelo e só na T4, ao custo de cerca de duas chamadas ao modelo por consulta (mediana de 1,47 s na T4, percentil 95 de 4,96 s). A v3 está integrada à API do *backend* do Capítulo 4, como abordagem padrão, e o `/api/health` mostra se a configuração no ar tem os mesmos *hashes* das rodadas avaliadas (Seção 4.4.6). Se a

latência for a restrição, a Saída Estruturada v3, com o mesmo validador, é a alternativa: acertou 87,8%, com mediana de 0,78 s, mas recusa com menos precisão e declara fora do escopo 5,0% das consultas do domínio. Se for preciso uma só chamada ao modelo, o degrau v3d — a decisão da v2 com as descrições corrigidas — acertou 75,2%, com 1,01 chamada por consulta, abaixo das duas etapas. Essa recomendação tem ressalvas próprias. Parte do acerto é do código do domínio, que as ferramentas e o validador trazem do manual que define o gabarito — para datas, a mesma tabela —, como mostra a distância para o degrau v3d, e esse código precisa acompanhar as mudanças do catálogo e das convenções de busca. O lote 2 sorteou nomes e códigos das mesmas listas que as ferramentas consultam, e o alcance delas em nomes fora dessas listas não foi medido. Em 25,0% das consultas, a chamada veio escrita no texto e foi interpretada pela v3, leitura que precisa ir para a produção junto com ela. A v3 foi desenvolvida em ciclos sobre os erros de uma amostra do lote 1 e medida com um só modelo, em consultas sintéticas, redigidas e anotadas por modelos da mesma família, e com um usuário simulado de resposta fixa. Com o Gemma 4 E2B e na estação de referência, a v3 foi medida só nas 62 consultas P e N, de dentro da amostra: acertou 78,0% delas, contra 47,5% da solução v1, com mediana de 1,75 s (Seção 5.6); o seu resultado num conjunto de teste com esse modelo ainda não foi medido, e, até lá, a recomendação do Gemma 4 E2B para GPU modesta vale com essa ressalva.

A recomendação de modelo tem três ressalvas. Ela vale para a configuração avaliada — sem raciocínio, sem exemplos no *prompt* e com as versões registradas nos manifestos —, e o desempenho com raciocínio ativado não foi medido; a escolha entre os modelos apoia-se nas 310 consultas, porque os dois lotes avaliaram só o Gemma 4 E4B. A distância para os modelos em nuvem indica margem de melhoria considerável, e ela não depende de serviços externos: os três modelos da referência têm pesos abertos, e o GPT-OSS 20B, por exemplo, é distribuído também para o Ollama, com cerca de 14 GB, e executa com cerca de 16 GB de memória (Ollama, 2025), o que o torna candidato natural a uma estação com mais VRAM; o *pipeline* permite repetir a comparação com o mesmo critério. E os modos de falha identificados devem ser observados no uso real antes da implantação em larga escala. O lote 1 já mostrou que as 310 subestimam, em linguagem corrente, a falsa recusa do *Tool Calling* v1, quase toda feita de pedidos de esclarecimento — 32,1% das consultas do domínio no lote 1, contra 18,8% nas 310, e 44,5% nas consultas com um só campo —, mas o texto dos lotes também é sintético, e a proporção real de consultas fora do catálogo continua desconhecida.

6 CONCLUSÃO

6.1 Síntese do Trabalho

Este Projeto Final de Curso aprimorou a camada de tradução linguística da solução de busca em linguagem natural sobre o acervo cartográfico da Diretoria de Serviço Geográfico (DSG), partindo do protótipo desenvolvido pelo 1º Centro de Geoinformação (1º CGEO). O protótipo demonstrou a viabilidade da abordagem, mas apresentava limitações documentadas pelo próprio cliente: *backend* em Node.js, Saída Estruturada via Instructor e Zod, um único modelo (Phi-4 14B), dicionário de normalização fixo no código, *fallback* por expressões regulares e ausência de um protocolo de avaliação.

A partir da análise crítica do protótipo, o trabalho entregou:

1. um *backend* em Python, com FastAPI e LangChain, que substitui o do protótipo e preserva o esquema do banco e o conjunto de parâmetros de busca;
2. a substituição da Saída Estruturada por *Tool Calling* nativo, em que o modelo decide se chama a ferramenta de busca e com quais argumentos, sem dicionário de normalização e sem exemplos resolvidos no *prompt*;
3. um *dataset* de avaliação com 310 consultas, construído em três camadas (protótipo, autores e geração programática), com gabarito definido por um manual de anotação escrito, leituras múltiplas explícitas para as consultas ambíguas e uma auditoria em três etapas — checagens automáticas, anotação independente às cegas e adjudicação registrada — que deixou todas as divergências com decisão justificada;
4. um *pipeline* de avaliação automatizado e reproduzível, que registra manifesto e traços de cada rodada, recalcula a pontuação contra o gabarito vigente e produz acurácia com intervalo de confiança, precisão, *recall* e F1 por campo, latência e testes de significância entre modelos;
5. a comparação de quatro modelos abertos executáveis localmente — Qwen 3 4B, Gemma 4 E4B, Gemma 4 E2B e Mistral Nemo 12B —, complementada por uma referência paralela com três modelos de maior porte servidos em nuvem;
6. um lote de validação independente (lote 1), de 1.929 consultas com gabarito fixado antes da redação e aceitas por anotação às cegas, sobre o qual as duas abordagens foram comparadas com a especificação da solução e com uma segunda especificação (v2), para separar o efeito da abordagem do efeito da especificação;

7. a v3 do *Tool Calling*, com ferramentas auxiliares determinísticas e validação com retorno, desenvolvida sobre as 310 consultas e os erros de uma amostra do lote 1 e testada, contra um controle de Saída Estruturada com as mesmas descrições e o mesmo validador, num segundo lote, de 945 consultas que o desenvolvimento não usou, com a configuração congelada e o plano de análise fixados antes da rodada.

6.2 Retomada dos Objetivos

O objetivo geral — aprimorar a camada de tradução linguística por meio do *Tool Calling* nativo, da avaliação comparativa de modelos locais e de métricas quantitativas — foi atendido pelos entregáveis definidos na Introdução:

- **Nova arquitetura de *backend*:** descrita no Capítulo 4, com as camadas de API, orquestração, persistência e avaliação, e com a lógica de consulta ao banco portada do protótipo;
- ***Dataset* de avaliação:** 310 consultas com manual de anotação e registro de auditoria (Capítulo 3 e Apêndice A); a auditoria mostrou, entre o gabarito e uma anotação independente feita às cegas, concordância quase perfeita nos campos e na detecção de ambiguidade e concordância substancial na decisão de chamar ou não a ferramenta, esta por causa de oito consultas fora do domínio que o gabarito tratava como observacionais, erro corrigido na adjudicação; cada divergência recebeu decisão registrada;
- ***Pipeline* de avaliação:** implementado nos comandos `pfc-dataset`, `pfc-auditar`, `pfc-avaliar` e `pfc-relatorio`, cobertos por testes automatizados (Capítulo 4);
- **Relatório comparativo e recomendação:** apresentados no Capítulo 5, com a recomendação de modelo e de abordagem na Seção 5.13;
- **Lotes de validação e v3:** os dois lotes, com gabarito fixado antes da redação (Seções 3.5 e 3.6), e a v3 do *Tool Calling* (Seção 3.6), implementada no pacote, no avaliador e na API do *backend*, em que é a abordagem padrão (Seção 4.4.6); a verificação das conclusões no lote 1 está na Seção 5.4, e o teste da v3 no lote 2, na Seção 5.5.

6.3 Principais Resultados

Os resultados do Capítulo 5 permitem responder às perguntas que motivaram o trabalho.

- **Qualidade dos modelos locais:** três dos quatro modelos — Gemma 4 E4B, Gemma 4 E2B e Qwen 3 4B — tiveram acurácias estatisticamente indistinguíveis, de 60,2%, 56,5% e 54,2%; o Gemma 4 E4B teve a maior acurácia e o maior F1 (0,793), e o Qwen 3 4B, o menor F1 dos três (0,666), por acrescentar campos não pedidos. O Mistral Nemo 12B, o maior dos quatro, ficou em 26,8%, sobretudo porque deixou de emitir a chamada da ferramenta, que em muitos casos descreveu em texto; o porte do modelo, portanto, não determinou o desempenho.
- **Tool Calling e Saída Estruturada:** com os mesmos modelos e consultas, numa só chamada, o *Tool Calling* não melhorou a extração (a v3, adiante, muda esse quadro). A Saída Estruturada foi mais precisa em três dos quatro modelos locais — 75,2% contra 60,2% no Gemma 4 E4B —, porque o *Tool Calling* dá ao modelo a liberdade de não buscar, e os modelos locais a usaram também em consultas legítimas; o *Tool Calling* só a superou no GPT-OSS 20B, e o efeito dependeu do modelo, não do porte. O ganho consistente do *Tool Calling* foi a recusa: três dos quatro modelos locais e dois dos três modelos em nuvem recusaram todas as oito consultas fora do domínio, e o Gemma 4 E2B, 62,5% delas, enquanto a Saída Estruturada, na forma avaliada nas 310 e pontuada literalmente, leva toda consulta a uma busca.
- **Lote de validação independente:** num segundo conjunto, o lote 1, de 1.929 consultas com gabarito fixado antes da redação, o Gemma 4 E4B repetiu a comparação com a especificação da solução (v1) e com a especificação v2, que explicita as convenções do manual e dá às duas abordagens uma forma de recusar. Com a v1, as duas empataram (61,5% com *Tool Calling* e 62,2% com Saída Estruturada; *p*-valor de 0,656), mas o empate soma duas diferenças de sinal contrário: nas consultas do domínio, a Saída Estruturada acertou 75,5%, e o *Tool Calling*, 48,9%, sobretudo porque deixou de buscar em 32,1% delas, quase sempre para pedir esclarecimento; nas 336 consultas fora do domínio, o *Tool Calling* recusou todas, e a Saída Estruturada, nenhuma. Pela pontuação do protocolo, em que toda resposta da Saída Estruturada é uma busca, o *Tool Calling* v1 só é mais preciso se mais de 18,0% das consultas estiverem fora do catálogo; se a aplicação tratasse como não busca a resposta sem nenhum campo do *schema*, a Saída Estruturada v1 recusaria 78,9% delas, e esse ponto subiria para 51,0%.
- **Especificação v2:** a v2 levou a falsa recusa do *Tool Calling* a 0,9%, mas introduziu erros de forma — a UF em sigla, o prefixo “MI” mantido na *keyword*, o limite acrescentado —, e a acurácia praticamente não mudou (62,8%; *p*-valor de 0,277): o que o *Tool Calling* ganhou ao buscar onde a v1 não buscava, perdeu nas consultas em que as duas versões buscaram, nas quais a acurácia caiu de 71,7% para 55,7%, e nas subespecificadas, em que passou a buscar com filtros indevidos. Nas 310, a v2 piorou as duas abordagens, embora tenha sido desenhada a partir dos erros

nesse conjunto. Com a v2, a ferramenta de recusa do *Tool Calling* teve precisão de 96,3%, contra 56,7% do campo de recusa da Saída Estruturada, que marcou também 17,8% das consultas do domínio; a Saída Estruturada, por sua vez, extraiu melhor nas consultas em que as duas abordagens buscaram (77,0% contra 55,8%).

- **Arquitetura em duas etapas:** as rodadas do lote 1 permitem simular, sem nova execução, uma combinação das duas vantagens, em que o *Tool Calling* v2 decide se busca ou recusa e, quando busca, valem os parâmetros que a Saída Estruturada v1 extraiu da mesma consulta. A combinação acertou 79,5% no lote 1 e 77,8% nas 310, acima das quatro configurações isoladas nos dois conjuntos. No lote 1, a diferença para a melhor delas, a Saída Estruturada v2, foi de +7,1 pontos percentuais (p -valor $< 0,001$); nas 310, a combinação ficou acima da Saída Estruturada v1 (p -valor de 0,008), numa diferença que vem inteiramente das consultas fora do domínio. Ela reúne a precisão de recusa do *Tool Calling* v2 (96,3%, com falsa recusa de 0,9%) e a extração da Saída Estruturada v1 (75,3% das consultas do domínio, contra 75,5% dela sozinha), e supera a Saída Estruturada v1 no lote 1 a partir de 0,1% de consultas fora do domínio, o que torna a escolha praticamente independente dessa proporção. O custo são duas chamadas em série quando há busca: latência mediana de 1,46 s na T4, contra 0,72 s da Saída Estruturada v1. Foi a melhor combinação das configurações v1 e v2 e, no lote 2, uma das referências secundárias da v3, cuja comparação principal foi com um controle de Saída Estruturada com as mesmas descrições e o mesmo validador.
- **Tool Calling v3, no conjunto de teste:** com ferramentas auxiliares que o modelo chama antes da busca (identificação de UF, município e folha, normalização de código e de escala, resolução de datas pela tabela do manual) e com o retorno de um validador que confere os parâmetros antes de executar, o *Tool Calling* acertou 94,4% das 945 consultas do lote 2 (IC 95%: 92,7% a 95,7%). Na comparação principal, fixada antes da rodada, o controle — a Saída Estruturada com as mesmas descrições e o mesmo validador, mas sem as ferramentas auxiliares — acertou 87,8% (p -valor $< 0,001$), e a arquitetura em duas etapas simulada com as rodadas do mesmo lote, 78,6%. A vantagem sobre o controle vem toda das consultas do domínio, em que o controle declarou fora do escopo 5,0% delas, contra 0,3% do *Tool Calling*, e errou mais parâmetros; o controle acertou mais nas consultas fora do domínio, que recusou todas, e nas subespecificadas (95,4% contra 92,3%). A partir dos 62,1% da v2, cada peça da v3 acrescentou acurácia, com diferença significativa, e os degraus medem quanto do resultado é do modelo e quanto é do código: com as descrições corrigidas, ainda numa só chamada, o modelo acertou 75,2%; as ferramentas auxiliares, com o usuário simulado, levaram a 83,1% (+7,8 p.p.), e o retorno do validador, a 94,4% (+11,3 p.p.). A vantagem do *Tool Calling* já estava na primeira decisão — a primeira

busca ou recusa, depois das ferramentas auxiliares e antes do retorno —, que acertou 83,0%, contra 74,6% no controle (numa comparação exploratória, feita depois do teste, 146 consultas acertadas só pelo *Tool Calling* e 67 só pelo controle; p -valor $< 0,001$), e o retorno do validador a reduziu um pouco: acrescentou mais ao controle (+13,2 p.p.) que ao *Tool Calling* (+11,4 p.p.) e, das consultas que consertou, deu ao modelo o valor a usar em 53,6% no *Tool Calling* e em 43,2% no controle. A recusa teve precisão de 98,7%, mas o seu *recall* caiu para 95,1%, contra 100,0% no *Tool Calling* v2, no degrau de uma chamada, no controle e nas duas etapas; o custo foram cerca de duas chamadas ao modelo por consulta (mediana de 1,47 s na T4). Nas 310 consultas, que orientaram o desenho e dão um resultado de dentro da amostra, a mesma configuração acertou 97,7%, e o controle, 94,1%; nos dois conjuntos, o *Tool Calling* v3 ficou acima do controle, e os dois acima das duas etapas e das configurações v1 e v2, mas a ordem entre estas mudou de um conjunto para o outro, e os degraus intermediários não foram executados no desenvolvimento.

- **O protótipo e a solução aprimorada:** o método do protótipo (Phi-4 14B, com dicionário e exemplos) acertou 58,8%, acurácia estatisticamente igual à dos três melhores modelos locais com *Tool Calling* numa só chamada, que são menores, mais de quinze vezes mais rápidos, dispensam o dicionário e recusam o que o catálogo não atende; cada falha passou a ser registrada e classificada, sem *fallback* que a mascarasse. Na solução de uma chamada (v1), o ganho em relação ao protótipo veio, portanto, da troca de modelo e da remoção do dicionário e dos exemplos, e não do *Tool Calling*. A v3 devolve ao sistema código de normalização, mas não como um dicionário aplicado à consulta antes do modelo: como ferramentas que o modelo consulta quando precisa e um validador que confere a saída, com cada intervenção registrada no traço da execução.
- **O que continua difícil:** numa só chamada (v1), datas relativas e ordenação estiveram entre as categorias de menor acurácia para os três modelos locais do patamar superior, e as consultas compostas, que exigem vários campos corretos ao mesmo tempo, não passaram de 53,1% em nenhum deles. Os modos de falha dominantes foram deixar de chamar a ferramenta — por descrição da chamada em texto, no Mistral Nemo 12B; por recusa indevida, no Qwen 3 4B; e por pedido de esclarecimento, nos modelos Gemma, em consultas amplas e, no Gemma 4 E4B, também em consultas com datas relativas e com vários critérios — e, sobretudo no Qwen 3 4B, acrescentar ordenação ou limite não pedidos. Com a v3, essas categorias subiram: numa análise por categoria exploratória, no lote 2, o Gemma 4 E4B acertou 91,4% das consultas de tempo relativo, 97,0% das de ordenação e 93,6% das compostas, contra 22,3%, 27,3% e 49,1% com a v1, com a aritmética de datas passada ao código, que usa a mesma tabela do gabarito; o que a v3 ainda erra está na Seção 5.5.

- **Hardware:** a qualidade praticamente não mudou entre a estação de referência e a GPU de nuvem; a latência, sim. Na estação de 4 GB, o Gemma 4 E2B teve latência mediana de 0,7 s, contra 2,6 s do Qwen 3 4B, com acurácia estatisticamente indistinguível (47,5% e 49,2%).
- **Referência em nuvem:** nas mesmas consultas, os modelos servidos pelo Groq acertaram 95,1% (Qwen 3.8 27B), 89,9% (GPT-OSS 120B) e 89,2% (GPT-OSS 20B), o que mostra que a tarefa, com a mesma ferramenta, o mesmo *prompt* e o mesmo gabarito, é resolvível por modelos de maior porte. O de maior acurácia, o Qwen 3.8 27B, foi avaliado sem raciocínio, como os modelos locais; a distância indica, portanto, sobretudo a margem de melhoria que modelos de maior capacidade podem trazer.
- **Recomendação:** na solução de uma chamada, recomenda-se, em estações com GPU modesta, o Gemma 4 E2B, pela acurácia estatisticamente indistinguível da dos outros dois melhores modelos locais e pela menor latência nos dois ambientes, com atenção à recusa das consultas fora do domínio; em estações com mais VRAM, o Gemma 4 E4B, que teve a maior acurácia e o maior F1; e o Qwen 3 4B quando predominarem consultas com códigos MI/INOM, tempo relativo ou ordenação. A recomendação de modelo se apoia nas 310 consultas, porque os lotes avaliaram só o Gemma 4 E4B. Quanto à abordagem, recomenda-se o *Tool Calling* v3 com o Gemma 4 E4B, medido só com esse modelo e só na T4: no conjunto de teste, ele superou o controle com Saída Estruturada, a arquitetura em duas etapas e todas as configurações v1 e v2. A troca entre capacidade de recusa e acurácia de extração que as configurações de uma chamada impunham não deixou de existir, mas mudou de forma: o *recall* da recusa caiu para 95,1%, e a v3 custa cerca de duas chamadas ao modelo por consulta. Com o Gemma 4 E2B, na estação de referência, a v3 foi medida só nas 62 consultas P e N, de dentro da amostra (78,0% delas, contra 47,5% da solução v1, com mediana de 1,75 s); ela é a abordagem padrão da API do *backend*, que permite voltar à solução de uma chamada por configuração (Seção 4.4.6). Se for preciso manter uma só chamada, o *Tool Calling* com as descrições corrigidas da v3, sem ferramentas auxiliares nem retorno, acertou 75,2% com 1,01 chamadas por consulta, em média; se a restrição for a latência, a alternativa é a Saída Estruturada v3, com as mesmas descrições e o mesmo validador, que acertou 87,8%, com latência mediana de 0,78 s na T4, e recusa com menos precisão (falsa recusa de 5,0%).

6.4 Limitações do Trabalho

Os resultados devem ser interpretados à luz das seguintes limitações:

- **Regularidade da camada gerada:** 248 das 310 consultas foram geradas por modelos de frase. A geração garante cobertura sistemática das combinações dos catálogos do domínio, mas essas consultas são linguisticamente mais regulares que as de usuários reais, o que pode favorecer os modelos; por isso os resultados são reportados também por camada (Tabela 24). A camada G responde por 247 das 306 consultas pontuadas, e a acurácia nela superou a das camadas P e N nos três modelos locais do patamar superior — no Gemma 4 E4B, 62,9%, contra 55,9% na camada N e 37,9% na P — e nos três modelos em nuvem; as métricas agregadas podem, por isso, superestimar o desempenho em consultas reais. O lote 1, redigido em oito registros de linguagem, confirma a ressalva para o *Tool Calling* v1, cuja falsa recusa passou de 18,8% nas 310 para 32,1% no lote 1 e cuja acurácia nas consultas simples caiu de 78,0% para 53,0%, mas não para a Saída Estruturada v1, que acertou 77,2% das consultas do domínio nas 310 e 75,5% no lote 1: a superestimação depende da abordagem.
- **Consultas não coletadas de usuários:** nenhuma consulta do *dataset* foi coletada do uso real do catálogo; as camadas P e N foram redigidas pela equipe do protótipo e pelos autores. Os dois lotes também são sintéticos: foram redigidos e anotados por agentes de linguagem de uma mesma família de modelos, e, no lote 1, uma amostra dos erros foi auditada por um único juiz, dessa mesma família; não houve juiz humano. A proporção real de consultas fora do domínio no BDGEx é desconhecida, e dela depende qual das duas abordagens v1 é mais precisa; a vantagem da arquitetura em duas etapas simulada, ao contrário, quase não depende dela, e a da v3 sobre o controle, que vem das consultas do domínio, só se inverteria se a maior parte das consultas estivesse fora dele.
- **Alcance da auditoria:** a anotação independente às cegas foi feita por um modelo de linguagem de propósito geral, e a conferência humana, feita por um dos autores, cobriu uma amostra de 40 consultas; isso verifica a consistência do gabarito com o manual, mas não substitui uma anotação dupla e completa por especialistas do domínio.
- **Ambientes de execução:** as rodadas completas foram executadas em GPU de nuvem, e a estação de referência executou as camadas P e N. A latência depende fortemente do *hardware* e é reportada por ambiente; valores obtidos em outras GPUs, com outra quantidade de VRAM, podem diferir.
- **Linhas de base:** a Saída Estruturada e o método do protótipo foram executados com uma repetição, contra três das rodadas com *Tool Calling* dos modelos locais; o método do protótipo foi reproduzido a partir do código original, com temperatura zero e sem a validação de locais no banco.

- **Alcance dos lotes:** os dois lotes avaliaram um único modelo, o Gemma 4 E4B, numa única GPU, a T4, com uma repetição por configuração, sem medir a variação entre execuções; a recomendação de modelo continua apoiada apenas nas 310 consultas, e a v3 não foi avaliada com outros modelos, entre eles o Gemma 4 E2B, recomendado para as estações com GPU modesta na solução de uma chamada.
- **O código do domínio na v3:** as ferramentas auxiliares e o validador da v3 codificam o manual que define o gabarito — para datas, usam a mesma tabela que resolve as regras de tempo —, e o lote 2 sorteou nomes e códigos das mesmas listas que as ferramentas consultam (a lista de municípios do IBGE e o índice do BDGEx). O resultado mede, portanto, o modelo junto com esse código, e o alcance do código em consultas reais, que depende de ele reconhecer nomes e expressões fora dessas listas, não foi medido. Os degraus dizem quanto é de cada um; a decomposição separa só o retorno do validador, porque a primeira decisão, que acertou 83,0%, já vem depois das ferramentas auxiliares, e o que o modelo faz só com a especificação é o degrau de uma chamada (75,2%). Fazem parte do sistema medido também a leitura das chamadas escritas como texto e o pedido “Responda chamando uma ferramenta”, que o laço envia uma vez quando o modelo responde só com texto. Em 236 consultas (25,0%), o modelo escreveu ao menos uma chamada no texto da resposta, em vez de usar o canal nativo, e a v3 a interpretou; 230 delas (97,5%) terminaram certas, e, na v1 e na v2, a mesma resposta contaria como não busca. Não se sabe quanto a v3 acertaria sem essa leitura, porque o diálogo seria outro; por isso, ela precisa ir para a produção junto com a v3. O usuário simulado responde sempre que não tem outros detalhes; um usuário real poderia acrescentar critérios, e o efeito do esclarecimento pode ser outro. A v3 foi desenvolvida sobre as 310 consultas e o lote 1, em ciclos sobre os erros de uma amostra deste, e os seus números de desenvolvimento são de dentro da amostra; o lote 2, construído com a mesma receita, mede a generalização para consultas novas, não para outra distribuição de consultas. A análise dos erros e por categoria no lote 2 é exploratória, feita depois do teste, e não mudou a v3. O código precisa ser mantido junto com o catálogo, e a latência da v3 na estação de referência foi medida só nas 62 consultas P e N, com a GPU compartilhada com outros programas.
- **Simulações e atribuições *post hoc*:** a arquitetura em duas etapas e a regra que trata como não busca a resposta sem nenhum campo do *schema* são simulações feitas sobre as rodadas existentes, e não *pipelines* executados; a regra do objeto vazio foi definida depois de ver os dados, e a latência da combinação é a soma das duas chamadas, sem medir *cache* nem sobreposição. As causas atribuídas aos erros de forma da v2 (a lista de siglas sem direção, a instrução de copiar o código exatamente como aparece, a menção do termo proibido e o exemplo de limite) não

foram testadas por ablação, e a comparação entre as duas v2 não controla o texto que enquadra a recusa, que difere entre elas. As consequências desses erros na SQL foram simuladas sobre os nomes das UFs e dos municípios, sem o acervo.

- **Critérios de pontuação e de contagem:** na categoria E, a pontuação aceita qualquer não busca, inclusive a recusa, o que favorece o *Tool Calling* v1 e a Saída Estruturada v2, que recusou todas as consultas subespecificadas do lote 1; contadas essas recusas como erro, a Saída Estruturada v2 ficaria em 64,9%. No lote 2, o mesmo critério favorece o controle da v3, que não buscou em 93,8% das subespecificadas, contra 29,2% do *Tool Calling* v3, e não explica, portanto, a vantagem da v3. Várias contagens de diagnóstico, como a dos pedidos de esclarecimento e a dos gabaritos discutíveis, dependem do critério adotado, e as análises por registro de linguagem, leitura múltipla e subtipo são *post hoc*, com subgrupos pequenos.
- **Configuração dos modelos:** os modelos locais e o Qwen 3.8 27B foram avaliados sem modo de raciocínio, e os dois GPT-OSS, que não permitem desligá-lo, com o menor esforço de raciocínio que aceitam; todos com temperatura zero e sem exemplos no *prompt*. O desempenho dos modelos locais com raciocínio ativado, e o de todos com *few-shot* ou com *prompts* ajustados a cada modelo, não foi medido.
- **Referência em nuvem:** os modelos do Groq executaram uma repetição, sob os limites de taxa do plano gratuito, e servem apenas como referência de modelos de maior porte; não atendem ao requisito de execução local, e sua latência inclui a rede.
- **Versões dos modelos:** os resultados valem para as versões registradas nos manifestos; o ecossistema de LLMs abertos evolui rapidamente, e o *pipeline* foi construído para que a avaliação possa ser refeita com novos modelos.

6.5 Trabalhos Futuros

Os resultados e a experiência adquirida apontam as seguintes direções. As duas primeiras, assim como as de interface e implantação, ajuste fino e estudo com usuários, decorrem das exclusões de escopo declaradas no Capítulo 3.

1. **Avaliação da recuperação em nível de produto:** medir a fidelidade do conjunto de produtos retornado pela busca (precisão, *recall* e F1 sobre os códigos das folhas), o que exige um gabarito de quais folhas cada consulta deveria devolver.
2. **Resolução avançada de nomes geográficos:** resolver os campos `state` e `city` em geometrias, para operações espaciais (`ST_Intersects`, `ST_Within`) contra as *bounding boxes* das folhas, com integração à malha do IBGE e a serviços de geocodificação.

3. ***Fallback* para produção:** nas configurações v1 e v2, falhas do modelo são registradas como métrica, sem recuperação por regras determinísticas — decisão adequada à avaliação; a v3 já recupera parte delas por código, com o retorno do validador, cujas intervenções ficam registradas no traço da execução. Em produção, convém ainda um caminho alternativo, como encaminhar a consulta bruta à busca textual do PostgreSQL.
4. **Lacunas do validador da v3:** a leitura exploratória dos erros que restaram no lote 2 mostra o que o validador não confere — a *keyword* preenchida com região, nome de projeto ou escala — e o que a tabela do manual não cobre, como algumas expressões de tempo, além das consultas fora do domínio em que o modelo buscou, 3 delas porque a recusa contestada devolveu ao modelo uma recusa correta. Acrescentar essas regras e rever a contestação é trabalho da mesma natureza, mas o efeito precisa ser medido num conjunto novo, porque o lote 2 já foi usado para identificá-las.
5. **A v3 em produção:** a v3 já é a abordagem padrão da API do *backend* (Seção 4.4.6), mas sobre a semente sintética; falta executá-la com o banco do acervo — as rodadas do lote 2 avaliaram os parâmetros, sem executar a SQL —; medir a sua latência na estação de referência, com, em média, 2,06 chamadas ao modelo por consulta; levar ao usuário real o pedido de esclarecimento, que aqui e na API tem uma resposta fixa, e avaliá-lo com usuários; e verificar com outros modelos, em especial o Gemma 4 E2B, se a vantagem sobre o controle se mantém.
6. **Modos de raciocínio e *prompts*:** avaliar os modelos com raciocínio ativado e com exemplos no *prompt*, medindo o ganho de qualidade contra o custo de latência.
7. **Modelos abertos de maior porte em GPU local:** avaliar localmente, numa GPU com 16 GB ou mais de VRAM, modelos de pesos abertos de maior porte, como o GPT-OSS 20B e o Qwen 3.8 27B, que na referência em nuvem acertaram 89,2% e 95,1%, para verificar se a margem observada na nuvem se mantém em execução local, dentro do requisito RNF1.
8. ***Dataset* com consultas reais:** ampliar o *dataset* com consultas coletadas de usuários do catálogo, anotadas por especialistas do domínio segundo o manual de anotação, com medida de concordância entre anotadores humanos.
9. **Interface e implantação:** desenvolver a interface gráfica, com mapa interativo, e integrar o *pipeline* ao sistema de produção da DSG, com autenticação, controle de acesso e monitoramento.
10. **Ajuste fino:** treinar por ajuste fino um modelo de menor porte para o domínio das consultas cartográficas brasileiras, o que pode reduzir o requisito de *hardware*.

11. **Estudo com usuários:** avaliar a solução com usuários reais do catálogo, medindo impacto na produtividade e na satisfação.
12. **Conversação em múltiplos turnos:** permitir que o usuário refine a busca a partir dos resultados, mantendo o contexto das consultas anteriores.

6.6 Considerações Finais

O trabalho mostra que a busca em linguagem natural sobre o acervo cartográfico da DSG pode ser construída com modelos de linguagem abertos, executados localmente e integrados por *Tool Calling*, sem dependência de serviços de nuvem e com o controle sobre os dados que o contexto da DSG e de suas organizações subordinadas exige. Mostra também que o *Tool Calling* não é, por si, garantia de melhor extração: numa só chamada, com os modelos locais avaliados, seu mérito estava na decisão explícita de buscar ou recusar, com precisão de recusa de 96,3% no lote 1, contra 56,7% do campo de recusa da Saída Estruturada. Usado como mecanismo — ferramentas auxiliares chamadas no meio da resposta e o retorno de um validador —, ele superou, num conjunto de teste independente, com o Gemma 4 E4B e em consultas sintéticas, a Saída Estruturada que recebeu as mesmas descrições e o mesmo validador, mas não as ferramentas auxiliares (94,4% contra 87,8%); a diferença mede o mecanismo junto com essas ferramentas, que também são código do domínio. A escolha entre as abordagens, ou a forma de combiná-las, deve ser feita com medidas, como as deste trabalho, e não por suposição: o resultado da mesma comparação mudou com a proporção de consultas fora do domínio de cada conjunto, uma especificação desenhada a partir dos erros das 310 piorou nesse mesmo conjunto, e o resultado de teste da v3 vem de um conjunto que o seu desenvolvimento não usou.

Os resultados convergem para uma conclusão que atravessa o trabalho: o diferencial estruturante da solução não é o modelo de linguagem, mas o conhecimento especializado do engenheiro cartógrafo. O modelo contribui com a compreensão da linguagem livre do usuário; a estrutura do domínio vem do especialista. Foi esse conhecimento que definiu o que conta como resposta certa — o manual de anotação, com as convenções de nomenclatura do INOM e do MI, as escalas do Sistema Cartográfico Nacional, os tipos de produto, os Centros de Geoinformação e as regras de tempo —, que construiu e auditou os conjuntos de avaliação e que, na v3, virou código: o índice de folhas do acervo, a lista de municípios do IBGE, a normalização de códigos e de escalas, a tabela de datas e o validador que confere cada parâmetro antes da busca. Com o mesmo modelo, esse código levou a acurácia no conjunto de teste de 75,2%, só com as descrições corrigidas, a 94,4%. A experiência da v2 mostra também a forma que esse conhecimento deve ter: escrito como instruções no *prompt* de um modelo pequeno, ele consertou alguns erros e criou outros;

escrito como código verificável, testado contra o gabarito e medido num conjunto que o desenvolvimento não viu, ele se somou ao modelo. A ferramenta de IA, sozinha, não substitui a Cartografia; é o engenheiro cartógrafo quem lhe dá a estrutura que a torna útil ao acervo.

Para o cliente, o resultado mais duradouro não é a classificação dos modelos avaliados, que envelhece com o ecossistema, mas o instrumento de avaliação: um *dataset* auditável, com gabarito justificado por regras escritas e divergências adjudicadas, e um *pipeline* reproduzível que permite comparar qualquer novo modelo, *prompt* ou configuração com o mesmo critério.

REFERÊNCIAS

- 1º Centro de Geoinformação. *Protótipo de busca de geoinformação utilizando Large Language Models*. 2025. <https://github.com/1cgeo/prototipo_busca_llm>. Repositório GitHub. Acesso em: mar. 2026.
- ABDIN, M.; ANEJA, J.; BEHL, H.; BUBECK, S.; ELDAN, R.; GUNASEKAR, S.; HARRISON, M.; HEWETT, R. J.; JAVAHERIPI, M.; KAUFFMANN, P. et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024. Disponível em: <<https://arxiv.org/abs/2412.08905>>.
- ARTSTEIN, R.; POESIO, M. Inter-coder agreement for computational linguistics. *Computational Linguistics*, v. 34, n. 4, p. 555–596, 2008.
- Brasil. *Decreto-Lei n.º 243, de 28 de fevereiro de 1967: Fixa as Diretrizes e Bases da Cartografia Brasileira e dá outras providências*. [S.l.], 1967. Institui o Sistema Cartográfico Nacional e as escalas-padrão da Cartografia Sistemática Terrestre Básica. Acesso em: set. 2026. Disponível em: <https://www.planalto.gov.br/ccivil_03/decreto-lei/1965-1988/del0243.htm>.
- Brasil. *Decreto n.º 89.817, de 20 de junho de 1984: Estabelece as Instruções Reguladoras das Normas Técnicas da Cartografia Nacional*. [S.l.], 1984. Acesso em: set. 2026. Disponível em: <https://www.planalto.gov.br/ccivil_03/decreto/1980-1989/d89817.htm>.
- Brasil. *Decreto n. 6.666, de 27 de novembro de 2008: Institui, no âmbito do Poder Executivo federal, a Infra-Estrutura Nacional de Dados Espaciais – INDE, e dá outras providências*. [S.l.], 2008. Publicado no Diário Oficial da União de 28 nov. 2008; redação alterada pelo Decreto n.º 12.402, de 13 de março de 2025. Acesso em: set. 2026. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2008/decreto/d6666.htm>.
- Brasil. Comissão Nacional de Cartografia (CONCAR). *Perfil de Metadados Geoespaciais do Brasil (Perfil MGB)*. [S.l.], 2009. Versão homologada em novembro de 2009; atualizada pelo Perfil MGB 2.0 (IBGE; DSG, 2021). Acesso em: set. 2026. Disponível em: <https://inde.gov.br/pdf/Perfil_MGB_homologado_nov2009_v1.pdf>.
- Brasil. Ministério da Defesa. Exército Brasileiro. Diretoria de Serviço Geográfico. *Especificação Técnica para Produtos de Conjuntos de Dados Geoespaciais (ET-PCDG)*. [S.l.], 2016. Norma EB80-N-72.001, 2.ª edição, aprovada pela Portaria n.º 008-DCT, de 10 de fevereiro de 2016. Acesso em: set. 2026. Disponível em: <<https://geoportal.eb.mil.br/portal/index.php/normas/et-pcdg>>.
- BROWN, T.; MANN, B.; RYDER, N.; SUBBIAH, M.; KAPLAN, J. D.; DHARIWAL, P.; NEELAKANTAN, A.; SHYAM, P.; SASTRY, G.; ASKELL, A. et al. Language models are few-shot learners. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2020. v. 33, p. 1877–1901. Disponível em: <<https://arxiv.org/abs/2005.14165>>.
- CHASE, H. *LangChain: Building applications with LLMs through composability*. 2022. <<https://github.com/langchain-ai/langchain>>. Acesso em: mar. 2026.

- CHEN, X.; LIN, M.; SCHÄRLI, N.; ZHOU, D. Teaching large language models to self-debug. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 8746–8825. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/2460396f2d0d421885997dd1612ac56b-Abstract-Conference.html>.
- COHEN, J. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, v. 20, n. 1, p. 37–46, 1960.
- COLVIN, S. *Pydantic: Data validation using Python type hints*. 2019. <<https://docs.pydantic.dev>>. Acesso em: mar. 2026.
- DIETTERICH, T. G. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, v. 10, n. 7, p. 1895–1923, 1998.
- Diretoria de Serviço Geográfico do Exército Brasileiro. *Acesso ao BDGEx*. 2025. <<https://geoportal.eb.mil.br/portal/index.php/bdgex/acesso-ao-bdgex>>. Geportal do Exército. Acesso em: set. 2026.
- Diretoria de Serviço Geográfico do Exército Brasileiro. *Missão da Diretoria de Serviço Geográfico*. 2025. <<https://geoportal.eb.mil.br/portal/index.php/chefia/missao>>. Geportal do Exército. Acesso em: set. 2026.
- DWORK, C.; FELDMAN, V.; HARDT, M.; PITASSI, T.; REINGOLD, O.; ROTH, A. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, v. 349, n. 6248, p. 636–638, 2015. Disponível em: <<https://doi.org/10.1126/science.aaa9375>>.
- FRANTAR, E.; ASHKBOOS, S.; HOEFLER, T.; ALISTARH, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2023. Disponível em: <<https://arxiv.org/abs/2210.17323>>.
- Gemma Team. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026. Google DeepMind. Disponível em: <<https://arxiv.org/abs/2607.02770>>.
- ggml contributors. *GGUF: file format specification*. 2023. <<https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>>. Formato introduzido no `llama.cpp` em agosto de 2023. Acesso em: set. 2026.
- GOU, Z.; SHAO, Z.; GONG, Y.; SHEN, Y.; YANG, Y.; DUAN, N.; CHEN, W. CRITIC: Large language models can self-correct with tool-interactive critiquing. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 57734–57811. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/fef126561bbf9d4467dbb8d27334b8fe-Abstract-Conference.html>.
- Groq. *Reasoning*. 2026. <<https://console.groq.com/docs/reasoning>>. Documentação do parâmetro `reasoning_effort`. Acesso em: set. 2026.
- HU, Y.; JANOWICZ, K.; PRASAD, S.; GAO, S. Metadata topic harmonization and semantic search for linked-data-driven geoportals: A case study using ArcGIS Online. *Transactions in GIS*, v. 19, n. 3, p. 398–416, 2015. Disponível em: <<https://doi.org/10.1111/tgis.12151>>.

- HUANG, J.; CHEN, X.; MISHRA, S.; ZHENG, H. S.; YU, A. W.; SONG, X.; ZHOU, D. Large language models cannot self-correct reasoning yet. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 32808–32824. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/8b4add8b0aa8749d80a34ca5d941c355-Abstract-Conference.html>.
- HUANG, S.; FANG, Z.; CHEN, Z.; YUAN, S.; YE, J.; ZENG, Y.; CHEN, L.; MAO, Q.; ZHAO, F. CRITICTOOL: Evaluating self-critique capabilities of large language models in tool-calling error scenarios. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, 2025. p. 26672–26704. Disponível em: <<https://aclanthology.org/2025.emnlp-main.1355/>>.
- KHATTAB, O.; SINGHVI, A.; MAHESHWARI, P.; ZHANG, Z.; SANTHANAM, K.; A, S. V.; HAQ, S.; SHARMA, A.; JOSHI, T. T.; MOAZAM, H. et al. DSPy: Compiling declarative language model calls into self-improving pipelines. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 54928–54958. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html>.
- LangChain AI. *LangGraph: Building stateful, multi-actor applications with LLMs*. 2024. <<https://github.com/langchain-ai/langgraph>>. Acesso em: mar. 2026.
- LI, H.; YUE, P.; WU, H.; TENG, B.; ZHAO, Y.; LIU, C. A question-answering framework for geospatial data retrieval enhanced by a knowledge graph and large language models. *International Journal of Digital Earth*, v. 18, n. 1, 2025. Art. 2510566. Disponível em: <<https://doi.org/10.1080/17538947.2025.2510566>>.
- LI, M.; ZHAO, Y.; YU, B.; SONG, F.; LI, H.; YU, H.; LI, Z.; HUANG, F.; LI, Y. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023. p. 3102–3116. Disponível em: <<https://aclanthology.org/2023.emnlp-main.187/>>.
- LI, Z.; NING, H. Autonomous GIS: the next-generation AI-powered GIS. *International Journal of Digital Earth*, v. 16, n. 2, p. 4668–4686, 2023. Disponível em: <<https://doi.org/10.1080/17538947.2023.2278895>>.
- LIU, J. *Instructor: Structured outputs for LLMs*. 2023. <<https://python.useinstructor.com>>. Acesso em: mar. 2026.
- LIU, X.; SHEN, S.; LI, B.; MA, P.; JIANG, R.; ZHANG, Y.; FAN, J.; LI, G.; TANG, N.; LUO, Y. A survey of text-to-SQL in the era of LLMs: Where are we, and where are we going? *IEEE Transactions on Knowledge and Data Engineering*, v. 37, n. 10, p. 5735–5754, 2025. Disponível em: <<https://ieeexplore.ieee.org/document/11095853>>.
- LU, J.; HOLLEIS, T.; ZHANG, Y.; AUMAYER, B.; NAN, F.; BAI, H.; MA, S.; MA, S.; LI, M.; YIN, G. et al. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In: *Findings of the Association for Computational Linguistics: NAACL 2025*. Albuquerque, New Mexico: Association for Computational Linguistics, 2025. p. 1160–1183. Disponível em: <<https://aclanthology.org/2025.findings-naacl.65/>>.

- MADAAN, A.; TANDON, N.; GUPTA, P.; HALLINAN, S.; GAO, L.; WIEGREFFE, S.; ALON, U.; DZIRI, N.; PRABHUMOYE, S.; YANG, Y. et al. Self-Refine: Iterative refinement with self-feedback. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023. v. 36, p. 46534–46594. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html>.
- MAI, G.; HUANG, W.; SUN, J.; SONG, S.; MISHRA, D.; LIU, N.; GAO, S.; LIU, T.; CONG, G.; HU, Y. et al. On the opportunities and challenges of foundation models for GeoAI (vision paper). *ACM Transactions on Spatial Algorithms and Systems*, v. 10, n. 2, p. 1–46, 2024. Disponível em: <<https://doi.org/10.1145/3653070>>.
- MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008. Disponível em: <<https://nlp.stanford.edu/IR-book/>>.
- MANVI, R.; KHANNA, S.; MAI, G.; BURKE, M.; LOBELL, D.; ERMON, S. GeoLLM: Extracting geospatial knowledge from large language models. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 38791–38807. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/a87f2df7c4ab0213c6ea228e7b7f0a4d-Abstract-Conference.html>.
- MCNEMAR, Q. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, v. 12, n. 2, p. 153–157, 1947.
- Mistral AI. *Mistral NeMo*. 2024. <<https://mistral.ai/news/mistral-nemo>>. Acesso em: set. 2026.
- NING, H.; LI, Z.; AKINBOYEWA, T.; LESSANI, M. N. An autonomous GIS agent framework for geospatial data retrieval. *International Journal of Digital Earth*, v. 18, n. 1, 2025. Art. 2458688. Disponível em: <<https://doi.org/10.1080/17538947.2025.2458688>>.
- Ollama. *gpt-oss:20b*. 2025. <<https://ollama.com/library/gpt-oss:20b>>. Biblioteca de modelos do Ollama. Acesso em: set. 2026.
- Ollama Team. *Ollama: Get up and running with large language models locally*. 2023. <<https://ollama.com>>. Acesso em: mar. 2026.
- Open Geospatial Consortium. *OpenGIS Catalogue Services Specification, version 2.0.2 (Corrigendum 2 Release)*. 2007. <<https://docs.ogc.org/is/07-006r1/07-006r1.pdf>>. OGC 07-006r1. Editores: Douglas Nebert, Arliss Whiteside e Panagiotis Vretanos. Acesso em: set. 2026.
- OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025. Disponível em: <<https://arxiv.org/abs/2508.10925>>.
- OUYANG, L.; WU, J.; JIANG, X.; ALMEIDA, D.; WAINWRIGHT, C.; MISHKIN, P.; ZHANG, C.; AGARWAL, S.; SLAMA, K.; RAY, A. et al. Training language models to follow instructions with human feedback. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2022. v. 35, p. 27730–27744. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html>.

- PATIL, S. G.; MAO, H.; YAN, F.; JI, C. C.-J.; SURESH, V.; STOICA, I.; GONZALEZ, J. E. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In: *Proceedings of the 42nd International Conference on Machine Learning*. PMLR, 2025. (Proceedings of Machine Learning Research, v. 267), p. 48371–48392. Disponível em: <<https://proceedings.mlr.press/v267/patil25a.html>>.
- PATIL, S. G.; ZHANG, T.; WANG, X.; GONZALEZ, J. E. Gorilla: Large language model connected with massive APIs. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2024. v. 37, p. 126544–126565. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html>.
- PostGIS Development Team. *PostGIS: Spatial and geographic objects for PostgreSQL*. 2001. <<https://postgis.net>>. Acesso em: mar. 2026.
- QIN, Y.; LIANG, S.; YE, Y.; ZHU, K.; YAN, L.; LU, Y.; LIN, Y.; CONG, X.; TANG, X.; QIAN, B. et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 9695–9717. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/28e50ee5b72e90b50e7196fde8ea260e-Abstract-Conference.html>.
- Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. Disponível em: <<https://arxiv.org/abs/2505.09388>>.
- Qwen Team. *Qwen3.8-27B*. 2026. <<https://huggingface.co/Qwen/Qwen3.8-27B>>. Model card no Hugging Face (Alibaba Cloud); pesos abertos sob licença Apache 2.0. Acesso em: set. 2026.
- RAMÍREZ, S. *FastAPI: Modern, fast web framework for building APIs with Python*. 2018. <<https://fastapi.tiangolo.com>>. Acesso em: mar. 2026.
- ROSS, H.; MAHABALESHWARKAR, A. S.; SUHARA, Y. When2Call: When (not) to call tools. In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Albuquerque, New Mexico: Association for Computational Linguistics, 2025. p. 3391–3409. Disponível em: <<https://aclanthology.org/2025.naacl-long.174/>>.
- SCHICK, T.; DWIVEDI-YU, J.; DESSÌ, R.; RAILEANU, R.; LOMELI, M.; HAMBRO, E.; ZETTLEMOYER, L.; CANCEDDA, N.; SCIALOM, T. Toolformer: Language models can teach themselves to use tools. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023. v. 36, p. 68539–68551. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html>.
- SHINN, N.; CASSANO, F.; GOPINATH, A.; NARASIMHAN, K.; YAO, S. Reflexion: Language agents with verbal reinforcement learning. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023. v. 36, p. 8634–8652. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html>.
- VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. In: *Advances in Neural*

Information Processing Systems. Curran Associates, Inc., 2017. v. 30. Disponível em: <<https://arxiv.org/abs/1706.03762>>.

WEI, J.; TAY, Y.; BOMMASANI, R.; RAFFEL, C.; ZOPH, B.; BORGEAUD, S.; YOGATAMA, D.; BOSMA, M.; ZHOU, D.; METZLER, D. et al. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. Disponível em: <<https://arxiv.org/abs/2206.07682>>.

WILSON, E. B. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, v. 22, n. 158, p. 209–212, 1927.

XAVIER, E. M. A.; MEYER, W. S.; LUNARDI, O. A. Banco de dados geográficos do Exército Brasileiro: arquitetura e resultados. In: *Jornadas Ibéricas de Infraestruturas de Dados Espaciais (JIIDE 2014)*. [s.n.], 2014. Acesso em: set. 2026. Disponível em: <https://www.idee.es/resources/presentaciones/JIIDE14/20141106/BancoDadosGeograficos_presentacion.pdf>.

YANG, C.; WANG, X.; LU, Y.; LIU, H.; LE, Q. V.; ZHOU, D.; CHEN, X. Large language models as optimizers. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2024. p. 12028–12068. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2024/hash/3339f19c5fcee3ad74502947a32be9e6-Abstract-Conference.html>.

YAO, S.; SHINN, N.; RAZAVI, P.; NARASIMHAN, K. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2025. p. 9965–10017. Disponível em: <https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html>.

YAO, S.; ZHAO, J.; YU, D.; DU, N.; SHAFRAN, I.; NARASIMHAN, K.; CAO, Y. ReAct: Synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2023. Disponível em: <<https://arxiv.org/abs/2210.03629>>.

YU, T.; ZHANG, R.; YANG, K.; YASUNAGA, M.; WANG, D.; LI, Z.; MA, J.; LI, I.; YAO, Q.; ROMAN, S. et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018. p. 3911–3921. Disponível em: <<https://aclanthology.org/D18-1425/>>.

ZHANG, Y.; WEI, C.; HE, Z.; YU, W. GeoGPT: An assistant for understanding and processing geospatial tasks. *International Journal of Applied Earth Observation and Geoinformation*, v. 131, 2024. Art. 103976. Disponível em: <<https://doi.org/10.1016/j.jag.2024.103976>>.

ZHAO, A.; HUANG, D.; XU, Q.; LIN, M.; LIU, Y.-J.; HUANG, G. ExpeL: LLM agents are experiential learners. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Association for the Advancement of Artificial Intelligence, 2024. v. 38, n. 17, p. 19632–19642. Disponível em: <<https://ojs.aaai.org/index.php/AAAI/article/view/29936>>.

APÊNDICE A – DATASET DE AVALIAÇÃO

Este apêndice documenta integralmente o *dataset* de avaliação descrito na Seção 3.3: o manual de anotação que define o gabarito, as 310 consultas com as respectivas leituras aceitas, o mapeamento das consultas aos requisitos funcionais e o registro da auditoria. As mesmas informações estão, em formato legível por máquina, no arquivo `data/dataset.json` do repositório (Apêndice C). As listagens deste apêndice são geradas pelo comando `pfc-dataset -apendice <diretório-do-texto>`, a partir das mesmas fontes que produzem esse arquivo (`manual_cases.py` e o gerador da camada G), sem transcrição manual.

Como ler as entradas.

Cada consulta traz o identificador, as categorias (Seção 3.3), o texto exatamente como é enviado ao modelo e as **leituras aceitas** do gabarito, na forma canônica da ferramenta. A notação segue o princípio P3 do manual: “A ou B” indica valor alternativo; “(opcional)” indica campo que pode faltar sem erro, mas que, se presente, precisa ter o valor indicado; “aceita-se também *city* no lugar de *state*” indica leitura estrutural alternativa. Expressões de tempo relativo aparecem pelo nome da regra (“ano corrente”, “semana passada”) e são resolvidas na data de referência da execução, segundo o Quadro 5. Na camada P, uma nota registra a linha de origem no arquivo do protótipo; nas camadas P e N, a nota registra também, quando há, a justificativa das leituras alternativas.

Escopo do gabarito.

O gabarito descreve os **parâmetros** que a consulta deve produzir, e não o conjunto de produtos que a busca resultante devolveria. A avaliação de recuperação em nível de produto está fora do escopo do trabalho (Seção 3.1.2).

A.1 Manual de anotação

O gabarito de cada consulta foi construído segundo as regras abaixo. Elas são a referência para a redação e a revisão das consultas, para a auditoria automática e para a anotação independente descritas na Seção A.14; qualquer anotação que as contrarie é considerada erro do gabarito, e não do modelo.

Princípios

- P1 — Evidência explícita.** O gabarito contém somente parâmetros para os quais existe um trecho da consulta que os justifica. Nada é inferido além do que o texto diz (“cartas de Manaus” não implica `state = "Amazonas"`).
- P2 — Forma canônica.** Todo valor é escrito na forma canônica da ferramenta `buscar_catalogo`: enumerados exatamente como no *schema*; estados e municípios por extenso, com acentos; datas em ISO 8601.
- P3 — Leituras múltiplas explícitas.** Quando a consulta admite mais de uma leitura razoável e nem o texto nem a descrição da ferramenta desempatam, o gabarito lista todas as leituras aceitas, sendo a primeira a preferencial. Há três mecanismos: *valor alternativo* (o campo aceita mais de um valor), *campo opcional* (pode faltar; se aparecer, precisa ter o valor indicado) e *leitura estrutural alternativa* (o mesmo trecho pode ocupar campos diferentes, como estado ou município).
- P4 — Fora do domínio.** Consultas sem intenção de busca no acervo cartográfico brasileiro têm como gabarito “não chamar a ferramenta”.
- P5 — Consistência.** A mesma expressão recebe a mesma anotação em todo o *dataset*.
- P6 — Casos observacionais.** Consultas cujo comportamento correto não é determinável a partir do texto (valor impossível de representar, comportamento esperado discutível) são executadas e reportadas, mas ficam fora das métricas principais. Consultas fora do domínio (P4) não são observacionais, pois o comportamento correto é determinado.

Regras por campo

- keyword** Código MI com seus sufixos, sem o prefixo “MI”; código INOM na grafia com hífen (grafias compactadas, como “sf22yd”, são normalizadas); nome próprio de carta quando precedido de “carta” ou “folha”. Se o nome da carta coincide com um município, aceita-se também `city`. Havendo dois códigos, aceita-se qualquer um deles, pois o *schema* comporta um só.
- scale** Escala explícita em qualquer grafia (“1:25.000”, “1:25000”, “25k”, “25 mil”). As convenções informadas na descrição da ferramenta valem como regra (“detalhada” → 1:25.000; “pequena escala” → 1:250.000). Expressões qualitativas não informadas ao modelo aceitam o conjunto de escalas que as satisfazem: “grande escala” → 1:25.000 ou maior; “média escala” → 1:50.000 ou 1:100.000; “maior que 100k” → 1:50.000 ou maior. Duas escalas explícitas: aceita-se qualquer uma.

productType Sinônimos informados na ferramenta (“topo” e “topográfica”; “orto” e “ortoimg”; “mdt”) e demais tipos pelo nome. “Cartas”, “mapas”, “produtos” e “folhas” sozinhos não definem tipo.

state, city Siglas e grafias sem acento são normalizadas. “Rio de Janeiro”, “São Paulo” e “rio” sem qualificador admitem as leituras estado ou município; siglas são sempre estado. Regiões (“sul”, “Nordeste”, “Amazônia Legal”) não são estados e não são anotadas, pois o *schema* não tem campo de região.

supplyArea Qualquer menção ao Centro de Geoinformação (“1º CGEO”, “1o cgeo”, “primeiro cgeo”, “dois cgeo”).

project Pelo nome ou pelos apelidos informados na ferramenta, com ou sem acento. A menção explícita a um projeto por um termo que o identifica sem ambiguidade no enumerado também vale como nome (“projeto rondonia” e “BCD de Rondônia”, sigla de Base Cartográfica Digital, para Base Cartográfica Digital de Rondônia). Expressão que designa o projeto só pelo termo distintivo do nome, sem a palavra “projeto”, sem sigla do nome e sem apelido informado (“cartografia sistemática” para Mapeamento Sistemático), torna o campo opcional: a consulta sustenta a leitura, mas a ferramenta não a informa.

Períodos Verbo de publicação (“publicad-”) decide `publicationPeriod`; verbo de criação (“criad-”, “feit-”, “elaborad-”, “produzid-”) decide `creationPeriod`; sem verbo que desempate, a leitura preferencial é `publicationPeriod` e `creationPeriod` é leitura estrutural alternativa, com o mesmo intervalo. O intervalo é resolvido com a data de referência da execução — a mesma informada ao modelo — segundo o Quadro 5.

Ordenação “Mais recente(s)”, “último(s)” → `DESC`; “mais antiga(s)”, “primeiro(s)”, “ordem cronológica” → `ASC`. Pista de criação ligada à ordenação (“primeiro mapeamento feito”, “última atualização”, “criadas mais recentemente”) → `creationDate`; pista de publicação (“últimas publicações”, “última carta publicada”) → `publicationDate`; sem pista, aceita-se qualquer dos dois campos. Número explícito → `limit`; singular sem número (“a carta mais recente”) → `limit = 1` opcional; plural sem número → sem `limit`. Ordenação nunca é anotada por padrão.

Consultas fora do domínio

A ferramenta não deve ser chamada quando a consulta trata de outro assunto (clima, preço, ficção), pede território fora do Brasil ou fora da Terra, ou não tem intenção

Quadro 5 – Leituras aceitas para as expressões de tempo relativo (D = data de referência da execução)

Expressão	Leituras aceitas (a primeira é a preferencial)
esse ano / este ano / deste ano	ano inteiro; 1º de janeiro até D
ano passado	ano anterior inteiro
2 anos atrás	o ano de dois anos antes, inteiro; D menos 2 anos até D
mês passado	mês anterior inteiro
este mês / deste mês	dia 1 até D; mês inteiro
semana passada	D-7 até D-1; D-7 até D; semana-calendário anterior (segunda a domingo)
esta semana / nesta semana	segunda-feira até D; segunda a domingo
hoje	D a D
últimos 3 (6) meses	D-90 (180) dias até D; mesmo dia, 3 (6) meses antes, até D
últimos 5 anos	D menos 5 anos até D; 1º de janeiro do ano-5 até D
desde AAAA	AAAA-01-01 até D; AAAA-01-01 sem limite final
depois de AAAA	início em AAAA-01-01 ou (AAAA+1)-01-01; fim em D ou sem limite final
antes de AAAA	sem limite inicial até (AAAA-1)-12-31
primeiro trimestre (deste ano)	janeiro a março do ano de D
segundo semestre do ano passado	julho a dezembro do ano anterior
último trimestre do ano passado	outubro a dezembro do ano anterior
trimestre passado	trimestre-calendário anterior ao de D

Fonte: Elaborado pelos autores. Implementação: `relative_time.py` do repositório.

de busca. Essas consultas formam a categoria F (fora do domínio) e entram nas métricas principais: a resposta é correta quando a ferramenta não é chamada. Consultas com valor impossível de representar (“escala 1:10.000.000”, “estado 42”) ou cujo comportamento esperado é discutível (“cartas do futuro”, “cartas topo no oceano atlântico”) são observacionais.

Rastreabilidade

Cada caso registra a origem. Na camada P, a fonte é o arquivo de testes do protótipo (`backend/evaluation/test-cases.ts`), com a linha da consulta; a anotação original da equipe do 1º CGEO é mantida como leitura preferencial, e as leituras alternativas acrescentadas seguem este manual. Na camada N, a consulta foi redigida pelos autores, e a justificativa de cada leitura alternativa fica registrada em nota. Na camada G, a consulta foi gerada pelo *script* gerador (semente 42), que registra a função geradora e o modelo de frase; o gabarito é conhecido por construção.

Procedimento de auditoria

1. **Checagens automáticas (pfc-auditar):** consultas duplicadas ou contraditórias; validade de enumerados e campos; resolução de todas as expressões de tempo em datas diversas; e um anotador baseado em regras, independente do gabarito, que localiza na consulta o trecho que justifica cada campo e aponta campo anotado sem evidência, evidência sem campo anotado e valor divergente.
2. **Anotação independente às cegas:** cada consulta é anotada novamente, sem acesso ao gabarito, a partir apenas deste manual e da definição da ferramenta. A concordância é medida em rigor crescente: decisão (chamar, não chamar ou observacional, com kappa de Cohen); leituras compatíveis nos dois sentidos; leitura preferencial idêntica; presença e valor de cada campo; e detecção de ambiguidade, também com kappa de Cohen.
3. **Adjudicação:** cada consulta com divergência em qualquer das medidas recebe uma decisão registrada — gabarito mantido, corrigido ou ampliado com leitura alternativa —, com justificativa por referência a este manual e com o gabarito anterior à decisão, a partir do qual a concordância anterior à adjudicação é recalculada.
4. O processo se repete até não restar divergência sem decisão.
5. **Revisão humana de amostra:** uma amostra estratificada por origem (40 consultas, semente 42) é conferida por uma pessoa, que marca se concorda com as leituras aceitas.

Extensões para o lote de validação

O lote de validação (Seção 3.5) cobre situações que as 310 consultas não tinham. As regras abaixo estendem o manual sem alterar nenhuma anotação das 310.

Consulta subespecificada (categoria E). A consulta pede produtos do acervo, mas não traz nenhum critério representável no *schema* (“quero ver as cartas do acervo”, “cartas do Nordeste”, “mapas para uma trilha”). O gabarito não tem parâmetros e aceita duas respostas: buscar sem filtros ou não buscar (pedir esclarecimento ou recusar). Uma chamada com qualquer parâmetro é erro. Difere de F porque há intenção de busca no acervo (“me ajuda” e “quero um mapa bonito” continuam F) e da categoria S porque não há nenhum critério (“ortoimagens” é consulta simples e exige a busca).

Tempo relativo parametrizado. As regras do Quadro 5 valem para qualquer quantidade e qualquer ano: “últimos N meses” ($D-30N$ dias até D ; mesmo dia, N meses

antes, até D); “últimos N anos” (D menos N anos até D; 1º de janeiro do ano— N até D); “últimos N dias” (D— N até D; D—($N - 1$) até D); “desde” e “a partir de AAAA”; “depois de” e “após AAAA”; “antes de” e “anteriores a AAAA”. Ano isolado (“em AAAA”, “de AAAA”) e intervalo de anos (“entre AAAA e BBBB”) têm leitura única, do primeiro ao último dia.

Municípios e UF. Qualquer município da lista oficial do IBGE é **city**; a sigla da UF junto ao município (“Campinas, SP”) anota também **state**. O lote não usa municípios homônimos de UF.

Nomes de folha reais. Nomes de folha do BDGEx precedidos de “carta” ou “folha” são **keyword**; quando coincidem com o nome de um município, vale a leitura estrutural **city**.

Armadilhas de vocabulário. “Carta”, “mapa”, “folha”, “escala” e “projeto” em outro sentido (“carta de vinhos”, “mapa astral”, “folha de pagamento”, “escala de plantão”, “projeto de lei”) não pedem produto cartográfico: categoria F.

A.2 Consultas do protótipo (P01–P22)

As 22 consultas do arquivo de testes do protótipo do 1º CGEO (**backend/evaluation/test-cases.ts**), na ordem do arquivo. A anotação original da equipe do 1º CGEO é mantida como leitura preferencial; as leituras alternativas acrescentadas seguem o manual. Os períodos relativos, que no arquivo original estavam fixados em datas calculadas para o início de 2025 (por exemplo, 2024-10-01 a 2024-12-31 para o “último trimestre do ano passado” de P15), são expressos pelas regras nomeadas do Quadro 5.

ID	P01
Categorias	M · C
Consulta	<i>“preciso da carta MI 2965-2-NE do segundo cgeo”</i>
Leituras aceitas	keyword: "2965-2-NE" supplyArea: "2º Centro de Geoinformação"
Nota	origem: test-cases.ts do protótipo, linha 6

ID	P02
Categorias	M · C
Consulta	<i>“folha MI 2866-3 em escala 50k”</i>
Leituras aceitas	keyword: "2866-3" scale: "1:50.000"
Nota	origem: test-cases.ts do protótipo, linha 16

ID	P03
Categorias	M · C · A
Consulta	<i>“MI 2901 do projeto olimpiadas”</i>
Leituras aceitas	keyword: "2901" project: "Olimpíadas Rio 2016"
Nota	origem: test-cases.ts do protótipo, linha 26

ID	P04
Categorias	M · C · A
Consulta	<i>“carta 530 em pequena escala”</i>
Leituras aceitas	keyword: "530" scale: "1:250.000"
Nota	origem: test-cases.ts do protótipo, linha 36

ID	P05
Categorias	M · C · T
Consulta	<i>“buscar folha SF-22-Y-D-II-4 do RS publicada esse ano”</i>
Leituras aceitas	keyword: "SF-22-Y-D-II-4" state: "Rio Grande do Sul" publicationPeriod: ano corrente
Nota	origem: test-cases.ts do protótipo, linha 48

ID	P06
Categorias	M · C · A
Consulta	<i>“carta SF-22-Y-D do tipo ortoimg”</i>
Leituras aceitas	keyword: "SF-22-Y-D" productType: "SCN Carta Ortoimagem"
Nota	origem: test-cases.ts do protótipo, linha 62

ID	P07
Categorias	M · C · A
Consulta	<i>“inom SF-22-Y-D-II-4-SE criado pelo primeiro cgeo”</i>
Leituras aceitas	keyword: "SF-22-Y-D-II-4-SE" supplyArea: "1° Centro de Geoinformação"
Nota	origem: test-cases.ts do protótipo, linha 72

ID	P08
Categorias	C
Consulta	<i>“carta topográfica Passo da Seringueira em 1:25.000”</i>
Leituras aceitas	keyword: "Passo da Seringueira" scale: "1:25.000" productType: "SCN Carta Topográfica Matricial"
Nota	origem: test-cases.ts do protótipo, linha 84

ID	P09
Categorias	C
Consulta	<i>“folha Porto Velho publicada em 2024”</i>
Leituras aceitas	keyword: "Porto Velho" publicationPeriod: 2024-01-01 a 2024-12-31 aceita-se também city no lugar de keyword
Nota	origem: test-cases.ts do protótipo, linha 95; nome de carta coincide com município: aceita-se também city

ID	P10
Categorias	C · A
Consulta	<i>“carta Vale do Guaporé do projeto rondonia”</i>
Leituras aceitas	keyword: "Vale do Guaporé" project: "Base Cartográfica Digital de Rondônia"
Nota	origem: test-cases.ts do protótipo, linha 108

ID	P11
Categorias	M · C · A
Consulta	<i>“MI 2965-2-NE e SF-22-Y-D do terceiro cgeo em grande escala”</i>
Leituras aceitas	keyword: "2965-2-NE"ou "SF-22-Y-D" supplyArea: "3° Centro de Geoinformação" scale: "1:25.000"ou "1:10.000"ou "1:5.000"ou "1:2.000"ou "1:1.000"
Nota	origem: test-cases.ts do protótipo, linha 120; dois códigos (o schema comporta um): aceita-se qualquer um; 'grande escala' aceita as escalas grandes

ID	P12
Categorias	M · C · T · A
Consulta	<i>“cartas do tipo topo com MI 530 ou SF-22 publicadas esse ano”</i>
Leituras aceitas	keyword: "530"ou "SF-22" productType: "SCN Carta Topográfica Matricial" publicationPeriod: ano corrente
Nota	origem: test-cases.ts do protótipo, linha 131; dois códigos: aceita-se qualquer um

ID	P13
Categorias	C · A
Consulta	<i>“todas as cartas em escala maior que 100k do rio grande do sul”</i>
Leituras aceitas	state: "Rio Grande do Sul" scale: "1:25.000"ou "1:50.000"ou "1:10.000"ou "1:5.000"ou "1:2.000"ou "1:1.000"
Nota	origem: test-cases.ts do protótipo, linha 147; 'maior que 100k' aceita qualquer escala maior que 1:100.000

ID	P14
Categorias	C · A
Consulta	<i>“mapeamento detalhado (25k ou 50k) de brasilia”</i>
Leituras aceitas	scale: "1:25.000"ou "1:50.000" city: "Brasília"
Nota	origem: test-cases.ts do protótipo, linha 156; duas escalas explícitas: aceita-se qualquer uma

ID	P15
Categorias	T · C · A
Consulta	<i>“cartas criadas no último trimestre do ano passado do segundo cgeo”</i>
Leituras aceitas	supplyArea: "2° Centro de Geoinformação" creationPeriod: 4º trimestre do ano anterior
Nota	origem: test-cases.ts do protótipo, linha 167

ID	P16
Categorias	T · C · A
Consulta	<i>“produtos da semana passada do terceiro cgeo”</i>
Leituras aceitas	supplyArea: "3º Centro de Geoinformação" publicationPeriod: semana passada aceita-se também creationPeriod no lugar de publicationPeriod
Nota	origem: test-cases.ts do protótipo, linha 179; sem verbo que decida publicação ou criação

ID	P17
Categorias	C · A
Consulta	<i>“cartas do mapeamento sistematico em goias”</i>
Leituras aceitas	project: "Mapeamento Sistemático" state: "Goiás"
Nota	origem: test-cases.ts do protótipo, linha 193

ID	P18
Categorias	C · A
Consulta	<i>“produtos da copa das confederacoes em fortaleza”</i>
Leituras aceitas	project: "Copa das Confederações" city: "Fortaleza"
Nota	origem: test-cases.ts do protótipo, linha 202

ID	P19
Categorias	O · C · A
Consulta	<i>“primeiro mapeamento feito em cuiaba”</i>
Leituras aceitas	city: "Cuiabá" sortField: "creationDate" sortDirection: "ASC" limit: 1 (opcional)
Nota	origem: test-cases.ts do protótipo, linha 213; singular sem número: limit 1 opcional

ID	P20
Categorias	O · C · A
Consulta	<i>“ultima atualizacao do para”</i>
Leituras aceitas	state: "Pará" sortField: "creationDate" sortDirection: "DESC" limit: 1 (opcional)
Nota	origem: test-cases.ts do protótipo, linha 223; singular sem número: limit 1 opcional

ID	P21
Categorias	O · T · C · A
Consulta	<i>“preciso das 5 cartas mais antigas do tipo ortoimg em escala detalhada do terceiro cgeo em pernambuco publicadas depois de 2020”</i>
Leituras aceitas	limit: 5 productType: "SCN Carta Ortoimagem" scale: "1:25.000" supplyArea: "3° Centro de Geoinformação" state: "Pernambuco" publicationPeriod: depois de 2020 sortField: "creationDate"ou "publicationDate" sortDirection: "ASC"
Nota	origem: test-cases.ts do protótipo, linha 235; 'mais antigas' sem pista ligada à ordenação: aceita-se publicationDate além do creationDate original

ID	P22
Categorias	M · O · T · C
Consulta	<i>“MI 2901 ou SF-22 do quarto cgeo no sudeste em pequena escala criado entre 2022 e 2023 ordem cronologica”</i>
Leituras aceitas	keyword: "2901"ou "SF-22" supplyArea: "4° Centro de Geoinformação" scale: "1:250.000" creationPeriod: 2022-01-01 a 2023-12-31 sortField: "creationDate"ou "publicationDate" sortDirection: "ASC"
Nota	origem: test-cases.ts do protótipo, linha 253; dois códigos; 'sudeste' é região (nada a anotar); ordem cronológica sem pista ligada

A.3 Consultas redigidas pelos autores (N01–N40)

Consultas redigidas para preencher lacunas de distribuição — em especial a categoria Simples, ausente na camada P —, exercitar as ambiguidades previstas no manual (estado ou capital, ordenação sem pista, regiões e grafias informais de escala, como “25k” e “cem k”) e cobrir a fronteira do domínio: N36 e N37 estão fora do domínio (categoria F), e N38 a N40 são observacionais.

ID	N01
Categorias	S
Consulta	<i>“cartas de São Paulo”</i>
Leituras aceitas	state: "São Paulo" aceita-se também city no lugar de state
Nota	'São Paulo' sem qualificador: estado ou capital

ID	N02
Categorias	S
Consulta	<i>“produtos em escala 1:50.000”</i>
Leituras aceitas	scale: "1:50.000"

ID	N03
Categorias	S
Consulta	<i>“ortoimagens”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem"

ID	N04
Categorias	S · A
Consulta	<i>“mapas do rj”</i>
Leituras aceitas	state: "Rio de Janeiro"

ID	N05
Categorias	S · A
Consulta	<i>“carta 25k”</i>
Leituras aceitas	scale: "1:25.000"

ID	N06
Categorias	S
Consulta	<i>“folha SF-22-Y-D”</i>
Leituras aceitas	keyword: "SF-22-Y-D"

ID	N07
Categorias	S · A
Consulta	<i>“modelos digitais de terreno”</i>
Leituras aceitas	productType: "MDT - RAM"

ID	N08
Categorias	S · A
Consulta	<i>“cartas topográficas”</i>
Leituras aceitas	productType: "SCN Carta Topográfica Matricial"

ID	N09
Categorias	S
Consulta	<i>“produtos do Amazonas”</i>
Leituras aceitas	state: "Amazonas"

ID	N10
Categorias	S · A
Consulta	<i>“mi 2901”</i>
Leituras aceitas	keyword: "2901"

ID	N11
Categorias	C
Consulta	<i>“cartas do 1º CGEO em Minas Gerais”</i>
Leituras aceitas	supplyArea: "1º Centro de Geoinformação" state: "Minas Gerais"

ID	N12
Categorias	C
Consulta	<i>“ortoimagens de Manaus em 1:50.000”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem" city: "Manaus" scale: "1:50.000"

ID	N13
Categorias	C · A
Consulta	<i>“mdt do para em 25k”</i>
Leituras aceitas	productType: "MDT - RAM" state: "Pará" scale: "1:25.000"

ID	N14
Categorias	M · C
Consulta	<i>“folha 2965-2 do Rio Grande do Sul”</i>
Leituras aceitas	keyword: "2965-2" state: "Rio Grande do Sul"

ID	N15
Categorias	M · C
Consulta	<i>“INOM SG-22-X-A”</i>
Leituras aceitas	keyword: "SG-22-X-A"

ID	N16
Categorias	M · C · T
Consulta	<i>“carta MI 3010 publicada em 2023”</i>
Leituras aceitas	keyword: "3010" publicationPeriod: 2023-01-01 a 2023-12-31

ID	N17
Categorias	T · A
Consulta	<i>“produtos publicados nos últimos 3 meses”</i>
Leituras aceitas	publicationPeriod: últimos 3 meses

ID	N18
Categorias	T · C
Consulta	<i>“cartas do ano passado em Roraima”</i>
Leituras aceitas	state: "Roraima" publicationPeriod: ano anterior, inteiro aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N19
Categorias	T · A
Consulta	<i>“atualizações deste mês”</i>
Leituras aceitas	publicationPeriod: mês corrente aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N20
Categorias	T · C · A
Consulta	<i>“cartas dos últimos 5 anos em escala 100k”</i>
Leituras aceitas	scale: "1:100.000" publicationPeriod: últimos 5 anos aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N21
Categorias	T · A
Consulta	<i>“qualquer coisa de hoje”</i>
Leituras aceitas	publicationPeriod: dia da execução aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N22
Categorias	T · C · A
Consulta	<i>“ortoimagens de 2 anos atrás”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem" publicationPeriod: ano de dois anos antes aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N23
Categorias	T · C · A
Consulta	<i>“produtos do trimestre passado do 4º CGEO”</i>
Leituras aceitas	supplyArea: "4º Centro de Geoinformação" publicationPeriod: trimestre anterior aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida publicação ou criação

ID	N24
Categorias	O · C
Consulta	<i>“3 cartas mais recentes de Curitiba”</i>
Leituras aceitas	city: "Curitiba" limit: 3 sortField: "publicationDate"ou "creationDate" sortDirection: "DESC"
Nota	'mais recentes' sem pista: aceita-se publicationDate ou creationDate

ID	N25
Categorias	O · C
Consulta	<i>“10 primeiras ortoimagens do Nordeste”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem" limit: 10 sortField: "publicationDate"ou "creationDate" sortDirection: "ASC"
Nota	'Nordeste' é região (nada a anotar); 'primeiras' sem pista

ID	N26
Categorias	O · C · A
Consulta	<i>“mais antigas em rondonia”</i>
Leituras aceitas	state: "Rondônia" sortField: "publicationDate"ou "creationDate" sortDirection: "ASC"
Nota	'mais antigas' sem pista

ID	N27
Categorias	O · C
Consulta	<i>“últimas 5 publicações”</i>
Leituras aceitas	limit: 5 sortField: "publicationDate" sortDirection: "DESC"

ID	N28
Categorias	O · M · C
Consulta	<i>“carta MI 2901 mais recente”</i>
Leituras aceitas	keyword: "2901" sortField: "publicationDate"ou "creationDate" sortDirection: "DESC" limit: 1 (opcional)
Nota	singular sem número: limit 1 opcional; 'mais recente' sem pista

ID	N29
Categorias	O · C · T
Consulta	<i>“três produtos mais antigos deste ano”</i>
Leituras aceitas	limit: 3 publicationPeriod: ano corrente sortField: "publicationDate"ou "creationDate" sortDirection: "ASC" aceita-se também creationPeriod no lugar de publicationPeriod
Nota	sem verbo que decida o período; 'mais antigos' sem pista

ID	N30
Categorias	A · C
Consulta	<i>“cartas topo do primeiro cgeo”</i>
Leituras aceitas	productType: "SCN Carta Topográfica Matricial" supplyArea: "1° Centro de Geoinformação"

ID	N31
Categorias	A · S
Consulta	<i>“orto banda p”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem Banda P Pol HH"

ID	N32
Categorias	A · C
Consulta	<i>“mapas do rio em cem k”</i>
Leituras aceitas	state: "Rio de Janeiro" scale: "1:100.000" aceita-se também city no lugar de state
Nota	'rio' é ambíguo entre o estado e a capital

ID	N33
Categorias	A · C
Consulta	<i>“cartografia sistematica sp”</i>
Leituras aceitas	project: "Mapeamento Sistemático"(opcional) state: "São Paulo"
Nota	'cartografia sistemática' designa o programa pelo termo distintivo do nome, sem ser nome nem apelido informado na ferramenta: project é opcional (regra de project; adjudicação)

ID	N34
Categorias	A · C
Consulta	<i>“ortos da amazonia legal”</i>
Leituras aceitas	productType: "SCN Carta Ortoimagem"
Nota	'Amazônia Legal' é região (nove estados), não estado: nada a anotar para ela

ID	N35
Categorias	A · M · C
Consulta	<i>“sf22yd do dois cgeo”</i>
Leituras aceitas	keyword: "SF-22-Y-D" supplyArea: "2° Centro de Geoinformação"

ID	N36
Categorias	F
Consulta	<i>“cartas fora do brasil”</i>
Leituras aceitas	não chamar a ferramenta (fora do território brasileiro, P4)

ID	N37
Categorias	F
Consulta	<i>“quero um mapa bonito”</i>
Leituras aceitas	não chamar a ferramenta (sem critério de busca, P4)

ID	N38
Categorias	S · A
Consulta	<i>“escala 1:10.000.000”</i>
Leituras aceitas	não chamar a ferramenta (observacional, P6: escala fora do enumerado da ferramenta)

ID	N39
Categorias	C · A
Consulta	<i>“cartas topo no oceano atlântico”</i>
Leituras aceitas	productType: "SCN Carta Topográfica Matricial" caso observacional (P6): executado e reportado, fora das métricas principais
Nota	área não terrestre: comportamento esperado discutível

ID	N40
Categorias	C · T
Consulta	<i>“cartas do futuro (ano 2050)”</i>
Leituras aceitas	publicationPeriod: 2050-01-01 a 2050-12-31 aceita-se também creationPeriod no lugar de publicationPeriod caso observacional (P6): executado e reportado, fora das métricas principais
Nota	data futura: comportamento esperado discutível

A.4 Geração programática da camada G

As 248 consultas da camada G são produzidas pelo *script* `scripts/generate_dataset_paper.py` do repositório (Apêndice C), a partir de catálogos

fechados do domínio: as 27 unidades da federação (com preposição, grafias usuais e, exceto para Sergipe, sigla), 12 municípios de referência sem homônimo entre as UFs, as 8 escalas do SCN (com apelidos como “25k”, “25 mil” e as convenções “detalhada” e “pequena escala”), os 9 tipos de produto das ET-PCDG, os 5 CGEOs, os 9 projetos institucionais, 12 códigos MI e 10 códigos INOM, 15 expressões de tempo (14 relativas e o intervalo absoluto “entre 2022 e 2023”) e os padrões de ordenação. Oito funções geradoras, uma por família, combinam esses catálogos em modelos de frase do português corrente.

Três propriedades tornam a camada auditável. A geração é **determinística**: um único gerador pseudoaleatório, com semente 42, consumido em ordem fixa, de modo que a mesma versão do *script* produz sempre as mesmas consultas. O gabarito é conhecido **por construção**: a frase é montada a partir dos parâmetros que deve produzir, e cada consulta registra no *dataset* a função geradora e o modelo de frase que a originaram. E **nenhuma consulta se repete**: a comparação é feita após normalização de caixa, acentos e pontuação, dentro da camada e em relação às camadas P e N. As convenções de anotação são as mesmas do manual; em particular, “São Paulo” e “Rio de Janeiro” por extenso recebem as leituras estado ou município, e frases sem verbo de publicação ou criação aceitam os dois campos de período.

As seções seguintes listam as 248 consultas por família. Os identificadores seguem o padrão $G\{\text{família}\}\{\text{sequência}\}$, em que a família é S (simples), C (compostas), M (código MI/INOM), T (tempo relativo), O (ordenação), A (informais), P (amplas, com muitos produtos) ou F (fronteira do domínio).

A.5 Consultas simples geradas por *templates* (G-S)

Total desta família: **61 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GS001	S	cartas do Acre	state: "Acre"
GS002	S · A	cartas de AL	state: "Alagoas"
GS003	S · A	cartas do AP	state: "Amapá"
GS004	S	cartas do Amazonas	state: "Amazonas"
GS005	S · A	cartas da bahia	state: "Bahia"
GS006	S · A	cartas do CE	state: "Ceará"
GS007	S · A	cartas do DF	state: "Distrito Federal"
GS008	S · A	cartas do ES	state: "Espírito Santo"
GS009	S	cartas de Goiás	state: "Goiás"
GS010	S · A	cartas do MA	state: "Maranhão"
GS011	S	cartas de Mato Grosso	state: "Mato Grosso"

ID	Cat.	Consulta	Leituras aceitas
GS012	S	cartas de Mato Grosso do Sul	state: "Mato Grosso do Sul"
GS013	S · A	cartas de MG	state: "Minas Gerais"
GS014	S	cartas do Pará	state: "Pará"
GS015	S · A	cartas da paraíba	state: "Paraíba"
GS016	S · A	produtos na escala 1:1000	scale: "1:1.000"
GS017	S · A	produtos na escala 1:2000	scale: "1:2.000"
GS018	S · A	produtos na escala 1:5000	scale: "1:5.000"
GS019	S · A	produtos na escala 1:10000	scale: "1:10.000"
GS020	S · A	produtos na escala 1:25000	scale: "1:25.000"
GS021	S · A	produtos na escala de 50 mil	scale: "1:50.000"
GS022	S · A	produtos na escala de 100 mil	scale: "1:100.000"
GS023	S · A	produtos em 250k	scale: "1:250.000"
GS024	S · A	cartas topográficas disponíveis	productType: "SCN Carta Topográfica Matricial"
GS025	S · A	cartas vetoriais disponíveis	productType: "SCN Carta Topográfica Vetorial"
GS026	S · A	ortoimagens disponíveis	productType: "SCN Carta Ortoimagem"
GS027	S · A	ortoimagens banda P HH disponíveis	productType: "SCN Carta Ortoimagem Banda P Pol HH"
GS028	S · A	ortoimagens banda X HH disponíveis	productType: "SCN Carta Ortoimagem Banda X Pol HH"
GS029	S · A	MDTs disponíveis	productType: "MDT — RAM"
GS030	S · A	MDSs disponíveis	productType: "MDS — RAM"
GS031	S · A	cartas CIRC disponíveis	productType: "CIRC"
GS032	S · A	mapas temáticos disponíveis	productType: "Cartas Temáticas Não SCN"
GS033	S · A	produtos do primeiro cgeo	supplyArea: "1° Centro de Geoinformação"
GS034	S · A	produtos do 2o cgeo	supplyArea: "2° Centro de Geoinformação"
GS035	S · A	produtos do 3o cgeo	supplyArea: "3° Centro de Geoinformação"
GS036	S · A	produtos do quarto cgeo	supplyArea: "4° Centro de Geoinformação"
GS037	S · A	produtos do 5o CGEO	supplyArea: "5° Centro de Geoinformação"
GS038	S	cartas de Brasília	city: "Brasília"
GS039	S	cartas de Manaus	city: "Manaus"
GS040	S	cartas de Porto Velho	city: "Porto Velho"
GS041	S	cartas de Cuiabá	city: "Cuiabá"
GS042	S	cartas de Fortaleza	city: "Fortaleza"
GS043	S	cartas de Salvador	city: "Salvador"
GS044	S	cartas de Belo Horizonte	city: "Belo Horizonte"
GS045	S	cartas de Curitiba	city: "Curitiba"
GS046	S	cartas de Porto Alegre	city: "Porto Alegre"

ID	Cat.	Consulta	Leituras aceitas
GS047	S	cartas de Recife	city: "Recife"
GS048	S · M	carta MI 2965-2-NE	keyword: "2965-2-NE"
GS049	S · M	carta MI 2866-3	keyword: "2866-3"
GS050	S · M	carta MI 2901	keyword: "2901"
GS051	S · M	carta MI 530	keyword: "530"
GS052	S · M	carta MI 3010	keyword: "3010"
GS053	S · M	carta MI 2965	keyword: "2965"
GS054	S · M	carta MI 2866-2-SO	keyword: "2866-2-SO"
GS055	S · M	carta MI 2901-4-NE	keyword: "2901-4-NE"
GS056	S · M	folha INOM SF-22-Y-D	keyword: "SF-22-Y-D"
GS057	S · M	folha INOM SF-22-Y-D-II	keyword: "SF-22-Y-D-II"
GS058	S · M	folha INOM SF-22-Y-D-II-4	keyword: "SF-22-Y-D-II-4"
GS059	S · M	folha INOM SF-22-Y-D-II-4-SE	keyword: "SF-22-Y-D-II-4-SE"
GS060	S · M	folha INOM SG-22-X-A	keyword: "SG-22-X-A"
GS061	S · M	folha INOM SF-23-V-C	keyword: "SF-23-V-C"

A.6 Consultas compostas geradas por *templates* (G-C)

Total desta família: **55 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GC001	C · A	cartas do amapá na escala 1:2.000	state: "Amapá"; scale: "1:2.000"
GC002	C · A	cartas de Mato Grosso do Sul em 25k	state: "Mato Grosso do Sul"; scale: "1:25.000"
GC003	C · A	cartas de santa catarina na escala 1:2.000	state: "Santa Catarina"; scale: "1:2.000"
GC004	C · A	cartas do Amapá na escala de 25 mil	state: "Amapá"; scale: "1:25.000"
GC005	C · A	cartas do RJ na escala 1:2000	state: "Rio de Janeiro"; scale: "1:2.000"
GC006	C · A	cartas de RO em 25k	state: "Rondônia"; scale: "1:25.000"
GC007	C · A	cartas do ES na escala 1:100000	state: "Espírito Santo"; scale: "1:100.000"
GC008	C · A	cartas da Paraíba em 50k	state: "Paraíba"; scale: "1:50.000"
GC009	C · A	cartas de mato grosso do sul na escala de 10 mil	state: "Mato Grosso do Sul"; scale: "1:10.000"
GC010	C · A	cartas de Goiás na escala 1:2000	state: "Goiás"; scale: "1:2.000"
GC011	C · A	cartas do Piauí na escala 1:10000	state: "Piauí"; scale: "1:10.000"
GC012	C · A	cartas da paraíba na escala 1:25000	state: "Paraíba"; scale: "1:25.000"
GC013	C · A	cartas de rondônia na escala 1:1000	state: "Rondônia"; scale: "1:1.000"
GC014	C · A	cartas do TO na escala 1:50000	state: "Tocantins"; scale: "1:50.000"
GC015	C · A	cartas de GO na escala de 10 mil	state: "Goiás"; scale: "1:10.000"
GC016	C · A	cartas de roraima na escala de 10 mil	state: "Roraima"; scale: "1:10.000"
GC017	C · A	cartas do paraná na escala 1:250000	state: "Paraná"; scale: "1:250.000"

ID	Cat.	Consulta	Leituras aceitas
GC018	C · A	cartas do Piauí em escala detalhada	state: "Piauí"; scale: "1:25.000"
GC019	C · A	cartas do rio de janeiro em 50k	state: "Rio de Janeiro"; scale: "1:50.000"; aceita-se também city no lugar de state
GC020	C · A	cartas da Bahia em 250k	state: "Bahia"; scale: "1:250.000"
GC021	C · A	cartas de SP na escala 1:2000	state: "São Paulo"; scale: "1:2.000"
GC022	C · A	cartas do RS na escala de 100 mil	state: "Rio Grande do Sul"; scale: "1:100.000"
GC023	C · A	cartas do amapá na escala de 100 mil	state: "Amapá"; scale: "1:100.000"
GC024	C · A	cartas da Paraíba em 25k	state: "Paraíba"; scale: "1:25.000"
GC025	C	cartas de Rondônia na escala 1:2.000	state: "Rondônia"; scale: "1:2.000"
GC026	C · A	cartas de São Paulo em 50k	state: "São Paulo"; scale: "1:50.000"; aceita-se também city no lugar de state
GC027	C · A	cartas do maranhão na escala 1:5.000	state: "Maranhão"; scale: "1:5.000"
GC028	C · A	cartas do Acre na escala 1:25000	state: "Acre"; scale: "1:25.000"
GC029	C · A	cartas de PE na escala 1:25000	state: "Pernambuco"; scale: "1:25.000"
GC030	C · A	cartas da bahia na escala 1:5000	state: "Bahia"; scale: "1:5.000"
GC031	C · A	mapas de Recife na escala 1:50000	city: "Recife"; scale: "1:50.000"
GC032	C	mapas de Brasília na escala 1:2.000	city: "Brasília"; scale: "1:2.000"
GC033	C · A	mapas de Fortaleza na escala 1:10000	city: "Fortaleza"; scale: "1:10.000"
GC034	C · A	mapas de Cuiabá na escala 1:2000	city: "Cuiabá"; scale: "1:2.000"
GC035	C · A	mapas de Belém em 250k	city: "Belém"; scale: "1:250.000"
GC036	C · A	mapas de Porto Alegre na escala 1:5000	city: "Porto Alegre"; scale: "1:5.000"
GC037	C · A	mapas de Goiânia na escala 1:250000	city: "Goiânia"; scale: "1:250.000"
GC038	C · A	mapas de Fortaleza em 100k	city: "Fortaleza"; scale: "1:100.000"
GC039	C · A	cartas temáticas de São Paulo	productType: "Cartas Temáticas Não SCN"; state: "São Paulo"; aceita-se também city no lugar de state
GC040	C · A	ortoimagens banda X HH de minas	productType: "SCN Carta Ortoimagem Banda X Pol HH"; state: "Minas Gerais"
GC041	C · A	cartas temáticas da PB	productType: "Cartas Temáticas Não SCN"; state: "Paraíba"
GC042	C · A	ortoimagens banda P HH do amapá	productType: "SCN Carta Ortoimagem Banda P Pol HH"; state: "Amapá"
GC043	C · A	cartas temáticas do Espírito Santo	productType: "Cartas Temáticas Não SCN"; state: "Espírito Santo"
GC044	C · A	cartas topográficas do Amapá	productType: "SCN Carta Topográfica Matricial"; state: "Amapá"
GC045	C · A	produtos do projeto copa das confederacoes no AP	project: "Copa das Confederações"; state: "Amapá"

ID	Cat.	Consulta	Leituras aceitas
GC046	C · A	produtos do projeto olimpiadas em PE	project: "Olimpíadas Rio 2016"; state: "Pernambuco"
GC047	C	produtos do projeto NGA-BECA no Distrito Federal	project: "NGA-BECA"; state: "Distrito Federal"
GC048	C · A	produtos do projeto beca no espírito santo	project: "NGA-BECA"; state: "Espírito Santo"
GC049	C · A	produtos do projeto copa das confederacoes no AM	project: "Copa das Confederações"; state: "Amazonas"
GC050	C · A	produtos do projeto BCD de Rondônia no Pará	project: "Base Cartográfica Digital de Rondônia"; state: "Pará"
GC051	C · A	cartas do 1º CGEO na escala 1:2.000	supplyArea: "1º Centro de Geoinformação"; scale: "1:2.000"
GC052	C · A	cartas do 3º cgeo na escala 1:2000	supplyArea: "3º Centro de Geoinformação"; scale: "1:2.000"
GC053	C · A	cartas do 2º cgeo em pequena escala	supplyArea: "2º Centro de Geoinformação"; scale: "1:250.000"
GC054	C · A	cartas do 2 CGEO na escala 1:25000	supplyArea: "2º Centro de Geoinformação"; scale: "1:25.000"
GC055	C · A	cartas do 1º CGEO em 250k	supplyArea: "1º Centro de Geoinformação"; scale: "1:250.000"

A.7 Consultas com código MI/INOM geradas por *templates* (G-M)

Total desta família: **28 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GM001	C · M	folha MI 2965-2-NE do Rio Grande do Sul	keyword: "2965-2-NE"; state: "Rio Grande do Sul"
GM002	C · M · A	folha MI 2866-3 do TO	keyword: "2866-3"; state: "Tocantins"
GM003	C · M · A	folha MI 2901 do AP	keyword: "2901"; state: "Amapá"
GM004	C · M · A	folha MI 530 do ceará	keyword: "530"; state: "Ceará"
GM005	C · M · A	folha MI 3010 do paraná	keyword: "3010"; state: "Paraná"
GM006	C · M · A	folha MI 2965 do distrito federal	keyword: "2965"; state: "Distrito Federal"
GM007	C · M · A	folha MI 2866-2-SO de AL	keyword: "2866-2-SO"; state: "Alagoas"

ID	Cat.	Consulta	Leituras aceitas
GM008	C · M · A	folha MI 2901-4-NE de MG	keyword: "2901-4-NE"; state: "Minas Gerais"
GM009	C · M · A	folha MI 531-1-NE de minas	keyword: "531-1-NE"; state: "Minas Gerais"
GM010	C · M	folha MI 2965-3 de Sergipe	keyword: "2965-3"; state: "Sergipe"
GM011	C · M · A	folha MI 2866-3-NO do maranhão	keyword: "2866-3-NO"; state: "Maranhão"
GM012	C · M	folha MI 3010-2-SE de Roraima	keyword: "3010-2-SE"; state: "Roraima"
GM013	C · M · A	INOM SF-22-Y-D do tipo temática	keyword: "SF-22-Y-D"; productType: "Cartas Temáticas Não SCN"
GM014	C · M · A	INOM SF-22-Y-D-II do tipo ortoimagem	keyword: "SF-22-Y-D-II"; productType: "SCN Carta Ortoimagem"
GM015	C · M · A	INOM SF-22-Y-D-II-4 do tipo banda X HH	keyword: "SF-22-Y-D-II-4"; productType: "SCN Carta Ortoimagem Banda X Pol HH"
GM016	C · M · A	INOM SF-22-Y-D-II-4-SE do tipo topográfica	keyword: "SF-22-Y-D-II-4-SE"; productType: "SCN Carta Topográfica Matricial"
GM017	C · M · A	INOM SG-22-X-A do tipo MDT	keyword: "SG-22-X-A"; productType: "MDT — RAM"
GM018	C · M · A	INOM SF-23-V-C do tipo topo	keyword: "SF-23-V-C"; productType: "SCN Carta Topográfica Matricial"
GM019	C · M · A	INOM SG-21-Z-B do tipo temática	keyword: "SG-21-Z-B"; productType: "Cartas Temáticas Não SCN"
GM020	C · M · A	INOM SH-22-W-A-III-1-NE do tipo topográfica	keyword: "SH-22-W-A-III-1-NE"; productType: "SCN Carta Topográfica Matricial"
GM021	C · M · A	INOM SG-22-Z-D-II-3 do tipo ortoimagem	keyword: "SG-22-Z-D-II-3"; productType: "SCN Carta Ortoimagem"
GM022	C · M · A	INOM SF-24-Y-A do tipo vetorial	keyword: "SF-24-Y-A"; productType: "SCN Carta Topográfica Vetorial"
GM023	C · M · A	MI 2866-2-SO produzido pelo 1 CGEO	keyword: "2866-2-SO"; supplyArea: "1° Centro de Geoinformação"
GM024	C · M · A	MI 2866-3 produzido pelo quinto cgeo	keyword: "2866-3"; supplyArea: "5° Centro de Geoinformação"
GM025	C · M · A	MI 2965-3 produzido pelo 3o cgeo	keyword: "2965-3"; supplyArea: "3° Centro de Geoinformação"
GM026	C · M · A	MI 530 produzido pelo 3o cgeo	keyword: "530"; supplyArea: "3° Centro de Geoinformação"
GM027	C · M · A	MI 2965-2-NE produzido pelo 3 CGEO	keyword: "2965-2-NE"; supplyArea: "3° Centro de Geoinformação"
GM028	C · M · A	MI 3010 produzido pelo segundo cgeo	keyword: "3010"; supplyArea: "2° Centro de Geoinformação"

A.8 Consultas com tempo relativo geradas por *templates* (G-T)

Total desta família: **41 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GT001	T	cartas publicadas esse ano	publicationPeriod: ano corrente
GT002	T	cartas publicadas no ano passado	publicationPeriod: ano anterior, inteiro
GT003	T	cartas publicadas no mês passado	publicationPeriod: mês anterior, inteiro
GT004	T	cartas publicadas na semana passada	publicationPeriod: semana passada
GT005	T	cartas publicadas nos últimos 3 meses	publicationPeriod: últimos 3 meses
GT006	T	cartas publicadas nos últimos 6 meses	publicationPeriod: últimos 6 meses
GT007	T	cartas publicadas nos últimos 5 anos	publicationPeriod: últimos 5 anos
GT008	T	cartas publicadas desde 2020	publicationPeriod: desde 2020
GT009	T	cartas publicadas depois de 2022	publicationPeriod: depois de 2022
GT010	T	cartas publicadas antes de 2020	publicationPeriod: antes de 2020
GT011	T	cartas publicadas entre 2022 e 2023	publicationPeriod: 2022-01-01 a 2023-12-31
GT012	T	cartas publicadas no primeiro trimestre deste ano	publicationPeriod: 1º trimestre do ano corrente
GT013	T	cartas publicadas no segundo semestre do ano passado	publicationPeriod: 2º semestre do ano anterior
GT014	T	cartas publicadas nesta semana	publicationPeriod: semana corrente
GT015	T	cartas publicadas hoje	publicationPeriod: dia da execução
GT016	C · T · A	cartas de pernambuco publicadas desde 2020	state: "Pernambuco"; publicationPeriod: desde 2020
GT017	C · T · A	cartas da bahia publicadas nos últimos 6 meses	state: "Bahia"; publicationPeriod: últimos 6 meses
GT018	C · T · A	cartas do AP produzidas no segundo semestre do ano passado	state: "Amapá"; creationPeriod: 2º semestre do ano anterior
GT019	C · T · A	cartas do MA publicadas no ano passado	state: "Maranhão"; publicationPeriod: ano anterior, inteiro
GT020	C · T	cartas da Paraíba publicadas esse ano	state: "Paraíba"; publicationPeriod: ano corrente
GT021	C · T · A	cartas de roraima elaboradas hoje	state: "Roraima"; creationPeriod: dia da execução
GT022	C · T · A	cartas de sergipe publicadas no primeiro trimestre deste ano	state: "Sergipe"; publicationPeriod: 1º trimestre do ano corrente
GT023	C · T	cartas de Rondônia publicadas depois de 2022	state: "Rondônia"; publicationPeriod: depois de 2022
GT024	C · T	cartas do Maranhão criadas nos últimos 3 meses	state: "Maranhão"; creationPeriod: últimos 3 meses

ID	Cat.	Consulta	Leituras aceitas
GT025	C · T · A	cartas da bahia publicadas antes de 2020	state: "Bahia"; publicationPeriod: antes de 2020
GT026	C · T · A	produtos do 3o cgeo publicados no ano passado	supplyArea: "3° Centro de Geoinformação"; publicationPeriod: ano anterior, inteiro
GT027	C · T · A	produtos do 3 CGEO publicados hoje	supplyArea: "3° Centro de Geoinformação"; publicationPeriod: dia da execução
GT028	C · T · A	produtos do 3º CGEO criados no primeiro trimestre deste ano	supplyArea: "3° Centro de Geoinformação"; creationPeriod: 1º trimestre do ano corrente
GT029	C · T · A	produtos do quarto cgeo publicados depois de 2022	supplyArea: "4° Centro de Geoinformação"; publicationPeriod: depois de 2022
GT030	C · T · A	produtos do 1º CGEO publicados no mês passado	supplyArea: "1° Centro de Geoinformação"; publicationPeriod: mês anterior, inteiro
GT031	C · T · A	produtos do terceiro cgeo criados nos últimos 3 meses	supplyArea: "3° Centro de Geoinformação"; creationPeriod: últimos 3 meses
GT032	C · T · A	produtos do 2 CGEO publicados antes de 2020	supplyArea: "2° Centro de Geoinformação"; publicationPeriod: antes de 2020
GT033	C · T · A	produtos do 5 CGEO publicados na semana passada	supplyArea: "5° Centro de Geoinformação"; publicationPeriod: semana passada
GT034	C · T	cartas na escala 1:5.000 publicadas depois de 2022	scale: "1:5.000"; publicationPeriod: depois de 2022
GT035	C · T · A	cartas na escala 1:5000 publicadas esse ano	scale: "1:5.000"; publicationPeriod: ano corrente
GT036	C · T · A	cartas na escala de 25 mil criadas no ano passado	scale: "1:25.000"; creationPeriod: ano anterior, inteiro
GT037	C · T · A	cartas em 50k publicadas no segundo semestre do ano passado	scale: "1:50.000"; publicationPeriod: 2º semestre do ano anterior
GT038	C · T · A	cartas na escala de 10 mil publicadas no mês passado	scale: "1:10.000"; publicationPeriod: mês anterior, inteiro
GT039	C · T	cartas na escala 1:2.000 elaboradas hoje	scale: "1:2.000"; creationPeriod: dia da execução
GT040	C · T · A	cartas na escala de 100 mil publicadas nesta semana	scale: "1:100.000"; publicationPeriod: semana corrente
GT041	C · T · A	cartas na escala 1:5000 publicadas nos últimos 6 meses	scale: "1:5.000"; publicationPeriod: últimos 6 meses

A.9 Consultas com ordenação geradas por *templates* (G-O)

Total desta família: **19 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GO001	C · O · A	a carta mais recente do CE	state: "Ceará"; sortField: "publicationDate"ou "creationDate"; sortDirection: "DESC"; limit: 1 (opcional)
GO002	C · O · A	a carta mais antiga do PA	state: "Pará"; sortField: "publicationDate"ou "creationDate"; sortDirection: "ASC"; limit: 1 (opcional)
GO003	C · O	as cartas mais recentes do Ceará	state: "Ceará"; sortField: "publicationDate"ou "creationDate"; sortDirection: "DESC"
GO004	C · O · A	as cartas mais antigas de mato grosso	state: "Mato Grosso"; sortField: "publicationDate"ou "creationDate"; sortDirection: "ASC"
GO005	C · O · A	a primeira carta produzida de sergipe	state: "Sergipe"; sortField: "creationDate"; sortDirection: "ASC"; limit: 1 (opcional)
GO006	C · O · A	a última carta publicada de GO	state: "Goiás"; sortField: "publicationDate"; sortDirection: "DESC"; limit: 1 (opcional)
GO007	C · O · A	as 3 cartas mais antigas de sergipe	state: "Sergipe"; sortField: "publicationDate"ou "creationDate"; sortDirection: "ASC"; limit: 3
GO008	C · O · A	as 5 publicações mais recentes de MG	state: "Minas Gerais"; sortField: "publicationDate"; sortDirection: "DESC"; limit: 5
GO009	C · O · A	as 10 cartas criadas mais recentemente do PR	state: "Paraná"; sortField: "creationDate"; sortDirection: "DESC"; limit: 10
GO010	C · O · A	cartas do distrito federal em ordem cronológica de publicação	state: "Distrito Federal"; sortField: "publicationDate"; sortDirection: "ASC"
GO011	C · O	as cartas temáticas mais recentes	productType: "Cartas Temáticas Não SCN"; sortField: "publicationDate"ou "creationDate"; sortDirection: "DESC"
GO012	C · O	as 3 cartas CIRC mais antigas	productType: "CIRC"; sortField: "publicationDate"ou "creationDate"; sortDirection: "ASC"; limit: 3
GO013	C · O	a última ortoimagem publicada	productType: "SCN Carta Ortoimagem"; sortField: "publicationDate"; sortDirection: "DESC"; limit: 1 (opcional)
GO014	C · O	a primeira carta topográfica vetorial elaborada	productType: "SCN Carta Topográfica Vetorial"; sortField: "creationDate"; sortDirection: "ASC"; limit: 1 (opcional)
GO015	C · O	cartas topográficas da mais nova para a mais antiga	productType: "SCN Carta Topográfica Matricial"; sortField: "publicationDate"ou "creationDate"; sortDirection: "DESC"

ID	Cat.	Consulta	Leituras aceitas
GO016	C · T · O	as cartas mais recentes publicadas entre 2022 e 2023	publicationPeriod: 2022-01-01 a 2023-12-31; sortField: "publicationDate"ou "creationDate"; sortDirection: "DESC"
GO017	C · T · O	a primeira carta criada na semana passada	creationPeriod: semana passada; sortField: "creationDate"; sortDirection: "ASC"; limit: 1 (opcional)
GO018	C · T · O	as 5 cartas mais antigas publicadas nos últimos 5 anos	publicationPeriod: últimos 5 anos; sortField: "publicationDate"ou "creationDate"; sortDirection: "ASC"; limit: 5
GO019	C · T · O	a última carta publicada nos últimos 6 meses	publicationPeriod: últimos 6 meses; sortField: "publicationDate"; sortDirection: "DESC"; limit: 1 (opcional)

A.10 Consultas ambíguas/informais adicionais (G-A)

Total desta família: **24 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GA001	S · A	mapas de pe	state: "Pernambuco"
GA002	S · A	mapas do pr	state: "Paraná"
GA003	S · A	mapas de sc	state: "Santa Catarina"
GA004	S · A	mapas de go	state: "Goiás"
GA005	S · A	mapas do ce	state: "Ceará"
GA006	S · A	mapas do pa	state: "Pará"
GA007	S · A	mapas de goias	state: "Goiás"
GA008	S · A	mapas de rondonia	state: "Rondônia"
GA009	S · A	mapas do piau	state: "Piauí"
GA010	S · A	mapas do amapa	state: "Amapá"
GA011	S · A	mapas do maranhao	state: "Maranhão"
GA012	S · A	mapas da paraiba	state: "Paraíba"
GA013	S · A	mapas de cuiaba	city: "Cuiabá"
GA014	S · A	mapas de goiania	city: "Goiânia"
GA015	S · A	mapas de belem	city: "Belém"
GA016	S · A	mapas de brasilia	city: "Brasília"
GA017	C · A	cartas do rio em 50k	state: "Rio de Janeiro"; scale: "1:50.000"; aceita-se também city no lugar de state
GA018	C · A	mapas detalhados do Amazonas	scale: "1:25.000"; state: "Amazonas"
GA019	S · A	cartas em pequena escala do sul	scale: "1:250.000"
GA020	S · A	produtos em média escala	scale: "1:50.000"ou "1:100.000"
GA021	C · A	cartas de Goiás em 25k	state: "Goiás"; scale: "1:25.000"
GA022	C · A	cartas do Tocantins em 50k	state: "Tocantins"; scale: "1:50.000"
GA023	C · A	cartas de Mato Grosso do Sul em 100k	state: "Mato Grosso do Sul"; scale: "1:100.000"

ID	Cat.	Consulta	Leituras aceitas
GA024	C · A	cartas de Pernambuco em 250k	state: "Pernambuco"; scale: "1:250.000"

A.11 Consultas amplas, com muitos produtos (G-P)

Total desta família: **13 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GP001	S	todas as cartas de Rondônia	state: "Rondônia"
GP002	S	todas as cartas do Tocantins	state: "Tocantins"
GP003	S	todas as cartas do Piauí	state: "Piauí"
GP004	S	todas as cartas de Mato Grosso	state: "Mato Grosso"
GP005	S	todas as cartas do Acre	state: "Acre"
GP006	S	todos os produtos do 1° Centro de Geoinformação	supplyArea: "1° Centro de Geoinformação"
GP007	S	todos os produtos do 2° Centro de Geoinformação	supplyArea: "2° Centro de Geoinformação"
GP008	S	todos os produtos do 3° Centro de Geoinformação	supplyArea: "3° Centro de Geoinformação"
GP009	S	todos os produtos do 4° Centro de Geoinformação	supplyArea: "4° Centro de Geoinformação"
GP010	S	todos os produtos do 5° Centro de Geoinformação	supplyArea: "5° Centro de Geoinformação"
GP011	C	cartas na escala 1:2.000 do Rio Grande do Norte	scale: "1:2.000"; state: "Rio Grande do Norte"
GP012	C	cartas na escala 1:5.000 de Alagoas	scale: "1:5.000"; state: "Alagoas"
GP013	C	cartas na escala 1:250.000 do Rio Grande do Sul	scale: "1:250.000"; state: "Rio Grande do Sul"

A.12 Consultas de fronteira do domínio (G-F)

Total desta família: **7 consultas**, listadas integralmente abaixo (a função geradora e o modelo de frase de cada consulta constam de `dataset.json`).

ID	Cat.	Consulta	Leituras aceitas
GF001	F	me ajuda	não chamar a ferramenta (sem intenção de busca)
GF002	F	cartas de Marte	não chamar a ferramenta (fora da Terra)
GF003	F	cartas da Argentina	não chamar a ferramenta (fora do território brasileiro)
GF004	F	qual a previsão do tempo para amanhã em Manaus?	não chamar a ferramenta (outro assunto)

ID	Cat.	Consulta	Leituras aceitas
GF005	F	quanto custa uma carta topográfica?	não chamar a ferramenta (pergunta de preço, não de busca)
GF006	F	mapa do tesouro pirata	não chamar a ferramenta (ficção)
GF007	S	cartas do estado 42	não chamar a ferramenta (observacional, P6: valor impossível de representar)

A.13 Requisitos exercitados pelas consultas

O Quadro 6 relaciona os grupos de consultas aos requisitos funcionais da Seção 4.2. Todas as consultas exercitam a tradução (RF2) e o registro dos traços de execução (RF5), pois cada rodada do *pipeline* de avaliação grava um registro por consulta; a troca do modelo entre rodadas, por parâmetro do *pipeline*, exercita ainda a substituíbilidade de modelo por configuração (RNF6). O recebimento pela rota REST (RF1), a execução da busca no banco (RF3) e a seleção do modelo por requisição (RF4) são exercitados pela API e pela demonstração ponta a ponta descritas no Capítulo 4, e não entram nas métricas, que avaliam a camada de tradução.

Quadro 6 – Grupos de consultas do *dataset* e aspectos da tradução que exercitam

Grupo de consultas	O que exige do modelo	Requisitos
Todas (310)	Preencher os campos da ferramenta sustentados pela consulta, na forma canônica, sem acrescentar campos	RF2, RF5, RNF6
Com código MI/INOM (M)	Preservar o código em keyword , normalizando grafias compactadas do INOM	RF2
Com tempo relativo (T)	Resolver a expressão em datas ISO a partir da data de referência informada e escolher entre período de publicação e de criação	RF2
Com ordenação (O)	Preencher sortField , sortDirection e, quando pedido, limit , sem ordenar quando a consulta não pede	RF2
Ambíguas/informais (A)	Normalizar siglas, grafias sem acento, apelidos e expressões qualitativas sem dicionário de normalização	RF2
Fora do domínio (F)	Reconhecer que o catálogo não atende ao pedido e não chamar a ferramenta	RF2
Observacionais (4)	Comportamento diante de valores impossíveis ou de pedidos de resposta discutível (reportado, não pontuado)	RF2, RF5

Fonte: Elaborado pelos autores. As contagens por categoria estão na Tabela 5.

A.14 Auditoria do *dataset*

A auditoria segue o procedimento descrito no manual de anotação (Seção A.1, subseção “Procedimento de auditoria”) e é reproduzível a partir do repositório. Seus artefatos ficam em `data/auditoria/`:

- `relatorio_auditoria.md` e `auditoria.json`: resultado das checagens automáticas (estrutura, rastreabilidade, anotador por regras e consistência), das medidas de concordância e da adjudicação;
- `auditoria_casos.csv`: uma linha por consulta, com o gabarito legível, as evidências localizadas pelo anotador por regras, os apontamentos e as divergências com a anotação independente;
- `anotacao_independente.json`: a anotação às cegas das 310 consultas, com a justificativa de cada uma;
- `adjudicacao.json`: as decisões de adjudicação, com o gabarito anterior a cada decisão;
- `revisao_humana_amostra.csv`: amostra estratificada por origem (40 consultas, somente 42), conferida manualmente por um dos autores.

O comando que refaz a auditoria e regenera as tabelas correspondentes (Tabelas 7 e 36 e Quadro 7) é:

```
pfc-auditar --anotacao-independente data/auditoria/anotacao_independente.json \
--paper <diretorio-do-texto>
```

A Tabela 36 detalha, por campo, a concordância entre a leitura preferencial do gabarito e a da anotação independente, e o Quadro 7 lista todas as decisões de adjudicação. Das quatro consultas com leitura preferencial divergente (Tabela 7), três — P13, P21 e P22 — decorrem da regra que mantém, na camada P, a anotação original da equipe do 1º CGEO como preferencial; na quarta, N33, o gabarito inclui na leitura preferencial o campo opcional `project`, que o anotador deixou de fora. Em todas, a leitura do anotador está entre as leituras aceitas.

A.15 Lote de validação: resultados complementares

Os artefatos do lote de validação (lote 1; Seções 3.5 e 5.4) ficam em `data/lote_validacao/`: os alvos (`alvos.json`), as tarefas e saídas de redação e de anotação, o relatório de montagem (`relatorio.md`), as consultas descartadas com

Quadro 7 – Decisões de adjudicação da auditoria do *dataset*

Consultas	Divergência	Decisão	Justificativa
N36, N37, GF001, GF002, GF003, GF004, GF005, GF006 <i>anotação independente</i>	o gabarito classificava a consulta como observacional; o anotador a tratou como fora do domínio, com gabarito determinado 'não chamar'	corrigido	P4 e seção "Consultas fora do domínio" do manual: consulta sem intenção de busca no acervo brasileiro tem comportamento correto determinado (não chamar); P6 reserva 'observacional' às consultas cujo comportamento correto não é determinável. A consulta passa às métricas principais, na categoria F (fora do domínio).
N33 <i>anotação independente</i>	o gabarito exigia project = Mapeamento Sistemático; o anotador anotou só state = São Paulo e listou project como leitura alternativa, pois 'cartografia sistemática' não é nome nem apelido informado na ferramenta	ampliado	As duas leituras são razoáveis (P3): a expressão designa o programa pelo termo distintivo do nome, mas não consta da descrição da ferramenta. project passa a campo opcional; a regra de project do manual passa a registrar o caso.
GF007 <i>anotação independente</i>	ambos classificam a consulta como observacional; o gabarito indica 'não chamar' e o anotador, 'chamar sem parâmetros'	mantido	Consulta observacional (P6, valor impossível de representar): o comportamento é reportado, mas não pontuado, de modo que a divergência não afeta nenhuma métrica. Mantém-se a indicação 'não chamar', coerente com N38 (escala 1:10.000.000).
P14 <i>consistência</i>	'detalhado' é anotado como 1:25.000 em todo o dataset, mas P14 aceita 1:25.000 ou 1:50.000	mantido	As escalas explícitas '(25k ou 50k)' prevalecem sobre o qualificativo 'detalhado' (regra de scale: duas escalas explícitas, aceita-se qualquer uma).
P13 <i>anotação independente (medida estrita)</i>	leitura preferencial diferente dentro do conjunto aceito: scale 1:25.000 no gabarito, 1:50.000 no anotador (ambas aceitas por 'maior que 100k')	mantido	Na camada P, a anotação original da equipe do 1º CGEO é mantida como leitura preferencial (manual, seção "Rastreabilidade"); a escolha da preferencial não afeta a pontuação.
P21, P22 <i>anotação independente (medida estrita)</i>	leitura preferencial diferente dentro do conjunto aceito: sortField creationDate no gabarito, publicationDate no anotador (ambos aceitos, sem pista ligada à ordenação)	mantido	Na camada P, a anotação original da equipe do 1º CGEO é mantida como leitura preferencial (manual, seção "Rastreabilidade"); a escolha da preferencial não afeta a pontuação.
GO016 <i>anotação independente (medida estrita)</i>	o gabarito aceita sortField publicationDate ou creationDate; o anotador aceitou só publicationDate	mantido	Em 'as cartas mais recentes publicadas entre 2022 e 2023', o verbo qualifica o filtro de período, não a ordenação; sem pista ligada à ordenação, aceitam-se os dois campos (regra de ordenação). É o mesmo raciocínio do anotador em P22.

Fonte: Elaborado pelos autores a partir de `data/auditoria/adjudicacao.json`.

Tabela 36 – Concordância por campo entre o gabarito e a anotação independente (leitura preferencial, após a adjudicação)

Campo	Gabarito	Anotador	Ambos	Kappa	Valor igual
keyword	62	62	62	1,000	62/62
scale	84	84	84	1,000	83/84
productType	44	44	44	1,000	44/44
state	130	130	130	1,000	130/130
city	27	27	27	1,000	27/27
supplyArea	40	40	40	1,000	40/40
project	11	10	10	0,951	10/10
publicationPeriod	51	51	51	1,000	51/51
creationPeriod	10	10	10	1,000	10/10
sortField	29	29	29	1,000	27/29
sortDirection	29	29	29	1,000	29/29
limit	21	21	21	1,000	21/21

Fonte: Elaborada pelos autores. Gabarito, Anotador e Ambos: consultas em que o campo está na leitura preferencial de cada um. Kappa de Cohen sobre a presença do campo. Valor igual: entre as consultas em que ambos o anotam; períodos comparados pelos limites, textos livres após normalização.

o motivo (`descartes.json`) e a auditoria dos erros (`auditoria_erros.json`, com os critérios alternativos para os gabaritos discutíveis em `auditoria_criterios.json`); a metodologia completa está em `docs/lote_validacao.md`. As tabelas deste apêndice são geradas por `python -m pfc_busca.evaluation.lote`, que calcula também, a partir das rodadas e sem chamada nova ao modelo, as duas configurações simuladas da Tabela 18 e as medidas complementares citadas na Seção 5.4 (seção 9.5 de `docs/lote_validacao.md`). Os artefatos do lote 2 (Seção 3.6), construído com os mesmos *scripts*, ficam em `data/lote_validacao_2/`; as rodadas do teste da v3 e o relatório do plano de análise, em `results/lote2/` (`consolidado/lote2.md`), gerados por `python -m pfc_busca.evaluation.lote2`; e o registro dos ciclos de desenvolvimento da v3, em `docs/v3_desenvolvimento.md` e `results/dev_v3/`.

Tabela 37 – Acurácia por categoria no lote de validação (Gemma 4 E4B)

Categoria	n	Tool Calling v1	Saída Estruturada v1	Tool Calling v2	Saída Estruturada v2
Simples	389	53,0%	79,2%	60,2%	55,5%
Compostas	955	52,4%	73,7%	49,9%	65,4%
Código MI/INOM	281	77,9%	78,3%	50,5%	76,5%
Tempo relativo	278	22,7%	65,8%	37,4%	61,2%
Ordenação	199	39,2%	77,4%	52,8%	73,4%
Ambíguas/informais	927	48,1%	72,9%	50,3%	61,1%
Subespecificadas	146	97,3%	74,0%	71,9%	100,0%
Fora do domínio	336	100,0%	0,0%	99,7%	100,0%
Todas	1929	61,5%	62,2%	62,8%	72,4%

Fonte: Elaborado pelos autores. Uma consulta pode pertencer a mais de uma categoria. Subespecificadas: buscar sem filtros ou não buscar são ambos corretos.

Tabela 38 – F1 por campo no lote de validação (Gemma 4 E4B)

Campo	Ocorr.	Tool Calling v1	Saída Estruturada v1	Tool Calling v2	Saída Estruturada v2
keyword	492	0,869	0,773	0,743	0,85
scale	415	0,821	0,969	0,963	0,92
productType	386	0,689	0,962	0,733	0,88
state	307	0,794	0,865	0,651	0,70
city	211	0,601	0,649	0,803	0,43
supplyArea	346	0,882	0,999	0,999	0,97
project	181	0,925	0,978	0,964	0,94
publicationPeriod	214	0,415	0,839	0,829	0,83
creationPeriod	106	0,538	0,545	0,829	0,84
sortField	199	0,699	0,941	0,959	0,97
sortDirection	199	0,748	0,977	0,959	0,98
limit	150	0,800	0,972	0,721	0,92
Ponderado		0,761	0,892	0,840	0,86
Macro		0,732	0,872	0,846	0,85

Fonte: Elaborado pelos autores. Ocorr.: ocorrências do campo no gabarito (a maior entre as configurações, porque os campos opcionais contam quando emitidos).

APÊNDICE B – ESPECIFICAÇÃO DOS AMBIENTES DE EXECUÇÃO

Este apêndice documenta o *hardware* e o *software* dos ambientes de execução descritos na Seção 3.7.2. Em todos eles, cada rodada grava um manifesto (`results/<modelo>/manifesto.json`) com as versões de *software*, a configuração de inferência, a data de referência e os *hashes* do *dataset* e da definição da ferramenta.

B.1 Estação de referência

Os dados da estação foram coletados executando-se, no próprio equipamento, os comandos PowerShell da Seção B.1.1, em setembro de 2026, com o *driver* NVIDIA usado nas rodadas da estação; a saída é transcrita, de forma condensada, na Seção B.1.2. O banco usado na demonstração ponta a ponta roda num contêiner Docker com a imagem `postgis/postgis:16-3.4` (PostgreSQL 16 com PostGIS 3.4), definida no `docker-compose.yml` do repositório; o `psql` listado é o cliente instalado na estação.

B.1.1 Comandos PowerShell de coleta

```
# CPU
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores,
    NumberOfLogicalProcessors, MaxClockSpeed

# Memória RAM (total em GB e detalhes por pente)
(Get-CimInstance Win32_PhysicalMemory |
    Measure-Object -Property Capacity -Sum).Sum / 1GB
Get-CimInstance Win32_PhysicalMemory | Select-Object Manufacturer,
    Capacity, Speed, ConfiguredClockSpeed, MemoryType, DeviceLocator

# GPU (geral)
Get-CimInstance Win32_VideoController | Select-Object Name,
    AdapterRAM, DriverVersion, VideoProcessor

# VRAM real (se GPU NVIDIA)
nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv

# Sistema operacional
Get-CimInstance Win32_OperatingSystem | Select-Object Caption,
    Version, BuildNumber, OSArchitecture

# Disco principal
```

Get-PhysicalDisk | Select-Object FriendlyName, MediaType, Size, BusType

Versoes de software

python --version

ollama --version

psql --version

docker --version

B.1.2 Saída coletada

CPU

Name	NumberOfCores	NumberOfLogicalProcessors	MaxClockSpeed
12th Gen Intel(R) Core(TM) i7-12700H	14	20	2300

RAM

RAM total (GB): 32

Pente 1: Kingston, 16 GB DDR4, 3200 MHz (Controller0-ChannelA-DIMM0)

Pente 2: Kingston, 16 GB DDR4, 3200 MHz (Controller1-ChannelA-DIMM0)

GPU

Name: NVIDIA GeForce RTX 3050 Laptop GPU

AdapterRAM 4 293 918 720; DriverVersion 32.0.16.1664

VideoProcessor NVIDIA GeForce RTX 3050 Laptop GPU

Name: Intel(R) Iris(R) Xe Graphics

AdapterRAM 2 147 479 552; DriverVersion 32.0.101.6790

VideoProcessor Intel(R) Iris(R) Xe Graphics Family

nvidia-smi

NVIDIA GeForce RTX 3050 Laptop GPU, 4096 MiB, 616.64

Sistema operacional

Microsoft Windows 11 Pro, versao 10.0.26200, Build 26200, 64 bits

Disco principal

INTEL SSDPEKNU512GZ, SSD NVMe, 512 GB

Software

Python 3.14.4

ollama version is 0.34.0

psql (PostgreSQL) 18.3

Docker version 29.6.1, build 8900f1d

B.2 Rodadas completas em GPU de nuvem

As rodadas completas dos quatro modelos locais foram executadas numa máquina virtual do Google Colab com GPU NVIDIA T4 (16 GB nominais de VRAM, dos quais o nvidia-smi informa 15.360 MiB; Tabela 11), pelo caderno notebooks/avaliacao_colab.ipynb do repositório. O caderno instala o Ollama pelo instalador oficial, baixa os modelos pelas *tags* do Ollama qwen3:4b-instruct-2507-q4_K_M, gemma4:e4b-it-qat, gemma4:e2b-it-qat e mistral-nemo:12b — para o

Qwen 3 4B e o Gemma 4 E2B, as mesmas da validação na estação de referência, com o mesmo *digest* registrado nos manifestos —, instala o pacote do projeto a partir do repositório e executa `pfc-avaliar` com três repetições por modelo, data de referência fixa (14 de setembro de 2026) e sem execução de SQL, pois a busca no banco não entra nas métricas. No mesmo ambiente, o caderno `notebooks/linha_de_base_colab.ipynb` executou as linhas de base com Saída Estruturada e o método do protótipo, com o modelo `phi4:14b`, uma repetição e a mesma data de referência. Os dois lotes de validação também rodaram nesse ambiente, com o Gemma 4 E4B e uma repetição: o caderno `notebooks/lote_validacao_colab.ipynb` executou as configurações v1 e v2 no lote 1 e as duas v2 nas 310, e o `notebooks/lote2_v3_colab.ipynb`, que confere o *hash* da v3 congelada antes de rodar, as configurações do plano de análise no lote 2 e a v3 e o seu controle nas 310. Nenhum serviço de LLM em nuvem é usado nesse ambiente: o Ollama executa dentro da máquina virtual, com os pesos baixados nela; somente a GPU é provida pela nuvem.

B.3 Referência em nuvem (Groq)

A referência em nuvem usa a API do Groq, compatível com a da OpenAI, por meio da classe `ChatOpenAI` do pacote `langchain-openai` (versão 1.6.6), com `langchain-core` 1.6.4, a partir da estação de referência. O *prompt* de sistema, a definição da ferramenta, a temperatura (zero) e o limite de 1024 *tokens* de saída são os mesmos das rodadas locais. O esforço de raciocínio foi fixado em `none` para o `qwen/qwen3.8-27b` e `low` para o `openai/gpt-oss-20b` e o `openai/gpt-oss-120b`, o menor valor aceito por cada modelo; o valor `none` desliga o raciocínio. Cada modelo executou as 310 consultas uma vez, com data de referência de 24 de setembro de 2026 e intervalo mínimo de 11 s entre chamadas; quando o provedor sinaliza o esgotamento do limite de taxa do plano gratuito, a execução aguarda o tempo indicado e retoma a mesma consulta, sem contá-la como erro. Uma chamada de ferramenta rejeitada pelo provedor por argumentos inválidos é registrada como erro do modelo. A linha de base com Saída Estruturada usa o mesmo cliente, sem ferramenta e com o modo JSON do provedor (`response_format` do tipo `json_object`), a data de referência de 24 de setembro de 2026 e o mesmo esforço de raciocínio de cada modelo.

APÊNDICE C – REPOSITÓRIO DE CÓDIGO E REPRODUÇÃO DOS EXPERIMENTOS

O código-fonte, o *dataset*, o manual de anotação, os registros de auditoria, os registros de execução de todas as rodadas e as fontes deste texto estão no repositório público do projeto, <<https://github.com/vhenequim/bdgex-busca-tool-calling>>. O código, o *dataset* e os artefatos de avaliação estão sob a licença MIT, e as partes derivadas do protótipo do 1º CGEO mantêm a licença MIT original, como registrado no arquivo `THIRD_PARTY_NOTICES.md`; o texto deste trabalho segue os termos de uso do IME impressos no verso da folha de rosto. A estrutura principal é a seguinte:

```
bdgex-busca-tool-calling/
  src/pfc_busca/
    schema.py          definicao da ferramenta buscar_catalogo (12 campos)
    prompts.py         prompt de sistema (com a data de referencia)
    agent.py           traducao via ChatOllama.bind_tools() (modelos locais)
    agent_groq.py      traducao via ChatOpenAI (referencia em nuvem)
    agent_estruturado.py linhas de base com Saida Estruturada
    v2.py              especificacao v2 (descricoes e recusar_consulta)
    v3.py              v3: laco de chamadas, retorno, recusa contestada
                      e controle com Saida Estruturada
    ferramentas.py     ferramentas auxiliares e validador da v3
    dados/             lista de municipios do IBGE usada pelas ferramentas
    tools.py, db.py    busca no PostgreSQL/PostGIS (psycopg)
    pipeline.py        ciclo completo: traducao, busca e resposta
    api.py             rotas POST /api/search e GET /api/health (FastAPI)
    evaluation/
      manual_cases.py  camadas P e N, com rastreabilidade e notas
      gabarito.py      leituras multiplas (valor alternativo, opcional, trocas)
      relative_time.py regras de tempo relativo (leituras aceitas)
      dataset_builder.py pfc-dataset: constroi e valida data/dataset.json
      audit.py         pfc-auditar: auditoria do dataset
      run_evaluation.py pfc-avaliar: execucao, manifesto, repontuacao
      metrics.py       acuracia, P/R/F1 por campo, latencia
      report.py        pfc-relatorio: tabelas, figuras, Wilson, McNemar
      conferencia.py   pfc-conferir: conferencia das rodadas
      comparacao.py    Tool Calling x Saida Estruturada (tabela e macros)
      lote.py          analise do lote 1 (tabelas e macros)
      lote2.py         analise do lote 2 (plano de analise da v3)
  scripts/generate_dataset_paper.py gerador da camada G (semente 42)
  scripts/lote_validacao/          coleta, alvos, redacao, anotacao e montagem
  scripts/v3/                     ciclos, autoteste do validador, decomposicao
  docs/manual_de_anotacao.md      manual de anotacao
```

docs/lote_validacao.md	metodologia dos lotes
docs/v3.md, docs/v3_desenvolvimento.md	desenho, plano e ciclos da v3
data/dataset.json	dataset (310 consultas)
data/lote_validacao.json	lote 1
data/lote_validacao_2.json	lote 2 (teste da v3)
data/auditoria/	artefatos da auditoria
results/	registros e manifestos das rodadas
notebooks/avaliacao_colab.ipynb	rodadas completas em GPU de nuvem
notebooks/linha_de_base_colab.ipynb	linhas de base em GPU de nuvem
notebooks/lote_validacao_colab.ipynb	rodadas do lote 1 (v1 e v2)
notebooks/lote2_v3_colab.ipynb	rodadas do lote 2 (confere o hash da v3)
tests/	testes automatizados (pytest)
texto/	fontes LaTeX e PDF deste trabalho

Os passos para reproduzir os resultados, a partir de um ambiente com Python 3.12 ou superior e o Ollama instalado, são:

```

pip install -e ".[dev]"          # instala o pacote, os comandos pfc-* e o pytest
pfc-dataset                     # reconstrói data/dataset.json (mesmo hash)
pfc-auditar --anotacao-independente data/auditoria/anotacao_independente.json
ollama pull qwen3:4b-instruct-2507-q4_K_M
pfc-avaliar --modelo qwen3:4b-instruct-2507-q4_K_M --repeticoes 3 \
    --hoje 2026-09-14 --sem-sql
pfc-conferir                    # confere as rodadas antes de consolidar
pfc-relatorio --modelos qwen3:4b-instruct-2507-q4_K_M,gemma4:e4b-it-qat,...
pytest                          # testes do calculo das metricas e do dataset

```

A pontuação é sempre recalculada contra o gabarito vigente, lido do arquivo `data/dataset.json`: o comando `pfc-relatorio` relê os registros de execução e reaplica o gabarito, de modo que as tabelas do Capítulo 5 podem ser regeneradas sem nova execução dos modelos.

As rodadas dos lotes e da v3 usam o mesmo `pfc-avaliar`, com a opção `--abordagem` (`tool_calling_v2`, `saida_estruturada_v2`, `tool_calling_v3d`, `tool_calling_v3a`, `tool_calling_v3` e `saida_estruturada_v3`) e a opção `--dataset` apontada para o arquivo do lote, executadas pelos cadernos da Seção B.2; as tabelas e as macros saem de `python -m pfc_busca.evaluation.lote` e `python -m pfc_busca.evaluation.lote2`. O índice de folhas do BDGEx que as ferramentas da v3 consultam é gerado por `scripts/v3/gerar_dados_ferramentas.py` a partir da coleta do catálogo e não é versionado, como a própria coleta, porque a política do BDGEx não trata de redistribuição.